



ESCOLA TÉCNICA DE CEILÂNDIA – ETC
CURSO TÉCNICO EM INFORMÁTICA

ANDERSON
GUSTAVO
LEONARDO
VITÓRIA

CONECTA VAGAS DF
PLATAFORMA DE VAGAS DE EMPREGO E CAPACITAÇÃO PROFISSIONAL DO
DISTRITO FEDERAL

CEILÂNDIA
2026



ESCOLA TÉCNICA DE CEILÂNDIA

**ESCOLA TÉCNICA DE CEILÂNDIA – ETC
CURSO TÉCNICO EM INFORMÁTICA**

ANDERSON
GUSTAVO
LEONARDO
VITÓRIA

**CONECTA VAGAS DF
PLATAFORMA DE VAGAS DE EMPREGO E CAPACITAÇÃO PROFISSIONAL DO
DISTRITO FEDERAL**

Projeto de Conclusão de Curso da
Escola Técnica de Ceilândia –
ETC, apresentado como requisito
parcial para a obtenção do título de
Técnico em Informática, sob
orientação do Prof. Esp. Cleyton
Ferreira da Silva

CEILÂNDIA
2026

ANDERSON
GUSTAVO
LEONARDO
VITÓRIA

Projeto de Conclusão de Curso da
Escola Técnica de Ceilândia –
ETC, apresentado como requisito
parcial para a obtenção do título de
Técnico em Informática, sob
orientação do Prof. Esp. Cleyton
Ferreira da Silva

Aprovado em ____/____/_____, pela comissão julgadora:

Prof. Esp. Adalberto (Coordenador dos Cursos de TI)

Prof. Esp. Cleyton Ferreira da Silva (Orientador)

Prof. Esp. xxxxxxxxxxxxxxxxxxxxxxxxx (Banca)

Prof. Esp. xxxxxxxxxxxxxxxxxxxxxxxxx (Banca)

Prof. Esp. xxxxxxxxxxxxxxxxxxxxxxxxx (Banca)

AGRADECIMENTOS

Primeiramente agradecemos a Deus pelo dom da vida. Agradecemos aos professores de todos os módulos, que contribuíram para que este projeto fosse concluído, principalmente àqueles que, com humildade e paciência, compartilharam seus conhecimentos com dedicação e respeito, valorizando e acreditando no nosso potencial.

Agradecemos também uns aos outros, pelo trabalho em equipe, e aos colegas de curso que, direta ou indiretamente, contribuíram para o desenvolvimento do trabalho, e a todos os profissionais da escola, que tornaram o ambiente educacional um lugar propício ao aprendizado.

DEDICATÓRIA

Dedicamos este trabalho a Deus, às nossas famílias e a todos que acreditaram que seríamos capazes de concluir o curso, incentivando-nos em cada etapa do desenvolvimento deste projeto.

CESSÃO DE DIREITOS

AUTORES: Anderson, Gustavo, Leonardo e Vitória.

TÍTULO: CONECTA VAGAS DF: PLATAFORMA DE VAGAS DE EMPREGO E CAPACITAÇÃO PROFISSIONAL DO DISTRITO FEDERAL.

DATA: ____/____/2026.

Nos termos da Lei Nº 9.610/1998 (Lei de Direitos Autorais) é concedida à Escola Técnica de Ceilândia – DF, permissão para reproduzir cópias deste trabalho e para emprestar ou permitir tais cópias somente para propósitos acadêmicos e científicos. Os autores reservam outros direitos de publicação e nenhuma parte deste trabalho de conclusão de curso pode ser reproduzida sem a autorização por escrito dos seguintes autores.

Anderson

Gustavo

Leonardo

Vitória

RESUMO

Este trabalho registra as etapas que percorremos para desenvolver o Conecta Vagas DF, uma plataforma web de vagas de emprego e capacitação profissional voltada ao Distrito Federal. O nosso propósito é diminuir a distância entre quem procura trabalho e quem oferece vagas, já que grande parte das oportunidades do DF circula de forma espalhada, em cartazes, grupos de mensagens e redes sociais, sem organização e sem indicar quais candidatos combinam mais com cada vaga. No sistema, o candidato envia o currículo e a plataforma lê o arquivo e preenche o perfil automaticamente, monta um portfólio profissional e calcula, por regras, uma nota de compatibilidade de 0 a 100 entre o candidato e cada vaga, recomendando cursos e e-books gratuitos que cobrem as competências que faltam. As empresas publicam vagas a partir do texto de um anúncio ou da foto de um cartaz, lida no próprio navegador, recebem as candidaturas ordenadas pela compatibilidade e consultam um banco de talentos. Onde as regras de leitura não têm pista, o sistema usa padrões tirados das vagas e dos cursos já cadastrados e mostra cada padrão usado. Os e-books ficam guardados na biblioteca da plataforma, e o candidato pode excluir a própria conta e todos os seus dados, como pede a LGPD. Desenvolvemos o projeto em PHP, no padrão MVC, com banco de dados MySQL/MariaDB e testes automáticos.

PALAVRAS-CHAVE: Emprego. Vagas. Currículo. Compatibilidade. Distrito Federal.

ABSTRACT

This work records the steps we took to develop Conecta Vagas DF, a web platform for job openings and professional training focused on the Federal District of Brazil. Our purpose is to reduce the distance between people looking for work and those offering jobs, since most opportunities in the Federal District circulate in a scattered way, on posters, messaging groups and social networks, without organization and without showing which candidates best match each opening. In the system, candidates upload their résumé and the platform reads the file and fills in their profile automatically, builds a professional portfolio and calculates, through rules, a compatibility score from 0 to 100 between the candidate and each job, recommending free courses and e-books that cover the missing skills. Companies publish jobs from the text of an advertisement or the photo of a poster, read inside the browser itself, receive applications sorted by compatibility and search a talent pool. Where the reading rules have no clue, the system uses patterns taken from the jobs and courses already registered and shows every pattern it used. E-books are stored in the platform's own library, and candidates can delete their own account and all their data, as required by the Brazilian data protection law (LGPD). We developed the project in PHP, following the MVC pattern, with a MySQL/MariaDB database and automated tests.

KEYWORDS: *Employment. Jobs. Résumé. Matching. Federal District.*

LISTA DE FIGURAS

Figura 1 – Diagrama de Caso de Uso Geral do Sistema Conecta Vagas DF (visão de cada personagem).....	52
Figura 2 – Diagrama de Caso de Uso Realizar Login.....	53
Figura 3 – Diagrama de Caso de Uso Acesso à Conta.....	53
Figura 4 – Diagrama de Caso de Uso Consultas Públicas.....	54
Figura 5 – Diagrama de Caso de Uso Perfil do Candidato.....	54
Figura 6 – Diagrama de Caso de Uso Candidatura.....	55
Figura 7 – Diagrama de Caso de Uso Manter Vaga.....	55
Figura 8 – Diagrama de Caso de Uso Área da Empresa.....	56
Figura 9 – Diagrama de Caso de Uso Planos.....	56
Figura 10 – Diagrama de Caso de Uso Manter Usuário.....	57
Figura 11 – Diagrama de Caso de Uso Manter Categoria.....	57
Figura 12 – Diagrama de Caso de Uso Manter Curso/E-book.....	58
Figura 13 – Diagrama de Caso de Uso Visualizar Pannel.....	58
Figura 14 – Diagrama de Caso de Uso Gerenciar Assinaturas.....	59
Figura 15 – Diagrama de Caso de Uso Padrões Automáticos da Extração.....	59
Figura 16 – Diagrama de Sequência Realizar Login.....	84
Figura 17 – Diagrama de Sequência Cadastrar Conta.....	85
Figura 18 – Diagrama de Sequência Pesquisar Vagas.....	86
Figura 19 – Diagrama de Sequência Enviar Currículo (extração e match).....	86
Figura 20 – Diagrama de Sequência Candidatar-se à Vaga.....	87
Figura 21 – Diagrama de Sequência Cancelar Candidatura.....	87
Figura 22 – Diagrama de Sequência Cadastrar Vaga (leitura do cartaz no navegador e extração).....	88
Figura 23 – Diagrama de Sequência Alterar Vaga.....	89
Figura 24 – Diagrama de Sequência Excluir Vaga.....	90
Figura 25 – Diagrama de Sequência Atualizar Status da Candidatura.....	91
Figura 26 – Diagrama de Sequência Assinar Plano.....	91
Figura 27 – Diagrama de Sequência Manter Usuário.....	92
Figura 28 – Diagrama de Sequência Excluir a Própria Conta (LGPD).....	93
Figura 29 – Diagrama de Sequência Trazer os PDFs para a Biblioteca.....	94
Figura 30 – Diagrama de Sequência Extração de Vaga com os Padrões Automáticos.....	94
Figura 31 – Diagrama de Classes de Domínio do Sistema Conecta Vagas DF.....	96
Figura 32 – Modelo Conceitual do Banco de Dados.....	98
Figura 33 – Modelo Lógico do Banco de Dados tcc_final (Diagrama Entidade-Relacionamento).....	100
Figura 34 – Tela Inicial.....	101
Figura 35 – Tela de Login.....	102
Figura 36 – Tela de Cadastro de Conta.....	102
Figura 37 – Tela de Consulta de Vagas.....	103
Figura 38 – Tela de Detalhe da Vaga com o Match.....	104

Figura 39 – Tela do Perfil do Candidato e Exclusão da Conta.....	104
Figura 40 – Tela de E-books.....	105
Figura 41 – Tela da Ficha do E-book com o Botão Baixar.....	106
Figura 42 – Tela de Planos e Assinaturas.....	106
Figura 43 – Tela de Cadastro de Vaga pela Empresa.....	107
Figura 44 – Tela do Relatório da Extração da Vaga.....	108
Figura 45 – Tela dos Padrões Automáticos Usados.....	109
Figura 46 – Tela da Prévia das Fichas com a Capa Tirada do PDF.....	109
Figura 47 – Tela do Painel do Administrador.....	110
Figura 48 – Tela de Manutenção de Usuários.....	111
Figura 49 – Arquitetura em camadas (MVC).....	121

LISTA DE TABELAS

Tabela 1 – Especificação do Caso de Uso Realizar Login.....	60
Tabela 2 – Especificação do Caso de Uso Cadastrar Conta.....	61
Tabela 3 – Especificação do Caso de Uso Recuperar Senha.....	62
Tabela 4 – Especificação do Caso de Uso Consultar Vagas.....	63
Tabela 5 – Especificação do Caso de Uso Consultar Cursos e E-books.....	63
Tabela 6 – Especificação do Caso de Uso Alterar Perfil do Candidato.....	64
Tabela 7 – Especificação do Caso de Uso Enviar Currículo (extração).....	64
Tabela 8 – Especificação do Caso de Uso Visualizar Portfólio e Match.....	65
Tabela 9 – Especificação do Caso de Uso Candidatar-se à Vaga.....	66
Tabela 10 – Especificação do Caso de Uso Cancelar Candidatura.....	66
Tabela 11 – Especificação do Caso de Uso Manter Perfil da Empresa.....	67
Tabela 12 – Especificação do Caso de Uso Cadastrar Vaga.....	68
Tabela 13 – Especificação do Caso de Uso Alterar Vaga.....	69
Tabela 14 – Especificação do Caso de Uso Excluir Vaga.....	69
Tabela 15 – Especificação do Caso de Uso Gerenciar Candidaturas Recebidas.....	70
Tabela 16 – Especificação do Caso de Uso Consultar Banco de Talentos.....	71
Tabela 17 – Especificação do Caso de Uso Assinar Plano.....	71
Tabela 18 – Especificação do Caso de Uso Cancelar Assinatura.....	72
Tabela 19 – Especificação do Caso de Uso Cadastrar Usuário.....	73
Tabela 20 – Especificação do Caso de Uso Alterar Usuário.....	73
Tabela 21 – Especificação do Caso de Uso Excluir Usuário.....	74
Tabela 22 – Especificação do Caso de Uso Ativar / Desativar Usuário.....	75
Tabela 23 – Especificação do Caso de Uso Cadastrar Categoria.....	75
Tabela 24 – Especificação do Caso de Uso Alterar Categoria.....	76
Tabela 25 – Especificação do Caso de Uso Excluir Categoria.....	76
Tabela 26 – Especificação do Caso de Uso Ativar / Desativar Categoria.....	77
Tabela 27 – Especificação do Caso de Uso Cadastrar Curso/E-book.....	78
Tabela 28 – Especificação do Caso de Uso Alterar Curso/E-book.....	78
Tabela 29 – Especificação do Caso de Uso Excluir Curso/E-book.....	79
Tabela 30 – Especificação do Caso de Uso Importar Fichas em Lote.....	79
Tabela 31 – Especificação do Caso de Uso Trazer os PDFs para a Biblioteca.....	80
Tabela 32 – Especificação do Caso de Uso Visualizar Pannel.....	81
Tabela 33 – Especificação do Caso de Uso Excluir a Própria Conta (LGPD).....	81
Tabela 34 – Especificação do Caso de Uso Gerenciar Assinaturas.....	82
Tabela 35 – Especificação do Caso de Uso Extrair com os Padrões Automáticos.....	83
Tabela 36 – Dicionário de dados da tabela usuarios.....	112
Tabela 37 – Dicionário de dados da tabela perfis.....	113
Tabela 38 – Dicionário de dados da tabela categorias.....	114
Tabela 39 – Dicionário de dados da tabela vagas.....	115
Tabela 40 – Dicionário de dados da tabela cursos.....	116
Tabela 41 – Dicionário de dados da tabela curriculos.....	116

Tabela 42 – Dicionário de dados da tabela matches.....	117
Tabela 43 – Dicionário de dados da tabela assinaturas.....	118
Tabela 44 – Dicionário de dados da tabela candidaturas.....	118
Tabela 45 – Dicionário de dados da tabela tentativas_login.....	119
Tabela 46 – Dicionário de dados da tabela redefinicoes_senha.....	119

LISTA DE QUADROS

Quadro 1 – Papéis profissionais e artefatos produzidos.....	29
Quadro 2 – Cronograma de atividades.....	34
Quadro 3 – Cronograma de software.....	35
Quadro 4 – Mapa de rotas.....	122
Quadro 5 – Casos de teste.....	124
Quadro 6 – Rastreabilidade entre requisito, caso de uso, código e banco.....	125
Quadro 7 – Caminho no código da navegação e busca de vagas.....	127
Quadro 8 – Caminho no código de cursos, e-books e vídeos.....	128
Quadro 9 – Caminho no código do cadastro.....	129
Quadro 10 – Caminho no código do login.....	130
Quadro 11 – Caminho no código da recuperação de senha.....	131
Quadro 12 – Caminho no código do perfil do candidato.....	132
Quadro 13 – Caminho no código do envio e leitura do currículo.....	134
Quadro 14 – Caminho no código da foto do currículo.....	135
Quadro 15 – Caminho no código do portfólio.....	136
Quadro 16 – Caminho no código do match.....	137
Quadro 17 – Caminho no código da candidatura.....	138
Quadro 18 – Caminho no código do cancelamento.....	139
Quadro 19 – Caminho no código dos planos.....	140
Quadro 20 – Caminho no código da exclusão de conta.....	141
Quadro 21 – Caminho no código do perfil da empresa.....	142
Quadro 22 – Caminho no código da publicação de vaga.....	143
Quadro 23 – Caminho no código do aviso de vaga parecida.....	144
Quadro 24 – Caminho no código de ativar, pausar e encerrar.....	145
Quadro 25 – Caminho no código das candidaturas recebidas.....	146
Quadro 26 – Caminho no código do banco de talentos e do download.....	147
Quadro 27 – Caminho no código do painel e dos gráficos.....	148
Quadro 28 – Caminho no código da manutenção de usuários.....	149
Quadro 29 – Caminho no código das categorias.....	149
Quadro 30 – Caminho no código do cadastro por ficha.....	151
Quadro 31 – Caminho no código da biblioteca de PDFs.....	152
Quadro 32 – Caminho no código das assinaturas.....	152
Quadro 33 – Caminho no código dos padrões automáticos.....	154
Quadro 34 – Caminho no código da limpeza diária.....	154
Quadro 35 – Caminho no código da doação por Pix.....	155
Quadro 36 – Pastas e arquivos da raiz do projeto.....	162
Quadro 37 – Funções de app/Core/Auth.php.....	164
Quadro 38 – Função registrada em app/Core/Autoloader.php.....	164
Quadro 39 – Método de app/Core/Controller.php.....	165
Quadro 40 – Funções de app/Core/Csrf.php.....	165
Quadro 41 – Classe e método públicos de app/Core/Database.php.....	166

Quadro 42 – Métodos de app/Core/Router.php.....	167
Quadro 43 – Funções de app/Core/Session.php.....	169
Quadro 44 – Constantes e funções de app/Core/Upload.php.....	170
Quadro 45 – Classe, função global e tratador de erro de app/Core/View.php.....	171
Quadro 46 – Passos de app/Core/bootstrap.php.....	172
Quadro 47 – Funções de app/Core/helpers.php.....	174
Quadro 48 – Tabela de rotas (public/index.php).....	177
Quadro 49 – Arquivos .htaccess e o que bloqueiam.....	178
Quadro 50 – Constantes de config/config.php.....	179
Quadro 51 – Os doze controladores e as áreas que atendem.....	181
Quadro 52 – Ações e métodos do AdminController.....	182
Quadro 53 – Ações do ArquivoController.....	187
Quadro 54 – Ações do AuthController.....	188
Quadro 55 – Ações do CandidaturaController.....	190
Quadro 56 – Ações do CurriculoController.....	191
Quadro 57 – Ações do CursoController.....	194
Quadro 58 – Ações e métodos do EmpresaController.....	195
Quadro 59 – Ações do HomeController.....	199
Quadro 60 – Ações do PasswordController.....	200
Quadro 61 – Ações do PerfilController.....	201
Quadro 62 – Ação do PlanosController.....	203
Quadro 63 – Ações do VagaController.....	204
Quadro 64 – Métodos públicos de AssinaturaDAO.....	207
Quadro 65 – Métodos públicos de CandidaturaDAO.....	209
Quadro 66 – Métodos públicos de CategoriaDAO.....	210
Quadro 67 – Métodos públicos de CurriculoDAO.....	211
Quadro 68 – Métodos públicos de CursoDAO.....	212
Quadro 69 – Métodos públicos de MatchDAO.....	213
Quadro 70 – Métodos públicos de PerfilDAO.....	214
Quadro 71 – Métodos públicos de UsuarioDAO.....	216
Quadro 72 – Métodos públicos de VagaDAO.....	218
Quadro 73 – Métodos públicos de PerfilDTO.....	220
Quadro 74 – Métodos públicos de UsuarioDTO.....	220
Quadro 75 – Relacionamentos.....	223
Quadro 76 – Métodos públicos do MatchService.....	225
Quadro 77 – Métodos públicos da classe Competencias.....	227
Quadro 78 – Métodos públicos da classe Portfolio.....	229
Quadro 79 – Métodos públicos da classe Pix.....	230
Quadro 80 – Métodos públicos da classe ExtracaoVaga.....	233
Quadro 81 – Métodos públicos da classe ExtracaoCurso.....	235
Quadro 82 – Métodos públicos da classe PadroesExtracao.....	238
Quadro 83 – Métodos públicos da classe PromptsPesquisa.....	240
Quadro 84 – Métodos públicos da classe FontesCursos.....	242
Quadro 85 – Arquivos da leitura de documentos e o papel de cada um.....	243

Quadro 86 – Limites de segurança da leitura de arquivos e imagens.....	244
Quadro 87 – Métodos públicos do LeitorDocumento.....	245
Quadro 88 – Métodos públicos do PdfTexto.....	249
Quadro 89 – Métodos e propriedades públicas do DocxTexto.....	254
Quadro 90 – Métodos públicos da OcrImagem.....	256
Quadro 91 – Métodos públicos da ExtracaoCurriculo.....	259
Quadro 92 – Etapas do ExtracaoCurriculo::extrairCampos().....	259
Quadro 93 – Seções reconhecidas no currículo e exemplos de títulos aceitos.....	261
Quadro 94 – Métodos públicos da AplicacaoCurriculo.....	264
Quadro 95 – Métodos públicos da ImagemRemota.....	267
Quadro 96 – Métodos da CapaPdf e papel do capa-pdf.ps1.....	270
Quadro 97 – Todas as views de app/Views, a tela que mostram, quem vê e os dados recebidos do controller.....	280
Quadro 98 – Jornadas de tests/jornadas.php, passo a passo.....	293

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
ANPD	Autoridade Nacional de Proteção de Dados
CDC	Código de Defesa do Consumidor
CLT	Consolidação das Leis do Trabalho
CNH	Carteira Nacional de Habilitação
CNPJ	Cadastro Nacional da Pessoa Jurídica
CSRF	Cross-Site Request Forgery (falsificação de requisição entre sites)
CSS	Cascading Style Sheets
DAO	Data Access Object (objeto de acesso a dados)
DF	Distrito Federal
DOC/DOCX	Formatos de documento do Microsoft Word
DTO	Data Transfer Object (objeto de transferência de dados)
EAD	Educação a Distância
ETC	Escola Técnica de Ceilândia
FK	Foreign Key (chave estrangeira)
HTML	HyperText Markup Language
HTTP/HTTPS	HyperText Transfer Protocol / HyperText Transfer Protocol Secure
IP	Internet Protocol (endereço de rede)
LGPD	Lei Geral de Proteção de Dados Pessoais
MVC	Model, View e Controller (modelo, visão e controlador)
OCR	Optical Character Recognition (reconhecimento óptico de caracteres)
OWASP	Open Worldwide Application Security Project
PCD	Pessoa com Deficiência
PDF	Portable Document Format
PDO	PHP Data Objects
PHP	PHP: Hypertext Preprocessor
PJ	Pessoa Jurídica
PK	Primary Key (chave primária)
RF	Requisito Funcional
RNF	Requisito Não Funcional
SGBD	Sistema de Gerenciamento de Banco de Dados
SQL	Structured Query Language

TI	Tecnologia da Informação
UC	Use Case (caso de uso)
UF	Unidade Federativa
UML	Unified Modeling Language (linguagem de modelagem unificada)
URL	Uniform Resource Locator (endereço de uma página)
VIP	Very Important Person (plano pago do candidato)
XSS	Cross-Site Scripting

SUMÁRIO

01 INTRODUÇÃO.....	25
1.1. OBJETIVOS.....	26
1.1.1. OBJETIVO GERAL.....	26
1.1.2. OBJETIVOS ESPECÍFICOS.....	26
1.2. JUSTIFICATIVA.....	27
02 DESENVOLVIMENTO.....	28
2.1. METODOLOGIA.....	28
2.2. FUNDAMENTAÇÃO TEÓRICA (REFERENCIAL TEÓRICO).....	29
2.3. TIPO DE PESQUISA.....	31
2.4. POPULAÇÃO E AMOSTRA.....	31
2.5. INSTRUMENTOS DE COLETA DE DADOS.....	32
2.6. TÉCNICAS DE ANÁLISE DOS DADOS.....	32
2.7. LEI GERAL DE PROTEÇÃO DE DADOS.....	32
2.8. CÓDIGO DE DEFESA DO CONSUMIDOR E/OUTRA LEGISLAÇÃO PERTINENTE À TEMÁTICA DO TRABALHO.....	33
2.9. CRONOGRAMA DE ATIVIDADES.....	34
2.10. CRONOGRAMA DE SOFTWARE.....	34
03 SITUAÇÃO PROBLEMA.....	36
04 EXPOSIÇÃO DO TEMA PRINCIPAL.....	37
05 PROPOSTA.....	38
06 ABRANGÊNCIA DO SISTEMA.....	40
6.1 Requisitos Funcionais.....	40
6.2 Requisitos Não Funcionais.....	46
6.3 Metodologia e Tecnologia Utilizada.....	49
07 RESULTADOS OBTIDOS / ESPERADOS.....	50
08 MODELO DO SOFTWARE.....	51
8.1 Diagramas de Caso de Uso.....	51
8.1.1 Diagrama de Caso de Uso Geral Sistema Conecta Vagas DF.....	51
8.1.2 Diagrama de Caso de Uso Específico Realizar Login.....	52
8.1.3 Diagrama de Caso de Uso Específico Acesso à Conta.....	53
8.1.4 Diagrama de Caso de Uso Específico Consultas Públicas.....	53
8.1.5 Diagrama de Caso de Uso Específico Perfil do Candidato.....	54

8.1.6 Diagrama de Caso de Uso Específico Candidatura.....	54
8.1.7 Diagrama de Caso de Uso Específico Manter Vaga.....	55
8.1.8 Diagrama de Caso de Uso Específico Área da Empresa.....	55
8.1.9 Diagrama de Caso de Uso Específico Planos.....	56
8.1.10 Diagrama de Caso de Uso Específico Manter Usuário.....	56
8.1.11 Diagrama de Caso de Uso Específico Manter Categoria.....	57
8.1.12 Diagrama de Caso de Uso Específico Manter Curso/E-book.....	58
8.1.13 Diagrama de Caso de Uso Específico Visualizar Pannel.....	58
8.1.14 Diagrama de Caso de Uso Específico Gerenciar Assinaturas.....	58
8.1.15 Diagrama de Caso de Uso Específico Padrões Automáticos da Extração.....	59
8.1.16 Especificação do Caso de Uso Realizar Login.....	59
8.1.17 Especificação do Caso de Uso Cadastrar Conta.....	61
8.1.18 Especificação do Caso de Uso Recuperar Senha.....	61
8.1.19 Especificação do Caso de Uso Consultar Vagas.....	62
8.1.20 Especificação do Caso de Uso Consultar Cursos e E-books.....	63
8.1.21 Especificação do Caso de Uso Alterar Perfil do Candidato.....	63
8.1.22 Especificação do Caso de Uso Enviar Currículo (extração).....	64
8.1.23 Especificação do Caso de Uso Visualizar Portfólio e Match.....	64
8.1.24 Especificação do Caso de Uso Candidatar-se à Vaga.....	65
8.1.25 Especificação do Caso de Uso Cancelar Candidatura.....	66
8.1.26 Especificação do Caso de Uso Manter Perfil da Empresa.....	67
8.1.27 Especificação do Caso de Uso Cadastrar Vaga.....	67
8.1.28 Especificação do Caso de Uso Alterar Vaga.....	68
8.1.29 Especificação do Caso de Uso Excluir Vaga.....	69
8.1.30 Especificação do Caso de Uso Gerenciar Candidaturas Recebidas.....	70
8.1.31 Especificação do Caso de Uso Consultar Banco de Talentos.....	70
8.1.32 Especificação do Caso de Uso Assinar Plano.....	71
8.1.33 Especificação do Caso de Uso Cancelar Assinatura.....	72
8.1.34 Especificação do Caso de Uso Cadastrar Usuário.....	72
8.1.35 Especificação do Caso de Uso Alterar Usuário.....	73
8.1.36 Especificação do Caso de Uso Excluir Usuário.....	73
8.1.37 Especificação do Caso de Uso Ativar / Desativar Usuário.....	74
8.1.38 Especificação do Caso de Uso Cadastrar Categoria.....	75

8.1.39 Especificação do Caso de Uso Alterar Categoria.....	75
8.1.40 Especificação do Caso de Uso Excluir Categoria.....	76
8.1.41 Especificação do Caso de Uso Ativar / Desativar Categoria.....	76
8.1.42 Especificação do Caso de Uso Cadastrar Curso/E-book.....	77
8.1.43 Especificação do Caso de Uso Alterar Curso/E-book.....	78
8.1.44 Especificação do Caso de Uso Excluir Curso/E-book.....	78
8.1.45 Especificação do Caso de Uso Importar Fichas em Lote.....	79
8.1.46 Especificação do Caso de Uso Trazer os PDFs para a Biblioteca.....	79
8.1.47 Especificação do Caso de Uso Visualizar Pannel.....	80
8.1.48 Especificação do Caso de Uso Excluir a Própria Conta (LGPD).....	81
8.1.49 Especificação do Caso de Uso Gerenciar Assinaturas.....	82
8.1.50 Especificação do Caso de Uso Extrair com os Padrões Automáticos.....	82
8.2 Diagramas de Sequência.....	84
8.2.1 Diagrama de Sequência Realizar Login.....	84
8.2.2 Diagrama de Sequência Cadastrar Conta.....	85
8.2.3 Diagrama de Sequência Pesquisar Vagas.....	86
8.2.4 Diagrama de Sequência Enviar Currículo (extração e match).....	86
8.2.5 Diagrama de Sequência Candidatar-se à Vaga.....	87
8.2.6 Diagrama de Sequência Cancelar Candidatura.....	87
8.2.7 Diagrama de Sequência Cadastrar Vaga (leitura do cartaz no navegador e extração)	88
8.2.8 Diagrama de Sequência Alterar Vaga.....	89
8.2.9 Diagrama de Sequência Excluir Vaga.....	90
8.2.10 Diagrama de Sequência Atualizar Status da Candidatura.....	91
8.2.11 Diagrama de Sequência Assinar Plano.....	91
8.2.12 Diagrama de Sequência Manter Usuário.....	92
8.2.13 Diagrama de Sequência Excluir a Própria Conta (LGPD).....	93
8.2.14 Diagrama de Sequência Trazer os PDFs para a Biblioteca.....	94
8.2.15 Diagrama de Sequência Extração de Vaga com os Padrões Automáticos.....	94
8.3 Diagrama de Classes.....	95
8.3.1 Diagrama de Classe Sistema Conecta Vagas DF.....	96
8.4 Modelo Conceitual do Banco de Dados.....	96
8.5 Modelo Lógico do Banco de Dados (Diagrama Entidade-Relacionamento).....	99
8.6 Telas.....	100

8.6.1 Tela Inicial Sistema Conecta Vagas DF.....	100
8.6.2 Tela de Login Sistema Conecta Vagas DF.....	101
8.6.3 Tela de Cadastro de Conta Sistema Conecta Vagas DF.....	102
8.6.4 Tela de Consulta de Vagas Sistema Conecta Vagas DF.....	103
8.6.5 Tela de Detalhe da Vaga com o Match Sistema Conecta Vagas DF.....	103
8.6.6 Tela do Perfil do Candidato e Exclusão da Conta Sistema Conecta Vagas DF.....	104
8.6.7 Tela de E-books Sistema Conecta Vagas DF.....	105
8.6.8 Tela da Ficha do E-book com o Botão Baixar Sistema Conecta Vagas DF.....	105
8.6.9 Tela de Planos e Assinaturas Sistema Conecta Vagas DF.....	106
8.6.10 Tela de Cadastro de Vaga pela Empresa Sistema Conecta Vagas DF.....	107
8.6.11 Tela do Relatório da Extração da Vaga Sistema Conecta Vagas DF.....	107
8.6.12 Tela dos Padrões Automáticos Usados Sistema Conecta Vagas DF.....	108
8.6.13 Tela da Prévia das Fichas com a Capa Tirada do PDF Sistema Conecta Vagas DF	109
8.6.14 Tela do Painel do Administrador Sistema Conecta Vagas DF.....	110
8.6.15 Tela de Manutenção de Usuários Sistema Conecta Vagas DF.....	110
8.7 Dicionário de Dados.....	111
8.8 Arquitetura do Sistema.....	119
8.9 Plano de Testes.....	123
8.10 Matriz de Rastreabilidade.....	125
8.11 Funcionalidades em Detalhe.....	126
8.11.1 Navegação e busca de vagas (Candidato).....	126
8.11.2 Cursos, e-books e vídeos: Baixar e Acessar (Candidato).....	127
8.11.3 Cadastro com aceite da LGPD (Candidato).....	128
8.11.4 Login com tolerância a erros e pausa por segurança (Candidato).....	129
8.11.5 Recuperação de senha (Candidato).....	130
8.11.6 Perfil do candidato.....	132
8.11.7 Envio e leitura do currículo, com relatório da extração (Candidato).....	133
8.11.8 Foto do currículo (Candidato).....	134
8.11.9 Portfólio (Candidato).....	135
8.11.10 Match com as vagas (Candidato).....	136
8.11.11 Candidatura e acompanhamento de status (Candidato).....	137
8.11.12 Cancelamento da candidatura (Candidato).....	138
8.11.13 Planos VIP e Premium (Candidato e Empresa).....	139

8.11.14 Exclusão da própria conta: LGPD (Candidato).....	140
8.11.15 Perfil da empresa.....	141
8.11.16 Publicação de vaga por formulário, por anúncio e por cartaz: OCR no navegador (Empresa).....	142
8.11.17 Aviso de vaga parecida (Empresa).....	143
8.11.18 Ativar, pausar e encerrar a vaga, e o limite do plano (Empresa).....	144
8.11.19 Candidaturas recebidas, ordenadas pelo match (Empresa).....	145
8.11.20 Banco de talentos e download de currículo (Empresa).....	146
8.11.21 Painel e gráficos (Administrador).....	147
8.11.22 Manutenção de usuários (Administrador).....	148
8.11.23 Categorias (Administrador).....	149
8.11.24 Cursos e e-books por ficha (uma e várias), imagem padrão e capa do PDF (Administrador).....	150
8.11.25 Biblioteca de PDFs (Administrador).....	151
8.11.26 Assinaturas (Administrador).....	152
8.11.27 Padrões automáticos das máquinas de extração (Sistema).....	153
8.11.28 Limpeza diária de arquivos (Sistema).....	154
8.11.29 Doação por Pix (Sistema).....	154
09 CONCLUSÃO.....	156
REFERÊNCIAS.....	158
APÊNDICE A – DOCUMENTAÇÃO DO CÓDIGO-FONTE.....	160
A.1 Organização das pastas do projeto.....	160
A.2 Núcleo da aplicação (app/Core).....	162
A.2.1 Auth.php.....	163
A.2.2 Autoloader.php.....	164
A.2.3 Controller.php.....	164
A.2.4 Csrf.php.....	165
A.2.5 Database.php.....	165
A.2.6 Router.php.....	166
A.2.7 Session.php.....	167
A.2.8 Upload.php.....	169
A.2.9 View.php.....	170
A.2.10 bootstrap.php.....	171
A.2.11 helpers.php.....	172

A.3 Porta de entrada, rotas e configuração.....	175
A.4 Controladores (app/Controllers).....	180
A.4.1 AdminController.....	182
A.4.2 ArquivoController.....	187
A.4.3 AuthController.....	188
A.4.4 CandidaturaController.....	189
A.4.5 CurrículoController.....	190
A.4.6 CursoController.....	193
A.4.7 EmpresaController.....	195
A.4.8 HomeController.....	199
A.4.9 PasswordController.....	200
A.4.10 PerfilController.....	201
A.4.11 PlanosController.....	203
A.4.12 VagaController.....	204
A.5 Modelos e objetos de dados (app/Models e app/DTO).....	205
A.5.1 AssinaturaDAO.....	206
A.5.2 CandidaturaDAO.....	208
A.5.3 CategoriaDAO.....	210
A.5.4 CurrículoDAO.....	210
A.5.5 CursoDAO.....	211
A.5.6 MatchDAO.....	212
A.5.7 PerfilDAO.....	213
A.5.8 UsuarioDAO.....	214
A.5.9 VagaDAO.....	217
A.5.10 PerfilDTO.....	219
A.5.11 UsuarioDTO.....	220
A.6 Banco de dados: scripts e dados de demonstração (database/).....	220
A.7 Serviços de negócio (app/Services).....	223
A.7.1 MatchService.php.....	223
A.7.2 Competencias.php.....	225
A.7.3 Portfolio.php.....	227
A.7.4 Pix.php.....	229
A.8 Extração de vagas e cursos e padrões automáticos.....	230

A.8.1 ExtracaoVaga.php.....	230
A.8.2 ExtracaoCurso.php.....	233
A.8.3 PadroesExtracao.php.....	236
A.8.4 PromptsPesquisa.php.....	238
A.8.5 FontesCursos.php.....	240
A.9 Leitura de documentos, currículo, imagens e capa do PDF.....	242
A.9.1 LeitorDocumento (app/Services/Extracao/LeitorDocumento.php).....	244
A.9.2 PdfTexto (app/Services/Extracao/PdfTexto.php).....	248
A.9.3 DocxTexto (app/Services/Extracao/DocxTexto.php).....	253
A.9.4 OcrImagem (app/Services/Extracao/OcrImagem.php).....	255
A.9.5 ExtracaoCurriculo (app/Services/Extracao/ExtracaoCurriculo.php).....	258
A.9.6 AplicacaoCurriculo (app/Services/Extracao/AplicacaoCurriculo.php).....	264
A.9.7 ImagemRemota (app/Services/Extracao/ImagemRemota.php).....	265
A.9.8 CapaPdf (app/Services/Extracao/CapaPdf.php e capa-pdf.ps1).....	270
A.10 Telas e componentes (app/Views).....	272
A.10.1 Layouts: header.php, footer.php e admin_nav.php.....	273
A.10.2 Partial de componentes do portal: componentes.php.....	274
A.10.3 Partial dos gráficos e peças do painel: graficos.php.....	275
A.10.4 Partials de apoio: icones.php e doacao.php.....	276
A.10.5 Partial do relatório da extração: relatorio_extracao.php e o relatório da vaga.....	277
A.10.6 Quadro geral das telas.....	278
A.10.7 A tela mais importante do candidato: perfil e portfólio.....	280
A.10.8 A visão geral com gráficos: admin/painel.php.....	281
A.10.9 Vagas com o relatório da extração: admin/vagas.php.....	282
A.10.10 Cursos com a prévia das fichas: admin/cursos.php.....	283
A.11 JavaScript e estilos (public/assets).....	284
A.11.1 app.js: comportamentos comuns de toda página.....	284
A.11.2 anuncios.js: cartaz ampliado, compartilhar e contador de caracteres.....	285
A.11.3 leitor-cartaz.js e o Tesseract.js (vendedor/tesseract): o OCR no navegador.....	286
A.11.4 painel.js: a dica dos gráficos.....	287
A.11.5 Bibliotecas de terceiros guardadas no site: vendedor/qrcode.js.....	287
A.11.6 app.css e site.css: a base visual e o layout do site.....	287
A.11.7 anuncios.css e portfolio.css: vagas e portfólio.....	288

A.11.8 painel.css: os dashboards do administrador e da empresa.....	288
A.11.9 Responsividade e acessibilidade: um fio comum nas cinco folhas de estilo.....	289
A.12 Testes automáticos e proteção do código.....	290
A.12.1 tests/lint.php.....	290
A.12.2 tests/smoke.php.....	290
A.12.3 tests/jornadas.php.....	292
A.12.4 tests/verificar.bat.....	293
A.12.5 .github/hooks/pre-commit.....	294
A.12.6 database/resetar_senhas.php.....	294

01 INTRODUÇÃO

Neste trabalho nós desenvolvemos e documentamos o Conecta Vagas DF, uma plataforma web que reúne num só lugar as vagas de emprego, os cursos e e-books de capacitação e os perfis profissionais dos candidatos do Distrito Federal. A ideia é aproximar quem procura trabalho das empresas que oferecem oportunidades.

Escolhemos esse tema depois de observar como as vagas circulam na nossa região. Boa parte delas é divulgada em cartazes, em grupos de mensagens e em perfis de redes sociais, de forma espalhada e sem padrão. O candidato precisa procurar em muitos lugares, não sabe se o seu perfil atende ao que a vaga pede e quase nunca recebe retorno. A empresa, por sua vez, recebe currículos sem organização e perde tempo para descobrir quais candidatos combinam mais com a vaga.

Os nossos principais objetivos são centralizar as vagas do DF, preencher o perfil automaticamente a partir do currículo, calcular a compatibilidade entre candidato e vaga, organizar as candidaturas com o acompanhamento do status e recomendar cursos e e-books que ajudem o candidato a se qualificar. Durante o desenvolvimento, o sistema ganhou também os padrões automáticos, tirados dos próprios cadastros, que ajudam a leitura de anúncios e de fichas de cursos quando as regras não têm pista, e um leitor de cartazes que funciona no próprio navegador, sem que ninguém precise instalar programas.

Para organizar o projeto, pensamos em quatro personagens. O Candidato é a pessoa que procura emprego; quem entra no site sem conta para ver vagas e cursos é, para nós, um candidato que ainda não se cadastrou. A Empresa é quem publica as vagas e acompanha as candidaturas. O Administrador cuida da plataforma inteira. O quarto personagem é o próprio Sistema, que faz uma parte do trabalho sozinho, como ler os documentos enviados, calcular o match, proteger as contas e limpar os arquivos que ficaram sem uso. Todos os diagramas deste documento usam esses quatro personagens.

Dividimos o trabalho em três partes. A primeira é o levantamento do problema e do que candidatos e empresas precisam. A segunda é a documentação do projeto, onde mostramos como o software foi pensado: requisitos, casos de uso, classes, sequências, modelo de dados e testes. A terceira é a programação do sistema.

Este documento segue o modelo de Projeto de Conclusão de Curso da Escola Técnica de Ceilândia. No capítulo 2 apresentamos a metodologia, a fundamentação teórica, o tipo de pesquisa, a legislação que se aplica ao sistema e os cronogramas. Nos capítulos 3, 4 e 5 tratamos do problema, do tema e da proposta. No capítulo 6 estão os requisitos e as tecnologias, no

capítulo 7 os resultados, no capítulo 8 o modelo do software, com os diagramas, as telas e o dicionário de dados, e no capítulo 9 a conclusão.

Ao final, esperamos que o software faça bem as funções descritas neste documento e que possa ser usado por candidatos, empresas e instituições do Distrito Federal que precisam de um jeito mais organizado de divulgar e encontrar oportunidades.

1.1. OBJETIVOS

1.1.1. OBJETIVO GERAL

Desenvolver uma plataforma web que centralize vagas de emprego e cursos de capacitação do Distrito Federal e que indique, de forma automática e explicável, a compatibilidade entre candidatos e vagas, com uma leitura de dados que usa o que já foi cadastrado para errar cada vez menos.

1.1.2. OBJETIVOS ESPECÍFICOS

- Centralizar a divulgação de vagas de emprego do Distrito Federal, com busca e filtros;
- Automatizar o preenchimento do perfil do candidato a partir do currículo enviado (PDF, DOC ou DOCX), inclusive a foto;
- Gerar automaticamente o portfólio profissional do candidato;
- Calcular a compatibilidade (match) entre candidato e vaga, com nota de 0 a 100 e explicação da nota;
- Organizar o processo de candidatura, com acompanhamento de status e retorno da empresa;
- Permitir que empresas publiquem vagas a partir do texto de um anúncio ou da imagem de um cartaz, lida no próprio navegador, sem instalar programas;
- Recomendar cursos e e-books que cubram as competências que faltam ao candidato, com os e-books disponíveis para baixar na biblioteca da plataforma;
- Completar a leitura de anúncios e de fichas de cursos com padrões automáticos tirados das vagas e dos cursos já cadastrados, de forma determinística e explicável;
- Garantir ao usuário o controle dos próprios dados, inclusive a exclusão da própria conta, conforme a LGPD;
- Oferecer ao administrador o controle de usuários, categorias, cursos, vagas, candidaturas e assinaturas;

- Verificar o sistema com testes automáticos executados antes de cada alteração do código.

1.2. JUSTIFICATIVA

Por juntar numa só plataforma a divulgação das vagas, o perfil profissional do candidato, o acompanhamento das candidaturas e a indicação de cursos, a nossa solução atende candidatos e empresas de forma organizada. Diferente de um simples mural de vagas, o sistema lê o currículo, monta o perfil e explica ao candidato por que ele combina ou não com cada vaga, mostrando o que falta e onde aprender.

Além disso, usamos tecnologias gratuitas e muito usadas no ensino técnico (PHP, MySQL e XAMPP), sem depender de serviços pagos e sem exigir que alguém instale programas, já que o leitor de cartazes roda no navegador de quem usa o site. Com isso, a plataforma pode ser implantada com baixo custo em escolas, instituições e órgãos da região.

Do ponto de vista acadêmico, o projeto nos permitiu colocar em prática o que estudamos no curso: levantamento de requisitos, modelagem em UML, banco de dados relacional, programação orientada a objetos, segurança de aplicações web e testes. Do ponto de vista profissional, o trabalho se parece com o de uma equipe de desenvolvimento de verdade, porque cada parte do sistema precisou ser planejada, documentada, testada e ligada às outras.

02 DESENVOLVIMENTO

2.1. METODOLOGIA

Desenvolvemos o projeto a partir do levantamento das dificuldades de candidatos e empresas do Distrito Federal na divulgação e na busca de vagas. Programamos de forma orientada a objetos, no padrão MVC (Model, View e Controller), e fizemos a modelagem em UML e no Modelo Entidade-Relacionamento.

O trabalho começou pela observação de como as vagas são anunciadas na região. Reunimos cartazes, anúncios de grupos de mensagens e páginas de vagas e anotamos que informações aparecem com mais frequência, como cargo, salário, local, tipo de contratação, requisitos, benefícios e contato. Essa observação serviu de base tanto para os requisitos quanto para as regras que o sistema usa para ler anúncios e currículos.

Em seguida escrevemos os requisitos funcionais e não funcionais e os casos de uso, sempre com os quatro personagens do sistema: Candidato, Empresa, Administrador e Sistema. A partir dos casos de uso desenhamos os diagramas de sequência, o diagrama de classes e o modelo de dados, primeiro o conceitual e depois o lógico, que virou o arquivo database/schema.sql com as 11 tabelas do banco tcc_final. Os diagramas foram escritos em PlantUML (PLANTUML, 2026), o que deixou cada diagrama guardado em texto junto com o código.

A programação foi feita em etapas. Primeiro montamos a base do MVC (roteador, sessão, conexão com o banco e as classes de acesso a dados), depois as telas públicas, o cadastro e o login, o perfil do candidato e a leitura do currículo, a área da empresa, o painel do administrador e, por fim, os recursos que vieram com o uso, como o leitor de cartazes no navegador, os padrões automáticos, a biblioteca de e-books e a exclusão da própria conta. A cada etapa o sistema era testado no XAMPP antes de seguir para a próxima.

Para não quebrar o que já funcionava, criamos testes automáticos. O tests/lint.php confere a sintaxe de todos os arquivos PHP, o tests/smoke.php faz 112 verificações rápidas das classes, das regras de negócio, do banco e das páginas, e o tests/jornadas.php usa o sistema como uma pessoa usaria, pelo navegador, em 18 verificações com uma conta temporária que é apagada no fim. Os três rodam juntos pelo tests/verificar.bat e também antes de cada commit, por um gancho do Git que impede gravar uma alteração que quebre o sistema.

Usamos o Git (GIT, 2026) para guardar o histórico do código e da documentação. Na reta final fizemos uma varredura completa do sistema, sem login e com as contas de candidato,

empresa e administrador, com testes de segurança, e só então fechamos esta documentação. Para que o sistema fosse documentado de forma completa, produzimos cada artefato pensando no profissional que seria responsável por ele numa equipe de desenvolvimento, como mostra o Quadro 1.

Papel	Responsabilidade	Artefato neste documento
Analista de Requisitos	Levantamento do problema, requisitos funcionais e não funcionais, casos de uso e suas especificações.	Capítulos 03 a 06; seção 8.1
Analista de Sistemas / Projetista UML	Diagrama de classes e diagramas de sequência.	Seções 8.2 e 8.3
Administrador de Banco de Dados (DBA)	Modelo conceitual, modelo lógico, dicionário de dados, índices e integridade.	Seções 8.4, 8.5 e 8.7
Arquiteto de Software	Arquitetura em camadas (MVC), roteamento e organização de pastas.	Seção 8.8
Desenvolvedor	Codificação em PHP, SQL, HTML, CSS e JavaScript; leitura de documentos, cálculo do match e leitor de cartaz no navegador.	Código-fonte
Analista de Dados (Extração)	Regras de leitura de anúncios, fichas e currículos, padrões automáticos (2 cadastros, 90%), nomes conhecidos e testes das extrações.	Capítulo 05; RF35, RF36 e RNF12
Analista de Segurança	Senhas, sessão, CSRF, uploads, controle de acesso, cabeçalhos de segurança e LGPD.	Seção 2.7; RNF03, RNF04 e RNF13
Analista de Testes (QA)	Plano de testes, teste rápido (smoke test), testes de jornada e gancho de commit.	Seção 8.9
Designer de Interface (UX/UI)	Telas, navegação, responsividade, carregador e mensagens ao usuário.	Seção 8.6; RNF05
Gerente de Projeto	Planejamento, cronogramas, rastreabilidade e acompanhamento das entregas.	Seções 2.9, 2.10 e 8.10

Quadro 1 – Papéis profissionais e artefatos produzidos
Fonte: Autoria própria (2026).

2.2. FUNDAMENTAÇÃO TEÓRICA (REFERENCIAL TEÓRICO)

A base do nosso trabalho é a engenharia de software. Sommerville (2018) define a engenharia de requisitos como o processo de descobrir, documentar e conferir o que o sistema deve fazer, e separa os requisitos funcionais, que descrevem os serviços que o sistema oferece, dos requisitos não funcionais, que tratam de qualidades como desempenho, segurança e

usabilidade. Seguimos essa divisão no capítulo 6. Pressman e Maxim (2016) também mostram que um software de qualidade depende de modelos que liguem o que foi pedido ao que foi construído, e foi isso que tentamos fazer com a matriz de rastreabilidade da seção 8.10.

Para a modelagem usamos a UML. Segundo Booch, Rumbaugh e Jacobson (2006), a UML é uma linguagem para visualizar, especificar, construir e documentar os artefatos de um sistema, e cada diagrama mostra o sistema de um ponto de vista. Guedes (2018) explica que o diagrama de casos de uso apresenta os atores e as funções que o sistema oferece a eles, que o diagrama de sequência mostra a troca de mensagens entre os objetos ao longo do tempo e que o diagrama de classes mostra a estrutura estática do software. Usamos esses três diagramas no capítulo 8.

No banco de dados seguimos a abordagem de Heuser (2009), que separa o modelo conceitual, independente do SGBD, do modelo lógico, que já mostra as tabelas, as colunas e as chaves. Elmasri e Navathe (2018) descrevem o modelo relacional e as restrições de integridade, como a chave primária e a chave estrangeira, que usamos para que um dado nunca aponte para um registro que não existe. No nosso banco, as chaves estrangeiras que ligam os dados a um usuário apagam esses dados junto com a conta, o que também ajuda a cumprir a LGPD.

A arquitetura segue o padrão MVC, que, conforme Sommerville (2018), separa a apresentação (a visão), a interação (o controlador) e os dados (o modelo), de forma que cada parte possa mudar sem afetar as outras. No PHP, o acesso ao banco foi feito com a extensão PDO e consultas preparadas, como recomenda o manual oficial da linguagem (THE PHP GROUP, 2026), e as senhas foram guardadas com a função `password_hash`. As proteções contra os ataques mais comuns em aplicações web, como injeção de SQL, XSS e falsificação de requisição (CSRF), seguem a lista OWASP Top Ten (OWASP FOUNDATION, 2026).

A leitura de cartazes usa o reconhecimento óptico de caracteres (OCR). Smith (2007) descreve o funcionamento do Tesseract, um dos motores de OCR de código aberto mais usados, que encontra as linhas de texto na imagem e reconhece as palavras. No nosso sistema usamos o Tesseract.js (NAPTHA, 2026), versão do Tesseract que roda dentro do navegador por meio do WebAssembly (W3C, 2026), de modo que ninguém precisa instalar programas no computador nem no servidor.

Para comparar textos, como as competências de um candidato e os requisitos de uma vaga, usamos a normalização de termos descrita por Manning, Raghavan e Schütze (2008): o texto é colocado em letras minúsculas, sem acentos e sem pontuação, para que palavras escritas de formas diferentes sejam reconhecidas como iguais. A mesma normalização é usada pelos padrões automáticos, que contam em que campo cada palavra costuma aparecer nas vagas e nos

cursos já cadastrados. Por fim, a parte metodológica do trabalho se apoia em Gil (2017) e em Marconi e Lakatos (2017), e a apresentação do documento segue as normas da ABNT para trabalhos acadêmicos, referências e citações (ABNT, 2011, 2018, 2023).

2.3. TIPO DE PESQUISA

Quanto à natureza, a nossa pesquisa é aplicada, porque tem o objetivo de produzir uma solução prática para um problema real: a dificuldade de encontrar e divulgar vagas no Distrito Federal. Quanto à abordagem, ela é mista. A maior parte da análise é qualitativa, já que estudamos como os anúncios são escritos e como candidatos e empresas usam a plataforma, mas também usamos dados quantitativos, como a nota de compatibilidade, a contagem dos padrões automáticos e o número de verificações aprovadas nos testes.

Quanto aos objetivos, a pesquisa é exploratória e descritiva. Segundo Gil (2017), a pesquisa exploratória busca deixar o problema mais claro e conhecido, e a descritiva descreve as características de um fenômeno ou de um grupo. Primeiro exploramos o problema, observando os anúncios e as necessidades dos usuários, e depois descrevemos em detalhes a solução construída. Quanto aos procedimentos, combinamos pesquisa bibliográfica (livros e normas), pesquisa documental (anúncios, cartazes, fichas de cursos, leis e documentação técnica) e testes do software em ambiente controlado.

2.4. POPULAÇÃO E AMOSTRA

A população de interesse é formada pelas pessoas que procuram trabalho no Distrito Federal e pelas empresas, principalmente as pequenas, que divulgam vagas na região. Como se trata do desenvolvimento de um software, não aplicamos questionários nem entrevistas formais com essa população.

A amostra usada para construir e validar o sistema foi formada por documentos: 62 vagas montadas a partir de cartazes e anúncios divulgados na região, 64 conteúdos de capacitação (43 cursos e 21 e-books gratuitos de fontes oficiais) e currículos de exemplo em PDF, DOC e DOCX. Escolhemos esses documentos porque eles mostram, na prática, a variedade de formatos que o sistema precisa ler. Para os testes de uso, criamos três contas de teste, uma de administrador, uma de empresa e uma de candidato, além da conta temporária usada pelos testes de jornada.

2.5. INSTRUMENTOS DE COLETA DE DADOS

O principal instrumento foi a observação dos canais em que as vagas circulam, como cartazes, grupos de mensagens e páginas de vagas, de onde tiramos os anúncios e cartazes analisados. Também fizemos análise documental das fichas de cursos e e-books de instituições oficiais, de currículos de exemplo, da legislação (LGPD, Código de Defesa do Consumidor e Lei de Direitos Autorais) e da documentação técnica das ferramentas usadas.

Durante o desenvolvimento, os próprios testes funcionaram como instrumentos de coleta. Os relatórios de extração mostram, campo por campo, o que o sistema conseguiu ler; os testes automáticos registram quantas verificações passaram; e a varredura final percorreu todas as páginas do sistema, sem login e com cada tipo de conta, registrando qualquer erro encontrado.

2.6. TÉCNICAS DE ANÁLISE DOS DADOS

Usamos técnicas mistas. Na parte qualitativa, fizemos uma análise de conteúdo dos anúncios, dos cartazes e dos currículos: identificamos os títulos de seção, as palavras-chave e os jeitos mais comuns de escrever cada informação, e transformamos isso nas regras de leitura do sistema. Na comparação entre o que foi pedido e o que foi entregue, conferimos cada requisito com os casos de uso, o código e as tabelas do banco, como mostra a matriz de rastreabilidade.

Na parte quantitativa, usamos contagens simples. Um padrão automático só vale quando um termo aparece em pelo menos dois cadastros e em pelo menos 90% deles no mesmo campo. O match soma pontos de competências (50), cargo (20), localização (15) e nível (15). E a qualidade do sistema foi medida pelo número de verificações aprovadas nos testes automáticos e pelo número de páginas sem erro na varredura final, apresentados no capítulo 7.

2.7. LEI GERAL DE PROTEÇÃO DE DADOS

A plataforma guarda dados pessoais de candidatos e de responsáveis por empresas, como nome, e-mail, telefone, cidade, formação, experiências e o próprio currículo. Por isso tratamos a Lei Geral de Proteção de Dados (Lei nº 13.709/2018) desde o começo do projeto. O Art. 5º da lei traz as definições que usamos para decidir o que fazer em cada parte do sistema (BRASIL, 2018).

Pelo inciso I, dado pessoal é a informação relacionada a pessoa natural identificada ou identificável, o que inclui quase tudo o que o candidato informa no perfil e no currículo. O inciso II define o dado pessoal sensível, como os dados de saúde; o sistema não pede esse tipo de informação, e o campo de informações adicionais, onde o candidato pode citar, se quiser, que é pessoa com deficiência, é opcional. O inciso V define o titular, que é a pessoa a quem os dados se referem, e o inciso X define o tratamento, que é qualquer operação feita com os dados, da coleta à eliminação.

O inciso XII define o consentimento como a manifestação livre, informada e inequívoca do titular. Por isso o cadastro só é concluído depois que a pessoa marca o aceite do termo de autorização e privacidade, e o sistema guarda a data desse aceite. O inciso XIV define a eliminação como a exclusão de dados armazenados em banco de dados. No nosso sistema, o candidato pode excluir a própria conta em Meu perfil, na parte Seus dados, confirmando com a senha. Nesse caso saem o perfil, os currículos e os arquivos, a foto, as candidaturas, os matches e as tentativas de login, o que atende também ao direito de eliminação previsto no Art. 18. Quando o administrador exclui um usuário, tudo também é removido em cascata.

O candidato também pode deixar o perfil privado, fora do banco de talentos, e o currículo só pode ser baixado pelo próprio candidato, pelo administrador e pela empresa autorizada. Seguindo o Art. 46, que pede medidas de segurança para proteger os dados, as senhas são guardadas com hash bcrypt, as consultas ao banco usam parâmetros preparados, os formulários têm token CSRF, os arquivos enviados ficam fora da pasta pública e os registros internos nunca guardam o e-mail inteiro, apenas uma versão mascarada (por exemplo, ca*****@conectavagas.com). Os padrões automáticos também não usam nenhum dado de currículo ou de perfil de candidato.

A fiscalização do cumprimento da lei cabe à Autoridade Nacional de Proteção de Dados (ANPD). Conforme o Art. 48, se acontecer um incidente de segurança que possa causar risco ou dano relevante aos titulares, o controlador deve comunicar o fato à ANPD e aos próprios titulares. Por isso registramos as falhas internas no log do servidor, sem o e-mail completo das pessoas, para que um incidente possa ser investigado sem expor ainda mais os dados.

2.8. CÓDIGO DE DEFESA DO CONSUMIDOR E/OUTRA LEGISLAÇÃO PERTINENTE À TEMÁTICA DO TRABALHO

A plataforma oferece dois planos pagos, o Candidato VIP (R\$ 9,90 por 30 dias) e a Empresa Premium (R\$ 49,90 por 30 dias). Nesta versão os planos são demonstrativos e não há

cobrança real, mas, como a ideia é que eles possam ser cobrados no futuro, observamos o Código de Defesa do Consumidor (Lei nº 8.078/1990). Pelos Art. 2º e 3º, o candidato ou a empresa que contrata um plano para uso próprio é consumidor, e quem mantém a plataforma passa a ser fornecedor do serviço (BRASIL, 1990).

O Art. 6º, inciso III, garante ao consumidor a informação adequada e clara sobre o serviço e o preço. Por isso a página de planos mostra o valor, a duração de 30 dias, os benefícios de cada plano e a informação de que não há fidelidade, e a assinatura pode ser cancelada a qualquer momento pelo próprio usuário. Se os planos passarem a ser cobrados, também será preciso respeitar o direito de arrependimento do Art. 49, já que a contratação acontece pela internet.

Outra lei importante para o trabalho é a Lei de Direitos Autorais (Lei nº 9.610/1998). A biblioteca da plataforma só guarda e-books gratuitos de fontes oficiais, e cada um mostra o crédito e o link da fonte original. Quando o conteúdo está em outro site, o botão é "Acessar" e o usuário é levado à página de origem, em vez de o arquivo ser copiado para a plataforma (BRASIL, 1998).

2.9. CRONOGRAMA DE ATIVIDADES

O Quadro 2 mostra o cronograma das atividades da pesquisa ao longo do semestre letivo.

Etapas	Julho/26	Agosto/26	Setembro/26	Outubro/26	Novembro/26
Escolha do tema	X				
Levantamento bibliográfico	X	X			
Elaboração do projeto		X			
Coleta de dados		X	X		
Análise dos dados			X		
Redação do trabalho			X	X	
Revisão final				X	
Apresentação e entrega				X	X

Quadro 2 – Cronograma de atividades
Fonte: Autoria própria (2026).

2.10. CRONOGRAMA DE SOFTWARE

O Quadro 3 mostra as fases do desenvolvimento do software no mesmo período.

Fase	Julho/26	Agosto/26	Setembro/26	Outubro/26	Novembro/26
Levantamento de requisitos	X	X			
Análise e modelagem		X	X		
Desenvolvimento (implementação)		X	X		
Testes e ajustes			X	X	
Documentação			X	X	
Entrega final				X	X

Quadro 3 – Cronograma de software
 Fonte: Autoria própria (2026).

03 SITUAÇÃO PROBLEMA

No Distrito Federal, grande parte das vagas de emprego é divulgada em cartazes afixados em comércios, em grupos de mensagens e em perfis de redes sociais. Algumas perguntas são feitas diariamente por quem procura trabalho: onde estão as vagas da minha região? O meu currículo atende ao que a vaga pede? O que preciso aprender para conseguir essa vaga? A empresa recebeu a minha candidatura?

Do lado das empresas, principalmente as pequenas, a situação se repete: os currículos chegam por canais diferentes, sem organização, e não há como saber rapidamente quais candidatos são mais compatíveis com a vaga. Todo esse controle é feito manualmente, o que gasta tempo e aumenta o risco de oportunidades e informações se perderem. Além disso, muitas ferramentas de leitura automática exigem a instalação de programas, o que afasta justamente os pequenos anunciantes, que só têm o celular ou um computador simples.

A partir dessas observações, formulamos o problema da nossa pesquisa da seguinte forma: como uma plataforma web pode reunir as vagas e os cursos do Distrito Federal e indicar, de forma automática e explicável, o quanto cada candidato combina com cada vaga, sem exigir que candidatos e pequenas empresas instalem programas ou digitem tudo de novo?

O problema relaciona duas variáveis principais: a forma como as informações de vagas e currículos chegam (espalhadas e em formatos variados, como foto de cartaz, texto de anúncio, PDF ou DOCX) e a qualidade da aproximação entre candidatos e empresas, medida pela nota de compatibilidade e pelo acompanhamento das candidaturas. Ele pode ser investigado de forma sistemática, porque o resultado de cada leitura automática e de cada cálculo de match pode ser conferido nos relatórios do sistema e nos testes.

04 EXPOSIÇÃO DO TEMA PRINCIPAL

O tema principal deste trabalho é o uso da tecnologia para aproximar as pessoas que procuram trabalho das oportunidades de emprego e de capacitação profissional. Esse tema é amplo e envolve tanto a divulgação das vagas quanto a preparação do candidato, já que muitas vezes a pessoa não consegue a vaga porque ainda falta alguma competência que um curso gratuito poderia oferecer.

A partir desse tema fizemos a delimitação. O sujeito da pesquisa são os candidatos e as empresas do Distrito Federal, principalmente os pequenos anunciantes, que hoje divulgam vagas em cartazes e grupos de mensagens. O objeto é uma plataforma web que reúne vagas, cursos e e-books, lê automaticamente currículos, anúncios, cartazes e fichas de cursos e calcula, por regras explicáveis, a compatibilidade entre candidato e vaga.

Também delimitamos o que o sistema não faz. Ele não substitui a análise da empresa, que continua decidindo quem chamar para a entrevista; a nota de compatibilidade serve de apoio, e o candidato vê o motivo de cada ponto. O sistema também não usa técnicas de treinamento com dados das pessoas: a leitura dos documentos segue regras escritas por nós e, onde a regra não tem pista, padrões contados na hora a partir das vagas e dos cursos já cadastrados, sem nunca usar dados de currículos.

Por isso o título do trabalho, Conecta Vagas DF: plataforma de vagas de emprego e capacitação profissional do Distrito Federal, é diferente do tema. O tema é a aproximação entre trabalho e qualificação por meio da tecnologia; o título resume o que construímos para tratar esse tema na realidade da nossa região.

05 PROPOSTA

A nossa proposta é construir um sistema web em que o candidato possa se cadastrar, enviar o currículo e ter o perfil preenchido automaticamente, ver o seu portfólio e a nota de compatibilidade com cada vaga, se candidatar, acompanhar o retorno das empresas e, quando quiser, excluir a própria conta. As empresas poderão publicar e gerenciar vagas, inclusive a partir de um cartaz, receber as candidaturas ordenadas pela compatibilidade e consultar o banco de talentos. O administrador vai controlar os usuários, as categorias, os cursos e e-books, as vagas, as candidaturas e as assinaturas.

O sistema também terá planos demonstrativos, sem cobrança real: o Candidato VIP, com candidaturas ilimitadas, selo no perfil e prioridade na lista que a empresa recebe, e a Empresa Premium, com vagas ilimitadas, destaque das vagas e o banco de talentos completo, com download de currículos. No plano gratuito, o candidato pode ter até 3 candidaturas em andamento e a empresa até 2 vagas ativas. Assim, conseguimos acompanhar melhor todo o processo, da divulgação da vaga até o retorno ao candidato.

A nossa hipótese básica é que reunir as vagas, os cursos e os perfis numa só plataforma, com leitura automática dos documentos e uma nota de compatibilidade explicada, diminui o trabalho de candidatos e empresas e torna a escolha das vagas e dos candidatos mais clara. Como hipóteses secundárias, consideramos que ler o cartaz no próprio navegador facilita a publicação de vagas pelos pequenos anunciantes, que explicar a nota de match ajuda o candidato a saber o que precisa aprender e que usar o que já foi cadastrado reduz os erros da leitura automática.

Para testar essa última hipótese, o sistema usa os padrões automáticos. A leitura de vagas, de cursos e de currículos funciona por regras: títulos de seção, palavras-chave e padrões que escrevemos no código. Quando a regra não tem pista, como numa linha do anúncio sem nenhuma palavra-chave, numa área de vaga ou de curso que ela não reconhece ou numa empresa ou instituição que ela não acha no texto, o Sistema usa padrões tirados na hora das vagas e dos cursos que já foram cadastrados e revisados por uma pessoa. O texto é normalizado (sem acento, em minúsculas, sem pontuação e com todo número trocado por #), e são consideradas as palavras e os pares de palavras vizinhas. Um termo vira padrão quando aparece em pelo menos dois cadastros diferentes e em pelo menos 90% deles no mesmo destino. A expressão "café da manhã", por exemplo, só aparece nos benefícios das vagas cadastradas, então uma linha nova com ela vai para Benefícios, mesmo sem palavra-chave na regra.

Os padrões não ficam guardados em tabela nem dependem de treino. Eles são montados a cada extração, então toda vaga ou curso salvo passa a contar na próxima, e o que é apagado deixa de contar. O Sistema também reconhece as empresas e as instituições de ensino que já estão cadastradas. O resultado é sempre o mesmo para o mesmo texto e os mesmos cadastros, e o relatório da extração da vaga mostra o quadro "Padrões automáticos usados", com o trecho, para onde ele foi e o padrão que decidiu. Deixamos o currículo de fora porque o perfil do candidato tem dados pessoais (LGPD), então a leitura dele segue só as regras.

A leitura de cartazes também não depende de programas instalados no servidor. O site entrega o Tesseract.js, que lê a imagem no próprio navegador e manda para o servidor só o texto com a posição das palavras. Enquanto o leitor carrega ou lê, um carregador amarelo mostra o andamento; quando ele está pronto, fica azul e o botão é liberado, o que também evita o clique duplo.

Para os cursos e e-books, o administrador pode colar a ficha de uma pesquisa, com título, tipo, instituição, link, PDF e imagem, ou várias fichas de uma vez, que aparecem numa prévia antes do cadastro. O Sistema abre cada link da ficha, identifica se é uma imagem, um PDF ou uma página e avisa quando um link não abriu como deveria. Quando o e-book não tem imagem, a capa é tirada da primeira página do PDF, usando o leitor de PDF do próprio Windows; se ele não estiver disponível, o Sistema usa a maior imagem da primeira página. Ao salvar, o PDF vai para a biblioteca da plataforma, e o conteúdo passa a mostrar o botão "Baixar", com o crédito da fonte original. Os conteúdos que ficam em outro site mostram o botão "Acessar". A falta de imagem nunca impede o cadastro: o conteúdo entra com a imagem padrão, que o administrador encontra depois pelo filtro "Com imagem padrão".

06 ABRANGÊNCIA DO SISTEMA

6.1 Requisitos Funcionais

Os requisitos funcionais descrevem os serviços que o sistema oferece, isto é, o que ele deve fazer a partir das entradas de cada personagem (SOMMERVILLE, 2018). A seguir estão os 40 requisitos funcionais do Conecta Vagas DF, com a prioridade de cada um.

RF [01] Realizar login

Prioridade: Essencial ☒ Importante ☐ Desejável ☐

O sistema deverá permitir que o usuário (candidato, empresa ou administrador) acesse o sistema com e-mail e senha cadastrados, aceitando a senha mesmo com o Caps Lock ligado, com a primeira letra trocada de maiúscula para minúscula ou com espaços sobrando no começo ou no fim.

RF [02] Cadastrar conta

Prioridade: Essencial ☒ Importante ☐ Desejável ☐

O sistema deverá permitir que o candidato ou a empresa crie a sua conta, informando nome, e-mail, senha e telefone, com aceite obrigatório do termo de autorização e privacidade (LGPD).

RF [03] Encerrar sessão

Prioridade: Essencial ☒ Importante ☐ Desejável ☐

O sistema deverá permitir que o usuário saia da sua conta (logout) de forma segura.

RF [04] Recuperar senha

Prioridade: Essencial ☐ Importante ☒ Desejável ☐

O sistema deverá permitir que o usuário solicite um link de redefinição de senha, válido por 30 minutos e de uso único, guardando no banco apenas o resumo (hash) do código do link.

RF [05] Consultar vagas

Prioridade: Essencial ☒ Importante ☐ Desejável ☐

O sistema deverá permitir consultar as vagas abertas por palavra-chave, categoria, cidade, tipo de contratação, nível e modelo de trabalho (presencial, remoto ou híbrido).

RF [06] Visualizar detalhes da vaga**Prioridade:** Essencial ☒ Importante ☐ Desejável ☐

O sistema deverá exibir os dados completos da vaga (descrição, requisitos, benefícios, salário, local e contato) e, para o candidato logado, a nota de compatibilidade.

RF [07] Consultar cursos e e-books**Prioridade:** Essencial ☐ Importante ☒ Desejável ☐

O sistema deverá permitir consultar cursos, e-books e vídeos de capacitação, com filtros por categoria, modalidade, nível e gratuidade, e baixar o PDF dos e-books guardados na biblioteca da plataforma.

RF [08] Alterar perfil do candidato**Prioridade:** Essencial ☒ Importante ☐ Desejável ☐

O sistema deverá permitir que o candidato cadastre e altere os seus dados profissionais: título, resumo, objetivo, cidade, formação, experiências, habilidades, idiomas, CNH, pretensão salarial e foto.

RF [09] Enviar currículo**Prioridade:** Essencial ☒ Importante ☐ Desejável ☐

O sistema deverá permitir que o candidato envie o currículo nos formatos PDF, DOC ou DOCX, com tamanho máximo de 10 MB.

RF [10] Extrair dados do currículo**Prioridade:** Essencial ☒ Importante ☐ Desejável ☐

O sistema deverá ler o currículo enviado e extrair automaticamente contato, experiências, formação, cursos, habilidades, idiomas, demais informações e a foto, preenchendo o perfil do candidato.

RF [11] Aplicar dados do relatório de extração**Prioridade:** Essencial ☐ Importante ☒ Desejável ☐

O sistema deverá exibir um relatório com o que foi encontrado, aplicado, mantido e o que faltou, permitindo ao candidato escolher quais dados substituir no perfil.

RF [12] Excluir currículo

Prioridade: Essencial ☐ Importante ☒ Desejável ☐

O sistema deverá permitir que o candidato exclua um currículo enviado.

RF [13] Gerar portfólio

Prioridade: Essencial ☐ Importante ☒ Desejável ☐

O sistema deverá montar automaticamente o portfólio profissional do candidato quando o cadastro estiver completo (telefone, título, cidade, resumo, objetivo, formação e habilidades), com linha do tempo, vagas compatíveis e cursos recomendados.

RF [14] Calcular match

Prioridade: Essencial ☒ Importante ☐ Desejável ☐

O sistema deverá calcular uma nota de compatibilidade de 0 a 100 entre o candidato e cada vaga ativa, considerando competências (50 pontos), cargo (20), localização (15) e nível (15), e explicar a nota.

RF [15] Recalcular match

Prioridade: Essencial ☐ Importante ☐ Desejável ☒

O sistema deverá permitir que o candidato solicite o recálculo do match e recalculá-lo automaticamente ao salvar o perfil, enviar ou excluir currículo e criar ou editar vagas.

RF [16] Candidatar-se à vaga

Prioridade: Essencial ☒ Importante ☐ Desejável ☐

O sistema deverá permitir que o candidato se candidate a uma vaga aberta, escolhendo um currículo e escrevendo uma mensagem de apresentação opcional, respeitando o limite do plano (até 3 candidaturas em andamento no plano gratuito).

RF [17] Cancelar candidatura

Prioridade: Essencial ☐ Importante ☒ Desejável ☐

O sistema deverá permitir que o candidato cancele uma candidatura que ainda não foi analisada pela empresa.

RF [18] Acompanhar candidaturas

Prioridade: Essencial ☒ Importante ☐ Desejável ☐

O sistema deverá exibir ao candidato o status de cada candidatura (enviada, em análise, entrevista, aprovado, rejeitado ou cancelada) e o retorno da empresa.

RF [19] Manter perfil da empresa

Prioridade: Essencial ☒ Importante ☐ Desejável ☐

O sistema deverá permitir que a empresa cadastre e altere nome fantasia, CNPJ, setor, site, telefone, cidade, descrição e logotipo.

RF [20] Cadastrar vaga

Prioridade: Essencial ☒ Importante ☐ Desejável ☐

O sistema deverá permitir que a empresa (ou o administrador) cadastre vagas, respeitando o limite do plano da empresa (até 2 vagas ativas no plano gratuito).

RF [21] Alterar vaga

Prioridade: Essencial ☒ Importante ☐ Desejável ☐

O sistema deverá permitir alterar os dados e o status (ativa, pausada ou encerrada) de uma vaga.

RF [22] Excluir vaga

Prioridade: Essencial ☒ Importante ☐ Desejável ☐

O sistema deverá permitir excluir uma vaga, removendo junto as candidaturas e os matches dela.

RF [23] Extrair vaga de anúncio ou cartaz

Prioridade: Essencial ☐ Importante ☒ Desejável ☐

O sistema deverá preencher o formulário da vaga a partir do texto colado de um anúncio ou da leitura (OCR) da imagem de um cartaz, feita no próprio navegador, sem instalar programas, com um carregador amarelo enquanto lê e azul quando está pronto, mostrar o relatório da extração e avisar quando já existir uma vaga aberta igual, para revisão antes de publicar.

RF [24] Gerenciar candidaturas recebidas

Prioridade: Essencial ☒ Importante ☐ Desejável ☐

O sistema deverá listar as candidaturas recebidas, ordenadas por plano VIP e maior match, e permitir alterar o status e registrar um retorno ao candidato.

RF [25] Consultar banco de talentos

Prioridade: Essencial ☐ Importante ☒ Desejável ☐

O sistema deverá permitir que a empresa pesquise perfis públicos de candidatos por termo e cidade.

RF [26] Baixar currículo

Prioridade: Essencial ☐ Importante ☒ Desejável ☐

O sistema deverá permitir o download do currículo apenas ao dono, ao administrador e à empresa autorizada.

RF [27] Assinar plano

Prioridade: Essencial ☐ Importante ☒ Desejável ☐

O sistema deverá permitir que o candidato assine o plano Candidato VIP (R\$ 9,90 por 30 dias) e a empresa assine o plano Empresa Premium (R\$ 49,90 por 30 dias), ambos demonstrativos.

RF [28] Cancelar assinatura

Prioridade: Essencial ☐ Importante ☒ Desejável ☐

O sistema deverá permitir cancelar a assinatura ativa, removendo os benefícios do plano.

RF [29] Manter usuários

Prioridade: Essencial ☒ Importante ☐ Desejável ☐

O sistema deverá permitir que o administrador cadastre, altere, ative, desative e exclua usuários.

RF [30] Manter categorias

Prioridade: Essencial ☐ Importante ☒ Desejável ☐

O sistema deverá permitir que o administrador cadastre, altere, ative, desative e exclua categorias de vagas e de cursos.

RF [31] Manter cursos e e-books

Prioridade: Essencial ☐ Importante ☒ Desejável ☐

O sistema deverá permitir que o administrador cadastre, altere, publique, oculte e exclua cursos, e-books e vídeos, inclusive preenchendo o formulário a partir do texto de divulgação ou da ficha de uma pesquisa.

RF [32] Visualizar painel**Prioridade:** Essencial ☐ Importante ☐ Desejável ☒

O sistema deverá exibir à empresa e ao administrador um painel com indicadores e gráficos de vagas, candidaturas e match.

RF [33] Excluir a própria conta**Prioridade:** Essencial ☒ Importante ☐ Desejável ☐

O sistema deverá permitir que o candidato exclua a própria conta, confirmando com a senha, removendo perfil, currículos e arquivos, foto, candidaturas, matches e tentativas de login (LGPD).

RF [34] Gerenciar assinaturas**Prioridade:** Essencial ☐ Importante ☒ Desejável ☐

O sistema deverá permitir que o administrador consulte as assinaturas com filtros e conceda, altere, cancele e exclua assinaturas de candidatos e empresas.

RF [35] Usar padrões automáticos na extração**Prioridade:** Essencial ☐ Importante ☒ Desejável ☐

O sistema deverá, onde as regras da extração não tiverem pista (linha da vaga sem palavra-chave, área da vaga ou do curso não reconhecida, empresa ou instituição não achada), decidir pelos padrões tirados das vagas e dos cursos já cadastrados (palavras e pares de palavras vistos em pelo menos 2 cadastros e em pelo menos 90% deles no mesmo destino) e reconhecer as empresas e as instituições já cadastradas.

RF [36] Mostrar os padrões usados**Prioridade:** Essencial ☐ Importante ☐ Desejável ☒

O sistema deverá mostrar no relatório da extração da vaga o quadro "Padrões automáticos usados", com o trecho, para onde ele foi e o padrão que decidiu, para a revisão antes de publicar.

RF [37] Guardar e-books na biblioteca**Prioridade:** Essencial ☐ Importante ☒ Desejável ☐

O sistema deverá baixar o PDF do e-book da fonte oficial (ou recebê-lo no cadastro) e guardá-lo na biblioteca da plataforma, com o botão "Baixar" e o crédito da fonte original; na prévia do cadastro, deverá abrir cada link da ficha (imagem, PDF ou página), avisar os links que não abriram e, no e-book sem imagem, usar como capa a primeira página do PDF. Sem imagem, o

conteúdo entra com a imagem padrão, que pode ser localizada pelo filtro "Com imagem padrão".

RF [38] Importar fichas em lote

Prioridade: Essencial ☐ Importante ☐ Desejável ☒

O sistema deverá permitir que o administrador cole várias fichas de cursos e e-books de uma vez, confira uma prévia e cadastre as marcadas.

RF [39] Apoiar o projeto por Pix

Prioridade: Essencial ☐ Importante ☐ Desejável ☒

O sistema deverá exibir, na página inicial e no rodapé, um QR Code Pix para doações voluntárias de apoio à plataforma, com o código montado pelo próprio sistema no padrão do Banco Central.

RF [40] Limpar arquivos sem uso

Prioridade: Essencial ☐ Importante ☐ Desejável ☒

O sistema deverá, no máximo uma vez por dia, quando o administrador abrir o painel, apagar os arquivos enviados que nenhum registro usa e que têm mais de 24 horas.

6.2 Requisitos Não Funcionais

Os requisitos não funcionais tratam das qualidades do sistema, como desempenho, segurança, usabilidade, confiabilidade e manutenção, e das tecnologias que ele deve usar (SOMMERVILLE, 2018). A seguir estão os 13 requisitos não funcionais do Conecta Vagas DF.

RNF [01] Desempenho

Prioridade: Essencial ☒ Importante ☐ Desejável ☐

O sistema deverá exibir as páginas em até 2 segundos em rede local, concluir a leitura de um currículo em até 15 segundos e a leitura de um cartaz no navegador em cerca de 10 segundos em um computador comum ligado na tomada.

RNF [02] Disponibilidade

Prioridade: Essencial ☒ Importante ☐ Desejável ☐

O sistema deverá permitir a disponibilidade de 24 horas todos os dias, quando hospedado em servidor.

RNF [03] Segurança**Prioridade:** Essencial ☒ Importante ☐ Desejável ☐

O sistema deverá armazenar senhas com hash bcrypt, usar consultas preparadas (PDO), token CSRF em todos os formulários, sessão com cookie HttpOnly e SameSite, cabeçalhos de segurança (Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Referrer-Policy e Permissions-Policy) e pausar o login por 5 minutos após 8 senhas erradas para o mesmo e-mail (ou 20 somando todos os endereços IP, ou 60 vindas de um mesmo IP).

RNF [04] Privacidade (LGPD)**Prioridade:** Essencial ☒ Importante ☐ Desejável ☐

O sistema deverá exigir o aceite do termo de autorização e privacidade no cadastro, permitir que o candidato torne o perfil privado e exclua a própria conta com todos os dados, excluir todos os dados do usuário quando a conta for excluída pelo administrador e não guardar o e-mail completo nos registros (logs).

RNF [05] Usabilidade**Prioridade:** Essencial ☒ Importante ☐ Desejável ☐

O sistema deverá funcionar nos navegadores Google Chrome, Mozilla Firefox e Microsoft Edge, adaptar-se a telas de computador e celular, exibir mensagens em português e mostrar um carregador (amarelo enquanto carrega, azul quando pronto) na leitura de cartazes e de fichas.

RNF [06] Compatibilidade**Prioridade:** Essencial ☒ Importante ☐ Desejável ☐

O sistema deverá funcionar em Windows 10 ou 11 com XAMPP (Apache com mod_rewrite, PHP 8.0 ou superior e MySQL/MariaDB) e também em Linux; ele foi testado no PHP 8.0.30 do XAMPP e, para publicação na internet, recomenda-se PHP 8.2 ou superior.

RNF [07] Requisitos de implementação**Prioridade:** Essencial ☒ Importante ☐ Desejável ☐

O sistema deverá ser implementado nas linguagens PHP, SQL, HTML, CSS e JavaScript, sem frameworks e sem programas a instalar: o leitor de cartazes (Tesseract.js) é servido pelo próprio site e roda no navegador.

RNF [08] Manutenibilidade**Prioridade:** Essencial ☐ Importante ☒ Desejável ☐

O sistema deverá seguir o padrão MVC, com uma classe de acesso a dados (DAO) por assunto e as regras de negócio separadas em serviços.

RNF [09] Armazenamento de arquivos**Prioridade:** Essencial ☒ Importante ☐ Desejável ☐

O sistema deverá aceitar currículos em PDF, DOC ou DOCX de até 10 MB, imagens JPG, PNG ou WEBP de até 3 MB (cartazes de até 8 MB) e PDFs da biblioteca de até 25 MB no envio e 150 MB quando baixados da fonte oficial, gravados fora da pasta pública com nome aleatório e tipo conferido pelo conteúdo.

RNF [10] Confiabilidade**Prioridade:** Essencial ☒ Importante ☐ Desejável ☐

O sistema deverá usar transações e chaves estrangeiras com exclusão em cascata (ON DELETE CASCADE) ou desvinculação (SET NULL) para manter a integridade dos dados.

RNF [11] Testabilidade**Prioridade:** Essencial ☐ Importante ☐ Desejável ☒

O sistema deverá possuir verificação de sintaxe (tests/lint.php), teste automático rápido (tests/smoke.php, 112 verificações) e testes de jornada pelo navegador (tests/jornadas.php, 18 verificações com conta temporária), reunidos em tests/verificar.bat e executados automaticamente antes de cada commit.

RNF [12] Previsibilidade dos padrões automáticos**Prioridade:** Essencial ☐ Importante ☒ Desejável ☐

Os padrões automáticos deverão ser determinísticos (o mesmo texto com os mesmos cadastros dá sempre o mesmo resultado), explicáveis (cada decisão registra o padrão que valeu), não usar dados de candidatos (LGPD), não depender de tabela nem de treino e, se o banco não responder, deixar a extração seguir só com as regras.

RNF [13] Segurança dos links externos

Prioridade: Essencial ☐ Importante ☒ Desejável ☐

Os links abertos pelo sistema (imagens e PDFs das fichas de cursos) só poderão ser endereços http ou https de servidores públicos, nunca da rede interna (o que vale também para cada redirecionamento), com HTTPS verificado e limite de tamanho e de tempo.

6.3 Metodologia e Tecnologia Utilizada

Para a programação do sistema usamos o PHP 8, no padrão MVC e sem frameworks. Escolhemos não usar frameworks de propósito, porque queríamos entender e mostrar como uma requisição chega ao servidor, como a rota é escolhida e como os dados vão do formulário até o banco e voltam para a tela. O sistema foi desenvolvido e testado no PHP 8.0.30 que vem no XAMPP; para publicar na internet, recomendamos o PHP 8.2 ou mais novo, porque o 8.0 não recebe mais correções de segurança.

As páginas foram construídas em HTML5, com a aparência e a adaptação para celular feitas em CSS3. O JavaScript cuida do comportamento das telas, como filtros, pré-visualizações, o carregador e os gráficos do painel. O banco de dados é o MySQL/MariaDB (ORACLE, 2026), acessado pela extensão PDO, com a linguagem SQL. Os arquivos PDF e DOCX são lidos por código PHP escrito por nós, e o código Pix de doação é montado pelo próprio sistema no padrão do Banco Central, com o QR Code desenhado no navegador.

A leitura de cartazes usa o Tesseract.js, que roda no navegador de quem usa o site por meio do WebAssembly, sem instalação; o Tesseract OCR (TESSERACT OCR, 2026) no servidor ficou apenas como reserva opcional. Os padrões automáticos são montados na hora, em PHP puro, a partir das vagas e dos cursos cadastrados, sem bibliotecas externas. A capa dos e-books é tirada da primeira página do PDF com o leitor de PDF que já vem no Windows, chamado pelo PowerShell.

Como aplicativos usamos o Visual Studio Code para escrever o código, o XAMPP (Apache, PHP e MySQL) como servidor local, com o módulo mod_rewrite do Apache ligado (THE APACHE SOFTWARE FOUNDATION, 2026), o phpMyAdmin e o MySQL Workbench para o banco de dados, o PlantUML para os diagramas e o Git e o GitHub para o controle de versões, com um gancho que roda os testes antes de cada commit. A modelagem foi feita em UML (casos de uso, classes e sequência) e no Modelo Entidade-Relacionamento, na notação de Chen adaptada por Heuser (2009).

07 RESULTADOS OBTIDOS / ESPERADOS

Esperamos entregar um produto que faça bem todas as funções descritas neste documento: que o candidato encontre vagas da sua região, tenha o perfil preenchido a partir do currículo e entenda a sua compatibilidade com cada vaga; que a empresa publique vagas com rapidez, até a partir de um cartaz, e receba as candidaturas já ordenadas; e que o administrador mantenha a plataforma organizada.

A versão que implementamos tem 11 tabelas, 12 controladores, 9 classes de acesso a dados, 17 serviços (leitura de documentos, padrões automáticos, capa de PDF, match, competências, portfólio e Pix) e 32 rotas. Os dados de demonstração trazem 20 categorias (13 de vagas e 7 de cursos), 62 vagas e 64 conteúdos de capacitação (43 cursos e 21 e-books, estes com o PDF guardado na biblioteca da plataforma), além das contas de teste de administrador, empresa e candidato.

Na varredura final, em 27/09/2026, rastreamos 1.701 páginas, sem login e com as contas de candidato, empresa e administrador, sem nenhum problema. A bateria de segurança (XSS, injeção de SQL, CSRF, sessão, uploads e permissões) foi aprovada, as tabelas do banco estavam íntegras e não houve erros de PHP no servidor. Na versão final, em 01/10/2026, o teste rápido passou em 112 de 112 verificações, inclusive as dos padrões automáticos e da capa do PDF, e os testes de jornada passaram em 18 de 18.

Para a sociedade, o resultado esperado é que a plataforma possa ser usada por escolas, instituições e órgãos da região como um ponto único de vagas e capacitação, ajudando principalmente quem está procurando o primeiro emprego e os pequenos anunciantes, que passam a publicar uma vaga apenas com a foto do cartaz.

08 MODELO DO SOFTWARE

8.1 Diagramas de Caso de Uso

O diagrama de caso de uso mostra como as pessoas usam o sistema. Cada caso de uso é uma tarefa que o sistema oferece, e cada ator é quem participa dela; todo caso de uso precisa ter pelo menos um ator (GUEDES, 2018). No nosso projeto os atores são quatro: o Candidato, a Empresa, o Administrador e o Sistema. Quem entra no site sem conta para ver vagas e cursos é tratado como um candidato que ainda não se cadastrou. O Sistema aparece como ator quando faz alguma coisa sozinho, como extrair os dados do currículo, calcular o match, aplicar os padrões automáticos, tirar a capa do PDF de um e-book e limpar os arquivos sem uso.

8.1.1 Diagrama de Caso de Uso Geral Sistema Conecta Vagas DF

A Figura 1 mostra o caso de uso geral organizado pela visão de cada personagem: em cada quadro aparecem os casos de uso de que o Candidato, a Empresa, o Administrador e o Sistema participam. Escolhemos esse formato porque vários casos de uso são compartilhados (o login, por exemplo, é usado pelos três personagens que têm conta) e, num quadro único, as ligações se cruzariam e atrapalhariam a leitura. Os casos de uso UC19 (excluir a própria conta), UC20 (gerenciar assinaturas) e UC21 (extrair com os padrões automáticos) foram incluídos nesta versão, e o detalhe de cada grupo aparece nos diagramas específicos a seguir.

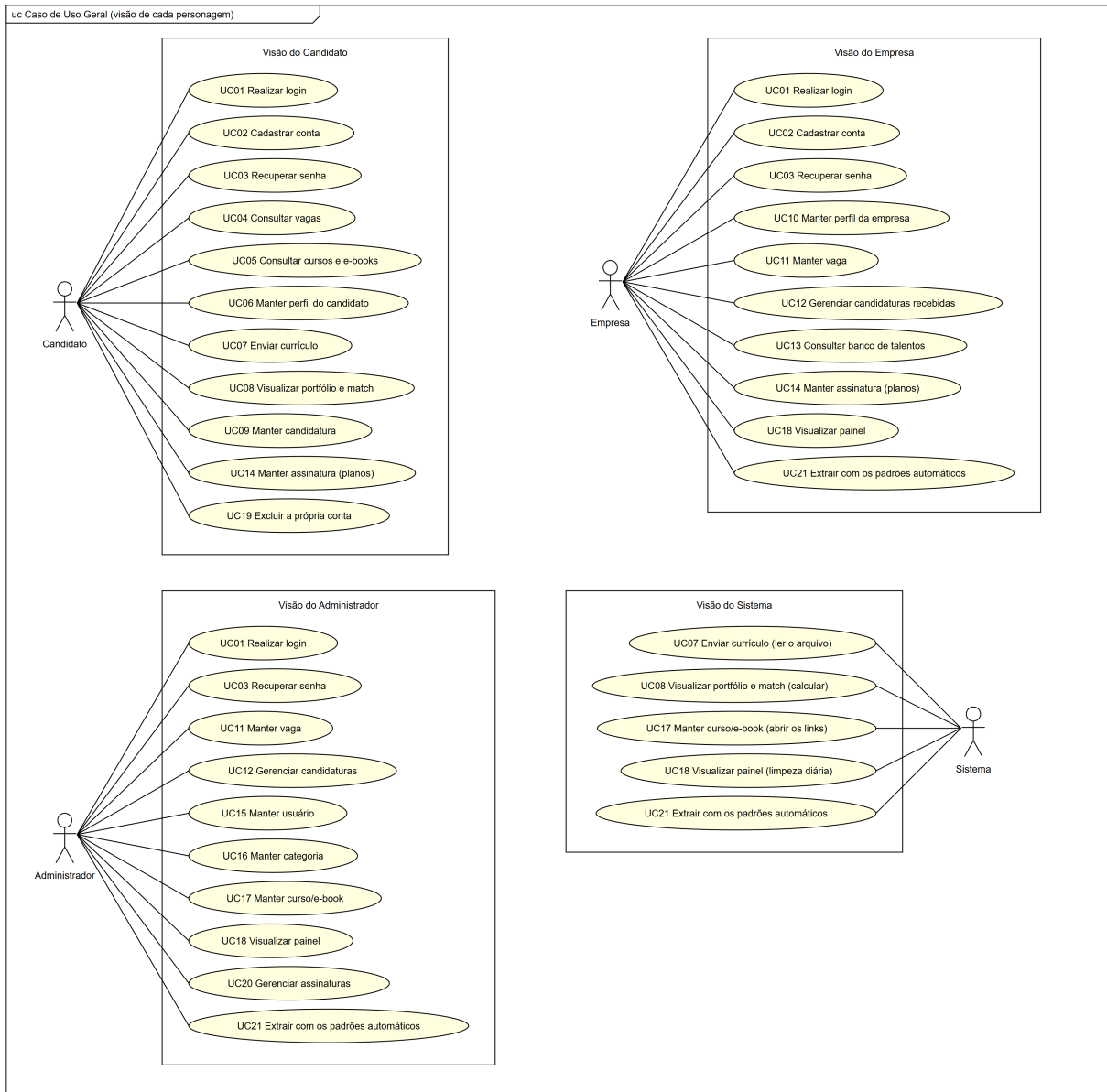


Figura 1 – Diagrama de Caso de Uso Geral do Sistema Conecta Vagas DF (visão de cada personagem)
Fonte: Autoria própria (2026).

8.1.2 Diagrama de Caso de Uso Específico Realizar Login

O diagrama da Figura 2 mostra o login, a pausa automática depois de tentativas repetidas e o encerramento da sessão.

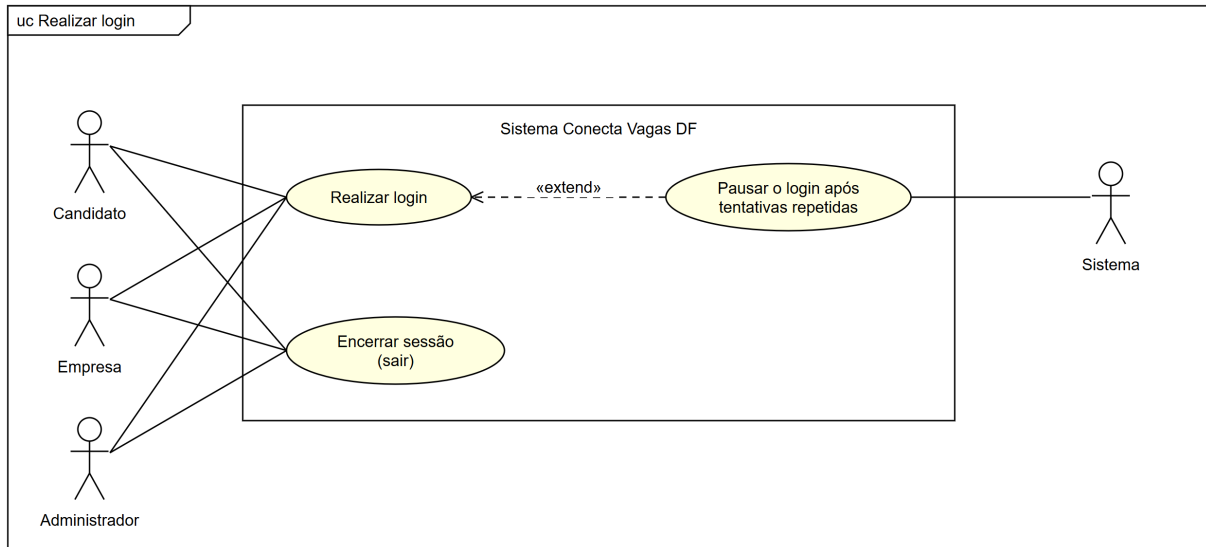


Figura 2 – Diagrama de Caso de Uso Realizar Login
Fonte: Autoria própria (2026).

8.1.3 Diagrama de Caso de Uso Específico Acesso à Conta

A Figura 3 reúne o cadastro da conta, com o aceite do termo LGPD, e a recuperação de senha.

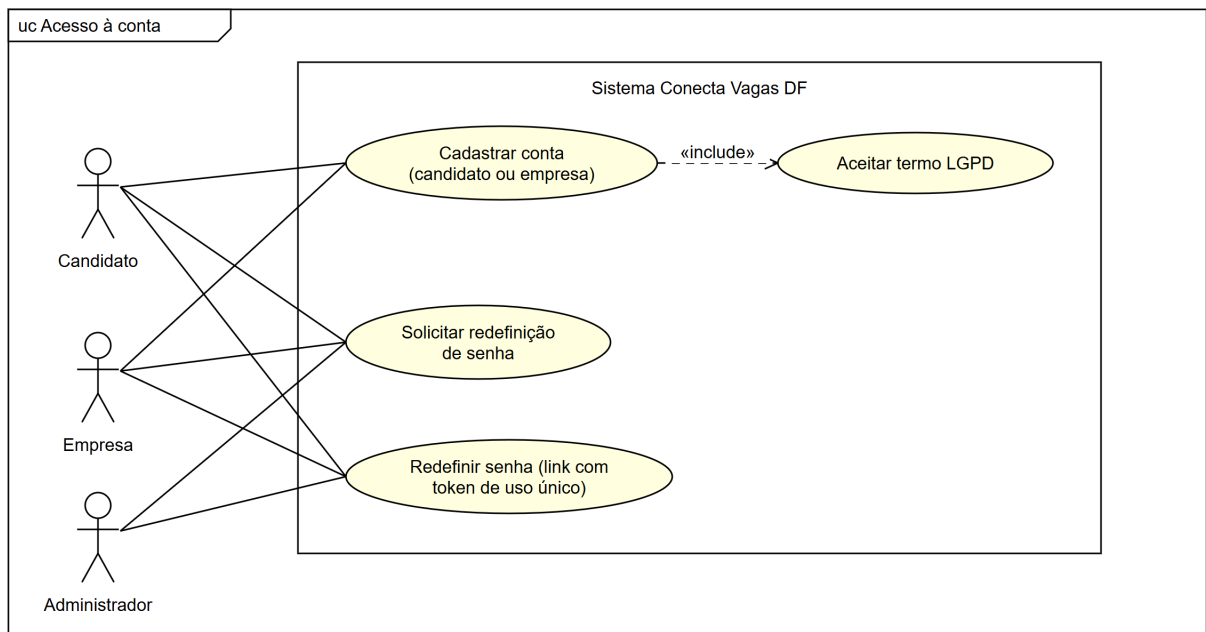


Figura 3 – Diagrama de Caso de Uso Acesso à Conta
Fonte: Autoria própria (2026).

8.1.4 Diagrama de Caso de Uso Específico Consultas Públicas

A Figura 4 mostra as consultas de vagas, cursos e e-books, que o candidato faz com ou sem conta, e o download do PDF da biblioteca.

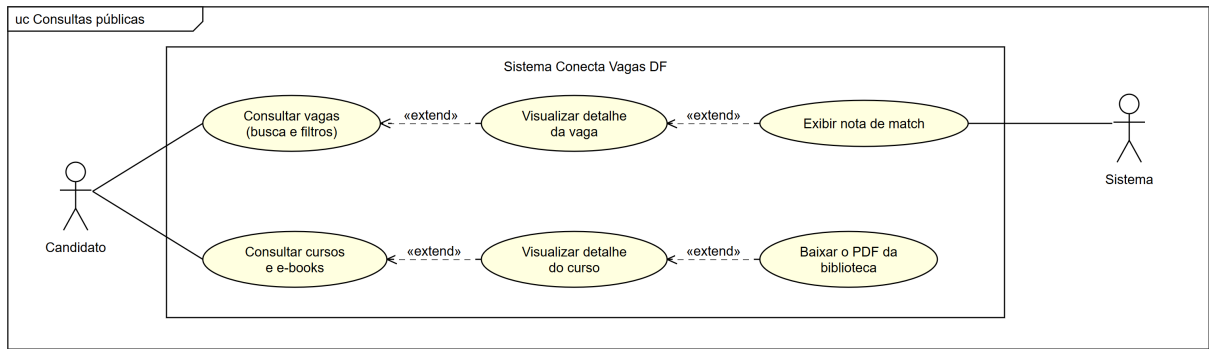


Figura 4 – Diagrama de Caso de Uso Consultas Públicas

Fonte: Autoria própria (2026).

8.1.5 Diagrama de Caso de Uso Específico Perfil do Candidato

A Figura 5 mostra o perfil do candidato: envio e leitura do currículo, portfólio, match e exclusão da própria conta.

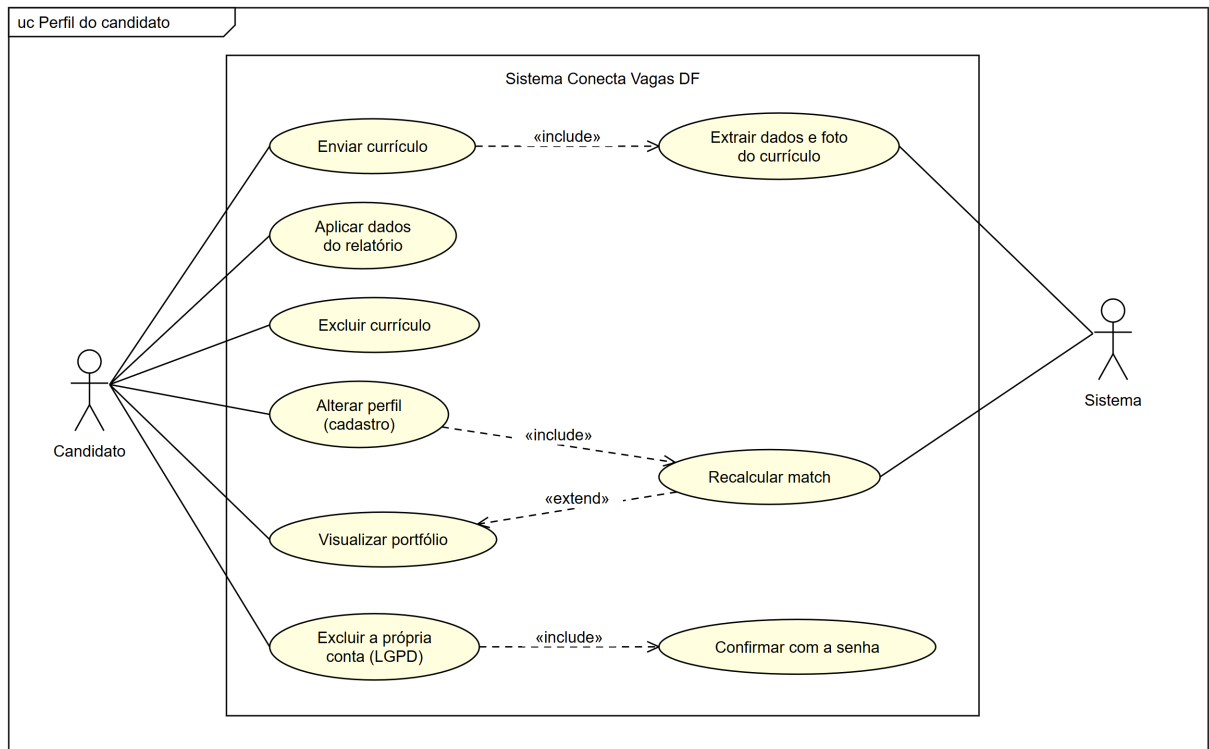


Figura 5 – Diagrama de Caso de Uso Perfil do Candidato

Fonte: Autoria própria (2026).

8.1.6 Diagrama de Caso de Uso Específico Candidatura

A Figura 6 mostra a candidatura à vaga, com a verificação do limite do plano, o cancelamento e o acompanhamento do status.

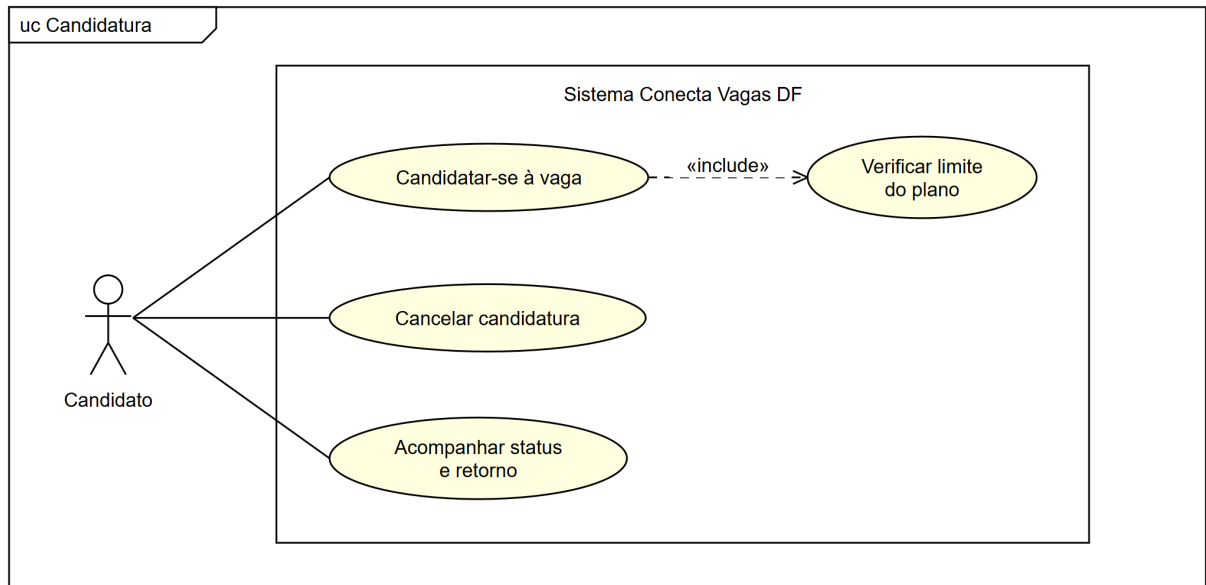


Figura 6 – Diagrama de Caso de Uso Candidatura
Fonte: Autoria própria (2026).

8.1.7 Diagrama de Caso de Uso Específico Manter Vaga

A Figura 7 mostra o cadastro, a alteração e a exclusão de vagas, com a leitura do anúncio ou do cartaz no navegador.

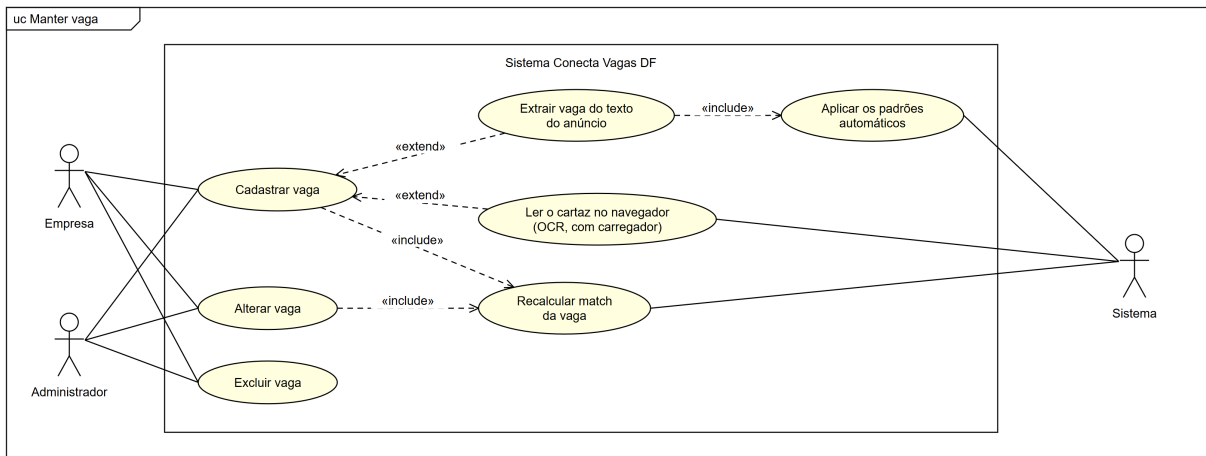


Figura 7 – Diagrama de Caso de Uso Manter Vaga
Fonte: Autoria própria (2026).

8.1.8 Diagrama de Caso de Uso Específico Área da Empresa

A Figura 8 mostra o que a empresa faz no painel: manter o perfil, gerenciar as candidaturas recebidas, baixar currículos e consultar o banco de talentos.

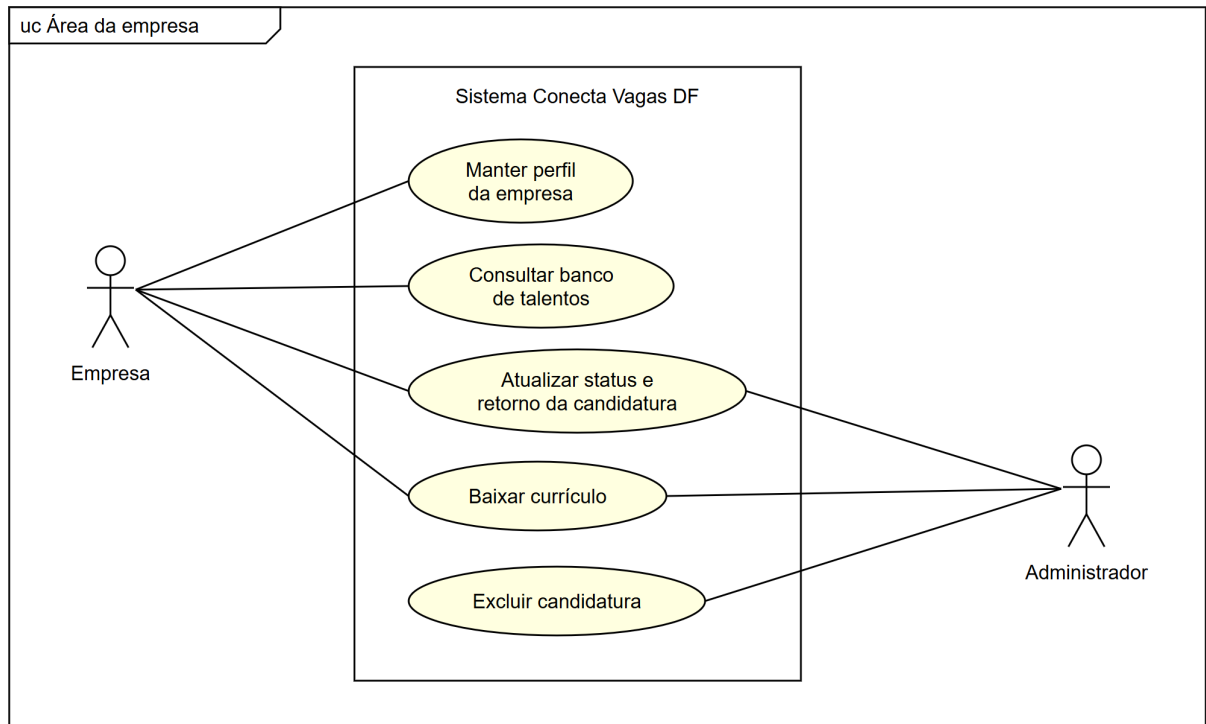


Figura 8 – Diagrama de Caso de Uso Área da Empresa
Fonte: Autoria própria (2026).

8.1.9 Diagrama de Caso de Uso Específico Planos

A Figura 9 mostra a assinatura e o cancelamento dos planos Candidato VIP e Empresa Premium.

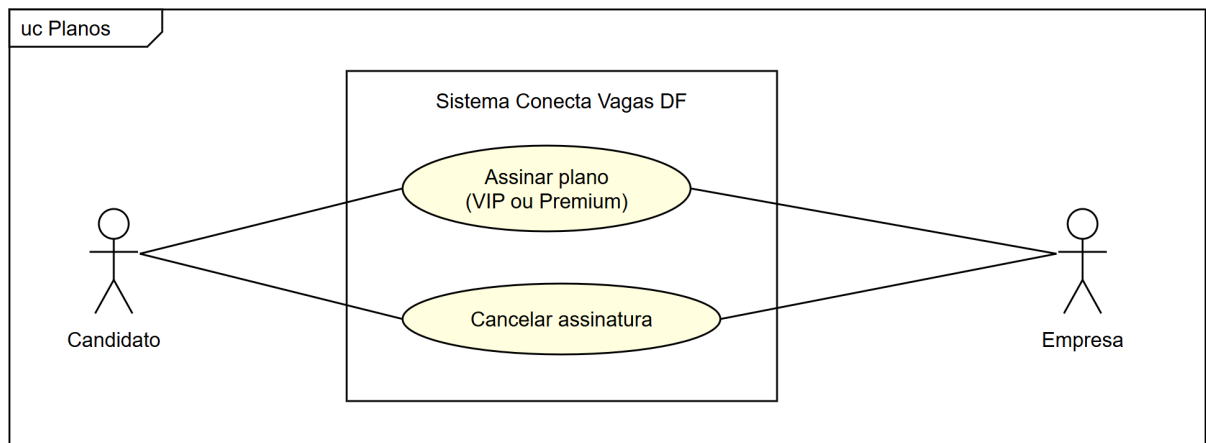


Figura 9 – Diagrama de Caso de Uso Planos
Fonte: Autoria própria (2026).

8.1.10 Diagrama de Caso de Uso Específico Manter Usuário

A Figura 10 mostra a manutenção de usuários pelo administrador.

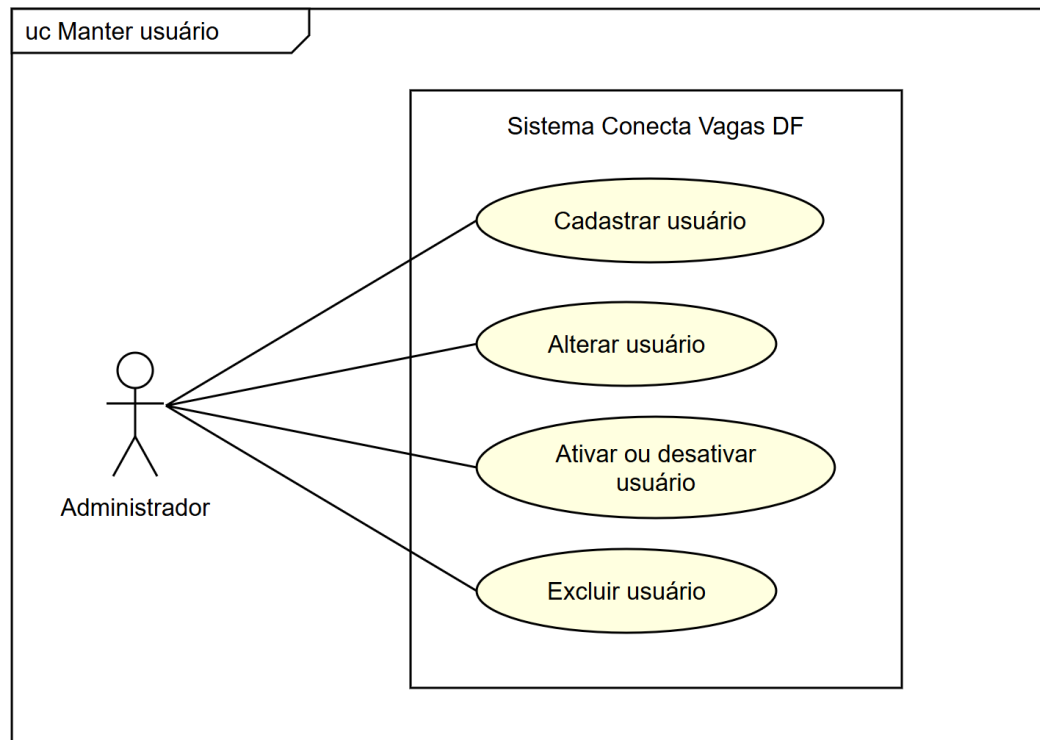


Figura 10 – Diagrama de Caso de Uso Manter Usuário
Fonte: Autoria própria (2026).

8.1.11 Diagrama de Caso de Uso Específico Manter Categoria

A Figura 11 mostra a manutenção das categorias de vagas e de cursos.

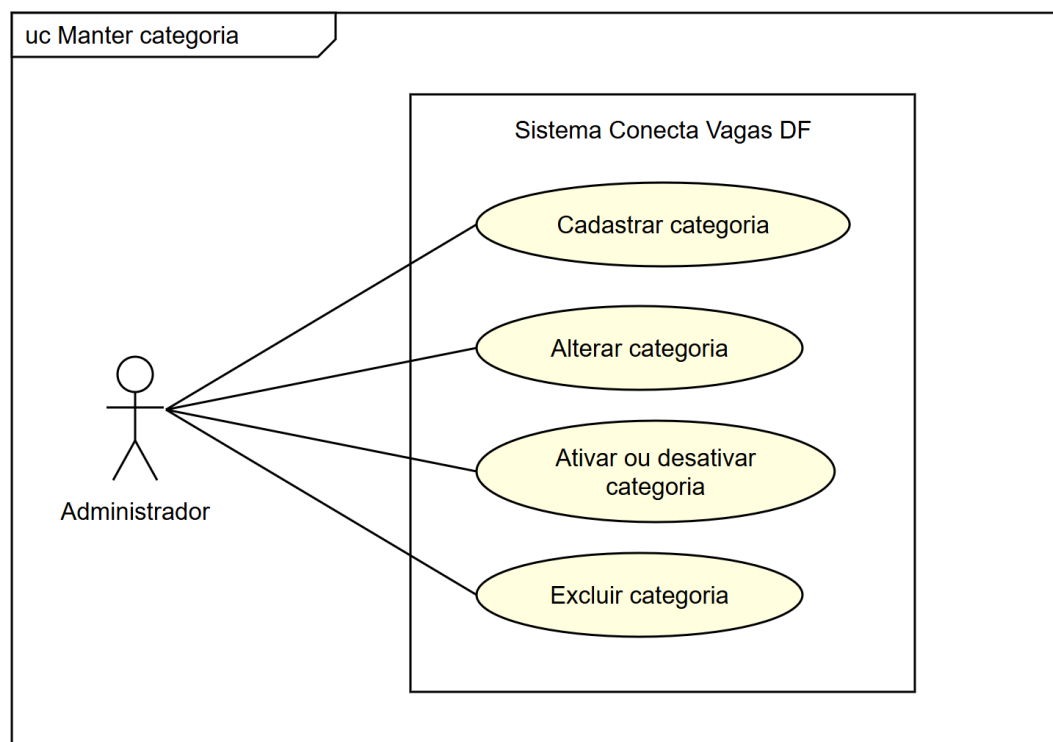


Figura 11 – Diagrama de Caso de Uso Manter Categoria
Fonte: Autoria própria (2026).

8.1.12 Diagrama de Caso de Uso Específico Manter Curso/E-book

A Figura 12 mostra a manutenção de cursos e e-books, com a leitura das fichas, a abertura dos links, a capa tirada do PDF e a biblioteca.

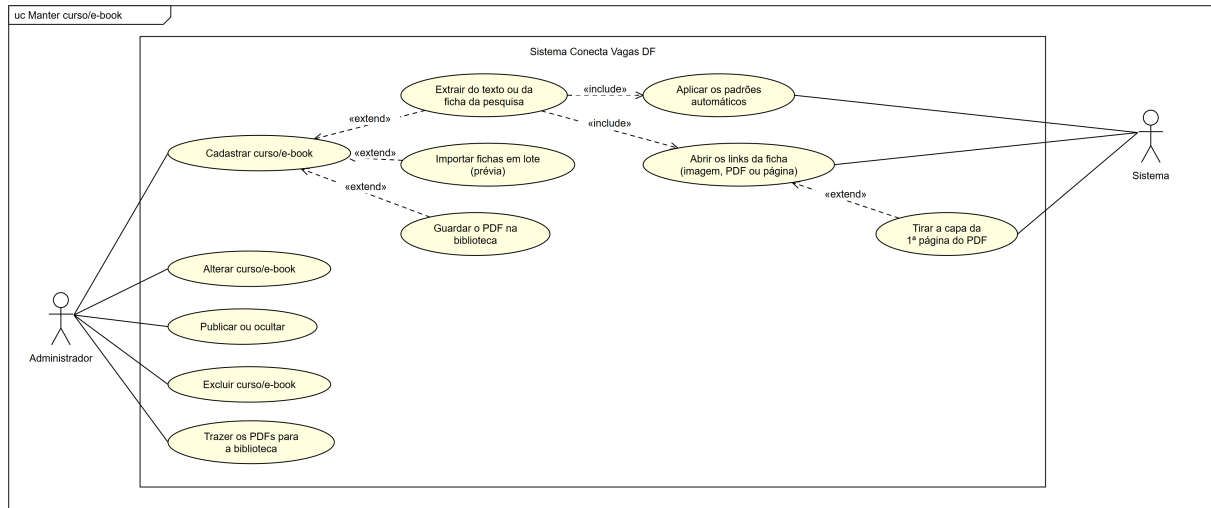


Figura 12 – Diagrama de Caso de Uso Manter Curso/E-book
Fonte: Autoria própria (2026).

8.1.13 Diagrama de Caso de Uso Específico Visualizar Painel

A Figura 13 mostra o painel com indicadores e gráficos e a limpeza diária dos arquivos sem uso, feita pelo Sistema.

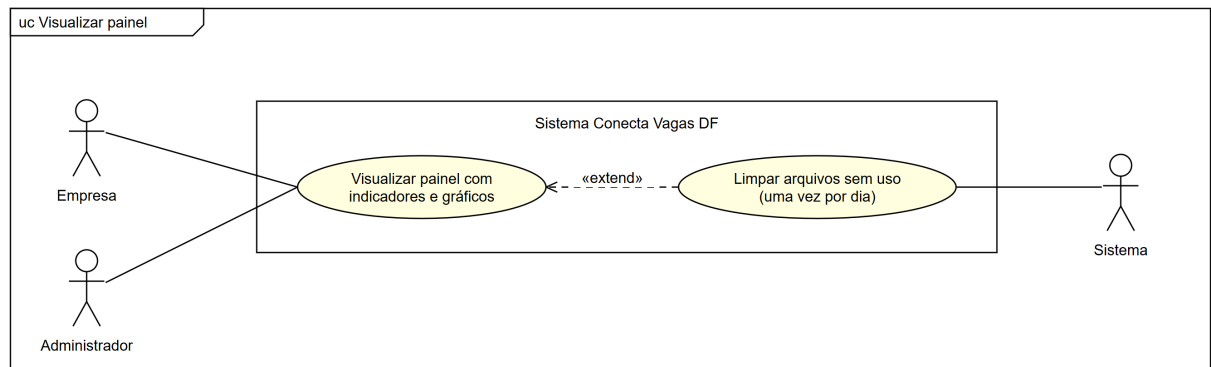


Figura 13 – Diagrama de Caso de Uso Visualizar Painel
Fonte: Autoria própria (2026).

8.1.14 Diagrama de Caso de Uso Específico Gerenciar Assinaturas

A Figura 14 mostra a gestão das assinaturas pelo administrador.

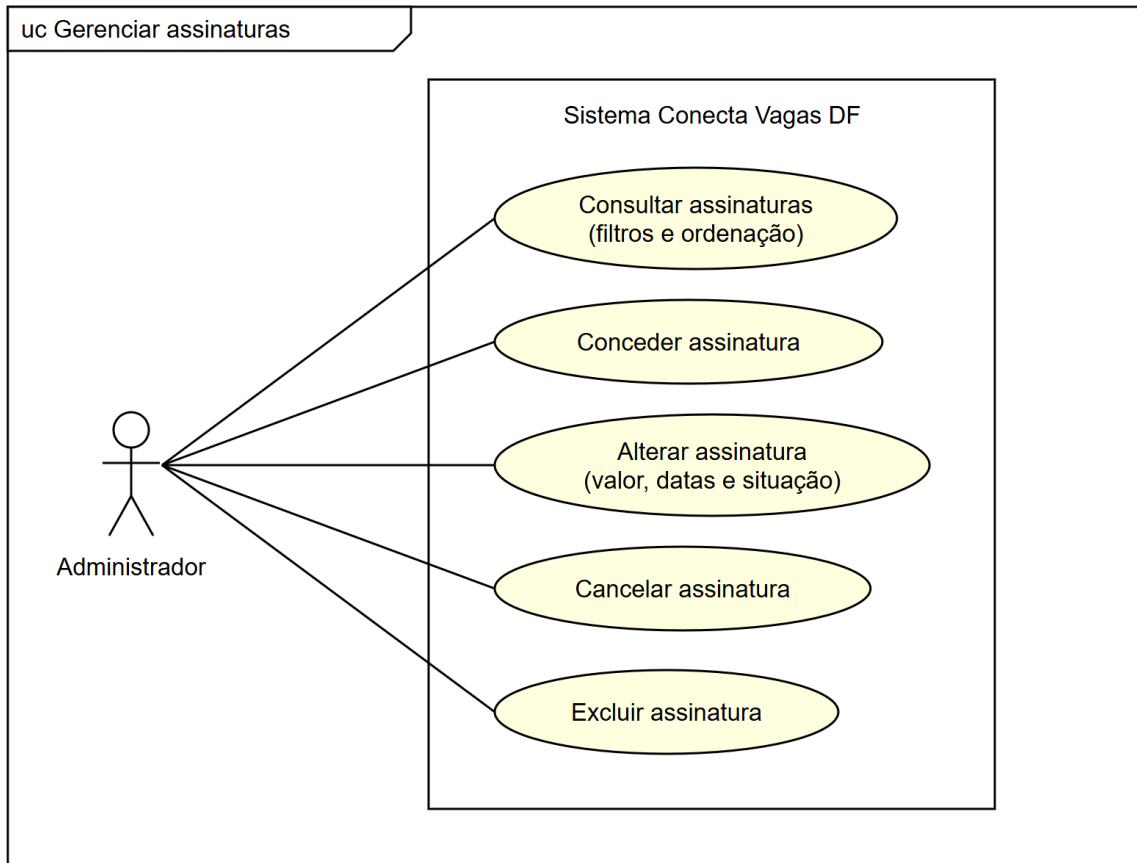


Figura 14 – Diagrama de Caso de Uso Gerenciar Assinaturas
Fonte: Autoria própria (2026).

8.1.15 Diagrama de Caso de Uso Específico Padrões Automáticos da Extração

A Figura 15 mostra como a Empresa e o Administrador usam a extração e onde o Sistema aplica os padrões automáticos.

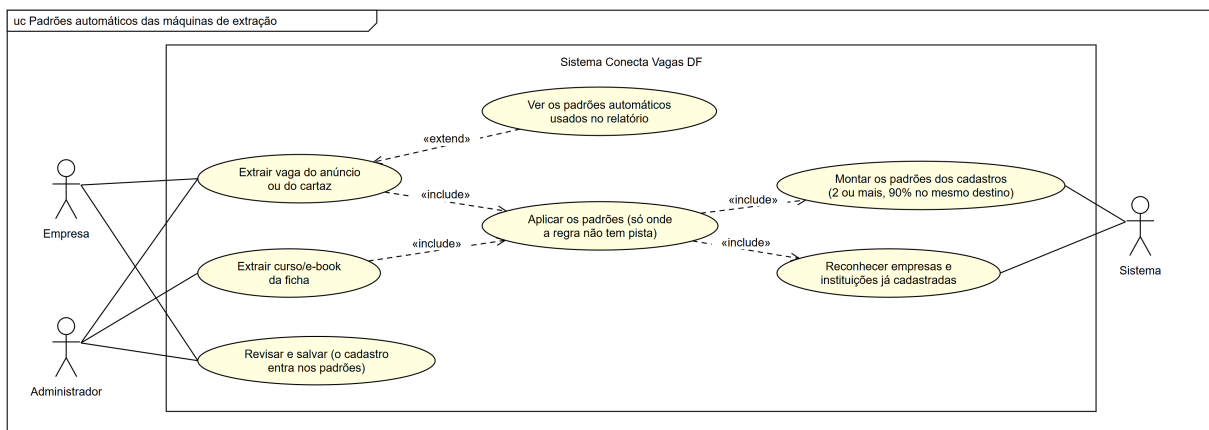


Figura 15 – Diagrama de Caso de Uso Padrões Automáticos da Extração
Fonte: Autoria própria (2026).

8.1.16 Especificação do Caso de Uso Realizar Login

Identificação: UC01

Nome: Realizar Login	
Ator Principal: Candidato, Empresa e Administrador	
Atores Secundários: Sistema (pausa do login após tentativas repetidas)	
Resumo: O ator informa e-mail e senha para entrar no sistema e ter acesso às funções do seu tipo de conta.	
Pré-Condições: O ator deverá estar cadastrado e ativo no sistema.	
Pós-Condições: O sistema exibirá o painel correspondente ao tipo de usuário.	
Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema
1. O ator informa e-mail e senha nos respectivos campos e clica no botão "Entrar".	2. O sistema verifica se o login está em pausa por tentativas repetidas e valida os dados informados, aceitando a senha mesmo com o Caps Lock ligado, a primeira letra trocada ou espaços nas pontas.
	3. O sistema inicia a sessão com um novo identificador e exibe o painel do usuário. O caso de uso é encerrado.
Fluxo Alternativo: dados incorretos	
Sequência Típica de Eventos	
Ator	Sistema
	2.1. O sistema verifica que os dados estão incorretos, registra a tentativa e exibe a mensagem "E-mail ou senha inválidos ou conta desativada." (no ambiente local, o motivo exato); quando faltam 3 tentativas ou menos, avisa quantas restam antes da pausa, retornando ao passo 1.
Fluxo Alternativo: login em pausa	
Sequência Típica de Eventos	
Ator	Sistema
	2.2. O sistema verifica que houve 8 senhas erradas seguidas e exibe a mensagem "Muitas tentativas de login sem sucesso. Por segurança, aguarde N minuto(s) e tente novamente, ou use "Esqueci minha senha"."
Fluxo Alternativo: encerrar sessão	
Sequência Típica de Eventos	
Ator	Sistema
3.1. O ator clica em "Sair".	3.2. O sistema encerra a sessão e exibe a página inicial.

Tabela 1 – Especificação do Caso de Uso Realizar Login
Fonte: Autoria própria (2026).

8.1.17 Especificação do Caso de Uso Cadastrar Conta

Identificação: UC02	
Nome: Cadastrar Conta	
Ator Principal: Candidato e Empresa (ainda sem conta)	
Atores Secundários: Não se aplica	
Resumo: O candidato ou a empresa cria a sua conta, aceita o termo de autorização e privacidade e já entra no sistema.	
Pré-Condições: O candidato ou a empresa ainda não deverá ter entrado no sistema.	
Pós-Condições: A conta e o perfil são criados e o usuário entra no sistema.	
Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema
1. O ator acessa a opção "Cadastro" no menu principal.	2. O sistema exibe o formulário de cadastro.
3. O ator escolhe o tipo (candidato ou empresa), preenche nome, e-mail, senha e telefone, marca o aceite do termo de privacidade e clica no botão "Cadastrar e montar perfil".	4. O sistema valida os dados e grava o usuário e o perfil na mesma transação.
	5. O sistema emite a mensagem "Cadastro realizado com sucesso!" e abre o perfil.
Fluxo Alternativo	
Sequência Típica de Eventos	
Ator	Sistema
	4.1. O sistema verifica que o e-mail já está cadastrado e emite a mensagem "Este e-mail já está cadastrado. Faça login ou use outro e-mail."
	4.2. O sistema verifica campos inválidos (e-mail, senha com menos de 6 ou mais de 72 caracteres ou termo não aceito), informa quais devem ser corrigidos e retorna ao passo 2.

Tabela 2 – Especificação do Caso de Uso Cadastrar Conta
Fonte: Autoria própria (2026).

8.1.18 Especificação do Caso de Uso Recuperar Senha

Identificação: UC03	
Nome: Recuperar Senha	
Ator Principal: Candidato, Empresa e Administrador	
Atores Secundários: Não se aplica	
Resumo: O ator que esqueceu a senha pede um link de redefinição e cadastra uma nova senha.	
Pré-Condições: O ator deverá possuir conta cadastrada.	
Pós-Condições: A senha é alterada, os links pendentes deixam de valer e as sessões abertas são encerradas.	

Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema
1. O ator clica em "Esqueci minha senha" e informa o e-mail.	2. O sistema gera um link com token válido por 30 minutos (no banco fica só o hash do token) e exibe a mensagem de confirmação, sem revelar se o e-mail existe; no modo de demonstração, o link aparece na tela.
3. O ator acessa o link e informa a nova senha.	4. O sistema valida o token e a senha.
	5. O sistema altera a senha e emite a mensagem "Senha alterada com sucesso. Entre com a nova senha."
Fluxo Alternativo	
Sequência Típica de Eventos	
Ator	Sistema
	2.1. Houve muitos pedidos seguidos; o sistema emite "Muitas tentativas. Aguarde alguns minutos e tente novamente."
	4.1. O sistema verifica que o link é inválido, já foi usado ou expirou e emite a mensagem "Link inválido, já utilizado ou expirado. Gere um novo."

Tabela 3 – Especificação do Caso de Uso Recuperar Senha
Fonte: Autoria própria (2026).

8.1.19 Especificação do Caso de Uso Consultar Vagas

Identificação: UC04	
Nome: Consultar Vagas	
Ator Principal: Candidato (com ou sem conta)	
Atores Secundários: Sistema (cálculo do match)	
Resumo: O ator pesquisa as vagas abertas, aplica filtros e abre os detalhes de uma vaga, com a nota de match quando está logado.	
Pré-Condições: Nenhuma.	
Pós-Condições: O sistema exibe a lista de vagas abertas que atendem aos filtros.	
Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema
1. O ator acessa a opção "Vagas" no menu principal.	2. O sistema exibe as vagas abertas, com destaque para as vagas Premium.
3. O ator informa palavra-chave e filtros (categoria, cidade, tipo, nível, modelo) e clica em "Buscar".	4. O sistema exibe as vagas encontradas, paginadas e, para o candidato logado, com a nota de match.
5. O ator clica em uma vaga.	6. O sistema exibe os detalhes da vaga, a explicação do match e os cursos recomendados.
Fluxo Alternativo	

Sequência Típica de Eventos	
Ator	Sistema
	4.1. O sistema não encontra vagas e exibe a mensagem informando que nenhuma vaga atende aos filtros.

Tabela 4 – Especificação do Caso de Uso Consultar Vagas
Fonte: Autoria própria (2026).

8.1.20 Especificação do Caso de Uso Consultar Cursos e E-books

Identificação: UC05	
Nome: Consultar Cursos e E-books	
Ator Principal: Candidato (com ou sem conta)	
Atores Secundários: Não se aplica	
Resumo: O ator consulta os cursos, e-books e vídeos de capacitação e baixa o PDF dos e-books guardados na biblioteca.	
Pré-Condições: Nenhuma.	
Pós-Condições: O sistema exibe a lista de cursos e e-books ativos.	
Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema
1. O ator acessa a opção "Cursos" ou "E-books" no menu principal.	2. O sistema exibe os cursos, e-books e vídeos ativos.
3. O ator aplica filtros (categoria, modalidade, nível, gratuito).	4. O sistema exibe os conteúdos encontrados.
5. O ator clica em um curso ou e-book.	6. O sistema exibe os detalhes, o botão "Baixar" (e-book guardado na biblioteca) ou "Acessar" (conteúdo em outro site) e as vagas relacionadas.
Fluxo Alternativo	
Sequência Típica de Eventos	
Ator	Sistema
	4.1. O sistema não encontra cursos e informa que nenhum conteúdo atende aos filtros.
Fluxo Alternativo: baixar da biblioteca	
Sequência Típica de Eventos	
Ator	Sistema
6.1. O ator clica em "Baixar".	6.2. O sistema entrega o PDF guardado na biblioteca da plataforma, com o crédito da fonte original.

Tabela 5 – Especificação do Caso de Uso Consultar Cursos e E-books
Fonte: Autoria própria (2026).

8.1.21 Especificação do Caso de Uso Alterar Perfil do Candidato

Identificação: UC06

Tabela 6 – Especificação do Caso de Uso Alterar Perfil do Candidato
Fonte: Autoria própria (2026).

8.1.22 Especificação do Caso de Uso Enviar Currículo (extração)

Identificação: UC07	
Nome: Enviar Currículo (extração)	
Ator Principal: Candidato	
Atores Secundários: Sistema (leitura do currículo e recálculo do match)	
Resumo: O candidato envia o currículo, e o Sistema lê o arquivo, preenche o perfil e mostra o relatório da extração para revisão.	
Pré-Condições: O ator deverá estar logado como candidato.	
Pós-Condições: O currículo é gravado, o perfil é preenchido e o relatório da extração é exibido.	
Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema
1. O ator seleciona o arquivo do currículo e clica em "Enviar".	2. O sistema valida tamanho (até 10 MB), extensão e conteúdo do arquivo.
	3. O sistema lê o texto do arquivo (PDF, DOCX ou DOC), extrai os dados só pelas regras (o currículo não entra nos padrões automáticos, por ter dados pessoais) e procura a foto do candidato.
	4. O sistema aplica os dados no perfil, recalcula o match, exibe o relatório da extração e a mensagem "Currículo lido: N dado(s) aplicado(s) ao perfil..."
5. O ator marca, no relatório, os dados que deseja substituir e clica em "Aplicar".	6. O sistema aplica as escolhas e recalcula o match.
Fluxo Alternativo	
Sequência Típica de Eventos	
Ator	Sistema
	2.1. O arquivo é maior que 10 MB ou de formato não aceito; o sistema emite "Envie o currículo em PDF, DOC ou DOCX." (ou a mensagem de tamanho) e retorna ao passo 1.
	3.1. O sistema não consegue ler o texto (imagem escaneada), salva o currículo e orienta o preenchimento manual do perfil.
	3.2. O perfil já tem foto; a foto encontrada no currículo aparece no relatório para o ator escolher se troca.

Tabela 7 – Especificação do Caso de Uso Enviar Currículo (extração)
Fonte: Autoria própria (2026).

8.1.23 Especificação do Caso de Uso Visualizar Portfólio e Match

Identificação: UC08

Nome: Visualizar Portfólio e Match	
Ator Principal: Candidato	
Atores Secundários: Sistema (montagem do portfólio e cálculo do match)	
Resumo: O candidato vê o portfólio montado automaticamente e a nota de match com cada vaga, com a explicação da nota.	
Pré-Condições: O cadastro do candidato deverá estar completo.	
Pós-Condições: O portfólio e as notas de match são exibidos.	
Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema
1. O ator acessa a aba "Portfólio".	2. O sistema monta o portfólio (linha do tempo, formação, links) e lista as vagas com a nota de match.
3. O ator clica em "Por que essa nota?".	4. O sistema exibe competências atendidas e faltantes, cargo, localização, nível e cursos recomendados.
Fluxo Alternativo	
Sequência Típica de Eventos	
Ator	Sistema
	2.1. O cadastro está incompleto; o sistema emite "Complete o cadastro para liberar o seu portfólio. Falta: ..." e abre o perfil.

Tabela 8 – Especificação do Caso de Uso Visualizar Portfólio e Match
 Fonte: Autoria própria (2026).

8.1.24 Especificação do Caso de Uso Candidatar-se à Vaga

Identificação: UC09.1	
Nome: Candidatar-se à Vaga	
Ator Principal: Candidato	
Atores Secundários: Não se aplica	
Resumo: O candidato se inscreve numa vaga aberta, escolhendo o currículo e escrevendo uma mensagem opcional.	
Pré-Condições: O ator deverá estar logado, possuir currículo enviado e a vaga deverá estar aberta.	
Pós-Condições: A candidatura é registrada com status "enviada".	
Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema
1. O ator acessa a vaga e clica em "Candidatar-se".	2. O sistema verifica o limite do plano e exibe o formulário.
3. O ator escolhe o currículo, escreve a mensagem de apresentação (opcional) e clica em "Enviar".	4. O sistema valida o currículo e grava a candidatura respeitando o limite.

	5. O sistema emite a mensagem "Candidatura enviada com sucesso! Acompanhe o status no seu perfil."
Fluxo Alternativo	
Sequência Típica de Eventos	
Ator	Sistema
	2.1. O ator já se candidatou; o sistema emite "Você já se candidatou a esta vaga."
	2.2. A vaga foi encerrada; o sistema emite "Esta vaga não está mais recebendo candidaturas."
	4.1. O limite de 3 candidaturas em andamento do plano gratuito foi atingido; o sistema emite a mensagem e direciona para a página de planos.

Tabela 9 – Especificação do Caso de Uso Candidatar-se à Vaga
Fonte: Autoria própria (2026).

8.1.25 Especificação do Caso de Uso Cancelar Candidatura

Identificação: UC09.2	
Nome: Cancelar Candidatura	
Ator Principal: Candidato	
Atores Secundários: Não se aplica	
Resumo: O candidato desiste de uma candidatura que ainda não foi analisada pela empresa.	
Pré-Condições: A candidatura deverá estar com status "enviada".	
Pós-Condições: A candidatura passa ao status "cancelada".	
Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema
1. O ator acessa "Minhas candidaturas" no perfil.	2. O sistema exibe as candidaturas e seus status.
3. O ator clica em "Cancelar".	4. O sistema emite a mensagem "Deseja realmente cancelar?".
5. O ator clica em "Confirmar".	6. O sistema cancela e emite a mensagem "Candidatura cancelada."
Fluxo Alternativo	
Sequência Típica de Eventos	
Ator	Sistema
5.1. O ator não confirma o cancelamento.	5.2. O sistema retorna ao passo 2.
	6.1. A candidatura já está em etapa avançada; o sistema emite "Não foi possível cancelar: a candidatura já está em uma etapa avançada."

Tabela 10 – Especificação do Caso de Uso Cancelar Candidatura
Fonte: Autoria própria (2026).

8.1.26 Especificação do Caso de Uso Manter Perfil da Empresa

Identificação: UC10	
Nome: Manter Perfil da Empresa	
Ator Principal: Empresa	
Atores Secundários: Não se aplica	
Resumo: A empresa cadastra e altera os seus dados, que aparecem nas vagas publicadas.	
Pré-Condições: O ator deverá estar logado como empresa.	
Pós-Condições: Os dados da empresa são exibidos nas vagas.	
Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema
1. O ator acessa a aba "Perfil da empresa".	2. O sistema exibe o formulário preenchido.
3. O ator preenche nome fantasia, CNPJ, setor, site, telefone, cidade, descrição e logotipo e clica em "Salvar".	4. O sistema valida os dados.
	5. O sistema grava e emite "Perfil da empresa atualizado. Esse nome aparece nas suas vagas."
Fluxo Alternativo	
Sequência Típica de Eventos	
Ator	Sistema
	4.1. O logotipo é inválido; o sistema emite "Logo inválida: use JPG, PNG ou WEBP de até 3 MB." e retorna ao passo 2.

Tabela 11 – Especificação do Caso de Uso Manter Perfil da Empresa
 Fonte: Autoria própria (2026).

8.1.27 Especificação do Caso de Uso Cadastrar Vaga

Identificação: UC11.1	
Nome: Cadastrar Vaga	
Ator Principal: Empresa e Administrador	
Atores Secundários: Sistema (leitura do anúncio ou do cartaz, padrões automáticos e match)	
Resumo: O ator publica uma vaga digitando os dados, colando o texto do anúncio ou enviando a foto do cartaz, que é lida no navegador.	
Pré-Condições: O ator deverá estar logado; a empresa deverá ter o perfil preenchido.	
Pós-Condições: A vaga é publicada e o match é calculado com os candidatos.	
Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema

1. O ator acessa a aba "Vagas" no painel.	2. O sistema exibe o formulário de cadastro e a lista de vagas e prepara o leitor de cartaz no navegador (carregador amarelo enquanto carrega, azul quando está pronto).
3. O ator cola o texto do anúncio ou escolhe a imagem do cartaz e clica em "Extrair dados do anúncio" ou "Ler cartaz e preencher".	4. O sistema lê o cartaz no próprio navegador (carregador amarelo com a porcentagem), envia o texto lido ao servidor, extrai título, empresa, salário, cidade, tipo, nível, requisitos, benefícios e contato, usa os padrões automáticos onde a regra não tem pista e preenche o formulário, com o relatório da extração e os "Padrões automáticos usados".
5. O ator revisa os dados e clica em "Salvar vaga".	6. O sistema valida os dados, verifica vaga duplicada e o limite do plano.
	7. O sistema grava a vaga, calcula o match e emite "Vaga publicada! Match calculado com N candidato(s)."
Fluxo Alternativo	
Sequência Típica de Eventos	
Ator	Sistema
	3.1. O ator clica em extrair sem texto e sem cartaz; o sistema emite "Cole o texto do anúncio (ou envie o cartaz) antes de extrair." e não gera relatório.
	3.2. O cartaz é inválido; o sistema emite "Cartaz inválido: envie JPG, PNG ou WEBP de até 8 MB."
	6.1. O sistema encontra campos inválidos (título vazio, salário máximo menor que o mínimo, data expirada) e informa quais corrigir.
	6.2. O limite de 2 vagas ativas do plano gratuito foi atingido; o sistema emite a mensagem e direciona para os planos.
	6.3. Existe vaga aberta igual; o sistema pede a confirmação "publicar mesmo assim".

Tabela 12 – Especificação do Caso de Uso Cadastrar Vaga

Fonte: Autoria própria (2026).

8.1.28 Especificação do Caso de Uso Alterar Vaga

Identificação: UC11.2
Nome: Alterar Vaga
Ator Principal: Empresa e Administrador
Atores Secundários: Sistema (recálculo do match)
Resumo: O ator altera os dados ou o status de uma vaga já publicada.
Pré-Condições: O ator deverá pesquisar a vaga; a empresa só altera as próprias vagas.
Pós-Condições: O sistema retornará à tela com a lista de vagas.

Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema
1. O ator clica no botão "Editar".	2. O sistema exibe os dados dentro de um formulário preenchido.
3. O ator, após editar, clica no botão "Salvar".	4. O sistema valida os dados inseridos.
	5. O sistema grava, recalcula o match e emite a mensagem "Vaga atualizada."
Fluxo Alternativo	
Sequência Típica de Eventos	
Ator	Sistema
	2.1. A vaga não pertence à empresa; o sistema nega o acesso (erro 403).
	4.1. O sistema informa que existem campos obrigatórios em branco ou inválidos.

Tabela 13 – Especificação do Caso de Uso Alterar Vaga
Fonte: Autoria própria (2026).

8.1.29 Especificação do Caso de Uso Excluir Vaga

Identificação: UC11.3	
Nome: Excluir Vaga	
Ator Principal: Empresa e Administrador	
Atores Secundários: Não se aplica	
Resumo: O ator exclui uma vaga, e junto saem as candidaturas e os matches dela.	
Pré-Condições: O ator deverá pesquisar a vaga.	
Pós-Condições: O sistema retornará à tela com a lista de vagas.	
Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema
1. O ator clica no botão "Excluir".	2. O sistema exibe os dados da vaga e a mensagem "Deseja realmente excluir?".
3. O ator clica no botão "Confirmar".	4. O sistema exclui a vaga, suas candidaturas e matches e emite "Vaga excluída (candidaturas e matches dela também)."
Fluxo Alternativo	
Sequência Típica de Eventos	
Ator	Sistema
3.1. O ator cancela a exclusão.	3.2. O sistema retorna ao passo 2 do fluxo principal.

Tabela 14 – Especificação do Caso de Uso Excluir Vaga
Fonte: Autoria própria (2026).

8.1.30 Especificação do Caso de Uso Gerenciar Candidaturas Recebidas

Identificação: UC12	
Nome: Gerenciar Candidaturas Recebidas	
Ator Principal: Empresa e Administrador	
Atores Secundários: Não se aplica	
Resumo: O ator vê as candidaturas ordenadas por compatibilidade, muda o status e escreve um retorno ao candidato.	
Pré-Condições: O ator deverá estar logado.	
Pós-Condições: O candidato visualiza o novo status e o retorno no perfil.	
Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema
1. O ator acessa a aba "Candidaturas".	2. O sistema exibe as candidaturas (VIP primeiro, depois maior match) com competências atendidas e faltantes.
3. O ator filtra por vaga ou status e abre uma candidatura.	4. O sistema exibe os dados do candidato e o link do currículo.
5. O ator altera o status, escreve o retorno e clica em "Salvar".	6. O sistema grava e emite "Candidatura atualizada. O candidato vê o novo status e o retorno no perfil dele."
Fluxo Alternativo	
Sequência Típica de Eventos	
Ator	Sistema
5.1. O administrador clica em "Excluir".	5.2. O sistema exclui e emite "Candidatura excluída."
	6.1. A candidatura é de outra empresa ou foi cancelada; o sistema emite "Não foi possível atualizar..."

Tabela 15 – Especificação do Caso de Uso Gerenciar Candidaturas Recebidas
Fonte: Autoria própria (2026).

8.1.31 Especificação do Caso de Uso Consultar Banco de Talentos

Identificação: UC13	
Nome: Consultar Banco de Talentos	
Ator Principal: Empresa e Administrador	
Atores Secundários: Não se aplica	
Resumo: O ator pesquisa os perfis públicos dos candidatos por termo e cidade.	
Pré-Condições: O ator deverá estar logado.	
Pós-Condições: O sistema exibe os perfis públicos encontrados.	
Fluxo Principal	
Sequência Típica de Eventos	

Ator	Sistema
1. O ator acessa a aba "Banco de talentos".	2. O sistema exibe os perfis públicos de candidatos.
3. O ator informa termo e cidade e clica em "Buscar".	4. O sistema exibe os perfis encontrados, com acesso ao portfólio.
Fluxo Alternativo	
Sequência Típica de Eventos	
Ator	Sistema
	4.1. A empresa sem plano Premium visualiza o banco de talentos de forma limitada (nome e contato ocultos), com convite para assinar o plano.

Tabela 16 – Especificação do Caso de Uso Consultar Banco de Talentos
Fonte: Autoria própria (2026).

8.1.32 Especificação do Caso de Uso Assinar Plano

Identificação: UC14.1	
Nome: Assinar Plano	
Ator Principal: Candidato e Empresa	
Atores Secundários: Não se aplica	
Resumo: O ator assina o plano pago do seu tipo de conta (Candidato VIP ou Empresa Premium), que é demonstrativo.	
Pré-Condições: O ator deverá estar logado e sem assinatura ativa.	
Pós-Condições: A assinatura fica ativa por 30 dias.	
Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema
1. O ator acessa a página "Planos".	2. O sistema exibe os planos gratuito e pago do seu tipo de conta.
3. O ator clica em "Assinar".	4. O sistema verifica o tipo de conta e se já existe plano ativo.
	5. O sistema registra a assinatura (VIP R\$ 9,90 ou Premium R\$ 49,90, demonstrativos) e emite "Parabéns! ... ativada com sucesso por 30 dias."
Fluxo Alternativo	
Sequência Típica de Eventos	
Ator	Sistema
	4.1. O plano já está ativo; o sistema emite "Seu plano VIP já está ativo." ou "O plano Empresa Premium já está ativo."

Tabela 17 – Especificação do Caso de Uso Assinar Plano
Fonte: Autoria própria (2026).

8.1.33 Especificação do Caso de Uso Cancelar Assinatura

Identificação: UC14.2	
Nome: Cancelar Assinatura	
Ator Principal: Candidato e Empresa	
Atores Secundários: Não se aplica	
Resumo: O ator cancela a assinatura ativa e volta ao plano gratuito.	
Pré-Condições: O ator deverá possuir assinatura ativa.	
Pós-Condições: Os benefícios do plano são removidos.	
Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema
1. O ator clica em "Cancelar assinatura".	2. O sistema emite a mensagem "Deseja realmente cancelar?".
3. O ator clica em "Confirmar".	4. O sistema cancela a assinatura, remove o destaque das vagas (empresa) e emite a confirmação.
Fluxo Alternativo	
Sequência Típica de Eventos	
Ator	Sistema
	4.1. Não existe assinatura ativa; o sistema emite "Não existe uma assinatura ativa para cancelar."

Tabela 18 – Especificação do Caso de Uso Cancelar Assinatura
 Fonte: Autoria própria (2026).

8.1.34 Especificação do Caso de Uso Cadastrar Usuário

Identificação: UC15.1	
Nome: Cadastrar Usuário	
Ator Principal: Administrador	
Atores Secundários: Não se aplica	
Resumo: O administrador cria uma conta de candidato, empresa ou administrador, já com o perfil.	
Pré-Condições: O administrador deverá estar logado.	
Pós-Condições: O sistema retornará à tela com a lista de usuários.	
Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema
1. O administrador acessa a aba "Usuários" no painel.	2. O sistema exibe o formulário de cadastro e a lista.
3. O administrador preenche nome, e-mail, senha, telefone, tipo e situação e clica no botão "Salvar".	4. O sistema valida os dados.
	5. O sistema emite a mensagem "Usuário criado com perfil."

Fluxo Alternativo	
Sequência Típica de Eventos	
Ator	Sistema
	4.1. O sistema verifica que campos obrigatórios não foram preenchidos ou são inválidos, informa quais devem ser corrigidos e retorna ao passo 2.

Tabela 19 – Especificação do Caso de Uso Cadastrar Usuário
Fonte: Autoria própria (2026).

8.1.35 Especificação do Caso de Uso Alterar Usuário

Identificação: UC15.2	
Nome: Alterar Usuário	
Ator Principal: Administrador	
Atores Secundários: Não se aplica	
Resumo: O administrador altera os dados de uma conta.	
Pré-Condições: O administrador deverá pesquisar o usuário.	
Pós-Condições: O sistema retornará à tela com a lista de usuários.	
Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema
1. O administrador clica no botão "Editar".	2. O sistema exibe os dados dentro de um formulário preenchido.
3. O administrador, após editar, clica no botão "Salvar".	4. O sistema valida os dados inseridos.
	5. O sistema emite a mensagem "Usuário atualizado."
Fluxo Alternativo	
Sequência Típica de Eventos	
Ator	Sistema
	4.1. O sistema impede rebaixar ou desativar o último administrador ativo e informa o motivo.

Tabela 20 – Especificação do Caso de Uso Alterar Usuário
Fonte: Autoria própria (2026).

8.1.36 Especificação do Caso de Uso Excluir Usuário

Identificação: UC15.3	
Nome: Excluir Usuário	
Ator Principal: Administrador	
Atores Secundários: Não se aplica	
Resumo: O administrador exclui uma conta, e todos os dados dela são removidos em cascata.	
Pré-Condições: O administrador deverá pesquisar o usuário.	

Pós-Condições: O usuário e todos os seus dados são removidos.	
Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema
1. O administrador clica no botão "Excluir".	2. O sistema exibe os dados do usuário e a mensagem "Deseja realmente excluir?".
3. O administrador clica no botão "Confirmar".	4. O sistema exclui a conta e emite "Usuário excluído (perfil, currículos, vagas e candidaturas foram removidos junto)."
Fluxo Alternativo	
Sequência Típica de Eventos	
Ator	Sistema
3.1. O administrador cancela a exclusão.	3.2. O sistema retorna ao passo 2.
	4.1. O administrador tenta excluir a própria conta; o sistema emite "Você não pode excluir a própria conta."
	4.2. É o último administrador ativo; o sistema emite "É preciso manter pelo menos um administrador ativo."

Tabela 21 – Especificação do Caso de Uso Excluir Usuário
Fonte: Autoria própria (2026).

8.1.37 Especificação do Caso de Uso Ativar / Desativar Usuário

Identificação: UC15.4	
Nome: Ativar / Desativar Usuário	
Ator Principal: Administrador	
Atores Secundários: Não se aplica	
Resumo: O administrador bloqueia ou libera o acesso de uma conta.	
Pré-Condições: O administrador deverá pesquisar o usuário.	
Pós-Condições: A situação da conta é alterada; a conta desativada perde o acesso na hora.	
Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema
1. O administrador clica em "Desativar" (ou "Ativar").	2. O sistema altera a situação e emite "Conta bloqueada: o login e a sessão aberta perdem o acesso." (ou "Conta ativada: o login volta a funcionar.")
Fluxo Alternativo	
Sequência Típica de Eventos	
Ator	Sistema
	2.1. O sistema impede desativar a própria conta ou o último administrador ativo e informa o motivo.

Tabela 22 – Especificação do Caso de Uso Ativar / Desativar Usuário
Fonte: Autoria própria (2026).

8.1.38 Especificação do Caso de Uso Cadastrar Categoria

Identificação: UC16.1	
Nome: Cadastrar Categoria	
Ator Principal: Administrador	
Atores Secundários: Não se aplica	
Resumo: O administrador cria uma categoria de vaga ou de curso.	
Pré-Condições: O administrador deverá estar logado.	
Pós-Condições: A categoria fica disponível para vagas ou cursos.	
Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema
1. O administrador acessa a aba "Categorias".	2. O sistema exibe o formulário e a lista.
3. O administrador informa nome e tipo (vaga ou curso) e clica em "Salvar".	4. O sistema valida os dados e emite a mensagem "Categoria salva."
Fluxo Alternativo	
Sequência Típica de Eventos	
Ator	Sistema
	4.1. O nome está vazio ou já existe para o mesmo tipo; o sistema informa o erro e retorna ao passo 2.

Tabela 23 – Especificação do Caso de Uso Cadastrar Categoria
Fonte: Autoria própria (2026).

8.1.39 Especificação do Caso de Uso Alterar Categoria

Identificação: UC16.2	
Nome: Alterar Categoria	
Ator Principal: Administrador	
Atores Secundários: Não se aplica	
Resumo: O administrador altera o nome ou o tipo de uma categoria.	
Pré-Condições: O administrador deverá selecionar a categoria.	
Pós-Condições: O sistema retornará à lista de categorias.	
Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema
1. O administrador clica em "Editar".	2. O sistema exibe o formulário preenchido.
3. O administrador altera e clica em "Salvar".	4. O sistema valida e emite "Categoria salva."
Fluxo Alternativo	
Sequência Típica de Eventos	

Ator	Sistema
	4.1. A categoria não existe mais; o sistema emite "Categoria não encontrada (pode ter sido excluída)."

Tabela 24 – Especificação do Caso de Uso Alterar Categoria
Fonte: Autoria própria (2026).

8.1.40 Especificação do Caso de Uso Excluir Categoria

Identificação: UC16.3	
Nome: Excluir Categoria	
Ator Principal: Administrador	
Atores Secundários: Não se aplica	
Resumo: O administrador exclui uma categoria; as vagas e os cursos que a usavam ficam sem categoria.	
Pré-Condições: O administrador deverá selecionar a categoria.	
Pós-Condições: Vagas e cursos que a usavam ficam sem categoria.	
Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema
1. O administrador clica em "Excluir".	2. O sistema emite "Deseja realmente excluir?".
3. O administrador clica em "Confirmar".	4. O sistema emite "Categoria excluída. Vagas e cursos que a usavam ficaram sem categoria."
Fluxo Alternativo	
Sequência Típica de Eventos	
Ator	Sistema
3.1. O administrador cancela a exclusão.	3.2. O sistema retorna à lista.

Tabela 25 – Especificação do Caso de Uso Excluir Categoria
Fonte: Autoria própria (2026).

8.1.41 Especificação do Caso de Uso Ativar / Desativar Categoria

Identificação: UC16.4	
Nome: Ativar / Desativar Categoria	
Ator Principal: Administrador	
Atores Secundários: Não se aplica	
Resumo: O administrador tira uma categoria dos filtros, ou a coloca de volta, sem mudar os itens que a usam.	
Pré-Condições: O administrador deverá selecionar a categoria.	
Pós-Condições: A categoria sai (ou volta) dos filtros; os itens continuam com ela.	
Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema

1. O administrador clica em "Desativar" (ou "Ativar").	2. O sistema emite "Categoria desativada: some dos filtros, mas os itens continuam com ela." (ou "Categoria ativada.")
Fluxo Alternativo	
Sequência Típica de Eventos	
Ator	Sistema
	2.1. A categoria não existe mais; o sistema emite "Categoria não encontrada."

Tabela 26 – Especificação do Caso de Uso Ativar / Desativar Categoria
Fonte: Autoria própria (2026).

8.1.42 Especificação do Caso de Uso Cadastrar Curso/E-book

Identificação: UC17.1	
Nome: Cadastrar Curso/E-book	
Ator Principal: Administrador	
Atores Secundários: Sistema (abertura dos links da ficha, capa do PDF e padrões automáticos)	
Resumo: O administrador cadastra um curso ou e-book digitando os dados ou colando a ficha, e o Sistema abre os links, tira a capa do PDF e guarda o arquivo na biblioteca.	
Pré-Condições: O administrador deverá estar logado.	
Pós-Condições: O conteúdo fica disponível na página de cursos; o PDF do e-book fica na biblioteca.	
Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema
1. O administrador acessa a aba "Cursos e e-books".	2. O sistema exibe a caixa "Extrair", o formulário e a lista.
3. O administrador cola o texto de divulgação ou a ficha da pesquisa e clica em "Extrair" (opcional).	4. O sistema preenche título, instituição, link, PDF, carga horária, preço, modalidade, nível e categoria (a área e a instituição que a regra não reconhece vêm dos padrões automáticos do catálogo), abre cada link da ficha e, no e-book sem imagem, tira a capa da 1ª página do PDF.
5. O administrador revisa, envia o PDF do e-book (opcional) e clica em "Salvar conteúdo".	6. O sistema valida, baixa a imagem e o PDF indicados, guarda o PDF na biblioteca com o crédito da fonte e emite "Conteúdo salvo."
Fluxo Alternativo	
Sequência Típica de Eventos	
Ator	Sistema
	6.1. O sistema verifica campos obrigatórios em branco ou link inválido, informa quais corrigir e retorna ao passo 2.
	6.2. Sem imagem, o conteúdo entra com a imagem padrão da plataforma e o sistema avisa para trocá-la depois em "Editar".

Tabela 27 – Especificação do Caso de Uso Cadastrar Curso/E-book
Fonte: Autoria própria (2026).

8.1.43 Especificação do Caso de Uso Alterar Curso/E-book

Identificação: UC17.2	
Nome: Alterar Curso/E-book	
Ator Principal: Administrador	
Atores Secundários: Não se aplica	
Resumo: O administrador altera os dados de um conteúdo ou o publica e oculta da área pública.	
Pré-Condições: O administrador deverá selecionar o conteúdo.	
Pós-Condições: O sistema retornará à lista.	
Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema
1. O administrador clica em "Editar".	2. O sistema exibe o formulário preenchido.
3. O administrador altera e clica em "Salvar conteúdo".	4. O sistema valida e emite "Conteúdo salvo."
Fluxo Alternativo	
Sequência Típica de Eventos	
Ator	Sistema
	4.1. O sistema informa que existem campos obrigatórios em branco.
Fluxo Alternativo: publicar ou ocultar	
Sequência Típica de Eventos	
Ator	Sistema
1.1. O administrador clica em "Ocultar" (ou "Publicar").	1.2. O sistema emite "Conteúdo ocultado: saiu da área pública." (ou "Conteúdo publicado: já aparece para os usuários.")

Tabela 28 – Especificação do Caso de Uso Alterar Curso/E-book
Fonte: Autoria própria (2026).

8.1.44 Especificação do Caso de Uso Excluir Curso/E-book

Identificação: UC17.3	
Nome: Excluir Curso/E-book	
Ator Principal: Administrador	
Atores Secundários: Não se aplica	
Resumo: O administrador exclui um conteúdo de capacitação.	
Pré-Condições: O administrador deverá selecionar o conteúdo.	
Pós-Condições: O conteúdo deixa de ser exibido.	
Fluxo Principal	
Sequência Típica de Eventos	

Ator	Sistema
1. O administrador clica em "Excluir".	2. O sistema emite "Deseja realmente excluir?".
3. O administrador clica em "Confirmar".	4. O sistema emite "Conteúdo excluído."
Fluxo Alternativo	
Sequência Típica de Eventos	
Ator	Sistema
3.1. O administrador cancela a exclusão.	3.2. O sistema retorna à lista.

Tabela 29 – Especificação do Caso de Uso Excluir Curso/E-book
Fonte: Autoria própria (2026).

8.1.45 Especificação do Caso de Uso Importar Fichas em Lote

Identificação: UC17.4	
Nome: Importar Fichas em Lote	
Ator Principal: Administrador	
Atores Secundários: Sistema (abertura dos links da ficha e capa do PDF)	
Resumo: O administrador cola várias fichas de uma vez, confere a prévia, já com as capas, e cadastra as que marcar.	
Pré-Condições: O administrador deverá estar logado.	
Pós-Condições: Os conteúdos marcados na prévia são cadastrados e publicados.	
Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema
1. O administrador cola várias fichas separadas por "---" e clica em "Extrair".	2. O sistema lê as fichas, abre cada link (imagem, PDF ou página), tira a capa do PDF dos e-books sem imagem, avisa os links que não abriram como deveriam e mostra a prévia com as fichas lidas.
3. O administrador confere a prévia, desmarca o que não quer e clica em "Cadastrar marcados".	4. O sistema baixa imagens e PDFs, cadastra e publica os marcados e informa quantos conteúdos foram cadastrados e quantos ficaram com o PDF na biblioteca (botão "Baixar").
Fluxo Alternativo	
Sequência Típica de Eventos	
Ator	Sistema
3.1. O administrador clica em "Descartar prévia".	3.2. O sistema descarta a prévia e retorna à lista.

Tabela 30 – Especificação do Caso de Uso Importar Fichas em Lote
Fonte: Autoria própria (2026).

8.1.46 Especificação do Caso de Uso Trazer os PDFs para a Biblioteca

Identificação: UC17.5
Nome: Trazer os PDFs para a Biblioteca
Ator Principal: Administrador
Atores Secundários: Sistema (download seguro dos PDFs)

Resumo: O administrador pede que o Sistema procure e baixe, das páginas oficiais, o PDF dos e-books que ainda abrem em outro site.	
Pré-Condições: O administrador deverá estar logado e o servidor deverá ter acesso à internet.	
Pós-Condições: Os e-books encontrados passam a ter o botão "Baixar".	
Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema
1. O administrador clica em "Trazer os PDFs para a biblioteca".	2. Para cada e-book que ainda abre em outro site, o sistema procura o PDF na página oficial e o baixa com segurança (endereço público, HTTPS verificado, até 150 MB).
	3. O sistema guarda os PDFs, credita a fonte original e informa quantos PDFs foram guardados na biblioteca e passaram a ter o botão "Baixar".
Fluxo Alternativo	
Sequência Típica de Eventos	
Ator	Sistema
	2.1. O link não leva a um PDF público; o e-book continua com "Acessar" e o sistema orienta enviar o arquivo em "Editar".

Tabela 31 – Especificação do Caso de Uso Trazer os PDFs para a Biblioteca
Fonte: Autoria própria (2026).

8.1.47 Especificação do Caso de Uso Visualizar Painel

Identificação: UC18	
Nome: Visualizar Painel	
Ator Principal: Empresa e Administrador	
Atores Secundários: Sistema (limpeza diária dos arquivos sem uso)	
Resumo: O ator vê os indicadores e gráficos do sistema; ao abrir o painel do administrador, o Sistema faz a limpeza diária.	
Pré-Condições: O ator deverá estar logado.	
Pós-Condições: O painel com indicadores é exibido.	
Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema
1. O ator acessa o "Painel".	2. O sistema calcula e exibe indicadores (vagas, candidaturas por status, candidaturas por dia, match médio, usuários por tipo) e gráficos.
3. O ator clica em um indicador.	4. O sistema direciona para a lista correspondente.
Fluxo Alternativo	
Sequência Típica de Eventos	
Ator	Sistema

	2.1. Para a empresa, o sistema exibe somente os dados das próprias vagas.
	2.2. Para o administrador, no máximo uma vez por dia, o sistema apaga os arquivos enviados que nenhum registro usa e que têm mais de 24 horas.

Tabela 32 – Especificação do Caso de Uso Visualizar Painei
Fonte: Autoria própria (2026).

8.1.48 Especificação do Caso de Uso Excluir a Própria Conta (LGPD)

Identificação: UC19	
Nome: Excluir a Própria Conta (LGPD)	
Ator Principal: Candidato	
Atores Secundários: Sistema (limite de tentativas de senha)	
Resumo: O candidato exclui a própria conta e todos os dados ligados a ela, confirmando com a senha.	
Pré-Condições: O ator deverá estar logado como candidato.	
Pós-Condições: A conta e todos os dados do candidato são removidos e a sessão é encerrada.	
Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema
1. O ator acessa "Meu perfil", na parte "Seus dados (LGPD)", e clica em "Excluir minha conta".	2. O sistema explica o que será apagado e pede a senha e a confirmação.
3. O ator informa a senha, marca a caixa de confirmação e confirma a exclusão.	4. O sistema confere a senha, com o mesmo limite de tentativas do login.
	5. O sistema apaga o perfil, os currículos e arquivos, a foto, as candidaturas, os matches e as tentativas de login, encerra a sessão e emite "Sua conta e todos os seus dados foram excluídos da plataforma."
Fluxo Alternativo	
Sequência Típica de Eventos	
Ator	Sistema
	4.1. A caixa de confirmação não foi marcada; o sistema emite "Marque a caixa de confirmação para excluir a conta." e retorna ao passo 2.
	4.2. A senha está incorreta; o sistema emite "Senha incorreta: a conta não foi excluída."
	4.3. Houve muitas senhas erradas; o sistema emite "Muitas senhas erradas. Aguarde alguns minutos e tente de novo."

Tabela 33 – Especificação do Caso de Uso Excluir a Própria Conta (LGPD)
Fonte: Autoria própria (2026).

8.1.49 Especificação do Caso de Uso Gerenciar Assinaturas

Identificação: UC20	
Nome: Gerenciar Assinaturas	
Ator Principal: Administrador	
Atores Secundários: Não se aplica	
Resumo: O administrador consulta as assinaturas e concede, altera, cancela ou exclui planos de candidatos e empresas.	
Pré-Condições: O administrador deverá estar logado.	
Pós-Condições: A assinatura é concedida, alterada, cancelada ou excluída do histórico.	
Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema
1. O administrador acessa a aba "Assinaturas".	2. O sistema exibe a lista com o resumo por plano, filtros (plano, situação, busca) e ordenação.
3. O administrador escolhe a conta, a duração em dias e o valor e clica em "Conceder".	4. O sistema valida e emite "Empresa Premium (ou Candidato VIP) concedido a NOME por N dias."
Fluxo Alternativo	
Sequência Típica de Eventos	
Ator	Sistema
	4.1. A conta está bloqueada; o sistema emite "Esta conta está bloqueada: ative-a antes de conceder um plano."
	4.2. A duração está fora de 1 a 3660 dias ou o valor é inválido; o sistema informa o erro.
Fluxo Alternativo: alterar, cancelar ou excluir	
Sequência Típica de Eventos	
Ator	Sistema
3.1. O administrador clica em "Editar", altera valor, datas ou situação e salva.	3.2. O sistema valida e emite "Assinatura atualizada."
3.3. O administrador clica em "Cancelar".	3.4. O sistema emite "Assinatura cancelada: a conta voltou ao plano gratuito (o histórico fica guardado)."
3.5. O administrador clica em "Excluir".	3.6. O sistema emite "Assinatura excluída do histórico."

Tabela 34 – Especificação do Caso de Uso Gerenciar Assinaturas
 Fonte: Autoria própria (2026).

8.1.50 Especificação do Caso de Uso Extrair com os Padrões Automáticos

Identificação: UC21
Nome: Extrair com os Padrões Automáticos
Ator Principal: Empresa e Administrador

Atores Secundários: Sistema (montagem e aplicação dos padrões)	
Resumo: Na extração de vagas e de cursos, o Sistema completa as regras com padrões tirados dos cadastros já existentes e mostra cada padrão usado.	
Pré-Condições: O ator deverá estar logado; os padrões saem das vagas e dos cursos já cadastrados.	
Pós-Condições: O formulário é preenchido e o relatório mostra os padrões automáticos usados; nada é salvo sem revisão.	
Fluxo Principal	
Sequência Típica de Eventos	
Ator	Sistema
1. O ator cola o anúncio (ou envia o cartaz) na aba "Vagas" e clica para extrair.	2. O sistema aplica as regras do código (títulos de seção, palavras-chave e padrões de texto).
	3. Para as linhas soltas sem palavra-chave, a área não reconhecida e a empresa não achada, o sistema monta os padrões a partir das vagas cadastradas (termo visto em pelo menos 2 vagas e em 90% delas no mesmo campo) e decide pelo padrão mais específico.
	4. O sistema preenche o formulário e mostra no relatório da extração o bloco "Padrões automáticos usados", com o trecho, para onde foi e o padrão que valeu.
5. O ator revisa e salva a vaga.	6. O sistema grava a vaga, que passa a contar nos padrões da próxima extração.
Fluxo Alternativo: sem padrão	
Sequência Típica de Eventos	
Ator	Sistema
	3.1. Nenhum padrão casa com o trecho, ou o padrão concorda com a regra; vale o palpite da regra (a linha fica na descrição).
Fluxo Alternativo: curso ou e-book	
Sequência Típica de Eventos	
Ator	Sistema
1.1. O administrador cola a ficha na aba "Cursos e e-books" e clica em "Extrair".	3.2. A área que a regra não reconhece vem do padrão dos títulos do catálogo, e a instituição não achada, das instituições já cadastradas.
Fluxo Alternativo: banco indisponível	
Sequência Típica de Eventos	
Ator	Sistema
	3.3. O sistema não consegue ler os cadastros; a extração segue só com as regras e o erro vai para o log.

Tabela 35 – Especificação do Caso de Uso Extrair com os Padrões Automáticos
 Fonte: Autoria própria (2026).

8.2 Diagramas de Sequência

Os diagramas de sequência mostram a troca de mensagens entre o ator e os objetos do sistema ao longo do tempo. Seguimos o padrão Boundary (a tela), Control (o controlador ou o serviço) e Entity (a classe de acesso a dados). A seguir estão os diagramas dos fluxos mais importantes do sistema.

8.2.1 Diagrama de Sequência Realizar Login

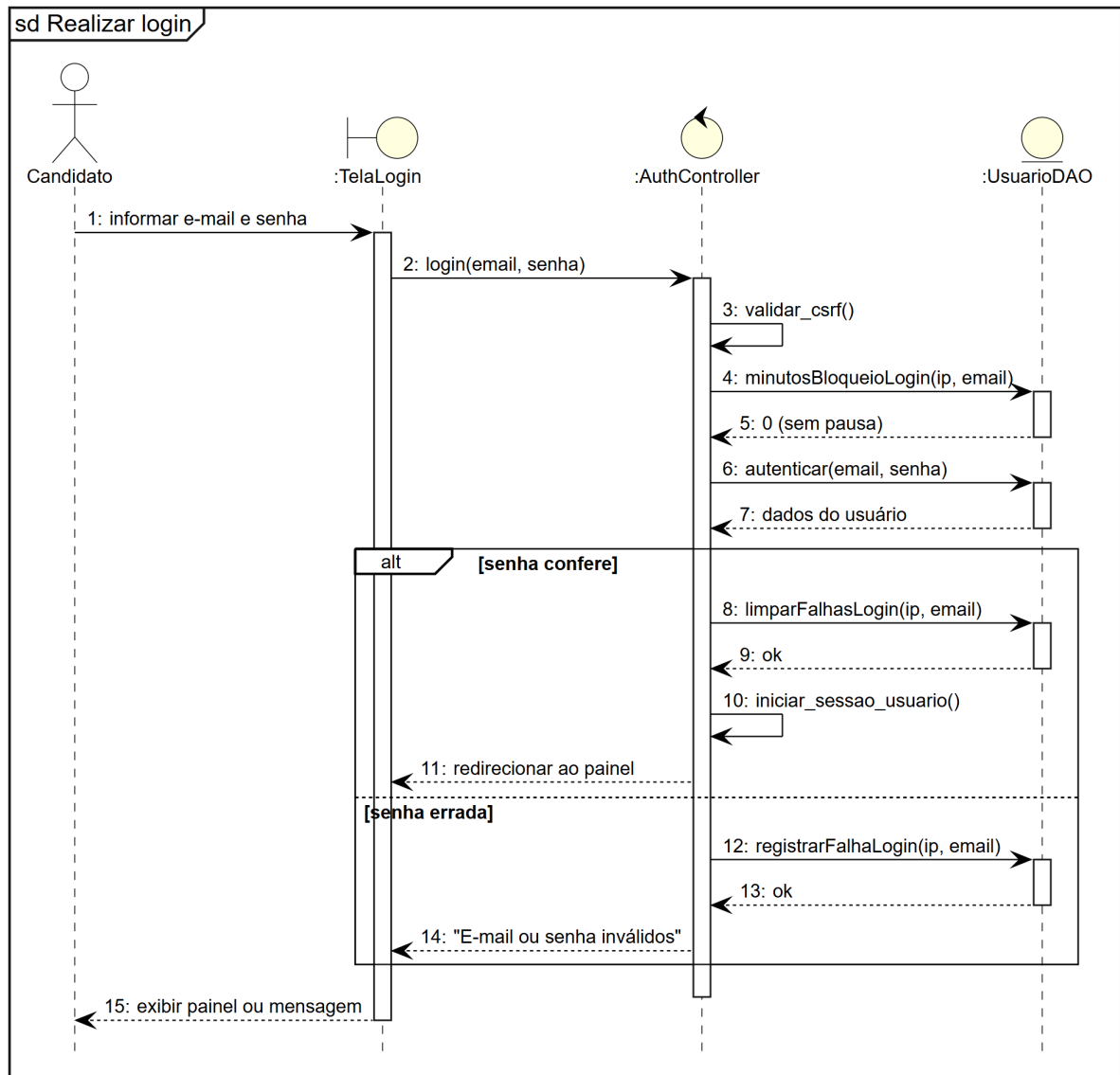


Figura 16 – Diagrama de Sequência Realizar Login
Fonte: Autoria própria (2026).

8.2.2 Diagrama de Sequência Cadastrar Conta

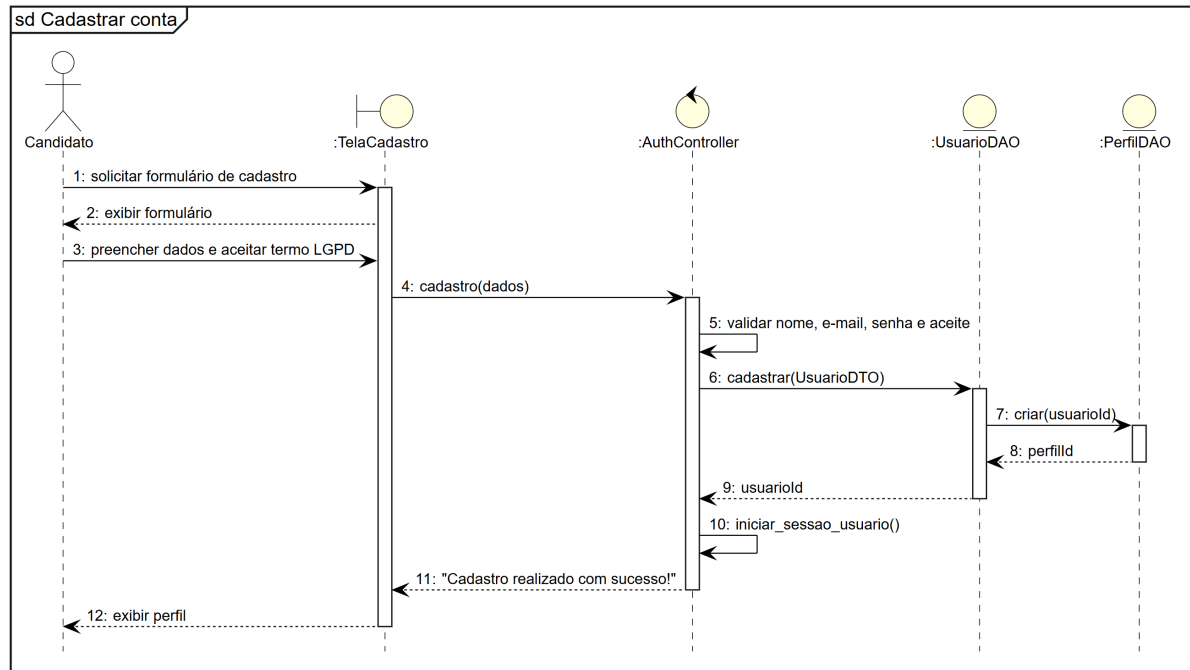


Figura 17 – Diagrama de Sequência Cadastrar Conta
Fonte: Autoria própria (2026).

8.2.3 Diagrama de Sequência Pesquisar Vagas

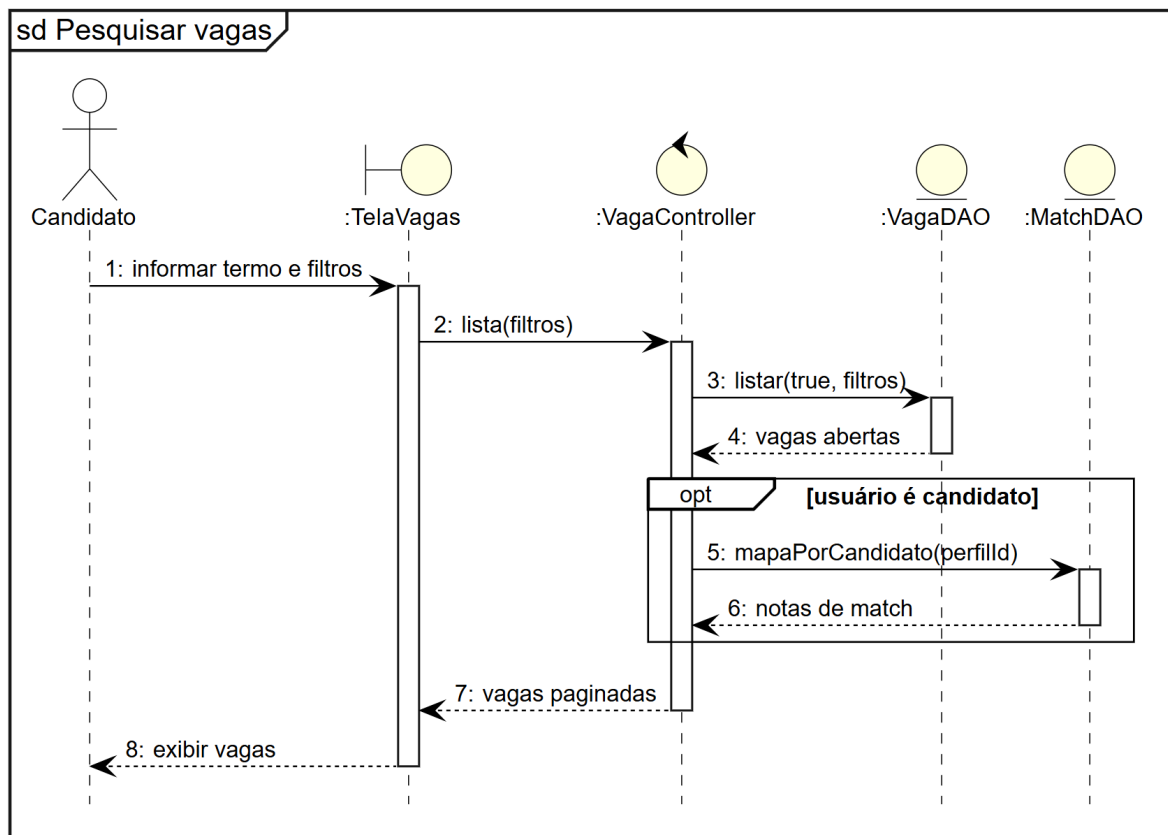


Figura 18 – Diagrama de Sequência Pesquisar Vagas
Fonte: Autoria própria (2026).

8.2.4 Diagrama de Sequência Enviar Currículo (extração e match)

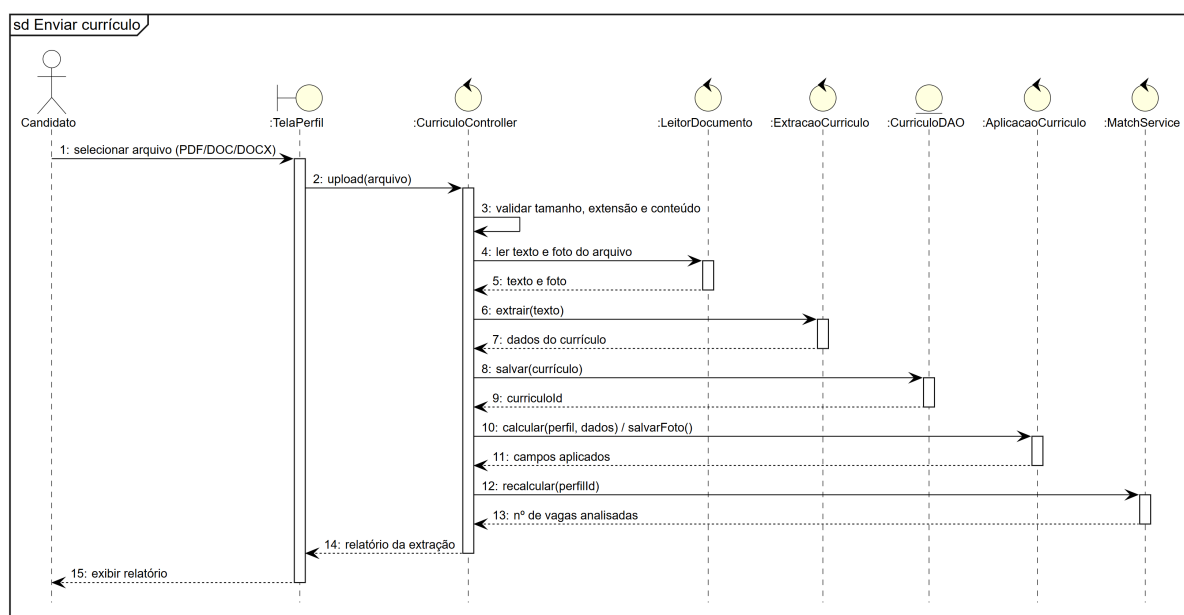


Figura 19 – Diagrama de Sequência Enviar Currículo (extração e match)
Fonte: Autoria própria (2026).

8.2.5 Diagrama de Sequência Candidatar-se à Vaga

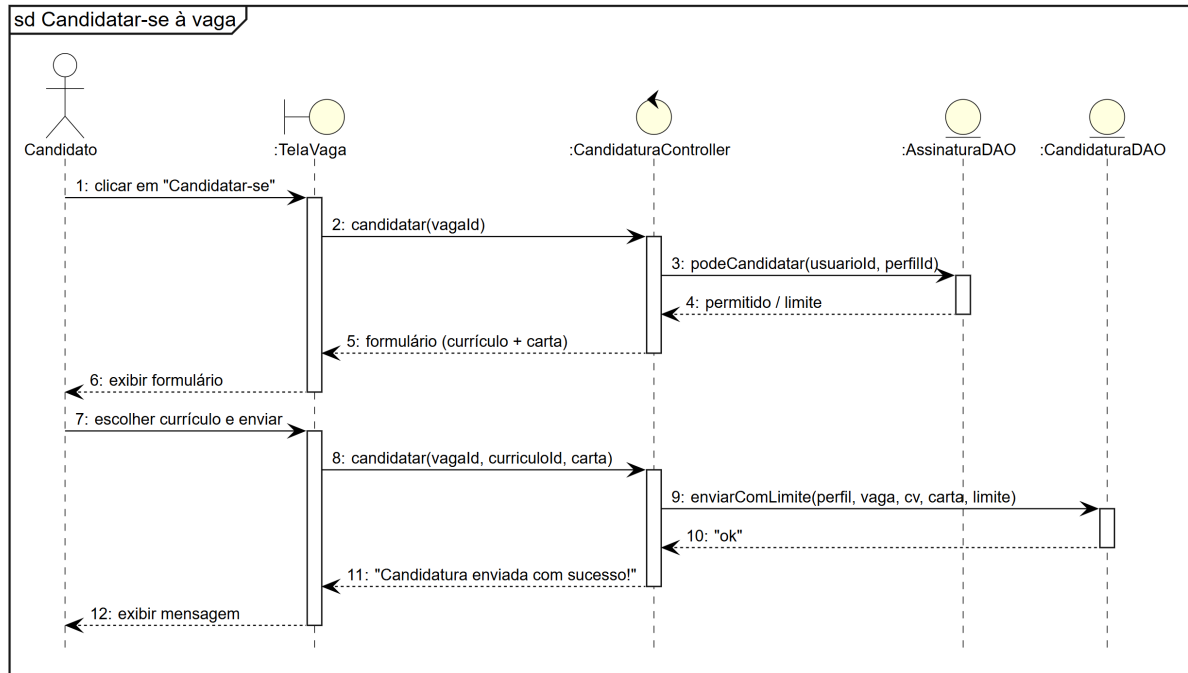


Figura 20 – Diagrama de Sequência Candidatar-se à Vaga
Fonte: Autoria própria (2026).

8.2.6 Diagrama de Sequência Cancelar Candidatura

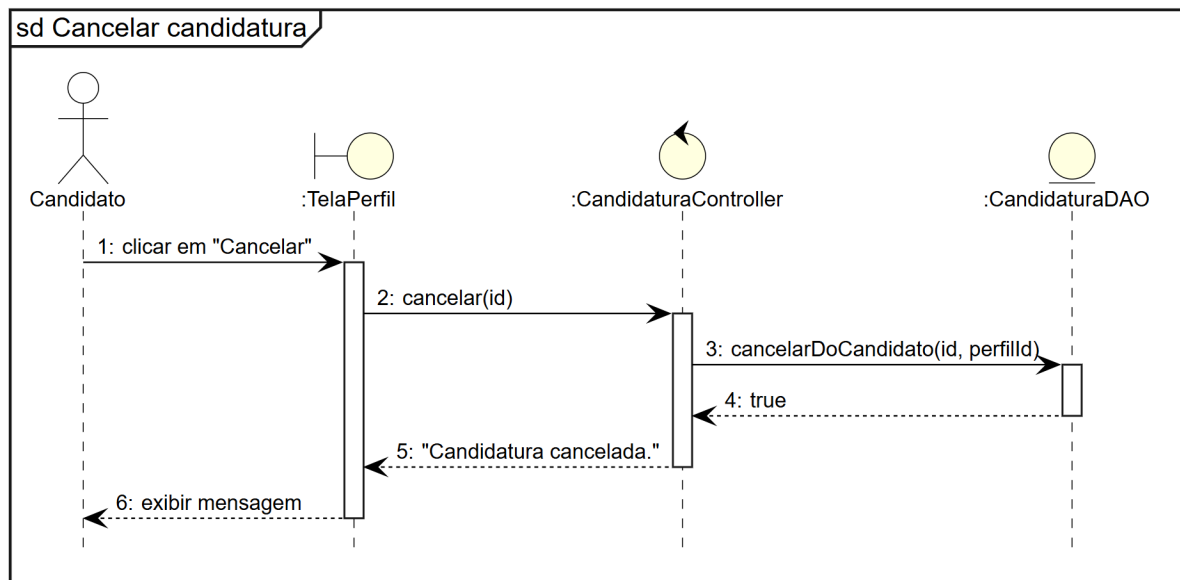


Figura 21 – Diagrama de Sequência Cancelar Candidatura
Fonte: Autoria própria (2026).

8.2.7 Diagrama de Sequência Cadastrar Vaga (leitura do cartaz no navegador e extração)

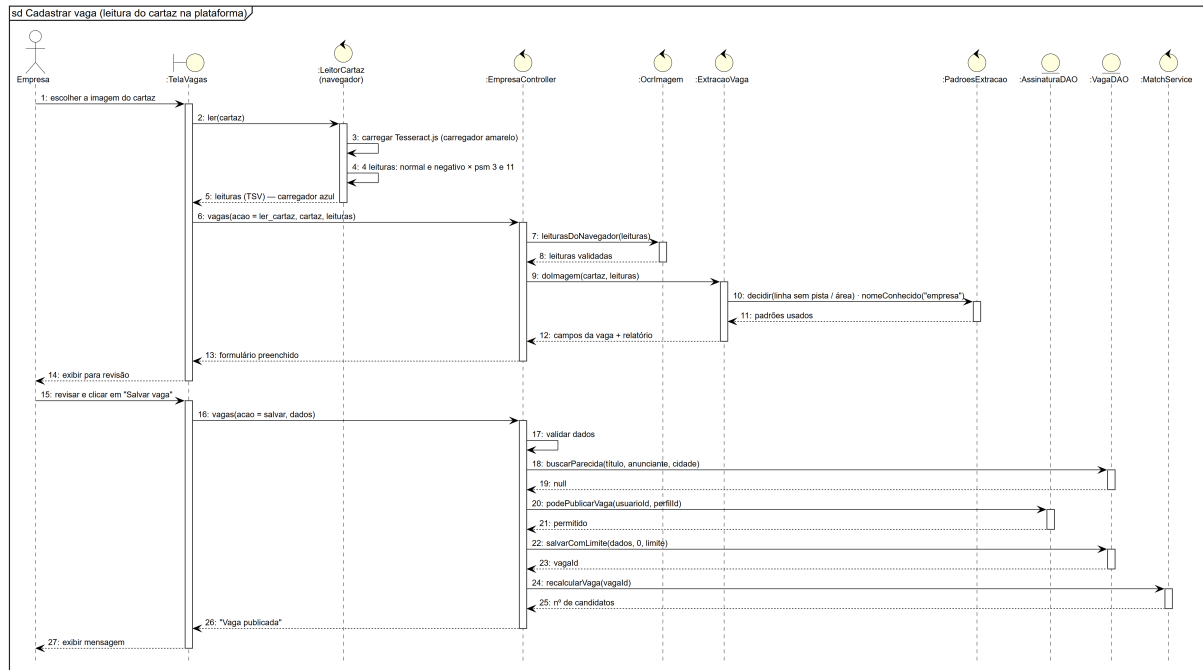


Figura 22 – Diagrama de Sequência Cadastrar Vaga (leitura do cartaz no navegador e extração)
Fonte: Autoria própria (2026).

8.2.8 Diagrama de Sequência Alterar Vaga

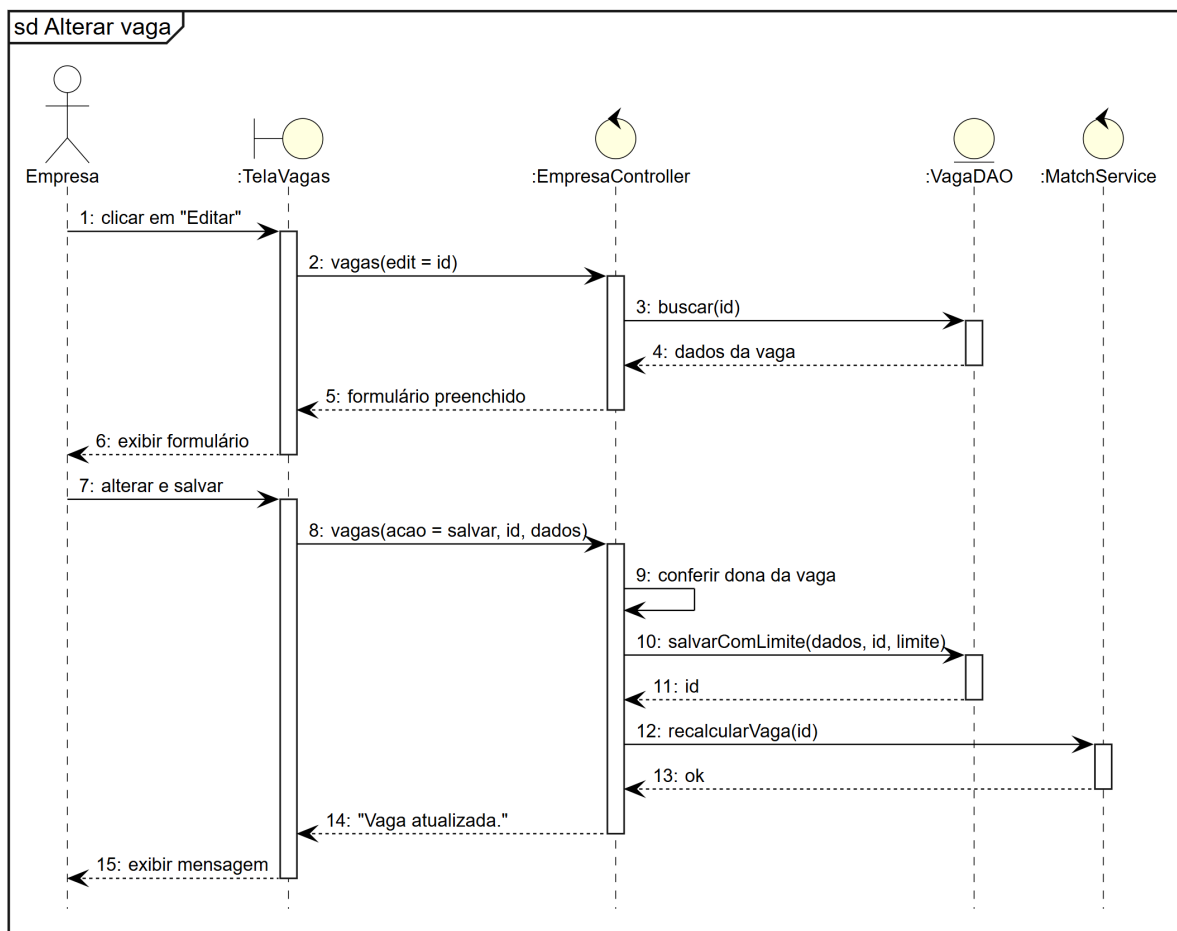


Figura 23 – Diagrama de Sequência Alterar Vaga

Fonte: Autoria própria (2026).

8.2.9 Diagrama de Sequência Excluir Vaga

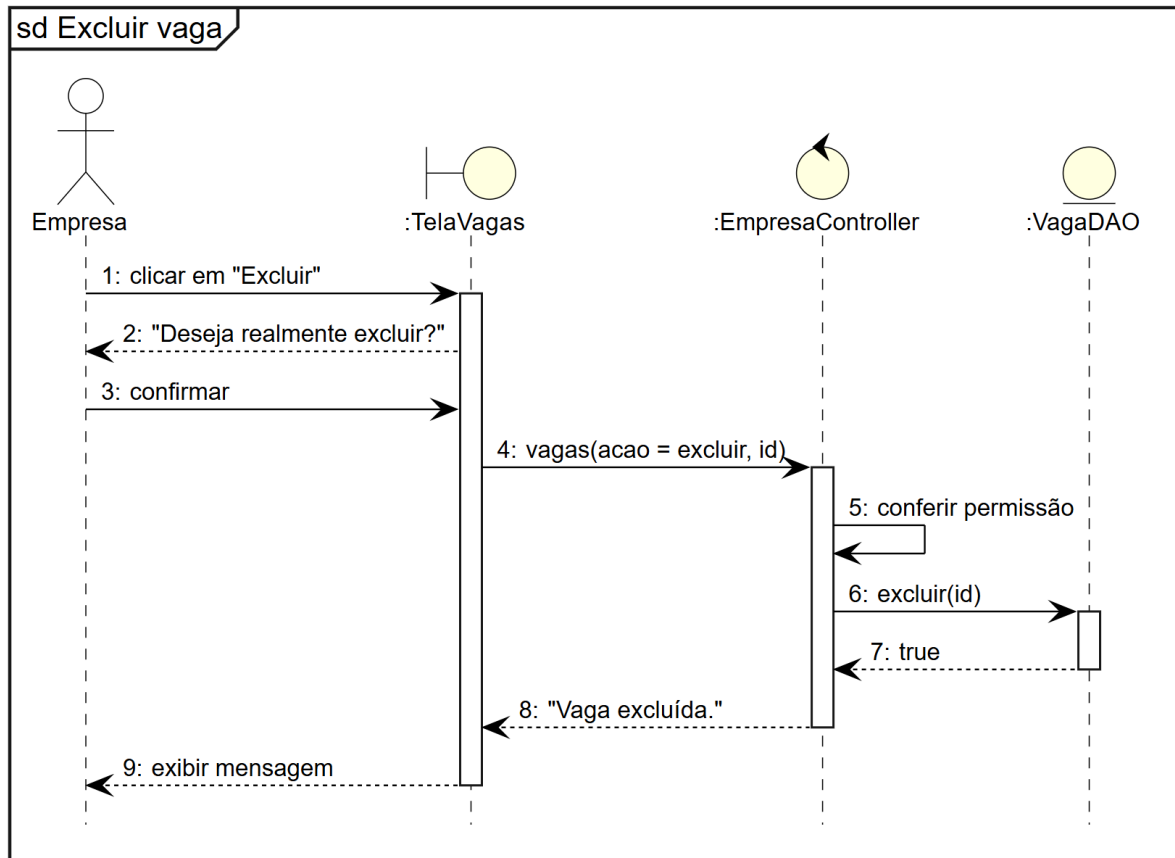


Figura 24 – Diagrama de Sequência Excluir Vaga
Fonte: Autoria própria (2026).

8.2.10 Diagrama de Sequência Atualizar Status da Candidatura

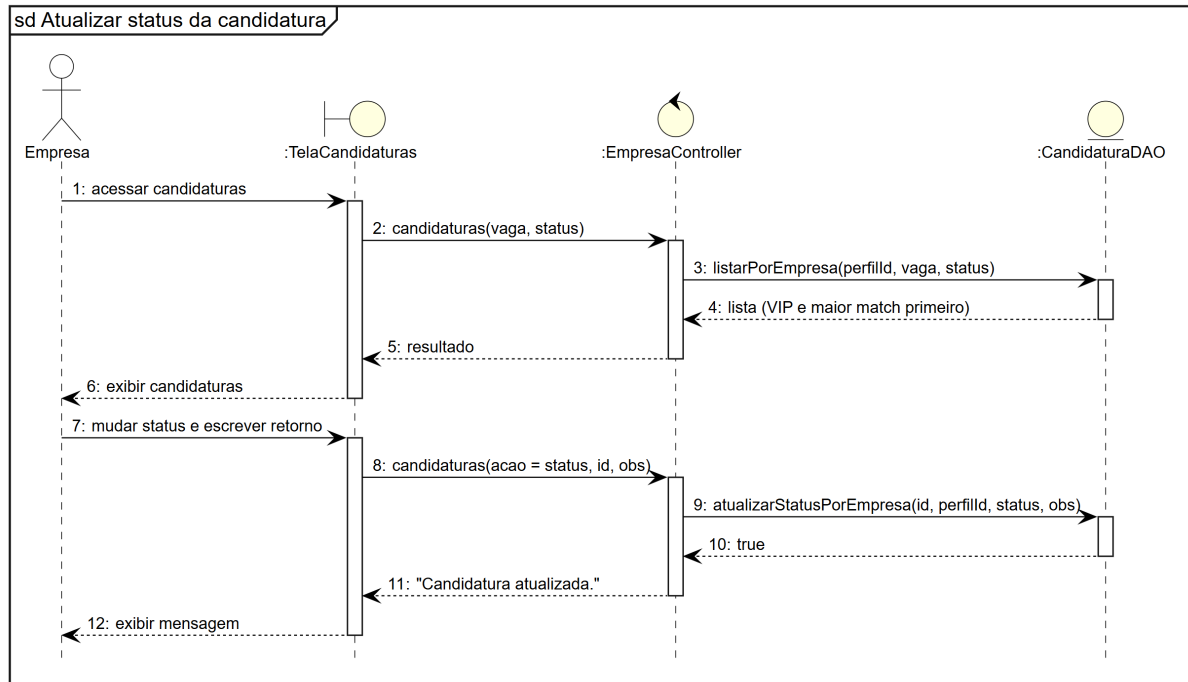


Figura 25 – Diagrama de Sequência Atualizar Status da Candidatura
Fonte: Autoria própria (2026).

8.2.11 Diagrama de Sequência Assinar Plano

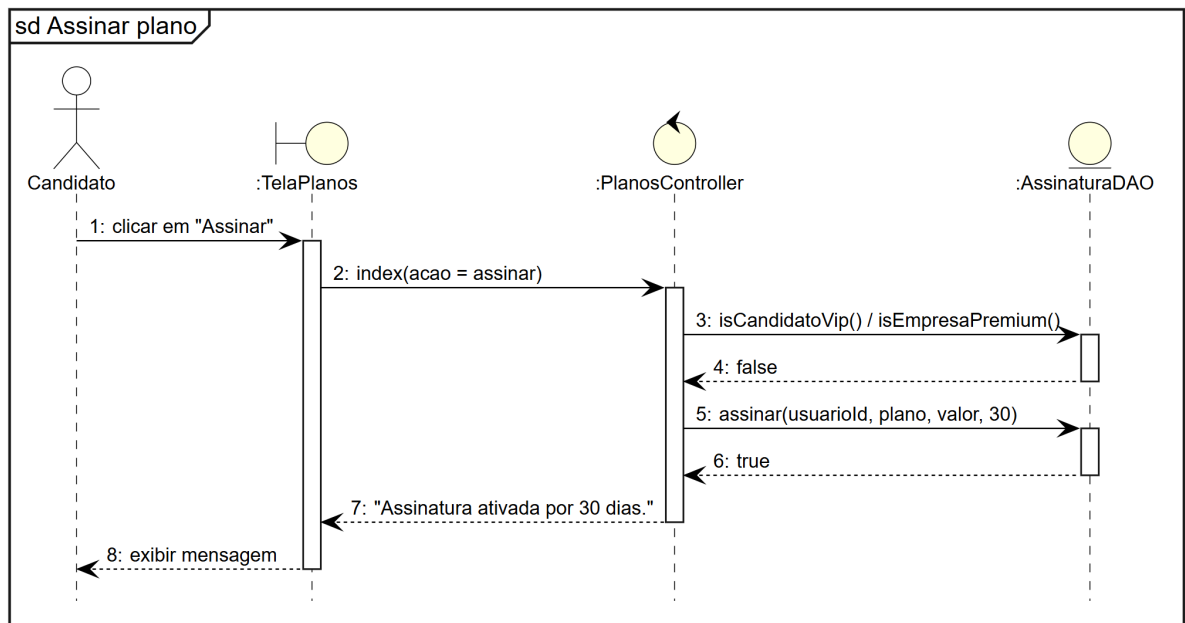


Figura 26 – Diagrama de Sequência Assinar Plano
Fonte: Autoria própria (2026).

8.2.12 Diagrama de Sequência Manter Usuário

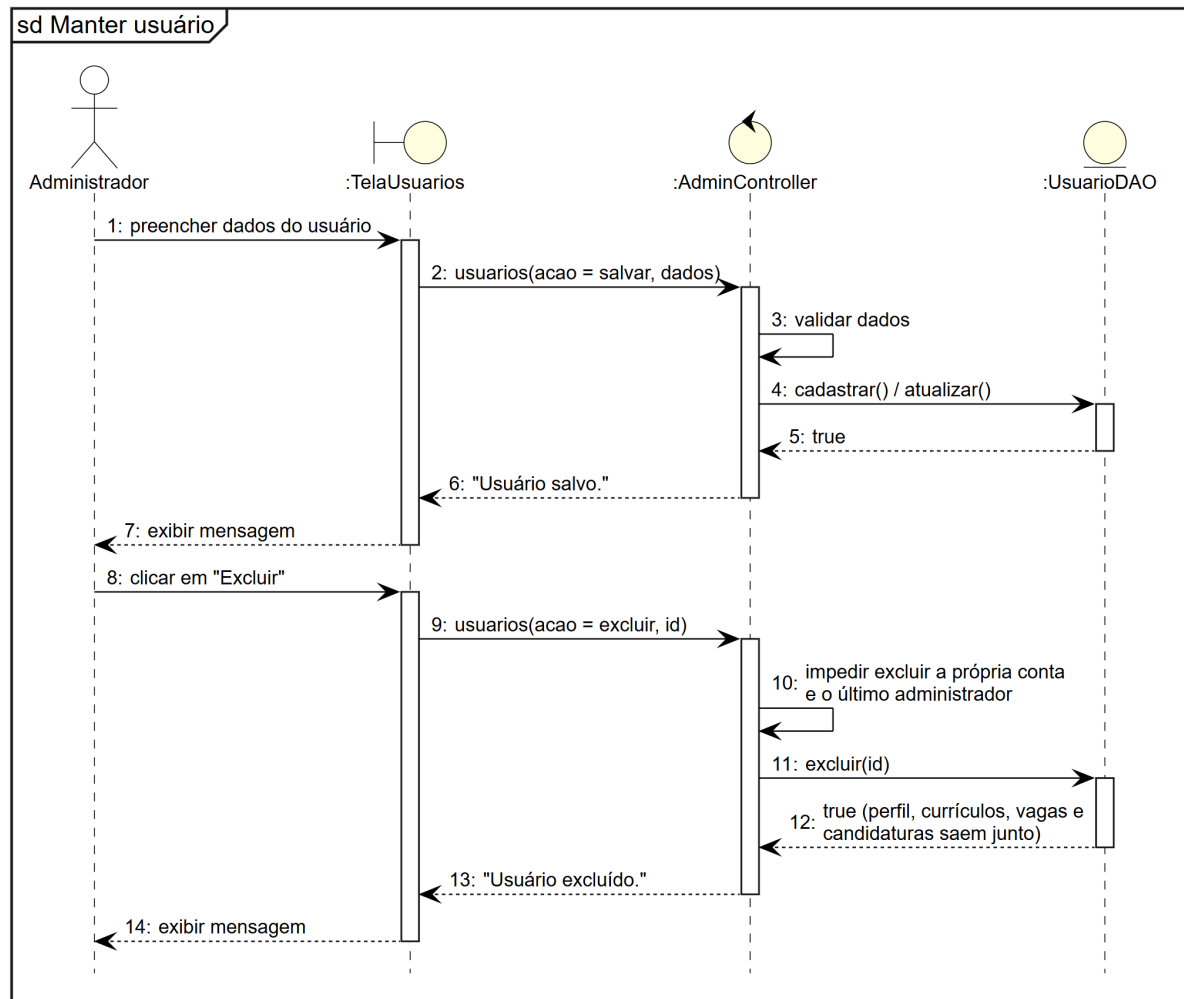


Figura 27 – Diagrama de Sequência Manter Usuário
Fonte: Autoria própria (2026).

8.2.13 Diagrama de Sequência Excluir a Própria Conta (LGPD)

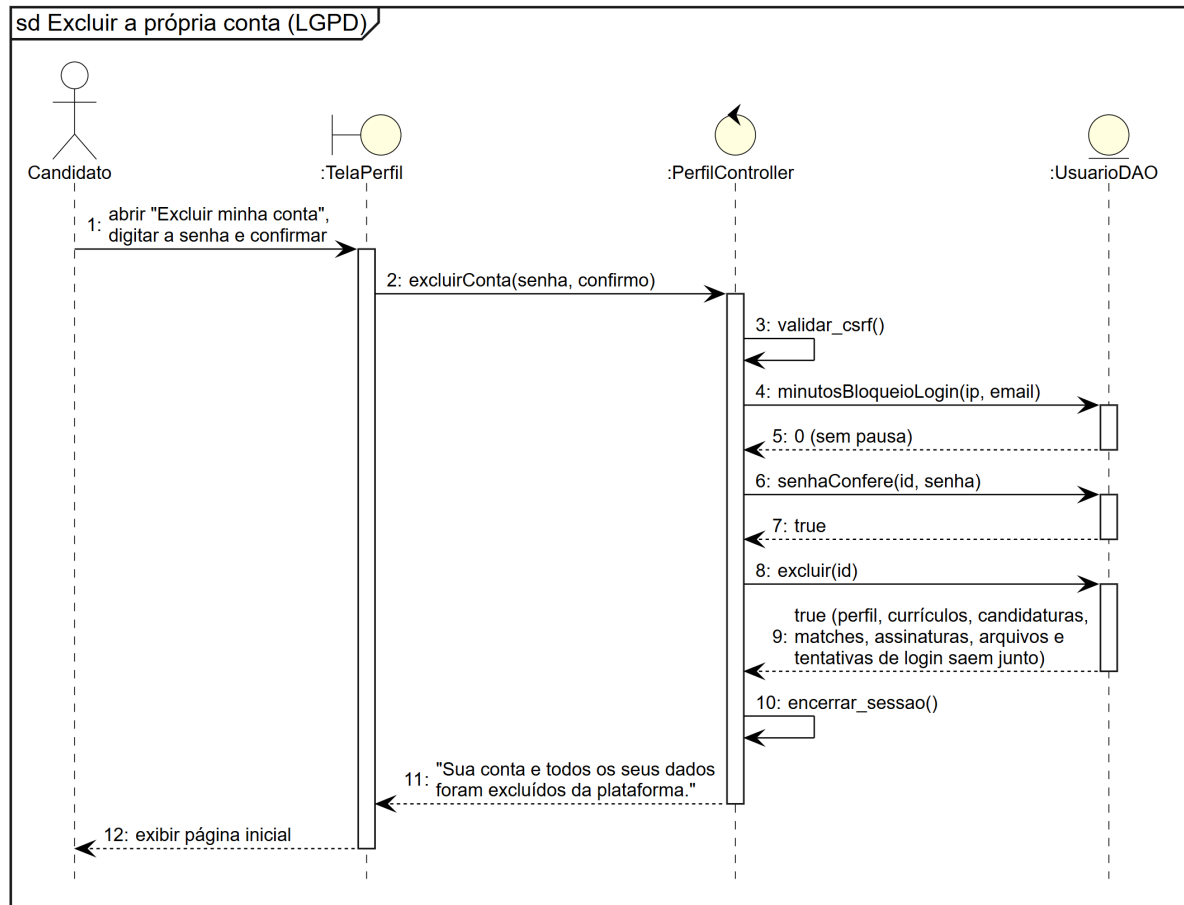


Figura 28 – Diagrama de Sequência Excluir a Própria Conta (LGPD)

Fonte: Autoria própria (2026).

8.2.14 Diagrama de Sequência Trazer os PDFs para a Biblioteca

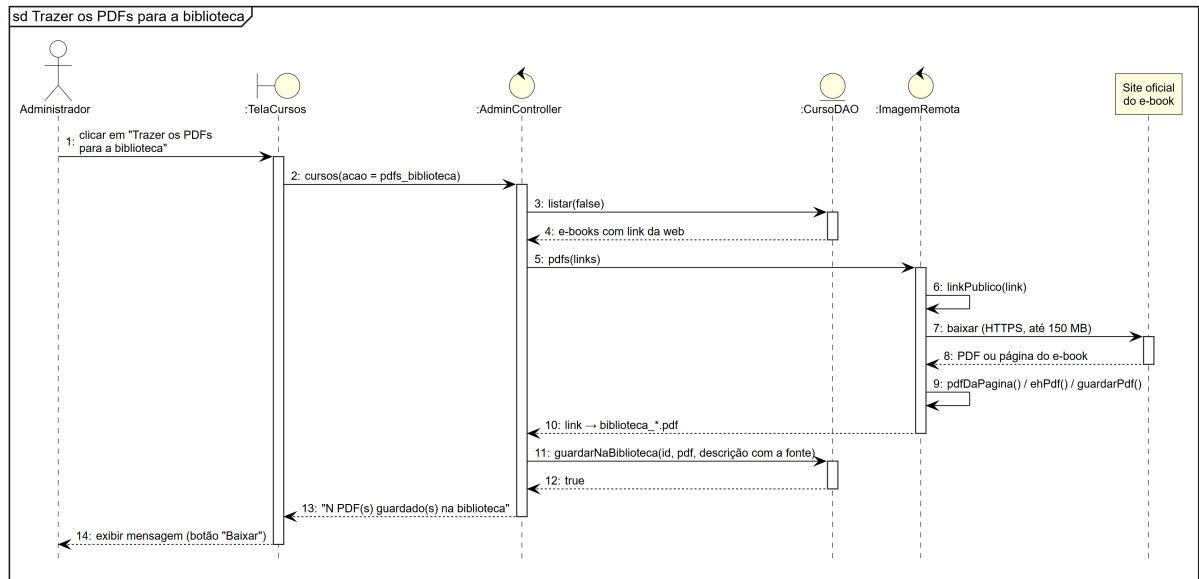


Figura 29 – Diagrama de Sequência Trazer os PDFs para a Biblioteca
Fonte: Autoria própria (2026).

8.2.15 Diagrama de Sequência Extração de Vaga com os Padrões Automáticos

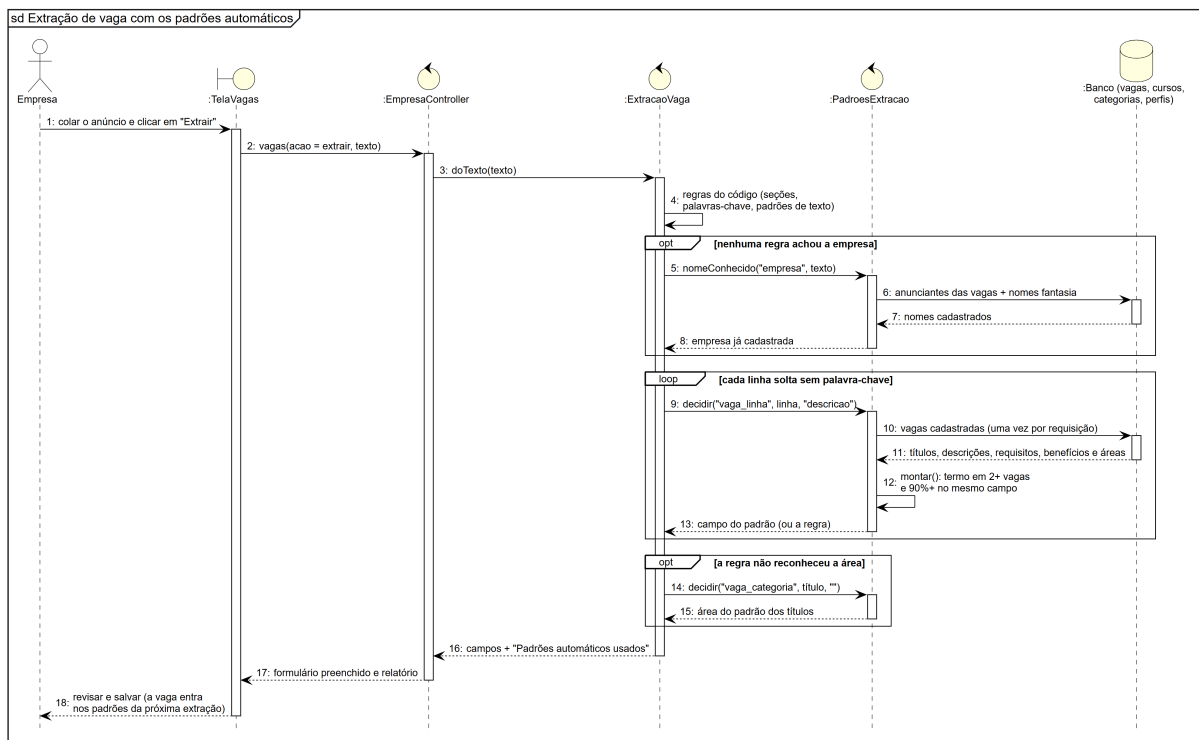


Figura 30 – Diagrama de Sequência Extração de Vaga com os Padrões Automáticos
Fonte: Autoria própria (2026).

8.3 Diagrama de Classes

O diagrama de classes de domínio mostra as classes que representam os objetos do Conecta Vagas DF, com os seus atributos e as associações entre elas, cada uma com a sua multiplicidade. Os atributos são os mesmos campos das tabelas do banco de dados, escritos no padrão de nomes das classes. No código, as operações sobre esses objetos ficam nas classes de acesso a dados (DAO) da pasta `app/Models` e nos objetos de transferência (DTO) da pasta `app/DTO`, que descrevemos método por método no Apêndice A, seção A.5. Os padrões automáticos não aparecem como classe de domínio porque não têm tabela: eles são montados na hora pelo serviço `PadroesExtracao`.

8.3.1 Diagrama de Classe Sistema Conecta Vagas DF

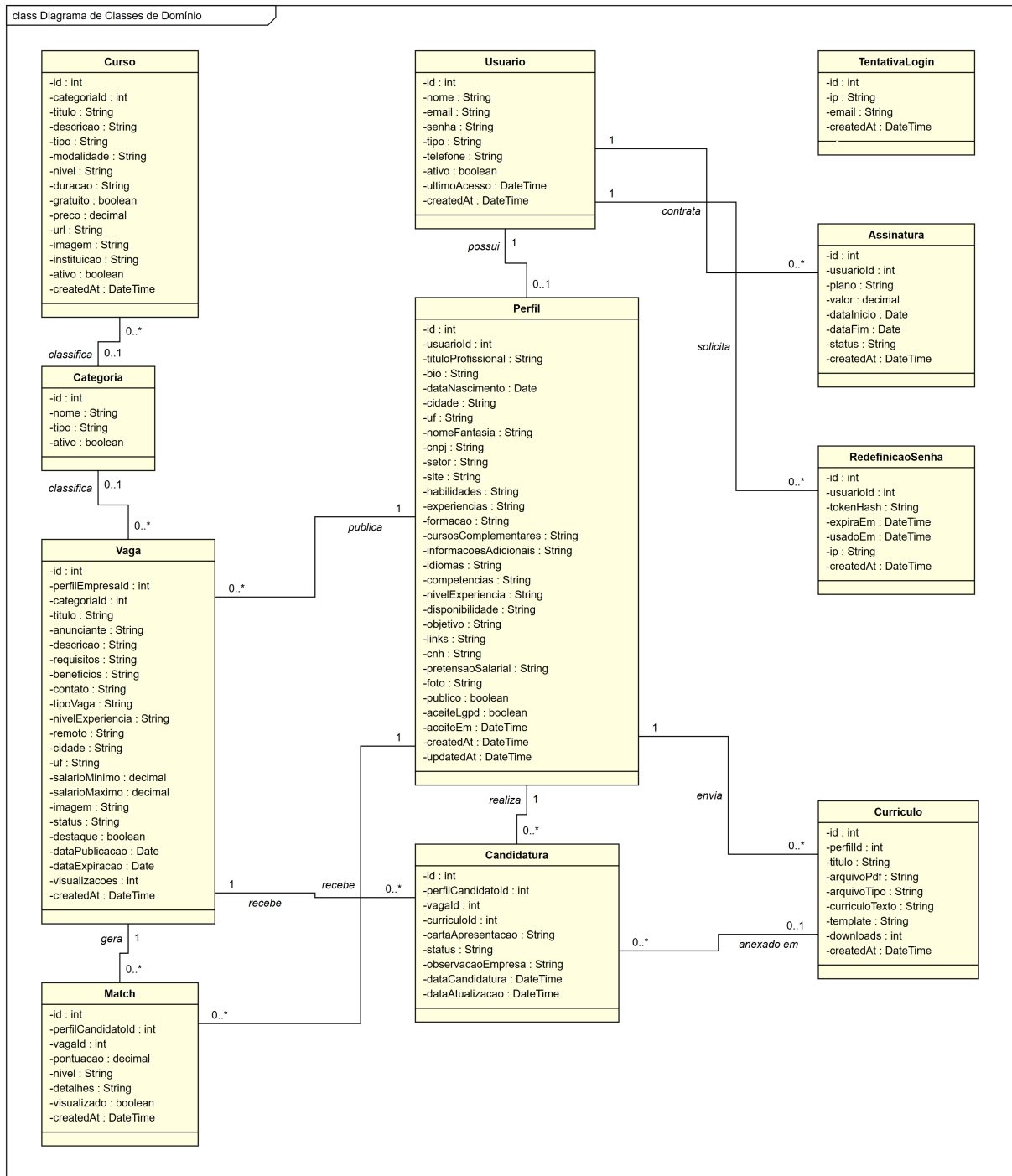


Figura 31 – Diagrama de Classes de Domínio do Sistema Conecta Vagas DF
Fonte: Autoria própria (2026).

8.4 Modelo Conceitual do Banco de Dados

Fizemos o modelo de dados em duas etapas. O modelo conceitual (Figura 32) mostra os dados do sistema sem depender do SGBD, na notação de Chen adaptada por Heuser (2009): entidades em retângulos, relacionamentos em losangos, atributos em círculos (o identificador

fica preenchido) e as cardinalidades mínima e máxima. PERFIL é especializado de forma exclusiva e total em CANDIDATO e EMPRESA, CANDIDATURA é uma entidade associativa entre CANDIDATO e VAGA, e COMBINA guarda a nota do match entre os dois.

Os padrões automáticos não são uma entidade, porque são montados na hora a partir de VAGA, CURSO, CATEGORIA e PERFIL, como indica a nota no diagrama. TENTATIVA DE LOGIN não se relaciona com as outras entidades: ela é o controle contra tentativas repetidas de senha por e-mail e IP.

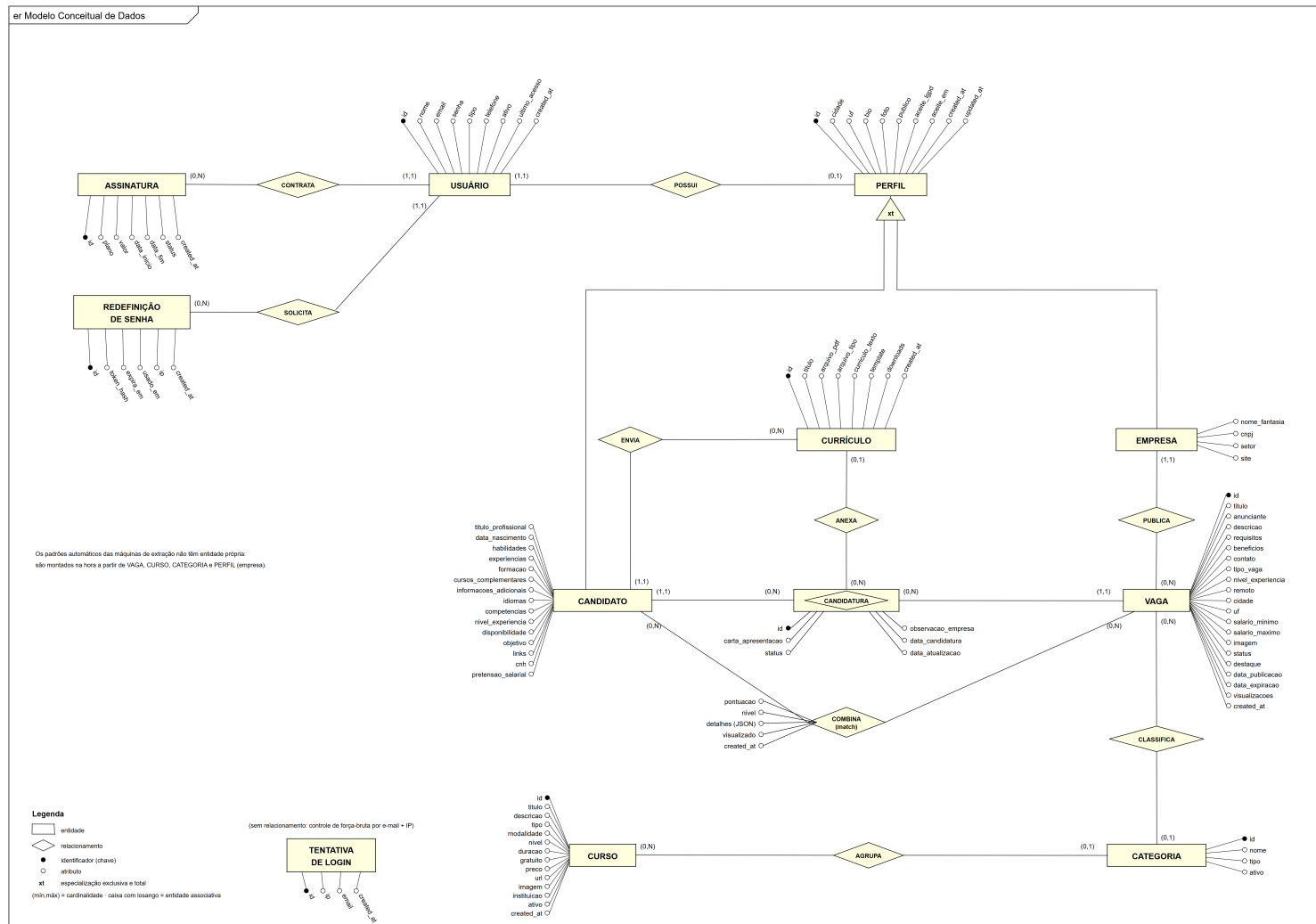


Figura 32 – Modelo Conceitual do Banco de Dados
Fonte: Autoria própria (2026).

8.5 Modelo Lógico do Banco de Dados (Diagrama Entidade-Relacionamento)

O modelo lógico (Figura 33) transforma o modelo conceitual nas 11 tabelas do banco tcc_final, no MySQL/MariaDB. Nós o geramos a partir do arquivo database/schema.sql, para mostrar exatamente as colunas e os tipos do banco (PK é a chave primária, FK a chave estrangeira, NN indica campo obrigatório e UQ campo único). A especialização de PERFIL virou uma tabela só (perfis), diferenciada pelo tipo do usuário; o relacionamento COMBINA virou a tabela matches, e a entidade associativa CANDIDATURA virou a tabela candidaturas, as duas com chave única (perfil_candidato_id, vaga_id).

As chaves estrangeiras que ligam os dados ao usuário usam ON DELETE CASCADE, então quem exclui a própria conta leva junto todos os seus dados pessoais, como pede a LGPD. As que ligam vagas e cursos a uma categoria usam ON DELETE SET NULL, para que a exclusão de uma categoria não apague as vagas e os cursos.

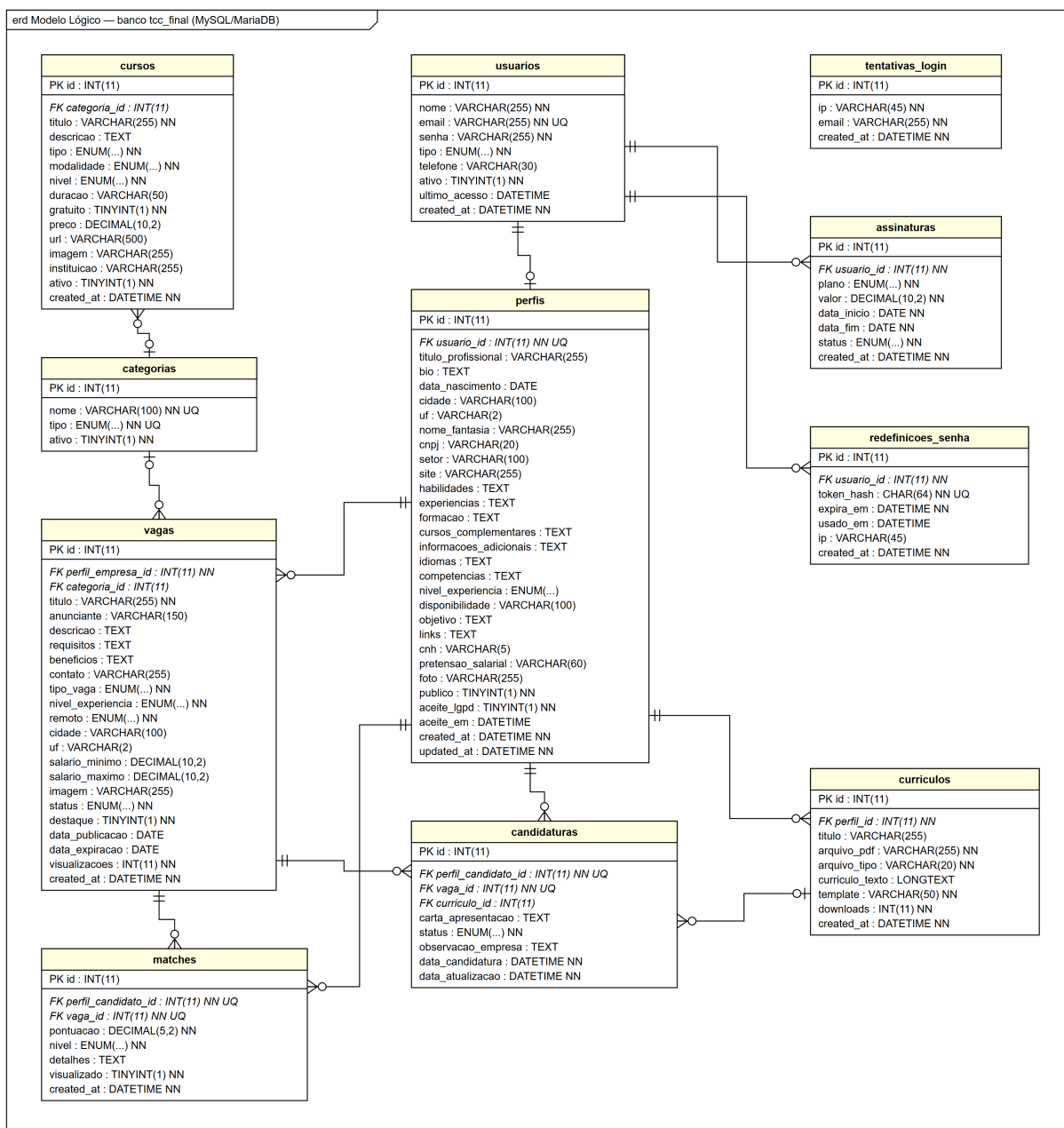


Figura 33 – Modelo Lógico do Banco de Dados tcc_final (Diagrama Entidade-Relacionamento)

Fonte: Autoria própria (2026).

8.6 Telas

A seguir estão as principais telas do sistema em funcionamento, capturadas da versão final no ambiente XAMPP com os dados de demonstração e as contas de teste.

8.6.1 Tela Inicial Sistema Conecta Vagas DF

A página inicial abre com fotos de Brasília, a busca de vagas e a vitrine das vagas abertas, seguidas dos cursos, dos planos e do convite discreto de doação por Pix.



Figura 34 – Tela Inicial
Fonte: Autoria própria (2026).

8.6.2 Tela de Login Sistema Conecta Vagas DF

Na tela de login o usuário informa e-mail e senha. O botão do olho, ao lado da senha, mostra o que foi digitado, e o login aceita a senha com o Caps Lock ligado ou com espaços sobrando.

Entrar na plataforma

Use seu e-mail e senha cadastrados.

Contas de teste (ambiente local):
 Admin: admin@conectavagas.com / Admin@123
 Empresa: empresa@conectavagas.com / Empresa@123
 Candidato: candidato@conectavagas.com / Candidato@123

E-mail

Senha

Entrar Criar conta

[Esqueci minha senha](#)

Conecta Vagas DF
Conectando talentos às melhores oportunidades de emprego e capacitação profissional no Distrito Federal.

Navegação
 Início
 Vagas de emprego
 Cursos gratuitos

Sua conta
 Cadastro
 Login
 Esqueci minha senha

Links úteis
 Fundação Bradesco
 SEBRAE
 Escola Virtual.Gov

Contato
 suporte@conectavagas.df
 Brasília - DF
 Termo de privacidade (LGPD)

Figura 35 – Tela de Login
 Fonte: Autoria própria (2026).

8.6.3 Tela de Cadastro de Conta Sistema Conecta Vagas DF

O cadastro pede nome, e-mail, telefone, senha e o tipo de conta, e só é concluído com o aceite do termo de autorização e privacidade.

Criar sua conta

O cadastro cria a conta e já abre o perfil para você completar as informações.

Nome completo / responsável

E-mail

Telefone

Senha (mínimo 6 caracteres)

Tipo de conta
Candidato (quero uma vaga)

☐ Li e aceito o termo de autorização e privacidade.

Cadastrar e montar perfil

Conecta Vagas DF
Conectando talentos às melhores oportunidades de emprego e capacitação profissional no Distrito Federal.

Navegação
 Início
 Vagas de emprego
 Cursos gratuitos

Sua conta
 Cadastro
 Login
 Esqueci minha senha

Links úteis
 Fundação Bradesco
 SEBRAE
 Escola Virtual.Gov

Contato
 suporte@conectavagas.df
 Brasília - DF
 Termo de privacidade (LGPD)

Figura 36 – Tela de Cadastro de Conta
 Fonte: Autoria própria (2026).

8.6.4 Tela de Consulta de Vagas Sistema Conecta Vagas DF

A consulta de vagas tem busca por palavra-chave e cidade, filtros por contratação, nível e modelo de trabalho e atalhos por área. Para o candidato logado, cada vaga mostra a nota de match.

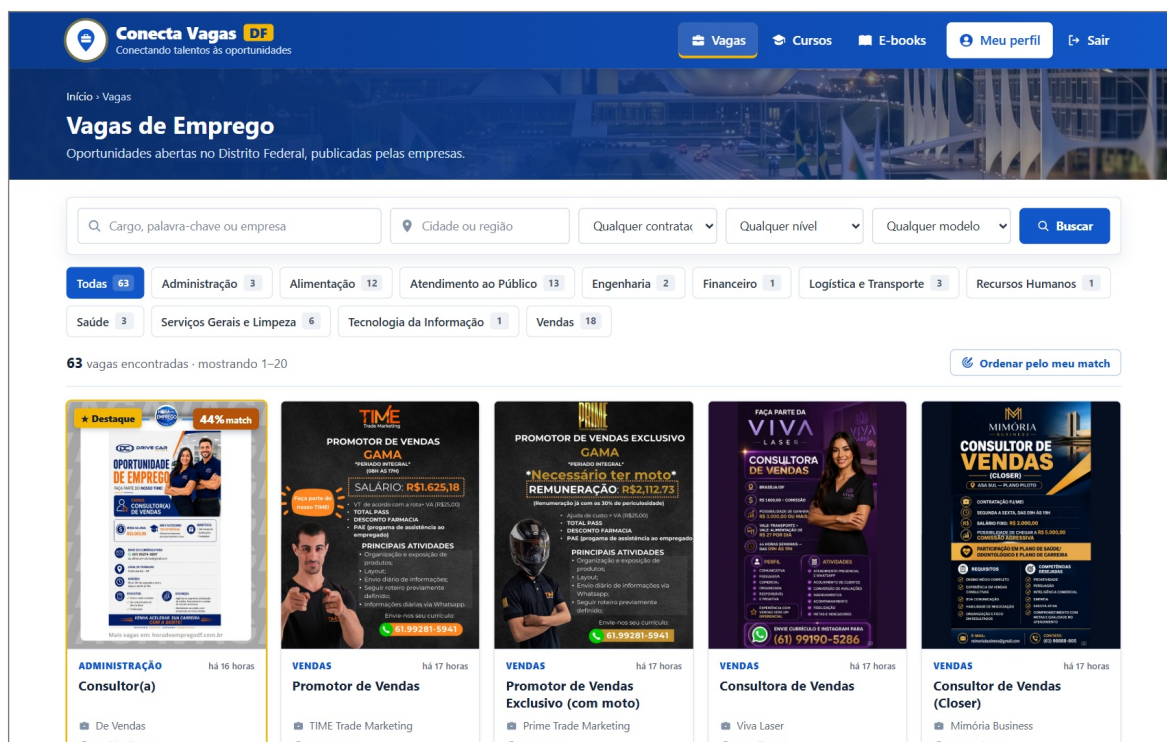


Figura 37 – Tela de Consulta de Vagas
Fonte: Autoria própria (2026).

8.6.5 Tela de Detalhe da Vaga com o Match Sistema Conecta Vagas DF

O detalhe da vaga mostra o cartaz, os dados da vaga, o contato do anúncio e, para o candidato logado, o quadro "Seu match", com os pontos de competências, cargo, localização e nível.

Conecta Vagas DF
Conectando talentos às oportunidades

Vagas Cursos E-books Meu perfil Sair

Início > Vagas > Administração > Consultor(a)

ADMINISTRAÇÃO há 16 horas

Consultor(a)

De Vendas

Ceilândia/DF Publicada em 01/10/2026 às 01h29 2 visualizações

SALÁRIO
R\$ 5.000,00

CLT Júnior Presencial **★ Destaque**

CONTATO DO ANÚNCIO

WhatsApp (61) 99274-8897

COMPARTILHE ESTA VAGA

WhatsApp Copiar link Enviar

Seu match 44%

Competências 20 / 50

Cargo diferente do seu histórico 0 / 20

Mesmo estado (DF) 9 / 15

Nível exatamente o pedido 15 / 15

Descrição da vaga

De Vendas está com inscrições abertas para a vaga de Consultor(a) em Ceilândia/DF. A contratação é pelo regime CLT, voltada a profissionais de nível júnior, com trabalho presencial. A remuneração informada é de R\$ 5.000,00.

De Vendas contrata Consultor(a) em Ceilândia.

DRIVE CAR
MOTORS
para desenvolvimento
na área.

SALÁRIO
R\$ 5.000,00

Empresa De Vendas

Publicado por Empresa Teste

Local Ceilândia/DF

Contratação CLT

Nível Júnior

Modelo Presencial

Candidatar-se

Cursos para aumentar seu

Figura 38 – Tela de Detalhe da Vaga com o Match

Fonte: Autoria própria (2026).

8.6.6 Tela do Perfil do Candidato e Exclusão da Conta Sistema Conecta Vagas DF

Em Meu perfil o candidato mantém os seus dados. Ao lado fica o quadro "Seus dados (LGPD)", com a opção "Excluir minha conta", que pede a senha e a confirmação.

Conecta Vagas DF
Conectando talentos às oportunidades

Vagas Cursos E-books Meu perfil Sair

Cadastro do perfil Campos com * são obrigatórios para liberar o Portfólio

Nome Candidato Teste

E-mail candidato@conectavagas.com

Título profissional * Candidato em busca de oportunidades

Nível de experiência Júnior

Telefone * (61) 97777-7777

Data de nascimento dd/mm/aaaa

Cidade / região * Brasília

UF DF

Resumo profissional *

Perfil de candidato para testes.

Objetivo profissional *

Buscar oportunidade profissional no DF.

Experiências — uma por bloco: Empresa, cargo e período (ex.: 2021 – 2023 ou 2025 – Atual)

Seus dados (LGPD)

Você pode apagar a sua conta quando quiser. Saem juntos o perfil, os currículos e arquivos, a foto, as candidaturas e o match. Não dá para desfazer.

Excluir minha conta

Digite sua senha para confirmar

☐ Entendo que a minha conta e os meus dados serão apagados para sempre.

Excluir minha conta

Figura 39 – Tela do Perfil do Candidato e Exclusão da Conta

Fonte: Autoria própria (2026).

8.6.7 Tela de E-books Sistema Conecta Vagas DF

A página de e-books lista os conteúdos gratuitos por área, com busca e filtros, cada um com a sua capa.

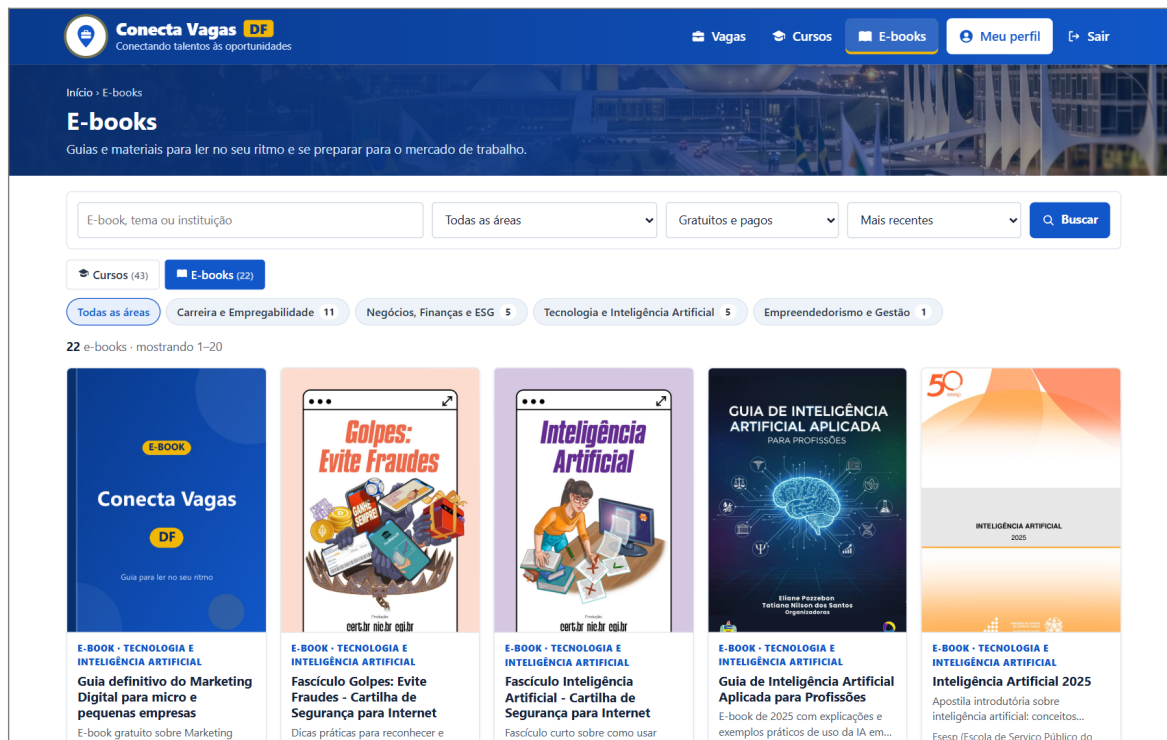


Figura 40 – Tela de E-books
Fonte: Autoria própria (2026).

8.6.8 Tela da Ficha do E-book com o Botão Baixar Sistema Conecta Vagas DF

A ficha do e-book guardado na biblioteca mostra o botão "Baixar PDF" e o crédito da fonte original. Os conteúdos que ficam em outro site mostram o botão "Acessar".



Figura 41 – Tela da Ficha do E-book com o Botão Baixar
Fonte: Autoria própria (2026).

8.6.9 Tela de Planos e Assinaturas Sistema Conecta Vagas DF

A página de planos mostra o plano gratuito e o plano pago de cada tipo de conta, com o preço, a duração e os benefícios.

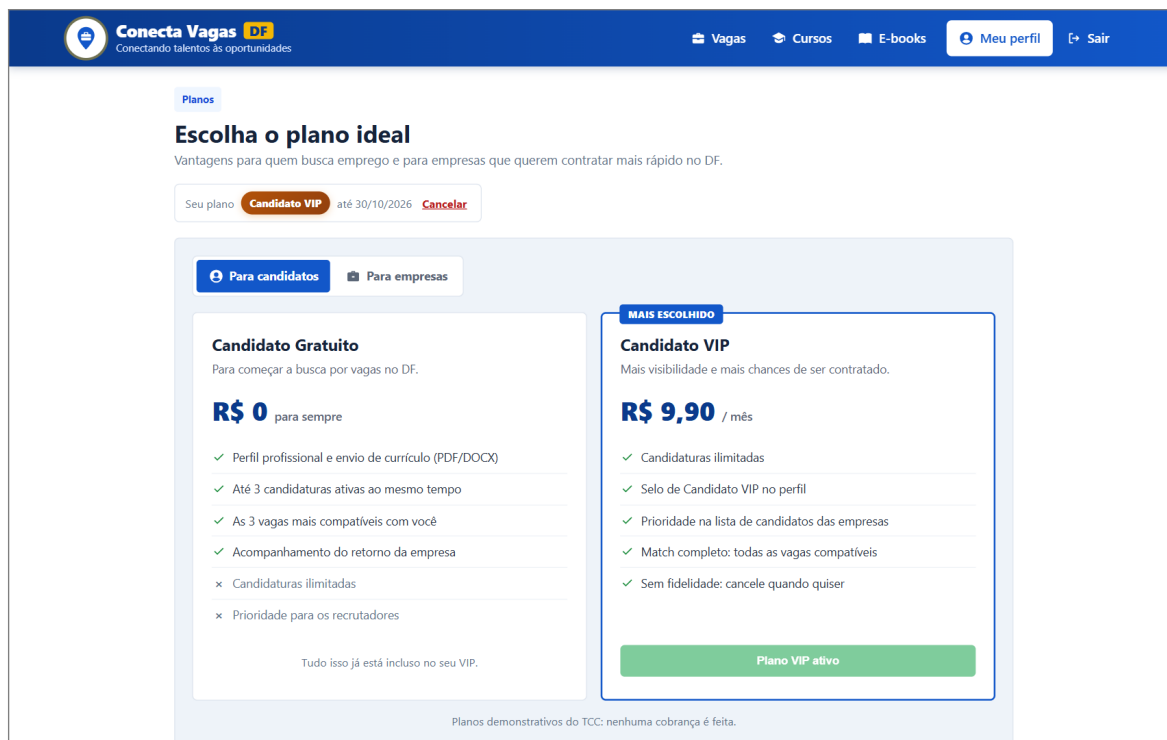


Figura 42 – Tela de Planos e Assinaturas
Fonte: Autoria própria (2026).

8.6.10 Tela de Cadastro de Vaga pela Empresa Sistema Conecta Vagas DF

Na aba Vagas a empresa envia o cartaz da vaga ou cola o texto do anúncio. O leitor de cartaz roda no navegador e o carregador fica azul quando ele está pronto.

Figura 43 – Tela de Cadastro de Vaga pela Empresa
Fonte: Autoria própria (2026).

8.6.11 Tela do Relatório da Extração da Vaga Sistema Conecta Vagas DF

Depois da leitura, o relatório mostra campo por campo o que foi lido do anúncio, o que recebeu um valor padrão e o que não foi encontrado. Nada é salvo antes da revisão.

Conecta Vagas DF
Conectando talentos às oportunidades

Vagas Cursos E-books Painel Sair

pause, edite ou exclua pela lista. Plano Básico: até 2 vagas ativas Fazer upgrade

▶ Máquina de extração: envie o cartaz da vaga (imagem) ou cole o anúncio — o formulário é preenchido

Pronto: anúncio lido — confira os campos e salve a vaga.

Relatório da extração da vaga 10 lidos do anúncio 2 valor padrão 2 faltando

Lido do texto do anúncio - nada foi salvo: revise o formulário abaixo e clique em "Salvar vaga".

▼ Ver campo por campo

Cargo (título)	Atendente de Loja	Lido do anúncio
Empresa anunciante	—	Não encontrado
Salário	R\$ 1.800,00	Lido do anúncio
Local	Taguatinga/DF	Lido do anúncio
Contratação	CLT	Valor padrão — confira
Nível	Júnior	Valor padrão — confira
Modelo de trabalho	Presencial	Lido do anúncio
Área (categoria)	Atendimento ao Público	Lido do anúncio
Descrição / atividades	Vaga de Atendente de Loja em Taguatinga. (+2 linhas)	Lido do anúncio
Requisitos	Ensino médio completo	Lido do anúncio
Benefícios	Salário: R\$ 1.800,00 (+1 linha)	Lido do anúncio
Contato do anúncio	Telefone (61) 99999-0000	Lido do anúncio
Quantidade de vagas	—	Não encontrado

Figura 44 – Tela do Relatório da Extração da Vaga
Fonte: Autoria própria (2026).

8.6.12 Tela dos Padrões Automáticos Usados Sistema Conecta Vagas DF

No fim do relatório aparece o quadro "Padrões automáticos usados". No exemplo, a linha "Café da manhã no local" foi para Benefícios pelo padrão "café manhã", tirado das vagas já cadastradas.

Conecta Vagas DF
Conectando talentos às oportunidades

Vagas Cursos E-books Painel Sair

Modelo de trabalho	Presencial	Lido do anúncio
Área (categoria)	Atendimento ao Público	Lido do anúncio
Descrição / atividades	Vaga de Atendente de Loja em Taguatinga. (+2 linhas)	Lido do anúncio
Requisitos	Ensino médio completo	Lido do anúncio
Benefícios	Salário: R\$ 1.800,00 (+1 linha)	Lido do anúncio
Contato do anúncio	Telefone (61) 99999-0000	Lido do anúncio
Quantidade de vagas	—	Não encontrado
Competências (match)	Atendimento ao cliente, Organização	Lido do anúncio

Padrões automáticos usados

LINHA "Café da manhã no local" → **Benefícios** (padrão: café manhã)

Tirados das vagas já cadastradas: valem onde a regra não tem pista e se atualizam a cada vaga salva.

Cadastrar vaga

Título: Atendente de Loja

Empresa anunciante (se a vaga for de outra empresa):

Contato do anúncio (WhatsApp, telefone ou e-mail):

Figura 45 – Tela dos Padrões Automáticos Usados
Fonte: Autoria própria (2026).

8.6.13 Tela da Prévia das Fichas com a Capa Tirada do PDF Sistema Conecta Vagas DF

Ao colar várias fichas de e-books, o administrador vê uma prévia antes de cadastrar. As capas da prévia foram tiradas da primeira página do PDF de cada e-book, e a situação avisa quando o conteúdo já está cadastrado.

Conecta Vagas DF
Conectando talentos às oportunidades

Vagas Cursos E-books Painel Sair

Colar aqui a ficha (uma ou várias, separadas por ---) ou o texto de divulgação:

Título:
Tipo:
Instituição:
Modalidade:

Extrair 1 ficha ou texto → preenche o formulário abaixo - várias fichas → prévia para cadastrar de uma vez

IMAGEM	TÍTULO	TIPO	INSTITUIÇÃO	MODALIDADE	ÁREA	LINK	
☐		Guia do Primeiro Emprego	E-book	Instituto Recicar	EAD	Carreira e Empregabilidade	recicar.org.br PDF → bibliote
☒		Dicas para o LinkedIn	E-book	Universidade de Caxias do Sul	EAD	Carreira e Empregabilidade	www.uces.br PDF → bibliote
☐		Passo a passo da Carteira de Trabalho Digital	E-book	Ministério do Trabalho e Emprego	EAD	Carreira e Empregabilidade	www.gov.br PDF → bibliote

☒ Guardar na nossa biblioteca o PDF dos e-books (o botão vira "Baixar")

Cadastrar marcados **Descartar prévia**

Novo curso

Título:

Instituição:

Categoria: Sem categoria

Formato: Curso

Modalidade: Nivel

Figura 46 – Tela da Prévia das Fichas com a Capa Tirada do PDF
Fonte: Autoria própria (2026).

8.6.14 Tela do Painel do Administrador Sistema Conecta Vagas DF

A visão geral do painel do administrador mostra os números do sistema e os gráficos de candidaturas, funil de seleção, vagas por área e vagas por região. As abas levam aos cadastros de usuários, categorias, cursos e e-books, assinaturas, vagas, candidaturas e banco de talentos.

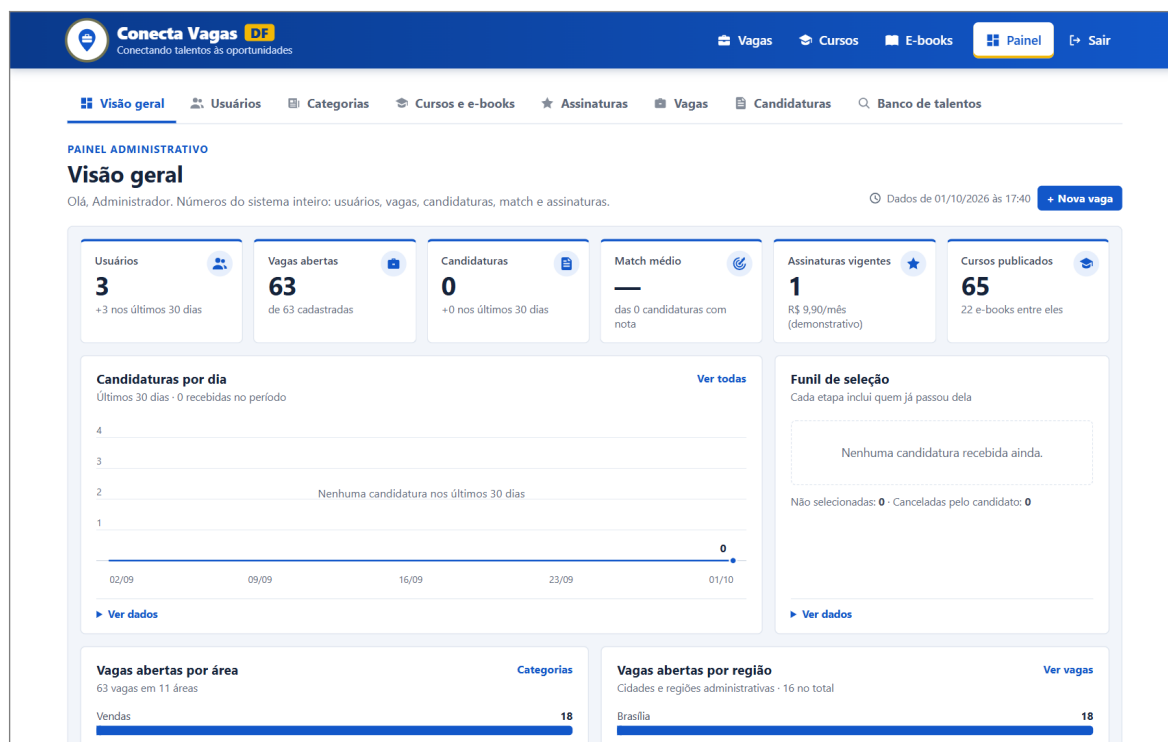


Figura 47 – Tela do Painel do Administrador
Fonte: Autoria própria (2026).

8.6.15 Tela de Manutenção de Usuários Sistema Conecta Vagas DF

Na aba Usuários o administrador cadastra, edita, bloqueia e exclui contas. O sistema não deixa excluir nem rebaixar a própria conta e mantém pelo menos um administrador ativo.

Figura 48 – Tela de Manutenção de Usuários

Fonte: Autoria própria (2026).

8.7 Dicionário de Dados

O dicionário de dados descreve cada atributo das 11 tabelas do banco `tcc_final` (MySQL/MariaDB, InnoDB, utf8mb4). Todas são criadas pelo arquivo `database/schema.sql`. Nas tabelas a seguir, PK indica a chave primária, FK a chave estrangeira, NOT NULL um campo obrigatório e UNIQUE um valor que não pode se repetir.

Tabela `usuarios`: Contas de acesso (candidato, empresa ou administrador), com e-mail, senha protegida e telefone.

Atributo	Descrição	Tipo	Tamanho	Formato	Restrições
id	Código que identifica o registro no banco	INT	11	Número	PK, NOT NULL, AUTO_INCREMENT
nome	Nome do usuário ou da empresa	VARCHAR	255	Texto	NOT NULL
email	E-mail usado no login	VARCHAR	255	Texto	NOT NULL, UNIQUE
senha	Hash bcrypt da senha	VARCHAR	255	Texto	NOT NULL
tipo	Tipo de conta	ENUM	—	'admin', 'candidato', 'empresa'	NOT NULL, DEFAULT 'candidato'
telefone	Telefone de contato	VARCHAR	30	(00) 00000-0000	NULL

Atributo	Descrição	Tipo	Tamanho	Formato	Restrições
ativo	Indica se a conta está ativa	TINYINT	1	0 ou 1	NOT NULL, DEFAULT 1
ultimo_acesso	Data e hora do último acesso	DATETIME	—	dd/mm/aaaa hh:mm	NULL
created_at	Data e hora de criação do registro	DATETIME	—	dd/mm/aaaa hh:mm	NOT NULL, DEFAULT CURRENT_TIMESTAMP

Tabela 36 – Dicionário de dados da tabela usuarios
Fonte: Autoria própria (2026).

Tabela perfis: Dados profissionais do candidato ou dados da empresa, um perfil para cada usuário.

Atributo	Descrição	Tipo	Tamanho	Formato	Restrições
id	Código que identifica o registro no banco	INT	11	Número	PK, NOT NULL, AUTO_INCREMENT
usuario_id	Código do usuário dono do perfil	INT	11	Número	FK para usuarios.id, NOT NULL, UNIQUE, ON DELETE CASCADE
titulo_profissional	Cargo ou título profissional do candidato	VARCHAR	255	Texto	NULL
bio	Resumo profissional ou descrição da empresa	TEXT	—	Texto	NULL
data_nascimento	Data de nascimento do candidato	DATE	—	dd/mm/aaaa	NULL
cidade	Cidade ou região administrativa	VARCHAR	100	Texto	DEFAULT 'Brasília'
uf	Sigla da unidade federativa	VARCHAR	2	AA	DEFAULT 'DF'
nome_fantasia	Nome fantasia da empresa	VARCHAR	255	Texto	NULL
cnpj	CNPJ da empresa	VARCHAR	20	00.000.000/00 00-00	NULL
setor	Setor de atuação da empresa	VARCHAR	100	Texto	NULL
site	Endereço do site da empresa	VARCHAR	255	URL	NULL
habilidades	Habilidades do candidato	TEXT	—	Texto	NULL
experiencias	Experiências profissionais	TEXT	—	Texto	NULL
formacao	Formação escolar e acadêmica	TEXT	—	Texto	NULL
cursos_complementares	Cursos complementares	TEXT	—	Texto	NULL

Atributo	Descrição	Tipo	Tamanho	Formato	Restrições
informacoes_adicionais	Informações adicionais (PCD, disponibilidade etc.)	TEXT	—	Texto	NULL
idiomas	Idiomas e níveis	TEXT	—	Texto	NULL
competencias	Competências comportamentais e técnicas	TEXT	—	Texto	NULL
nivel_experiencia	Nível de experiência do candidato	ENUM	—	'estagiario', 'junior', 'pleno', 'senior'	DEFAULT 'junior'
disponibilidade	Disponibilidade de horário ou mudança	VARCHAR	100	Texto	NULL
objetivo	Objetivo profissional	TEXT	—	Texto	NULL
links	Links (LinkedIn, GitHub, portfólio)	TEXT	—	URL	NULL
cnh	Categoria da CNH	VARCHAR	5	A, B, AB...	NULL
pretensao_salarial	Pretensão salarial	VARCHAR	60	Texto	NULL
foto	Caminho da foto ou logotipo	VARCHAR	255	Texto	NULL
publico	Indica se o perfil aparece no banco de talentos	TINYINT	1	0 ou 1	NOT NULL, DEFAULT 1
aceite_lgpd	Aceite do termo de privacidade	TINYINT	1	0 ou 1	NOT NULL, DEFAULT 0
aceite_em	Data e hora do aceite	DATETIME	—	dd/mm/aaaa hh:mm	NULL
created_at	Data e hora de criação do registro	DATETIME	—	dd/mm/aaaa hh:mm	NOT NULL, DEFAULT CURRENT_TIMESTAMP
updated_at	Data e hora da última alteração	DATETIME	—	dd/mm/aaaa hh:mm	NOT NULL, ON UPDATE CURRENT_TIMESTAMP

Tabela 37 – Dicionário de dados da tabela perfis
Fonte: Autoria própria (2026).

Tabela categorias: Áreas usadas para classificar as vagas e os cursos.

Atributo	Descrição	Tipo	Tamanho	Formato	Restrições
id	Código que identifica o registro no banco	INT	11	Número	PK, NOT NULL, AUTO_INCREMENT
nome	Nome da categoria	VARCHAR	100	Texto	NOT NULL, UNIQUE (nome, tipo)
tipo	Indica se é categoria de vaga ou de curso	ENUM	—	'vaga', 'curso'	NOT NULL

Atributo	Descrição	Tipo	Tamanho	Formato	Restrições
ativo	Indica se a categoria está ativa	TINYINT	1	0 ou 1	NOT NULL, DEFAULT 1

Tabela 38 – Dicionário de dados da tabela categorias
Fonte: Autoria própria (2026).

Tabela vagas: Vagas publicadas pelas empresas ou pelo administrador.

Atributo	Descrição	Tipo	Tamanho	Formato	Restrições
id	Código que identifica o registro no banco	INT	11	Número	PK, NOT NULL, AUTO_INCREMENT
perfil_empresa_id	Código do perfil da empresa que publicou	INT	11	Número	FK para perfis.id, NOT NULL, ON DELETE CASCADE
categoria_id	Código da categoria da vaga	INT	11	Número	FK para categorias.id, NULL, ON DELETE SET NULL
titulo	Título (cargo) da vaga	VARCHAR	255	Texto	NOT NULL
anunciante	Empresa do anúncio (curadoria)	VARCHAR	150	Texto	NULL
descricao	Descrição das atividades	TEXT	—	Texto	NULL
requisitos	Requisitos da vaga	TEXT	—	Texto	NULL
beneficios	Benefícios oferecidos	TEXT	—	Texto	NULL
contato	Contato do anúncio	VARCHAR	255	Texto	NULL
tipo_vaga	Tipo de contratação	ENUM	—	'clt', 'pj', 'estagio', 'temporario'	NOT NULL, DEFAULT 'clt'
nivel_experiencia	Nível pedido	ENUM	—	'estagiario', 'junior', 'pleno', 'senior'	NOT NULL, DEFAULT 'junior'
remoto	Modelo de trabalho	ENUM	—	'presencial', 'remoto', 'hibrido'	NOT NULL, DEFAULT 'presencial'
cidade	Cidade da vaga	VARCHAR	100	Texto	NULL
uf	UF da vaga	VARCHAR	2	AA	NULL
salario_minimo	Salário mínimo oferecido	DECIMAL	10,2	0.000,00	NULL
salario_maximo	Salário máximo oferecido	DECIMAL	10,2	0.000,00	NULL
imagem	Caminho do cartaz da vaga	VARCHAR	255	Texto	NULL

Atributo	Descrição	Tipo	Tamanho	Formato	Restrições
status	Situação da vaga	ENUM	—	'ativa', 'pausada', 'encerrada'	NOT NULL, DEFAULT 'ativa'
destaque	Vaga em destaque (Premium)	TINYINT	1	0 ou 1	NOT NULL, DEFAULT 0
data_publicacao	Data de publicação	DATE	—	dd/mm/aaaa	NULL
data_expiracao	Data de expiração	DATE	—	dd/mm/aaaa	NULL
visualizacoes	Quantidade de visualizações	INT	11	Número	NOT NULL, DEFAULT 0
created_at	Data e hora de criação do registro	DATETIME	—	dd/mm/aaaa hh:mm	NOT NULL, DEFAULT CURRENT_TIME STAMP

Tabela 39 – Dicionário de dados da tabela vagas
Fonte: Autoria própria (2026).

Tabela cursos: Cursos, e-books e vídeos de capacitação.

Atributo	Descrição	Tipo	Tamanho	Formato	Restrições
id	Código que identifica o registro no banco	INT	11	Número	PK, NOT NULL, AUTO_INCREMENT
categoria_id	Código da categoria do curso	INT	11	Número	FK para categorias.id, NULL, ON DELETE SET NULL
titulo	Título do curso	VARCHAR	255	Texto	NOT NULL
descricao	Descrição do conteúdo	TEXT	—	Texto	NULL
tipo	Tipo de conteúdo	ENUM	—	'curso', 'ebook', 'video'	NOT NULL, DEFAULT 'curso'
modalidade	Modalidade	ENUM	—	'ead', 'presencial', 'hibrido'	NOT NULL, DEFAULT 'ead'
nivel	Nível do curso	ENUM	—	'iniciante', 'intermediario', 'avancado'	NOT NULL, DEFAULT 'iniciante'
duracao	Carga horária	VARCHAR	50	Texto	NULL
gratuito	Indica se é gratuito	TINYINT	1	0 ou 1	NOT NULL, DEFAULT 1
preco	Preço do curso	DECIMAL	10,2	0.000,00	NULL
url	Link da instituição	VARCHAR	500	URL	NULL
imagem	Caminho da capa	VARCHAR	255	Texto	NULL

Atributo	Descrição	Tipo	Tamanho	Formato	Restrições
instituicao	Instituição ofertante	VARCHAR	255	Texto	NULL
ativo	Indica se o conteúdo está visível	TINYINT	1	0 ou 1	NOT NULL, DEFAULT 1
created_at	Data e hora de criação do registro	DATETIME	—	dd/mm/aaaa hh:mm	NOT NULL, DEFAULT CURRENT_TIMESTAMP

Tabela 40 – Dicionário de dados da tabela cursos
Fonte: Autoria própria (2026).

Tabela currículos: Currículos enviados pelos candidatos e o texto extraído deles.

Atributo	Descrição	Tipo	Tamanho	Formato	Restrições
id	Código que identifica o registro no banco	INT	11	Número	PK, NOT NULL, AUTO_INCREMENT
perfil_id	Código do perfil do candidato	INT	11	Número	FK para perfis.id, NOT NULL, ON DELETE CASCADE
titulo	Título do currículo	VARCHAR	255	Texto	NULL
arquivo_pdf	Caminho do arquivo enviado	VARCHAR	255	Texto	NOT NULL
arquivo_tipo	Formato do arquivo	VARCHAR	20	pdf, doc, docx	NOT NULL
curriculo_texto	Texto extraído do arquivo	LONGTEXT	—	Texto	NULL
template	Modelo visual	VARCHAR	50	Texto	NOT NULL, DEFAULT 'moderno'
downloads	Quantidade de downloads	INT	11	Número	NOT NULL, DEFAULT 0
created_at	Data e hora de criação do registro	DATETIME	—	dd/mm/aaaa hh:mm	NOT NULL, DEFAULT CURRENT_TIMESTAMP

Tabela 41 – Dicionário de dados da tabela currículos
Fonte: Autoria própria (2026).

Tabela matches: Nota de compatibilidade entre cada candidato e cada vaga, com a explicação.

Atributo	Descrição	Tipo	Tamanho	Formato	Restrições
id	Código que identifica o registro no banco	INT	11	Número	PK, NOT NULL, AUTO_INCREMENT

Atributo	Descrição	Tipo	Tamanho	Formato	Restrições
perfil_candidato_id	Código do perfil do candidato	INT	11	Número	FK para perfis.id, NOT NULL, ON DELETE CASCADE
vaga_id	Código da vaga	INT	11	Número	FK para vagas.id, NOT NULL, ON DELETE CASCADE, UNIQUE (perfil_candidato_id, vaga_id)
pontuacao	Nota de compatibilidade (0 a 100)	DECIMAL	5,2	000,00	NOT NULL, DEFAULT 0
nivel	Faixa da nota	ENUM	—	'excelente', 'alto', 'medio', 'baixo'	NOT NULL, DEFAULT 'baixo'
detalhes	Explicação da nota	TEXT	—	JSON	NULL
visualizado	Indica se o candidato já viu	TINYINT	1	0 ou 1	NOT NULL, DEFAULT 0
created_at	Data e hora de criação do registro	DATETIME	—	dd/mm/aaaa hh:mm	NOT NULL, DEFAULT CURRENT_TIMESTAMP

Tabela 42 – Dicionário de dados da tabela matches
Fonte: Autoria própria (2026).

Tabela assinaturas: Planos Candidato VIP e Empresa Premium contratados (demonstrativos).

Atributo	Descrição	Tipo	Tamanho	Formato	Restrições
id	Código que identifica o registro no banco	INT	11	Número	PK, NOT NULL, AUTO_INCREMENT
usuario_id	Código do usuário assinante	INT	11	Número	FK para usuarios.id, NOT NULL, ON DELETE CASCADE
plano	Plano assinado	ENUM	—	'assinante', 'empresa'	NOT NULL
valor	Valor do plano	DECIMAL	10,2	0.000,00	NOT NULL, DEFAULT 0
data_inicio	Início da vigência	DATE	—	dd/mm/aaaa	NOT NULL
data_fim	Fim da vigência	DATE	—	dd/mm/aaaa	NOT NULL
status	Situação da assinatura	ENUM	—	'ativa', 'cancelada', 'expirada'	NOT NULL, DEFAULT 'ativa'

Atributo	Descrição	Tipo	Tamanho	Formato	Restrições
created_at	Data e hora de criação do registro	DATETIME	—	dd/mm/aaaa hh:mm	NOT NULL, DEFAULT CURRENT_TIME STAMP

Tabela 43 – Dicionário de dados da tabela assinaturas
Fonte: Autoria própria (2026).

Tabela candidaturas: Inscrições dos candidatos nas vagas e o retorno da empresa.

Atributo	Descrição	Tipo	Tamanho	Formato	Restrições
id	Código que identifica o registro no banco	INT	11	Número	PK, NOT NULL, AUTO_INCREMENT
perfil_candidato_id	Código do perfil do candidato	INT	11	Número	FK para perfis.id, NOT NULL, ON DELETE CASCADE
vaga_id	Código da vaga	INT	11	Número	FK para vagas.id, NOT NULL, ON DELETE CASCADE, UNIQUE (perfil_candidato_id, vaga_id)
curriculo_id	Código do currículo enviado	INT	11	Número	FK para curriculos.id, NULL, ON DELETE SET NULL
carta_apresentacao	Carta de apresentação	TEXT	—	Texto	NULL
status	Etapa da candidatura	ENUM	—	'enviada', 'em_analise', 'entrevista', 'aprovado', 'rejeitado', 'cancelada'	NOT NULL, DEFAULT 'enviada'
observacao_empresa	Retorno da empresa ao candidato	TEXT	—	Texto	NULL
data_candidatura	Data e hora do envio	DATETIME	—	dd/mm/aaaa hh:mm	NOT NULL, DEFAULT CURRENT_TIME STAMP
data_atualizacao	Data e hora da última mudança	DATETIME	—	dd/mm/aaaa hh:mm	NOT NULL, ON UPDATE CURRENT_TIME STAMP

Tabela 44 – Dicionário de dados da tabela candidaturas
Fonte: Autoria própria (2026).

Tabela tentativas_login: Senhas erradas, usadas para pausar o login depois de tentativas repetidas.

Atributo	Descrição	Tipo	Tamanho	Formato	Restrições
id	Código que identifica o registro no banco	INT	11	Número	PK, NOT NULL, AUTO_INCREMENT
ip	Endereço IP da tentativa	VARCHAR	45	IPv4/IPv6	NOT NULL
email	E-mail informado na tentativa	VARCHAR	255	Texto	NOT NULL
created_at	Data e hora de criação do registro	DATETIME	—	dd/mm/aaaa hh:mm	NOT NULL, DEFAULT CURRENT_TIMESTAMP

Tabela 45 – Dicionário de dados da tabela tentativas_login
Fonte: Autoria própria (2026).

Tabela redefinicoes_senha: Pedidos de nova senha, guardando só o resumo (hash) do código do link.

Atributo	Descrição	Tipo	Tamanho	Formato	Restrições
id	Código que identifica o registro no banco	INT	11	Número	PK, NOT NULL, AUTO_INCREMENT
usuario_id	Código do usuário	INT	11	Número	FK para usuarios.id, NOT NULL, ON DELETE CASCADE
token_hash	Hash SHA-256 do token do link	CHAR	64	Hexadecimal	NOT NULL, UNIQUE
expira_em	Data e hora de expiração do link	DATETIME	—	dd/mm/aaaa hh:mm	NOT NULL
usado_em	Data e hora em que o link foi usado	DATETIME	—	dd/mm/aaaa hh:mm	NULL
ip	IP de quem solicitou	VARCHAR	45	IPv4/IPv6	NULL
created_at	Data e hora de criação do registro	DATETIME	—	dd/mm/aaaa hh:mm	NOT NULL, DEFAULT CURRENT_TIMESTAMP

Tabela 46 – Dicionário de dados da tabela redefinicoes_senha
Fonte: Autoria própria (2026).

8.8 Arquitetura do Sistema

O sistema segue o padrão MVC em PHP puro. Todas as requisições passam pelo arquivo public/index.php (o front controller), que abre a sessão, confere a conta logada, envia os cabeçalhos de segurança e usa o roteador para chamar a ação certa do controller. Os 12 controllers consultam os models (9 DAOs) e os services (as regras de negócio: leitura de

documentos, padrões automáticos, capa de PDF, match, competências, portfólio e Pix) e exibem as views. No navegador, o Tesseract.js, servido pelo próprio site, lê a imagem do cartaz e manda para o servidor só o texto com a posição das palavras. Os arquivos enviados ficam em storage/uploads, fora da pasta pública.

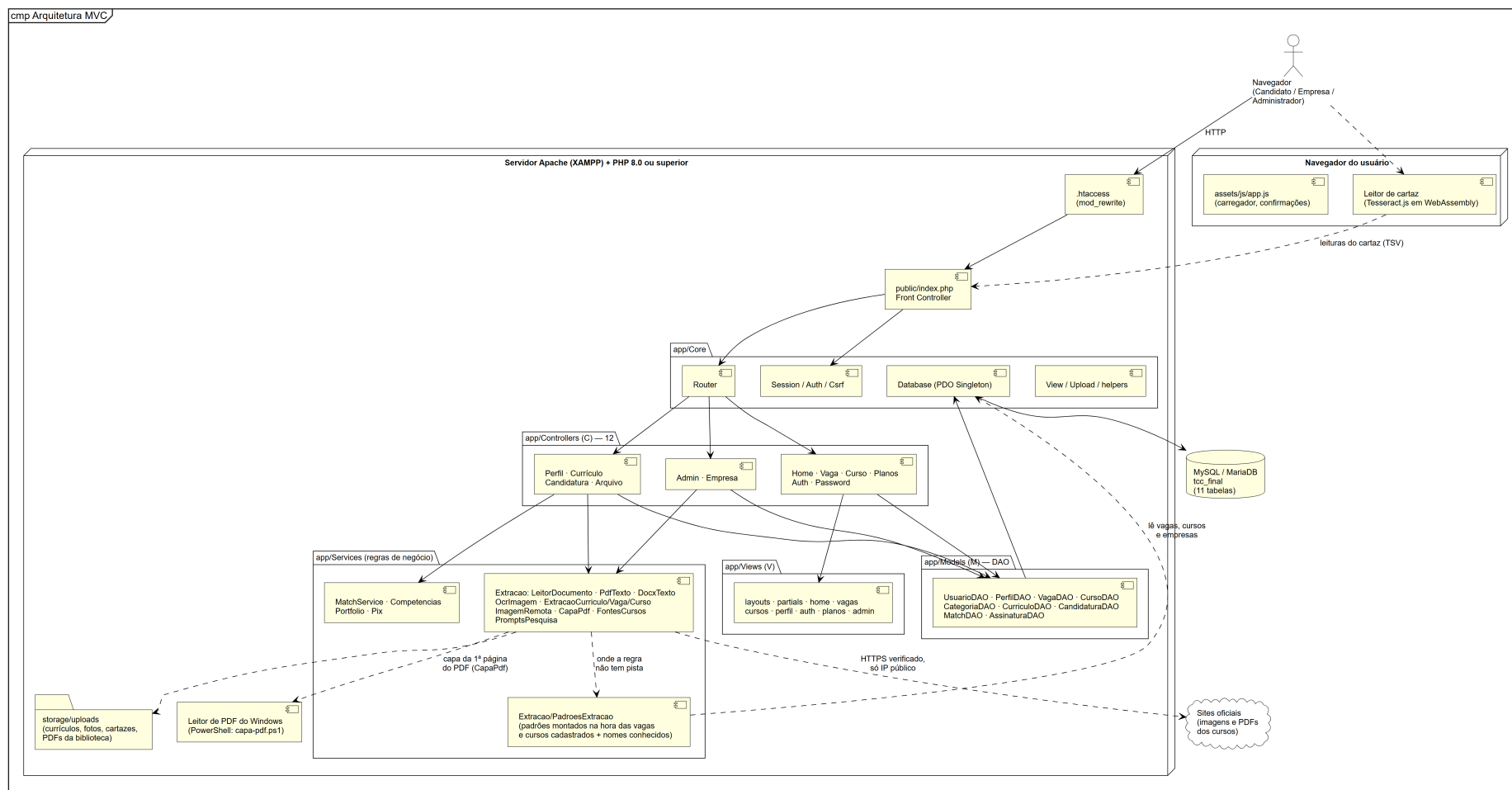


Figura 49 – Arquitetura em camadas (MVC)
Fonte: Autoria própria (2026).

O Quadro 4 lista as rotas do sistema, o controller e a ação que atendem cada endereço e quem pode acessá-lo.

Endereço	Controller::ação	Acesso
index.php (raiz do site)	HomeController::index	Todos
contrato.php	HomeController::contrato	Todos
vagas.php	VagaController::lista	Todos
vaga.php	VagaController::detalhe	Todos
cursos.php	CursoController::lista	Todos
curso.php	CursoController::detalhe	Todos
planos.php	PlanosController::index	Todos (assinar: logado)
login.php	AuthController::login	Candidato, Empresa e Administrador (sem login)
cadastro.php	AuthController::cadastro	Candidato e Empresa (sem conta)
view/usuario/logout.php	AuthController::logout	Logado
esqueci_senha.php	PasswordController::esqueci	Candidato, Empresa e Administrador (sem login)
redefinir_senha.php	PasswordController::redefinir	Quem recebeu o link
view/perfil/index.php	PerfilController::index	Logado
view/perfil/salvar.php	PerfilController::salvar	Candidato
view/perfil/portfolio.php	PerfilController::portfolio	Candidato; empresa e administrador (perfil público ou candidatura)
view/perfil/recalcular_match.php	PerfilController::recalcularMatch	Candidato
view/perfil/conta_excluir.php	PerfilController::excluirConta	Candidato (a própria conta)
view/perfil/curriculo_upload.php	CurriculoController::upload	Candidato
view/perfil/aplicar_extracao.php	CurriculoController::aplicarExtracao	Candidato
view/perfil/curriculo_excluir.php	CurriculoController::excluir	Candidato
candidatar.php	CandidaturaController::candidatar	Candidato
view/perfil/candidatura_cancelar.php	CandidaturaController::cancelar	Candidato
download.php	ArquivoController::download	Dono, administrador e empresa autorizada
admin/index.php	AdminController::painel	Empresa e Administrador
admin/pages/usuarios.php	AdminController::usuarios	Administrador
admin/pages/categorias.php	AdminController::categorias	Administrador
admin/pages/cursos.php	AdminController::cursos	Administrador
admin/pages/assinaturas.php	AdminController::assinaturas	Administrador
admin/pages/vagas.php	EmpresaController::vagas	Empresa e Administrador
admin/pages/candidaturas.php	EmpresaController::candidaturas	Empresa e Administrador
admin/pages/talentos.php	EmpresaController::talentos	Empresa e Administrador
admin/pages/empresa_perfil.php	EmpresaController::perfil	Empresa
assets/uploads/* (imagens)	ArquivoController::imagem	Todos (fotos, logos e cartazes)

Quadro 4 – Mapa de rotas
Fonte: Autoria própria (2026).

8.9 Plano de Testes

O plano de testes reúne os casos de teste funcionais e de segurança que executamos no sistema, além dos testes automáticos: o `tests/lint.php`, que confere a sintaxe de todos os arquivos PHP; o `tests/smoke.php`, o teste rápido, com 112 verificações; e o `tests/jornadas.php`, em que uma conta temporária usa o sistema pelo navegador em 18 verificações e é apagada no fim. Os três rodam juntos pelo `tests/verificar.bat` e antes de cada commit, pelo gancho `.githubhooks/pre-commit`.

Executamos e aprovamos os casos do Quadro 5 na varredura final de 27/09/2026; os casos dos padrões automáticos e da capa do PDF (CT25, CT26, CT30 e CT31) e os testes automáticos foram conferidos de novo na versão final, em 01/10/2026.

ID	Caso de teste	Entrada / passos	Resultado esperado	Requisito
CT01	Login válido	E-mail e senha de uma conta de teste	Sessão iniciada e painel do perfil exibido	RF01
CT02	Login inválido e pausa	8 tentativas com senha errada em 5 minutos	Aviso quando faltam 3; na 8ª, pausa de 5 minutos	RF01, RNF03
CT03	Cadastro sem aceite LGPD	Formulário completo sem marcar o termo	Cadastro recusado com a mensagem do termo	RF02, RNF04
CT04	Cadastro com e-mail repetido	E-mail já existente	Mensagem "Este e-mail já está cadastrado..."	RF02
CT05	Busca de vagas com filtros	Termo "vendedor", cidade "Taguatinga"	Somente vagas abertas compatíveis com os filtros	RF05
CT06	Envio de currículo PDF	Currículo PDF de 2 páginas com foto	Perfil e foto preenchidos, relatório exibido e match recalculado	RF09, RF10, RF14
CT07	Envio de arquivo inválido	Arquivo .exe renomeado para .pdf ou maior que 10 MB	Envio recusado com mensagem de formato ou tamanho	RF09, RNF09
CT08	Liberação do portfólio	Perfil com todos os campos obrigatórios	Aba Portfólio liberada e notas de match exibidas	RF13, RF14
CT09	Limite do plano gratuito (candidato)	4ª candidatura em andamento sem VIP	Candidatura recusada e redirecionamento para Planos	RF16, RF27
CT10	Cancelar candidatura	Candidatura com status "enviada"	Status alterado para "cancelada"	RF17
CT11	Extração de vaga por texto	Texto de anúncio com salário, cidade e requisitos	Formulário preenchido para revisão, sem gravar	RF23
CT12	Limite do plano gratuito (empresa)	3ª vaga ativa sem Premium	Publicação recusada e redirecionamento para Planos	RF20, RF27
CT13	Empresa altera vaga de outra empresa	Acesso a ?edit= de vaga alheia	Acesso negado (HTTP 403)	RF21, RNF03
CT14	Atualizar status da candidatura	Status "entrevista" com retorno	Candidato vê o novo status e o retorno	RF18, RF24
CT15	Download de currículo sem permissão	download.php por outro candidato ou empresa sem vínculo	Acesso negado (HTTP 403)	RF26, RNF03

ID	Caso de teste	Entrada / passos	Resultado esperado	Requisito
CT16	Excluir usuário	Exclusão pelo administrador	Remoção em cascata de perfil, currículos, vagas e candidaturas; as vagas dele deixam de contar nos padrões automáticos	RF29, RNF10
CT17	Formulário sem token CSRF	POST sem o campo csrf	Página de erro 403	RNF03
CT18	Teste automático (smoke test)	php tests/smoke.php	Todas as verificações com "OK" (112 de 112)	RNF11
CT19	Leitura de cartaz no navegador	Imagem JPG de um cartaz no painel, em Vagas	Carregador amarelo com a porcentagem, depois azul; formulário preenchido, sem instalar programas	RF23, RNF05, RNF07
CT20	Extrair com a caixa vazia	Extrair sem texto e sem cartaz	Mensagem pedindo o texto; nenhum relatório em branco	RF23
CT21	Excluir a própria conta	Senha correta e caixa de confirmação marcada	Conta e dados removidos (perfil, currículos, candidaturas, matches, tentativas de login); sessão encerrada	RF33, RNF04
CT22	Excluir conta sem permissão	Senha errada; ou id de outra conta enviado no formulário	Conta não excluída; o id enviado é ignorado	RF33, RNF03
CT23	Biblioteca de e-books	"Trazer os PDFs para a biblioteca"	E-books com o botão "Baixar" e o crédito da fonte original	RF07, RF37
CT24	Importar fichas em lote	Três fichas separadas por "---"	Prévia com 3 itens e cadastro dos marcados	RF31, RF38
CT25	Padrão automático de linha	Vagas cadastradas com "Uniforme" nos Benefícios e anúncio novo com "Uniforme fornecido pela empresa"	A linha vai para Benefícios e aparece em "Padrões automáticos usados"	RF35, RF36, RNF12
CT26	Nome conhecido	Anúncio que só cita uma empresa já cadastrada	Empresa preenchida, com "empresa já cadastrada" no relatório	RF23, RF35
CT27	Conceder assinatura (administrador)	Premium por 30 dias a uma empresa ativa	Assinatura ativa e benefícios liberados	RF34
CT28	Testes de jornada	php tests/jornadas.php	18 de 18 com "OK"; a conta temporária é apagada no fim	RNF11
CT29	Varredura completa	Rastrear todas as páginas sem login e com cada tipo de conta	1.701 páginas sem erro	RNF05, RNF11
CT30	Capa do e-book pelo PDF	PDF com imagem na 1ª página	Capa gerada pela 1ª página (leitor do Windows ou, sem ele, a imagem da página)	RF37, RF38
CT31	A regra continua mandando	Linha com palavra-chave de requisito ("Experiência com uniforme")	A linha fica em Requisitos, mesmo com padrão de outro campo	RF35, RNF12

Quadro 5 – Casos de teste
Fonte: Autoria própria (2026).

8.10 Matriz de Rastreabilidade

A matriz de rastreabilidade do Quadro 6 liga cada requisito funcional ao caso de uso, ao código que o implementa e às tabelas do banco utilizadas.

Requisito	Caso de uso	Controller / Service	Tabelas
RF01, RF03	UC01	AuthController::login / logout	usuarios, tentativas_login
RF02	UC02	AuthController::cadastro	usuarios, perfis
RF04	UC03	PasswordController::esqueci / redefinir	usuarios, redefinicoes_senha
RF05, RF06	UC04	VagaController::lista / detalhe	vagas, categorias, matches
RF07	UC05	CursoController::lista / detalhe	cursos, categorias
RF08	UC06	PerfilController::salvar	perfis, usuarios
RF09 a RF12	UC07	CurriculoController::upload / aplicarExtracao / excluir; LeitorDocumento; ExtracaoCurriculo	curriculos, perfis
RF13 a RF15	UC08	PerfilController::portfolio / recalcularMatch; MatchService; Portfolio	matches, vagas, perfis
RF16 a RF18	UC09.1, UC09.2	CandidaturaController::candidatar / cancelar	candidaturas, assinaturas
RF19	UC10	EmpresaController::perfil	perfis
RF20 a RF23	UC11.1 a UC11.3	EmpresaController::vagas; ExtracaoVaga; OcrImagem; leitor de cartaz (Tesseract.js)	vagas, matches
RF24, RF26	UC12	EmpresaController::candidaturas; ArquivoController::download	candidaturas, curriculos
RF25	UC13	EmpresaController::talentos	perfis
RF27, RF28	UC14.1, UC14.2	PlanosController::index; AssinaturaDAO	assinaturas, vagas
RF29	UC15.1 a UC15.4	AdminController::usuarios	usuarios (cascata)
RF30	UC16.1 a UC16.4	AdminController::categorias	categorias
RF31	UC17.1 a UC17.3	AdminController::cursos; ExtracaoCurso	cursos
RF32	UC18	AdminController::painel	vagas, candidaturas, matches, usuarios
RF33	UC19	PerfilController::excluirConta; UsuarioDAO	usuarios (cascata), tentativas_login
RF34	UC20	AdminController::assinaturas; AssinaturaDAO	assinaturas
RF35	UC21 (UC11.1, UC17.1)	ExtracaoVaga; ExtracaoCurso; PadroesExtracao	vagas, cursos, categorias, perfis
RF36	UC21	EmpresaController::vagas (quadro "Padrões automáticos usados")	vagas
RF37	UC05, UC17.1, UC17.5	AdminController::cursos; ImagemRemota; CapaPdf; CursoDAO	cursos
RF38	UC17.4	AdminController::cursos; ExtracaoCurso	cursos
RF39	Não se aplica	HomeController::index; Pix	Não se aplica
RF40	UC18	AdminController::painel (manutenção diária)	curriculos, perfis, vagas, cursos (arquivos em uso)

Quadro 6 – Rastreabilidade entre requisito, caso de uso, código e banco

Fonte: Autoria própria (2026).

8.11 Funcionalidades em Detalhe

Nesta seção descrevemos cada funcionalidade do Conecta Vagas DF individualmente, do jeito que ela existe hoje no código. Organizamos as subseções pelos quatro personagens do projeto, na ordem em que eles aparecem no dia a dia do sistema: primeiro o Candidato (da navegação sem conta até a exclusão dos próprios dados), depois a Empresa (perfil, publicação de vagas e seleção de candidatos), depois o Administrador (painel, usuários, categorias, catálogo de cursos e assinaturas) e por fim o Sistema, que é como chamamos as tarefas que a própria aplicação faz sozinha, sem ninguém clicar em nada. Para cada funcionalidade explicamos quem a usa e onde ela fica no site (o item de menu, a tela e o endereço), o passo a passo do que a pessoa faz e do que o Sistema faz em resposta, o caminho dentro do código (a rota em `public/index.php`, o método do Controller, os Services e DAOs chamados, as tabelas lidas e gravadas e a tela de `app/Views` exibida no final), as regras e limites que valem para aquela funcionalidade e as mensagens que aparecem para a pessoa, inclusive quando algo dá errado. Sempre que ajuda a enxergar o fluxo de uma vez, fechamos a funcionalidade com um quadro "Caminho no código", coluna por coluna.

8.11.1 Navegação e busca de vagas (Candidato)

Esta é a porta de entrada do site e não exige conta: para nós, quem navega sem se cadastrar já é um candidato, só que ainda não cadastrado. A pessoa chega pelo item "Vagas" do menu principal (`app/Views/layouts/header.php`), pelo endereço `vagas.php`, ou clicando num cartão de vaga na página inicial ou na própria lista, que abre `vaga.php?id=<id>`.

Na lista, a pessoa digita um termo de busca, escolhe cidade, área (categoria), tipo de contratação, nível de experiência e modelo de trabalho (presencial, remoto ou híbrido), e pode trocar a ordem para "por match" quando está logada como candidata. O `VagaController::lista()` lê esses filtros com `get_str()/(int) get_str()`, consulta a `CategoriaDAO::listar('vaga', true)` para montar as abas de área e a `VagaDAO::listar(true, $filtros)` para buscar as vagas abertas que casam com eles; sem nenhum filtro marcado, reaproveita a mesma lista completa (evita rodar a consulta duas vezes). Se o candidato estiver logado, o controller também busca o mapa de match (`MatchDAO::mapaPorCandidato`) e as candidaturas já enviadas (`CandidaturaDAO::listarPorCandidato`) para marcar a porcentagem de compatibilidade e o status em cada cartão. A lista é paginada em 20 cartões por página e mostra, acima da grade, etiquetas dos filtros em uso, cada uma com um link para removê-la sozinha. A tela é `app/Views/vagas/lista.php`.

Ao abrir uma vaga específica, o `VagaController::detalhe()` busca o registro com `VagaDAO::buscar($id)` e confere se ela está aberta (`VagaDAO::estaAberta()`, que olha o status, o prazo de inscrição e se a empresa dona continua com a conta ativa). Vaga pausada ou encerrada só é mostrada para a empresa dona e para o administrador; para qualquer outra pessoa, ou para um id que não existe, a página responde 404 com a tela `vagas/nao_encontrada.php`. Quando a vaga está aberta e quem está vendo não é a própria empresa, o sistema soma uma visualização (`VagaDAO::incrementarVisualizacao()`) uma única vez por sessão, guardando a marca em `$_SESSION['vagas_vistas']`, justamente para um F5 não inflar artificialmente o indicador de "Desempenho" que a empresa vê no painel. A página de detalhe monta as competências pedidas pela vaga (`Competencias::daVaga()`), o match do candidato logado (se houver), cursos recomendados para cobrir o que falta e uma lista de vagas relacionadas, priorizando a mesma área. A tela é `vagas/detalhe.php`.

Etapa	Arquivo e método	O que acontece
Pedido da lista	<code>public/index.php</code> (rota <code>vagas.php</code>) chama <code>VagaController::lista()</code>	o roteador identifica o endereço e executa a ação
Filtros e busca	<code>VagaController::lista()</code> , <code>VagaDAO::listar()</code> , <code>CategoriaDAO::listar()</code>	lê os filtros da URL e consulta as tabelas vagas e categorias
Match do candidato	<code>MatchDAO::mapaPorCandidato()</code> , <code>CandidaturaDAO::listarPorCandidato()</code>	marca a % de compatibilidade e o status já enviado em cada cartão
Tela da lista	<code>app/Views/vagas/lista.php</code>	grade de cartões paginada, com abas de área e filtros ativos
Página da vaga	<code>VagaController::detalhe()</code> , <code>VagaDAO::buscar()/estaAberta()/incrementarVisualizacao()</code>	confere a permissão, conta a visualização e monta o anúncio
Tela da vaga	<code>app/Views/vagas/detalhe.php</code>	cartaz, descrição, requisitos, benefícios, contato e vagas relacionadas

Quadro 7 – Caminho no código da navegação e busca de vagas
Fonte: Autoria própria (2026).

8.11.2 Cursos, e-books e vídeos: Baixar e Acessar (Candidato)

O catálogo de capacitação também é público. Cada formato tem a sua própria página, sem misturar: "Cursos" no menu abre `cursos.php`, "E-books" abre `cursos.php?tipo=ebook`, e os vídeos ficam em `cursos.php?tipo=video` (sem item próprio no menu, junto da aba de e-books). O `CursoController::lista()` lê o tipo pedido com `enum_val()`, os filtros de busca, categoria e gratuidade, e consulta `CursoDAO::listar(true, [...])` para montar três coisas ao mesmo tempo: a contagem de cada aba (Cursos, E-books, Vídeos), os atalhos de área que têm conteúdo naquele formato e a grade paginada do formato atual. Quando o candidato logado está vendo a aba de

cursos, o sistema ainda monta uma vitrine de "Recomendados para você": olha as cinco vagas com melhor match (MatchDAO::listarPorCandidato) e recomenda, pela Competencias::cursosPara(), os cursos que cobrem as competências que mais faltam. A tela é app/Views/cursos/lista.php.

Ao abrir um conteúdo (curso.php?id=), o CursoController::detalhe() busca o registro (CursoDAO::buscar) e, se ele estiver oculto (ativo=0), só mostra para o administrador, para qualquer outra pessoa é 404 (cursos/nao_encontrado.php). A página monta as competências que o conteúdo desenvolve (Competencias::doCurso), as vagas que pedem essas competências, outros conteúdos do mesmo formato e, para o candidato logado, o que ainda falta para ele. O botão de acesso é decidido pela função pt_acesso_conteudo() (app/Views/partials/componentes.php): quando a url do conteúdo é um PDF da nossa biblioteca (reconhecido por eh_pdf_biblioteca(), que só aceita o padrão assets/uploads/biblioteca_*.pdf), o botão é "Baixar" e o arquivo é entregue pelo próprio site, pelo ArquivoController::imagem(), com o atributo download; quando o conteúdo mora em outro site, o botão é "Acessar" (ou "Acessar e-book"/"Acessar vídeo", pela pt_cta_curso()) e abre o link oficial numa aba nova, com rel="noopener". A tela é cursos/detalhe.php.

Etapa	Arquivo e método	O que acontece
Lista por formato	public/index.php (rota cursos.php) chama CursoController::lista()	separa cursos, e-books e vídeos sem misturar os formatos
Consulta e recomendação	CursoDAO::listar(), Competencias::cursosPara(), MatchDAO::listarPorCandidato()	busca o catálogo e recomenda cursos para quem está logado
Tela da lista	app/Views/cursos/lista.php	abas, atalhos de área e grade paginada
Página do conteúdo	CursoController::detalhe(), CursoDAO::buscar()	confere se está publicado e monta a ficha do conteúdo
Botão de acesso	pt_acesso_conteudo(), eh_pdf_biblioteca(), ArquivoController::imagem()	decide entre "Baixar" (PDF da biblioteca) e "Acessar" (link externo)

Quadro 8 – Caminho no código de cursos, e-books e vídeos

Fonte: Autoria própria (2026).

8.11.3 Cadastro com aceite da LGPD (Candidato)

O cadastro fica no item "Cadastro" do menu, endereço cadastro.php, e é a porta de entrada tanto para quem vai usar o site como candidato quanto para quem vai publicar vagas como empresa (o tipo de conta é escolhido no próprio formulário). A pessoa preenche nome, e-mail, senha, telefone e marca a caixa de aceite do termo de autorização e privacidade, cujo texto fica em contrato.php (HomeController::contrato(), tela home/contrato.php).

No POST, o `AuthController::cadastro()` valida os campos: nome obrigatório, e-mail no formato correto e com até 255 caracteres, senha entre 6 e 72 caracteres (72 é o limite do próprio `bcrypt`) e a caixa `aceite_lgpd` marcada, sem ela, a mensagem "É preciso aceitar o termo de autorização e privacidade." aparece e o cadastro não é feito. O tipo da conta só pode ser "candidato" ou "empresa" (`enum_val()` filtra qualquer outro valor para "candidato"): não existe like no formulário público para criar um administrador. Passando as validações, `UsuarioDAO::cadastrar()` grava o usuário com a senha já em `password_hash()` (`bcrypt`) e, dentro da MESMA transação, cria o perfil padrão (`PerfilDAO::criar()`, que começa em Brasília/DF, público e com o aceite da LGPD já registrado com a data e hora de agora). Se o e-mail já existir (inclusive por uma condição de corrida, detectada pelo código de erro 23000 da chave única da coluna email), a mensagem é "Este e-mail já está cadastrado. Faça login ou use outro e-mail."

Dando certo, o sistema já abre a sessão da pessoa (`iniciar_sessao_usuario()`, que gera um novo identificador de sessão e um novo token CSRF, evitando fixação de sessão) e a leva direto para o próximo passo: o candidato vai para o próprio perfil (`view/perfil/index.php?novo=1`) e a empresa vai para os dados da empresa (`admin/pages/empresa_perfil.php`). A tela do formulário é `auth/cadastro.php`.

Etapa	Arquivo e método	O que acontece
Formulário	<code>public/index.php</code> (rota <code>cadastro.php</code>) chama <code>AuthController::cadastro()</code>	exibe <code>auth/cadastro.php</code> (GET) ou recebe o envio (POST)
Validação e aceite	<code>AuthController::cadastro()</code>	confere nome, e-mail, senha e a caixa <code> aceite_lgpd</code>
Gravação em transação	<code>UsuarioDAO::cadastrar()</code> , <code>PerfilDAO::criar()</code>	grava em <code>usuarios</code> (senha com <code>password_hash()</code>) e em perfis, juntos
Sessão nova	<code>iniciar_sessao_usuario()</code> (<code>app/Core/Session.php</code>)	abre a sessão já logada, com novo id de sessão e novo token CSRF
Redirecionamento	<code>AuthController::cadastro()</code>	candidato vai ao próprio perfil; empresa vai ao perfil da empresa

Quadro 9 – Caminho no código do cadastro
Fonte: Autoria própria (2026).

8.11.4 Login com tolerância a erros e pausa por segurança (Candidato)

O login fica no item "Login" do menu, endereço `login.php` (`AuthController::login()`). Quem já está logado é redirecionado para o painel certo do seu tipo de conta (`destinoPainel()`); senão, a tela `auth/login.php` pede e-mail e senha, com um botão em forma de olho ao lado do campo de senha para a pessoa conferir o que digitou antes de enviar.

No POST, antes de qualquer outra coisa o sistema confere se aquele e-mail (ou aquele IP) está pausado por tentativas demais (`UsuarioDAO::minutosBloqueioLogin()`): o bloqueio soma três contadores na tabela `tentativas_login` dentro da janela de tempo configurada

(LOGIN_JANELA_MINUTOS), o mesmo e-mail a partir do mesmo IP, o mesmo IP somando todos os e-mails, e o mesmo e-mail somando todos os IPs (ataque distribuído), e libera sozinho quando as tentativas saem da janela, nunca é permanente. Passando o bloqueio, a autenticação (UsuarioDAO::autenticar()) compara a senha com password_verify() contra o hash gravado, mas não testa só o que foi digitado: UsuarioDAO::variantesSenha() também testa a senha sem espaços nas pontas, com a primeira letra trocada de maiúscula para minúscula (ou o contrário) e com todas as letras na caixa trocada, como quando o Caps Lock está ligado, no máximo quatro variantes, e elas também são testadas quando o e-mail nem existe, para gastar o mesmo tempo de resposta e não revelar quais e-mails têm conta. Se a conta não existir, a senha estiver errada ou a conta estiver desativada, o login falha, a tentativa é registrada (UsuarioDAO::registrarFalhaLogin()) e a mensagem é sempre a mesma genérica ("E-mail ou senha inválidos ou conta desativada.") para quem acessa de outro computador; só no ambiente local (DEBUG, acesso pelo próprio computador) é que a tela conta o motivo exato (sem conta, senha errada ou conta desativada), para ajudar nos testes. A cada tentativa que resta cada vez menos, a mensagem também avisa: "Restam N tentativa(s) antes de uma pausa de 5 minutos." e, ao estourar o limite, "Login pausado por 5 minutos por segurança."

Autenticando com sucesso, o sistema limpa as tentativas daquele e-mail e IP (UsuarioDAO::limparFalhasLogin()), abre a sessão (iniciar_sessao_usuario()) e, se o algoritmo de hash da senha estiver desatualizado (não o custo do bcrypt, só o algoritmo), refaz o hash sem invalidar as outras sessões abertas. A pessoa é levada para o painel do seu tipo de conta, ou de volta para onde estava (por exemplo, "Entrar para assinar" nos planos, com voltar=planos).

Etapa	Arquivo e método	O que acontece
Formulário	public/index.php (rota login.php) chama AuthController::login()	exibe auth/login.php (GET) ou recebe e-mail e senha (POST)
Pausa por tentativas	UsuarioDAO::minutosBloqueioLogin() (tabela tentativas_login)	bloqueia por alguns minutos após tentativas demais
Autenticação tolerante	UsuarioDAO::autenticar(), UsuarioDAO::variantesSenha()	confere a senha e até 4 variantes comuns de digitação
Falha registrada	UsuarioDAO::registrarFalhaLogin()	grava a tentativa errada para contar o próximo bloqueio
Sessão aberta	iniciar_sessao_usuario() (app/Core/Session.php)	novo id de sessão, novo token CSRF e marca da senha atual

Quadro 10 – Caminho no código do login
Fonte: Autoria própria (2026).

8.11.5 Recuperação de senha (Candidato)

O link "Esqueci minha senha" leva a esqueci_senha.php (PasswordController::esqueci()). Como o projeto é acadêmico e roda sem servidor de e-mail

configurado, o fluxo é demonstrativo: a resposta na tela é sempre a mesma, exista ou não o e-mail informado ("link enviado"), para não revelar quais contas existem, e cada conta só pode pedir até três links por hora (limite dentro de `UsuarioDAO::criarTokenRedefinicao()`). O pedido também passa pelo mesmo limite de tentativas do login (`minutosBloqueioLogin()`, sem somar os outros IPs), para não virar uma ferramenta de descobrir e-mails cadastrados.

Quando o e-mail existe e está dentro do limite, o sistema gera um token aleatório de 32 bytes, grava no banco só o hash SHA-256 dele (nunca o token em si) com validade de 30 minutos (`UsuarioDAO::REDEFINICAO_MINUTOS`) e invalida qualquer link pendente anterior daquela conta. No modo de demonstração (DEBUG), o link completo é registrado em `storage/logs/redefinicoes_senha.log` com o e-mail mascarado (`mascarar_email()`, ex.: "ca***@dominio.com") e, só quando o acesso é feito do próprio computador (localhost), o link também aparece na tela, para facilitar o teste sem precisar abrir o log. Fora do modo de demonstração, nada disso acontece: o link seria entregue por e-mail de verdade.

O link abre `redefinir_senha.php?token=...` (`PasswordController::redefinir()`). O `UsuarioDAO::buscarPorTokenRedefinicao()` confere o formato do token, o hash, se ainda não foi usado e se não expirou; sem isso, a mensagem é "Link inválido, já utilizado ou expirado. Gere um novo." Com o token válido, a pessoa digita a nova senha (mesmas regras de 6 a 72 caracteres) e a confirmação; `UsuarioDAO::redefinirSenha()` marca o token como usado (uso único, inclusive contra duas abas abertas com o mesmo link) e grava o novo `password_hash()`. Qualquer sessão aberta daquela conta é encerrada (a marca da senha mudou), as tentativas de login daquele e-mail são zeradas, e a pessoa é levada ao login com a mensagem "Senha alterada com sucesso. Entre com a nova senha."

Etapas	Arquivo e método	O que acontece
Pedido do link	<code>public/index.php</code> (rota <code>esqueci_senha.php</code>) chama <code>PasswordController::esqueci()</code>	gera o token e grava só o hash SHA-256 em <code>redefinicoes_senha</code>
Entrega demonstrativa	<code>registrar_log()</code> (<code>storage/logs/redefinicoes_senha.log</code>)	no modo DEBUG, grava o link; no localhost, também mostra na tela
Uso do link	<code>PasswordController::redefinir()</code> , <code>UsuarioDAO::buscarPorTokenRedefinicao()</code>	confere validade, prazo de 30 minutos e uso único
Nova senha	<code>UsuarioDAO::redefinirSenha()</code>	grava o novo <code>password_hash</code> e invalida o token e as sessões antigas

Quadro 11 – Caminho no código da recuperação de senha
Fonte: Autoria própria (2026).

8.11.6 Perfil do candidato

"Meu perfil" é o item de menu que leva ao próprio cadastro profissional, em `view/perfil/index.php` (`PerfilController::index()`), acessível só a quem está logado como candidato (`exigirLogin()` mais a conferência `isCandidato()`; empresa e administrador são redirecionados para o painel). A tela reúne, na mesma página, a caixa de envio do currículo (que dispara a máquina de extração, descrita na próxima funcionalidade) e o formulário manual com todos os campos do perfil: título profissional, resumo, cidade, UF, nível de experiência, habilidades, experiências, formação, cursos complementares, informações adicionais, idiomas, competências, disponibilidade, objetivo, links, CNH, pretensão salarial e a foto.

Ao salvar (`PerfilController::salvar()`, POST para `view/perfil/salvar.php`), o sistema valida a UF (sigla de 2 letras) e a categoria da CNH (A, B, C, D, E ou as combinadas, como AB), filtra os links para só aceitar um endereço web por linha, e trata a foto como opcional (JPG/PNG/WEBP até 3 MB, `salvar_imagem_enviada()`): se a pessoa enviar uma foto nova, a antiga só é apagada depois que o perfil é salvo com sucesso com a nova, para nunca perder a referência no meio do caminho. `PerfilDAO::salvar()` grava tudo na tabela `perfis`; o telefone é gravado separadamente em `usuarios` (`UsuarioDAO::atualizarTelefone()`), porque pertence à conta, não ao perfil profissional. Depois de salvar, o `MatchService::recalcular()` roda de novo com todas as vagas abertas.

O `Portfolio::validarCadastro()` decide se o cadastro já está completo (telefone, título profissional, cidade, resumo, objetivo, formação e habilidades preenchidos): completando-o, a aba "Portfólio" aparece no topo do menu e a mensagem muda de "Cadastro salvo. Para liberar o Portfólio, falta: ..." para "Cadastro validado! A aba Portfólio foi liberada no topo...". A tela é `app/Views/perfil/index.php`.

Etapa	Arquivo e método	O que acontece
Tela do perfil	<code>public/index.php</code> (rota <code>view/perfil/index.php</code>) chama <code>PerfilController::index()</code>	reúne envio de currículo e formulário manual do cadastro
Salvar o formulário	<code>PerfilController::salvar()</code> , <code>PerfilDAO::salvar()</code>	valida UF/CNH/links, grava em <code>perfis</code> e trata a foto enviada
Telefone da conta	<code>UsuarioDAO::atualizarTelefone()</code>	grava o telefone em <code>usuarios</code> , separado do perfil
Match em dia	<code>MatchService::recalcular()</code>	recalcula a compatibilidade com todas as vagas ativas
Validação do cadastro	<code>Portfolio::validarCadastro()</code>	decide se a aba Portfólio já pode aparecer no menu

Quadro 12 – Caminho no código do perfil do candidato
Fonte: Autoria própria (2026).

8.11.7 Envio e leitura do currículo, com relatório da extração (Candidato)

Na tela "Meu perfil", a pessoa escolhe o arquivo do currículo (PDF, DOC ou DOCX, até o limite configurado em `MAX_FILE_SIZE`) e envia pelo formulário que chama `view/perfil/curriculo_upload.php` (`CurriculoController::upload()`). O controller confere o tamanho e a extensão, e ainda confirma que o CONTEÚDO do arquivo bate com a extensão (assinatura `%PDF` para PDF, `PK\x03\x04` para DOCX, o cabeçalho OLE para DOC), um arquivo renomeado não passa disso. O arquivo é salvo com um nome aleatório em `storage/uploads` e, até ser gravado no banco, é tratado como provisório: se a requisição parar no meio (um erro, falta de memória), um `register_shutdown_function()` apaga o arquivo órfão.

A extração em si usa `ExtracaoCurriculo::extrair()` (que por dentro chama `LeitorDocumento`, `PdfTexto` ou `DocxTexto`, dependendo do formato) para tirar o texto do arquivo, e `ExtracaoCurriculo::extrairCampos()` para separar nome, e-mail, telefone, data de nascimento, cidade, UF, links, CNH, pretensão salarial, disponibilidade, PCD, nível (calculado pelo tempo somado das experiências), experiências, formação, cursos, habilidades e idiomas. Os títulos de seção são reconhecidos mesmo sem acento, em maiúsculas, numerados, com ícones ou com as letras espaçadas. Em seguida, `AplicacaoCurriculo::calcular()` decide, campo por campo, o que fazer: campo vazio no perfil recebe o valor extraído (status "aplicado"); campo já preenchido é mantido, a não ser que a pessoa marque a caixa "Substituir" (status "mantido" ou "sugerido", guardado como pendente para a pessoa decidir depois); listas (como habilidades) são mescladas sem duplicar.

Terminada a aplicação (`AplicacaoCurriculo::salvar()`), o sistema atualiza o telefone e sugere o nome da conta quando ele difere do currículo (sem aplicar sozinho: o candidato confirma depois), grava o registro em currículos (`CurriculoDAO::salvar()`), com o texto extraído guardado para a máquina de match reutilizar e recalcula o match (`MatchService::recalcular()`). Tudo o que foi encontrado, aplicado, mantido ou não encontrado é guardado em `$_SESSION['relatorio_extracao']` e mostrado no topo do portfólio (redirect para `view/perfil/portfolio.php?relatorio=1`), na tela `app/Views/partials/relatorio_extracao.php`: cada linha do relatório mostra o campo, o valor lido, o valor atual e o status. Se o arquivo não tiver texto legível (por exemplo, uma página escaneada), o currículo é salvo mesmo assim, mas a mensagem avisa que o perfil precisa ser preenchido manualmente. Erros de leitura (arquivo corrompido, ou um arquivo que "incha" demais ao ser aberto, uma espécie de bomba de descompressão) cancelam o envio, apagam o arquivo e mostram uma mensagem orientando a salvar o arquivo de novo e reenviar.

A tela do relatório tem um botão "usar o dado do currículo" por campo pendente, que chama `view/perfil/aplicar_extracao.php` (`CurriculoController::aplicarExtracao()`): a pessoa marca os campos que quer trocar pelo valor do currículo (inclusive o nome da conta e a foto) e o sistema aplica só esses, atualiza o relatório na sessão e recalcula o match de novo, se algo mudou no perfil.

Etapa	Arquivo e método	O que acontece
Envio do arquivo	<code>public/index.php</code> (rota <code>view/perfil/curriculo_upload.php</code>) chama <code>CurriculoController::upload()</code>	confere tamanho, extensão e conteúdo real do arquivo
Extração do texto	<code>ExtracaoCurriculo::extrair()/extrairCampos()</code> (com <code>LeitorDocumento</code> , <code>PdfTexto</code> , <code>DocxTexto</code>)	lê o PDF/DOC/DOCX e separa nome, contato, experiências, formação etc.
Aplicação no perfil	<code>AplicacaoCurriculo::calcular()/salvar()</code> , <code>PerfilDAO::salvar()</code>	preenche o vazio, mantém o preenchido e mescla as listas
Gravação do currículo	<code>CurriculoDAO::salvar()</code> (tabela <code>curriculos</code>)	guarda o arquivo e o texto extraído
Match em dia	<code>MatchService::recalcular()</code>	recalcula a compatibilidade com as vagas ativas
Relatório	<code>\$_SESSION['relatorio_extracao']</code> , <code>app/Views/partials/relatorio_extracao.php</code>	mostra o que foi aplicado, mantido ou não encontrado
Aplicar pendências	<code>CurriculoController::aplicarExtracao()</code>	aplica, campo a campo, o que a pessoa escolheu do relatório

Quadro 13 – Caminho no código do envio e leitura do currículo
Fonte: Autoria própria (2026).

8.11.8 Foto do currículo (Candidato)

A busca pela foto acontece dentro do próprio envio do currículo, sem nenhuma tela separada: `LeitorDocumento::extrairFoto()` junta as imagens embutidas no arquivo (as do DOCX ficam em `word/media`; as do PDF são recompostas pelo `PdfTexto` a partir de JPEGs e imagens comprimidas, remontadas em PNG) e dá uma nota para cada uma, favorecendo tons de pele, variedade de cores e proporção de retrato, e descartando o que tem cara de logotipo, ícone, banner ou página inteira escaneada. A imagem com a melhor nota é a escolhida.

O `CurriculoController::upload()` só chama essa busca quando o texto do currículo foi lido com sucesso. Encontrando uma foto, ela é salva como JPEG de até 800 pixels (`AplicacaoCurriculo::salvarFoto()`). Se o perfil ainda não tinha foto, ela já entra como a foto do perfil (status "aplicado" no relatório); se o perfil já tinha uma foto, a encontrada no currículo fica apenas como pendente no relatório (status "mantido"), com uma miniatura, para o candidato escolher se quer trocar pelo botão "usar o dado do currículo"

(CurriculoController::aplicarExtracao()), que troca o campo foto do perfil e só então apaga a foto antiga (apagar_upload_sem_uso()), nunca antes.

Etapa	Arquivo e método	O que acontece
Busca da foto	LeitorDocumento::extrairFoto() (com PdfTexto::imagens(), word/media do DOCX)	reúne as imagens do arquivo e escolhe a que tem cara de foto de rosto
Aplicação	AplicacaoCurriculo::salvarFoto() (), CurriculoController::upload()	perfil sem foto: aplica direto; perfil com foto: guarda como pendente
Troca pelo relatório	CurriculoController::aplicarExtracao(), apagar_upload_sem_uso()	troca a foto do perfil e só depois apaga a antiga, se o candidato escolher

Quadro 14 – Caminho no código da foto do currículo
Fonte: Autoria própria (2026).

8.11.9 Portfólio (Candidato)

O portfólio é a vitrine profissional montada automaticamente a partir dos dados do cadastro, em view/perfil/portfolio.php (PerfilController::portfolio()). Sem o parâmetro id, é o portfólio do próprio candidato logado; com ?id=<perfil>, é o portfólio de outro candidato, visto por quem tiver permissão. A aba só aparece no menu depois que Portfolio::validarCadastro() considera o cadastro completo; enquanto isso, a pessoa é mandada de volta ao formulário com a lista do que falta.

A montagem fica inteiramente dentro da classe Portfolio (app/Services/Portfolio.php): Portfolio::experiencias() lê o texto solto digitado (ou extraído do currículo) e separa cada experiência em empresa, cargo, período e descrição, entendendo formatos bem diferentes de escrita (período no fim ou no início da linha, "Cargo – Empresa", linhas com rótulo "Empresa:"/"Cargo:"/"Período:", datas como "Jan/2020 – Atual" ou "03/2018 a 12/2019") e ordenando as mais recentes primeiro. Portfolio::formacao() faz o mesmo para a formação acadêmica (curso, instituição e situação: cursando, concluído, incompleto, com o ano quando existe), e Portfolio::lista()/links() organizam habilidades, cursos complementares, idiomas e links em listas limpas, sem repetição.

Quem vê o portfólio e o que vê depende de quem está olhando: o próprio dono e o administrador veem tudo, inclusive a máquina de match (explicada na próxima funcionalidade); uma empresa que já recebeu candidatura daquele candidato também vê o contato; empresa Premium vê o portfólio completo de qualquer candidato com perfil público; empresa sem Premium e sem candidatura recebida vê só uma prévia (nome mascarado, sem foto, sem contato, sem experiências e sem formação), igual à prévia do Banco de Talentos; pessoas sem conta e outros candidatos veem a versão pública completa, mas sem os dados de contato. Um candidato pode deixar o próprio perfil privado (campo "publico" do cadastro), e nesse caso só

o dono, o administrador e quem já recebeu candidatura dele conseguem abrir o portfólio, os demais recebem acesso negado. A tela é `app/Views/perfil/portfolio.php`.

Etapa	Arquivo e método	O que acontece
Abertura	<code>public/index.php</code> (rota <code>view/perfil/portfolio.php</code>) chama <code>PerfilController::portfolio()</code>	decide o portfólio do dono ou de outro candidato, pelo parâmetro <code>id</code>
Validação do cadastro	<code>Portfolio::validarCadastro()</code>	sem o cadastro completo, a aba ainda não é liberada
Montagem do conteúdo	<code>Portfolio::experiencias()/formacao()/lista()/links()</code>	transforma o texto do perfil em linha do tempo, formação e listas
Controle de quem vê o quê	<code>PerfilController::portfolio()</code> , <code>CandidaturaDAO::empresaRecebeuDoCandidato()</code> , <code>AssinaturaDAO::isEmpresaPremium()</code>	decide entre portfólio completo, prévia ou acesso negado
Tela	<code>app/Views/perfil/portfolio.php</code>	exibe a vitrine, com ou sem os dados de contato, conforme a permissão

Quadro 15 – Caminho no código do portfólio
Fonte: Autoria própria (2026).

8.11.10 Match com as vagas (Candidato)

O match é a nota de 0 a 100 que mede o quanto o perfil do candidato combina com cada vaga aberta, calculada pelo `MatchService` (`app/Services/MatchService.php`) e gravada por vaga em `matches.detalhes` (`MatchDAO::salvar()`), em formato JSON, para explicar a nota sem precisar recalculá-la a cada vez que alguém olha.

A pontuação é dividida em quatro partes, somando até 100: Competências (peso 50), comparando as competências que a vaga pede (`Competencias::daVaga()`) com as que o candidato tem, vindas do perfil e do texto do currículo (`Competencias::doPerfil()`); Cargo (peso 20), comparando palavras do título da vaga com o título profissional/objetivo do candidato (peso cheio) ou com o histórico de experiências, formação e habilidades (60% do peso); Localização (peso 15), valendo 15 pontos para a mesma cidade ou vaga remota, 9 para o mesmo estado e 6 quando o candidato tem disponibilidade para mudança; e Nível (peso 15), comparando o nível de experiência do candidato com o pedido pela vaga, com pontuação cheia quando é igual e menor quando o candidato tem menos experiência que o pedido. O resultado final é classificado por `MatchService::classificar()`: 75 pontos ou mais é "excelente", 55 ou mais é "alto", 35 ou mais é "médio", e menos que isso é "baixo". A classe calcula ainda "observações" que não mudam a nota, mas aparecem como explicação: CNH pedida pela vaga, vaga aberta para PCD e vagas que envolvem viagem, cruzando com o que o currículo ou o perfil trouxeram.

O recálculo acontece sozinho em vários momentos, ao enviar ou excluir um currículo, ao salvar o perfil, e quando a empresa cria ou edita uma vaga, e também pode ser pedido na

hora pelo botão "Recalcular match" do portfólio (PerfilController::recalcularMatch()), POST para view/perfil/recalcular_match.php), que mostra a mensagem "Match recalculado: N vaga(s) analisada(s)." O candidato vê a nota e a explicação ("Por que essa nota?") em cada vaga compatível do portfólio e na própria página da vaga; a empresa vê a mesma nota, com as competências atendidas e as que faltam, ao lado de cada candidatura recebida.

Etapa	Arquivo e método	O que acontece
Cálculo da nota	MatchService::calcular(), Competencias::daVaga()/doPerfil()	soma competências, cargo, localização e nível, até 100 pontos
Recálculo em lote	MatchService::recalcular() (por candidato), MatchService::recalcularVaga() (por vaga)	refaz a nota do candidato com todas as vagas, ou da vaga com todos os candidatos
Gravação	MatchDAO::salvar() (tabela matches, coluna detalhes em JSON)	grava a pontuação, o nível (baixo a excelente) e a explicação
Botão manual	PerfilController::recalcularMatch()	refaz o match na hora, a pedido do candidato
Exibição	app/Views/perfil/portfolio.php, app/Views/vagas/detalhe.php, app/Views/admin/candidaturas.php	mostra a nota e a explicação para candidato e empresa

Quadro 16 – Caminho no código do match

Fonte: Autoria própria (2026).

8.11.11 Candidatura e acompanhamento de status (Candidato)

Na página da vaga, o botão de candidatura leva a `candidatar.php?vaga_id=<id>` (CandidaturaController::candidatar()), só para quem está logado como candidato. O controller garante o perfil (PerfilDAO::obterOuCriar()), confere que a vaga existe e está aberta (senão: "Esta vaga não está mais recebendo candidaturas.") e que não existe já uma candidatura ativa para ela (senão: "Você já se candidatou a esta vaga."). No formulário (GET), a pessoa escolhe qual dos seus currículos enviar e pode escrever uma carta de apresentação de até 5000 caracteres; a tela é `app/Views/vagas/candidatar.php`.

No envio (POST), o sistema confere de novo o token CSRF e a permissão do plano (AssinaturaDAO::podeCandidatar()): no plano gratuito, o candidato pode ter até 3 candidaturas ativas (enviada, em análise ou entrevista) ao mesmo tempo; o plano VIP não tem limite. A contagem e a gravação acontecem na mesma transação (CandidaturaDAO::enviarComLimite(), que trava a linha do perfil com FOR UPDATE), para dois envios simultâneos não conseguirem passar do limite. Se o limite já foi atingido, a mensagem é "Você atingiu o limite de 3 candidaturas ativas do Plano Gratuito. Torne-se Candidato VIP para enviar candidaturas ilimitadas!" e a pessoa é levada a `planos.php`. É a mesma mensagem na conferência e na

transação, guardada uma única vez em `AssinaturaDAO::AVISO_LIMITE_CANDIDATURAS`. Dando certo, a mensagem é "Candidatura enviada com sucesso! Acompanhe o status no seu perfil." e a candidatura entra com o status "enviada".

O acompanhamento é feito na própria tela "Meu perfil" (lista de candidaturas do `CandidaturaDAO::listarPorCandidato()`), que mostra o título da vaga, a empresa, a data e o status atual: enviada, em análise, entrevista, aprovado ou rejeitado, status que só a empresa muda (próxima seção), ou cancelada, que só o próprio candidato escolhe (próxima funcionalidade). Quando a empresa muda o status e deixa uma observação, o candidato vê as duas coisas no mesmo lugar.

Etapa	Arquivo e método	O que acontece
Abertura do formulário	<code>public/index.php</code> (rota <code>candidatar.php</code>) chama <code>CandidaturaController::candidatar()</code>	confere login, vaga aberta e se já existe candidatura ativa
Limite do plano	<code>AssinaturaDAO::podeCandidatar()</code>	até 3 candidaturas ativas no plano gratuito; sem limite no VIP
Envio com limite seguro	<code>CandidaturaDAO::enviarComLimite()</code> (tabela candidaturas)	conta e grava na mesma transação, travando a linha do perfil
Acompanhamento	<code>CandidaturaDAO::listarPorCandidato()</code> , <code>app/Views/perfil/index.php</code>	mostra o status e a observação da empresa no perfil do candidato

Quadro 17 – Caminho no código da candidatura
Fonte: Autoria própria (2026).

8.11.12 Cancelamento da candidatura (Candidato)

Só o próprio candidato cancela uma candidatura, pelo botão correspondente na lista de candidaturas do seu perfil, que envia um POST para `view/perfil/candidatura_cancelar.php` (`CandidaturaController::cancelar()`). O cancelamento só é aceito enquanto a candidatura ainda está numa etapa inicial: `CandidaturaDAO::cancelarDoCandidato()` só atualiza o registro quando o status é "enviada" ou "em_analise" e pertence ao perfil do candidato que está pedindo; candidatura já em entrevista, aprovada, rejeitada ou já cancelada não é afetada pela mesma chamada.

Dando certo, a mensagem é "Candidatura cancelada." e a pessoa volta para `view/perfil/index.php#candidaturas`; quando a condição não bate (etapa avançada, candidatura de outro perfil ou id inexistente), a mensagem é "Não foi possível cancelar: a candidatura já está em uma etapa avançada." Uma candidatura cancelada libera a vaga na contagem do limite do plano gratuito e pode ser enviada de novo mais tarde: `CandidaturaDAO::enviarComLimite()` detecta que já existe um registro cancelado para aquele par candidato/vaga e o reaproveita

(atualiza o status para "enviada" de novo, com o currículo e a carta atuais), em vez de criar uma linha duplicada, e é justamente para isso que existe a chave única (candidato, vaga) na tabela.

Etapa	Arquivo e método	O que acontece
Pedido de cancelamento	public/index.php (rota view/perfil/candidatura_cancelar.php) chama CandidaturaController::cancelar()	confere login, tipo de conta e token CSRF
Regra de etapa	CandidaturaDAO::cancelarDoCandidato()	só cancela enviada ou em_analise, e só do próprio candidato
Reenvio posterior	CandidaturaDAO::enviarComLimite()	candidatura cancelada é reaproveitada numa nova candidatura à mesma vaga

Quadro 18 – Caminho no código do cancelamento

Fonte: Autoria própria (2026).

8.11.13 Planos VIP e Premium (Candidato e Empresa)

A tela de planos (planos.php, PlanosController::index()) mostra lado a lado o Candidato VIP (R\$ 9,90) e o Empresa Premium (R\$ 49,90), os dois por 30 dias, sem fidelidade, com o plano atual da conta (se houver), o histórico de assinaturas (AssinaturaDAO::listarPorUsuario()) e as vantagens de cada plano. Como é um trabalho acadêmico, a cobrança é demonstrativa: não existe nenhuma integração com meio de pagamento, a assinatura é simplesmente gravada no banco depois da confirmação.

Ao clicar em assinar (POST com acao=assinar_candidato ou acao=assinar_empresa), o sistema confere se a pessoa está logada (senão: "Faça login para assinar um plano.") e se o tipo da conta combina com o plano (um candidato não assina o Empresa Premium, e vice-versa). AssinaturaDAO::assinar() cancela qualquer assinatura ativa anterior da mesma conta (nunca duas ao mesmo tempo) e grava uma nova, com data de início hoje e data de fim 30 dias depois. A confirmação é festiva: "Parabéns! Sua assinatura VIP de Candidato foi ativada com sucesso por 30 dias." ou a equivalente para a empresa. Cancelar (acao=cancelar) mantém o histórico (AssinaturaDAO::cancelar(), que só muda o status para "cancelada") e, no caso da empresa, tira o destaque das vagas dela na hora (AssinaturaDAO::removerDestaqueSemPremium()), mas as vagas que já estavam abertas continuam abertas, só as próximas publicações passam a respeitar o limite do plano básico.

As regras de cada plano são conferidas sempre no servidor, nunca só na tela: o candidato gratuito tem até 3 candidaturas ativas e vê as três vagas de melhor match; o VIP não tem limite, aparece primeiro na lista da empresa e vê todas as vagas compatíveis, além do selo de VIP no perfil. A empresa básica tem até 2 vagas abertas; a Premium não tem limite, as vagas ficam em destaque na busca e na página inicial, e o banco de talentos abre por completo (nome, contato

e download do currículo). Toda assinatura vencida é marcada como "expirada" automaticamente na primeira consulta de cada requisição (`AssinaturaDAO::expirarAssinaturas()`), sem precisar de nenhuma tarefa agendada.

Etapa	Arquivo e método	O que acontece
Tela de planos	<code>public/index.php</code> (rota <code>planos.php</code>) chama <code>PlanosController::index()</code>	mostra o plano atual, o histórico e as vantagens de cada plano
Assinar	<code>AssinaturaDAO::assinar()</code> (tabela assinaturas)	cancela a assinatura anterior e grava a nova, por 30 dias
Cancelar	<code>AssinaturaDAO::cancelar()</code> , <code>removerDestaqueSemPremium()</code>	mantém o histórico e tira o destaque das vagas da empresa
Conferência das regras	<code>AssinaturaDAO::podeCandidatar()</code> , <code>podePublicarVaga()</code> , <code>isCandidatoVip()</code> , <code>isEmpresaPremium()</code>	aplica os limites e as vantagens de cada plano no servidor
Expiração automática	<code>AssinaturaDAO::expirarAssinaturas()</code>	marca como expirada toda assinatura vencida, a cada requisição

Quadro 19 – Caminho no código dos planos

Fonte: Autoria própria (2026).

8.11.14 Exclusão da própria conta: LGPD (Candidato)

Pensando na LGPD, o próprio candidato apaga a conta inteira em "Meu perfil", seção "Seus dados, Excluir minha conta", que envia um POST para `view/perfil/conta_excluir.php` (`PerfilController::excluirConta()`). A tela pede a senha atual (com a mesma tolerância de digitação e a mesma pausa por tentativas do login, conferidas por `UsuarioDAO::senhaConfere()` e `UsuarioDAO::minutosBloqueioLogin()`) e uma caixa de confirmação explícita.

Sem a caixa marcada, a mensagem é "Marque a caixa de confirmação para excluir a conta."; com a senha errada, "Senha incorreta: a conta não foi excluída." e a tentativa é contada como uma falha de login, para o mesmo mecanismo de pausa valer aqui também. Passando as duas conferências, `UsuarioDAO::excluir()` apaga a linha de usuarios, o que arrasta em cascata (chaves estrangeiras ON DELETE CASCADE) o perfil, os currículos, as candidaturas, os matches, as assinaturas e os pedidos de troca de senha; os arquivos enviados associados (currículos, foto do perfil, imagens de vagas enviadas pela própria conta) são apagados do disco em seguida, e as tentativas de login daquele e-mail também somem. Nenhum dado da pessoa fica para trás.

Depois de excluída a conta, a sessão é encerrada e uma sessão nova e vazia é aberta só para mostrar a confirmação: "Sua conta e todos os seus dados foram excluídos da plataforma." A pessoa é levada à página inicial. Empresas não têm esse botão: pedem a exclusão ao administrador, que a faz pela tela de usuários (próxima seção).

Etapa	Arquivo e método	O que acontece
Pedido de exclusão	public/index.php (rota view/perfil/conta_excluir.php) chama PerfilController::excluirConta()	confere a pausa de login, a senha e a caixa de confirmação
Exclusão em cascata	UsuarioDAO::excluir() (tabela usuarios e FKs ON DELETE CASCADE)	apaga a conta, o perfil, currículos, candidaturas, matches, assinaturas
Arquivos e rastros	apagar_upload_sem_uso(), DELETE FROM tentativas_login	remove os arquivos enviados e as tentativas de login daquele e-mail
Encerramento da sessão	encerrar_sessao(), iniciar_sessao()	fecha a sessão da conta apagada e abre uma nova só para a confirmação

Quadro 20 – Caminho no código da exclusão de conta

Fonte: Autoria própria (2026).

8.11.15 Perfil da empresa

A empresa mantém os próprios dados em admin/pages/empresa_perfil.php (EmpresaController::perfil()), aba "Perfil da empresa" do painel (app/Views/layouts/admin_nav.php), acessível só a contas do tipo empresa. O formulário reúne nome fantasia (que passa a aparecer em todas as vagas publicadas por aquela conta), CNPJ, setor, site, cidade, UF, uma descrição da empresa e o logotipo.

No POST, o sistema confere que o nome fantasia foi informado, que o CNPJ (quando preenchido) tem exatamente 14 dígitos, e já devolve formatado como 00.000.000/0000-00, que o site (quando preenchido) começa com http:// ou https:// (url_http_valida()), para nunca aceitar um link "javascript:") e que a UF tem 2 letras. O logotipo segue a mesma regra das demais imagens do site (JPG/PNG/WEBP até 3 MB, salvar_imagem_enviada()), e a logo antiga só é apagada depois que a nova foi salva com sucesso. PerfilDAO::salvar() grava tudo, sempre com o perfil marcado como público e com o aceite da LGPD renovado (são campos herdados do mesmo formulário usado pelo candidato); o telefone, como no perfil do candidato, é gravado aparte em usuarios.

Dando certo, a mensagem é "Perfil da empresa atualizado. Esse nome aparece nas suas vagas."; qualquer erro de validação impede o salvamento e mostra a lista de problemas. A tela é app/Views/admin/empresa_perfil.php.

Etapa	Arquivo e método	O que acontece
Tela do perfil	public/index.php (rota admin/pages/empresa_perfil.php) chama EmpresaController::perfil()	exigirLogin() mais a conferência isEmpresa()
Validação	EmpresaController::perfil(), url_http_valida()	confere nome fantasia, CNPJ (14 dígitos), site e UF

Etapa	Arquivo e método	O que acontece
Logotipo	salvar_imagem_enviada(), apagar_upload_sem_uso()	troca a logo só depois que o perfil é salvo com a nova
Gravação	PerfilDAO::salvar() (tabela perfis), UsuarioDAO::atualizarTelefone() ()	grava os dados da empresa e o telefone da conta

Quadro 21 – Caminho no código do perfil da empresa
Fonte: Autoria própria (2026).

8.11.16 Publicação de vaga por formulário, por anúncio e por cartaz: OCR no navegador (Empresa)

A tela de vagas do painel (admin/pages/vagas.php, EmpresaController::vagas()) é tanto o CRUD de vagas quanto a máquina de extração de vagas. A empresa (ou o administrador, por qualquer empresa) tem três jeitos de preencher o formulário: digitando cada campo manualmente; colando na caixa "Colar texto" o anúncio recebido pelo WhatsApp, Instagram ou de um site; ou enviando a foto do cartaz da vaga ("Ler cartaz"). Nos dois últimos casos, nada é salvo direto, a extração só preenche o formulário para revisão.

Quando a pessoa cola o texto e pede para extrair (acao=extrair), ExtracaoVaga::doTexto() aplica as regras escritas a partir de dezenas de cartazes reais do DF e separa cargo, empresa anunciante, salário (sem confundir com vale-transporte ou vale-refeição), cidade, tipo de contratação, nível, modelo de trabalho, contato, quantidade de vagas e a área, junto com a descrição, os requisitos e os benefícios. Onde a regra não acha uma pista, uma linha sem palavra-chave conhecida, uma área ou uma empresa que a regra não reconhece, entra o Sistema por meio dos padrões automáticos (PadroesExtracao, detalhados em sua própria seção). ExtracaoVaga::relatorio() monta, campo a campo, o que foi lido do anúncio, o que ficou com o valor padrão (ex.: nível "Júnior" quando o anúncio não diz) e o que não foi encontrado, junto com os padrões automáticos usados.

Quando a pessoa envia o cartaz (acao=ler_cartaz), a imagem é validada e salva (salvar_imagem_enviada(), até 8 MB) e se torna, desde já, a imagem da vaga no formulário. A leitura do texto é feita pelo próprio navegador de quem envia, com o Tesseract.js (OCR em WebAssembly) servido pelo site, em quatro leituras diferentes (público/assets/js/leitor-cartaz.js); o resultado em TSV viaja junto com o cartaz e o servidor confere e aplica a mesma calibragem em OcrImagem::leiturasDoNavegador()/montar(), uma leitura forjada ou inválida é ignorada, e nesse caso o servidor lê sozinho com o Tesseract instalado nele, se houver (OcrImagem::disponivel()). Com o texto em mãos (do navegador ou do servidor),

ExtracaoVaga::doImagem() aplica as mesmas regras do texto colado. O texto lido fica numa caixa editável, para corrigir um erro do OCR e extrair de novo sem precisar reenviar o cartaz.

Terminada a extração (por texto ou por cartaz), o formulário aparece preenchido para a empresa revisar e só então publicar (o botão de salvar de fato). Ao salvar, o sistema confere o título obrigatório, a UF, o intervalo de salário (mínimo não maior que o máximo) e a data de expiração (não pode já ter passado para uma vaga que entra como ativa); o campo "Destaque" só é aceito de contas Premium ou do administrador. A imagem definitiva pode ser um arquivo enviado agora, o cartaz que acabou de ser lido, a imagem que a vaga já tinha, ou um caminho de assets/img (EmpresaController::imagemDoFormulario()). Cartazes lidos na sessão e não aproveitados em nenhuma vaga são apagados ao sair da tela ou ao extrair de novo (EmpresaController::descartarCartazesLidos()).

Etapa	Arquivo e método	O que acontece
Tela de vagas	public/index.php (rota admin/pages/vagas.php) chama EmpresaController::vagas()	CRUD de vagas e as três formas de preencher o formulário
Extração por texto	ExtracaoVaga::doTexto(), PadroesExtracao::decidir()	separa os campos do anúncio colado e usa os padrões automáticos onde falta pista
OCR do cartaz	public/assets/js/leitor-cartaz.js (Tesseract.js), OcrImagem::leiturasDoNavegador()/montar()	lê o texto no navegador e confere a leitura no servidor
Extração por imagem	ExtracaoVaga::doImagem()	aplica as mesmas regras de texto ao conteúdo lido do cartaz
Relatório da extração	ExtracaoVaga::relatorio()	mostra o que foi lido, o que ficou no padrão e o que faltou
Publicação	VagaDAO::salvar()/salvarComLimite() (tabela vagas)	grava a vaga só depois da revisão da empresa

Quadro 22 – Caminho no código da publicação de vaga
Fonte: Autoria própria (2026).

8.11.17 Aviso de vaga parecida (Empresa)

Antes de publicar (ou de reativar) uma vaga como ativa, VagaDAO::buscarParecida() procura, entre as vagas já abertas da mesma empresa, uma com o mesmo título, o mesmo anunciante e a mesma cidade (sem diferenciar maiúsculas nem acentos, pela collation do banco). O objetivo é simples: evitar que o mesmo cartaz ou o mesmo anúncio, lido ou digitado duas vezes, vire duas vagas publicadas.

Encontrando uma vaga parecida, o EmpresaController::vagas() acrescenta aos erros do formulário a mensagem "Já existe uma vaga aberta igual: #<id>, <título>. Se for outra vaga, marque 'publicar mesmo assim'." e não salva nada, a tela volta com os dados

preenchidos para a pessoa decidir. Se for realmente outra vaga (duas lojas diferentes com o mesmo cargo na mesma cidade, por exemplo), a caixa "publicar mesmo assim" (campo `publicar_duplicada`) pula essa conferência só para aquele envio.

Etapa	Arquivo e método	O que acontece
Conferência antes de publicar	<code>EmpresaController::vagas()</code>	roda só quando a vaga está entrando ou continuando como ativa
Busca da vaga parecida	<code>VagaDAO::buscarParecida()</code> (tabela <code>vagas</code>)	mesmo título, anunciante e cidade entre as vagas já abertas
Decisão da empresa	campo <code>publicar_duplicada</code> do formulário	pula o aviso quando a empresa confirma que é mesmo outra vaga

Quadro 23 – Caminho no código do aviso de vaga parecida
Fonte: Autoria própria (2026).

8.11.18 Ativar, pausar e encerrar a vaga, e o limite do plano (Empresa)

Cada vaga na lista do painel tem botões para ativar, pausar ou encerrar com um clique só (`painel_botoes()`, em `app/Core/helpers.php`), cada um um formulário POST com token CSRF para `admin/pages/vagas.php`, tratado por `EmpresaController::vagas()` nas ações `acao=ativar`, `acao=pausar` e `acao=encerrar`. Pausar tira a vaga da busca pública sem apagar nada ("Vaga pausada: saiu da busca, mas continua salva."); encerrar marca o fim definitivo daquela vaga ("Vaga encerrada."); ativar reabre uma vaga pausada, encerrada ou com o prazo vencido.

O limite do plano básico (2 vagas abertas por empresa) só é conferido quando a vaga está entrando como ativa e ainda não estava aberta antes, ou seja, ao publicar uma vaga nova, ao reativar uma pausada/encerrada e ao renovar uma vaga vencida com uma data nova; uma vaga que já estava aberta e só teve outro campo editado não entra nessa conta de novo. A contagem e a gravação acontecem na mesma transação, com a linha do perfil da empresa travada por `FOR UPDATE` (`VagaDAO::alterarStatus()` para o botão de um clique, `VagaDAO::salvarComLimite()` para o formulário completo), para duas publicações simultâneas não conseguirem passar do limite. Batendo no limite, a mensagem é "Sua empresa atingiu o limite de 2 vagas ativas do Plano Básico Gratuito. Assine o Plano Empresa Premium para publicar vagas ilimitadas!" e a pessoa é levada a `planos.php`; a empresa Premium e o administrador nunca têm limite (`AssinaturaDAO::podePublicarVaga()`).

Toda vez que uma vaga entra ou volta a ficar ativa, o Sistema recalcula o match dela com todos os candidatos (`MatchService::recalcularVaga()`), e a mensagem de sucesso já informa quantos candidatos foram analisados (ex.: "Vaga ativada. Match calculado com 12 candidato(s)."). Se a vaga tem uma data de expiração que já passou, a mensagem ainda avisa

para editar a vaga e renovar a data, senão ela não volta a aparecer na busca mesmo com o status "ativa".

Etapa	Arquivo e método	O que acontece
Botão de um clique	app/Views/admin/vagas.php (painel_botoes()), POST para admin/pages/vagas.php	cada ação é um formulário próprio, com token CSRF
Mudança de status	EmpresaController::vagas(), VagaDAO::alterarStatus()	troca entre ativa, pausada e encerrada
Limite do plano básico	AssinaturaDAO::podePublicarVaga(), VagaDAO::salvarComLimite()/alterarStatus()	conta e grava na mesma transação, travando a linha do perfil da empresa
Match em dia	MatchService::recalcularVaga()	recalcula a compatibilidade da vaga com todos os candidatos ao reativar

Quadro 24 – Caminho no código de ativar, pausar e encerrar
Fonte: Autoria própria (2026).

8.11.19 Candidaturas recebidas, ordenadas pelo match (Empresa)

A tela admin/pages/candidaturas.php (EmpresaController::candidaturas()) reúne as candidaturas recebidas pela empresa (ou, para o administrador, de toda a plataforma), com filtro por vaga e por status e opções de ordem: a padrão é "Relevância (VIP e match)", candidatos VIP primeiro, depois do maior match para o menor (já assim na própria consulta SQL, CandidaturaDAO::listarPorEmpresa()/listarTodas()), e ainda dá para ordenar por maior match, mais recentes, mais antigas ou nome do candidato (A–Z).

Cada linha mostra o nome do candidato (ou os dados mascarados, conforme a visibilidade do perfil), o título profissional, a nota de match com as competências atendidas e as que faltam, o currículo enviado (para abrir ou baixar) e um menu para mudar o status: enviada, em análise, entrevista, aprovado ou rejeitado, a empresa NUNCA marca "cancelada" (CandidaturaDAO::STATUS_EMPRESA deixa essa opção de fora; cancelar é só do candidato) e também não altera uma candidatura que o candidato já cancelou. Cada mudança pode vir com uma observação de até 2000 caracteres, que o candidato lê junto com o novo status no próprio perfil. Só o administrador exclui candidaturas (acao=excluir); para a empresa essa opção simplesmente não aparece como válida.

Dando certo, a mensagem é "Candidatura atualizada. O candidato vê o novo status e o retorno no perfil dele."; não sendo possível (candidatura inexistente, de outra empresa, ou já cancelada pelo candidato), a mensagem explica exatamente isso. A tela é app/Views/admin/candidaturas.php.

Etapa	Arquivo e método	O que acontece
Lista ordenada	public/index.php (rota admin/pages/candidaturas.php) chama EmpresaController::candidaturas()	CandidaturaDAO::listarPorEmpresa()/listarTodas(), já ordenadas por VIP e match
Mudança de status	CandidaturaDAO::atualizarStatusPorEmpresa()/atualizarStatus()	empresa nunca marca cancelada; administrador pode qualquer status
Exclusão	CandidaturaDAO::excluir()	só o administrador
Tela	app/Views/admin/candidaturas.php	filtro por vaga e status, nota de match e botão de status

Quadro 25 – Caminho no código das candidaturas recebidas

Fonte: Autoria própria (2026).

8.11.20 Banco de talentos e download de currículo (Empresa)

O Banco de Talentos (admin/pages/talentos.php, EmpresaController::talentos()) reúne os candidatos com perfil público (PerfilDAO::listarTalentos()), com busca por termo e por cidade, e ordenação por relevância (VIP primeiro, depois os mais recentes), cargo ou cidade; só quem tem Premium pode ordenar pelo nome, porque é o único plano que vê o nome de verdade. Sem o plano Premium, a busca por termo também é mais restrita: ela só confere o cargo, as habilidades, as competências, as experiências e os cursos do candidato, nunca o nome nem os links, assim a busca não é usada para confirmar se uma pessoa específica tem conta ali.

O acesso completo (nome, telefone, e-mail e o botão de baixar o currículo) é um recurso do plano Empresa Premium; no plano básico, os cartões aparecem com o nome mascarado e sem contato, a mesma prévia usada no portfólio. O download do currículo, em qualquer caso, passa pelo ArquivoController::download() (rota download.php?id=), que confere se quem está pedindo é o próprio candidato, o administrador, uma empresa que já recebeu candidatura com aquele currículo (CurrículoDAO::enviadoParaEmpresa()) ou uma empresa Premium abrindo o currículo de um perfil público e ativo; fora isso, a resposta é "Acesso restrito" (403). O arquivo é entregue com o cabeçalho certo (PDF abre no navegador; DOC/DOCX sempre baixam) e, quando quem abre não é o próprio dono, o sistema registra mais uma visualização (CurrículoDAO::registrarDownload()).

Etapa	Arquivo e método	O que acontece
Lista de talentos	public/index.php (rota admin/pages/talentos.php) chama EmpresaController::talentos()	PerfilDAO::listarTalentos(), com busca restrita sem o Premium
Prévia ou completo	AssinaturaDAO::isEmpresaPremium()	decide entre nome/contato completos ou dados mascarados

Etapa	Arquivo e método	O que acontece
Download do currículo	public/index.php (rota download.php) chama <code>ArquivoController::download()</code>	confere dono, admin, empresa que recebeu a candidatura ou Premium
Entrega do arquivo	<code>ArquivoController::download()</code> , <code>CurriculoDAO::registrarDownload()</code>	envia o arquivo com o cabeçalho certo e conta a visualização

Quadro 26 – Caminho no código do banco de talentos e do download
Fonte: Autoria própria (2026).

8.11.21 Painel e gráficos (Administrador)

A primeira tela depois do login do administrador (ou da empresa) é `admin/index.php` (`AdminController::painel()`), aba "Visão geral" do painel. Para o administrador, os números são de toda a plataforma; para a empresa, só das próprias vagas e candidaturas (o controller já filtra pelo `perfil_empresa_id` em todas as consultas). Os indicadores vêm sempre já agrupados no próprio SQL (`GROUP BY` nos DAOs), nunca com uma consulta por linha: `VagaDAO::resumoPainel()` (totais de vagas por situação), `CandidaturaDAO::contarPorStatus()/contarPorDia()/resumoMatch()` (candidaturas por status, por dia nos últimos 30 dias e por faixa de match) e, só para o administrador, `UsuarioDAO::contarPorTipo()`, `VagaDAO::contarPorCategoria()/contarPorCidade()`, `MatchDAO::resumoVagasAbertas()` e `AssinaturaDAO::resumoPorPlano()`.

Os gráficos (`app/Views/partials/graficos.php`) são desenhados em SVG simples pelo próprio PHP, sem nenhuma biblioteca de gráficos: candidaturas por dia (linha), usuários novos e vagas por área (barras), situação das candidaturas (funil: Recebidas, Analisadas, Entrevista, Aprovadas, cada etapa contando quem chegou nela ou passou dela, sem contar as canceladas) e assinaturas por plano. A empresa vê, no lugar dos gráficos gerais, o "Desempenho" de cada vaga dela (`VagaDAO::desempenhoPorEmpresa()`): candidaturas recebidas, quantas aguardam análise, visualizações e o match médio de quem se candidatou.

Só para o administrador, é nesta tela que o Sistema confere e roda, no máximo uma vez por dia, a manutenção automática (`manutencao_diaria()`, explicada em sua própria seção), por isso ela só acontece quando alguém com acesso ao painel completo abre essa página, nunca na área pública do site.

Etapa	Arquivo e método	O que acontece
Abertura do painel	public/index.php (rota admin/index.php) chama <code>AdminController::painel()</code>	administrador vê tudo; empresa vê só as próprias vagas e candidaturas

Etapa	Arquivo e método	O que acontece
Indicadores agrupados	VagaDAO::resumoPainel(), CandidaturaDAO::contarPorStat us()/contarPorDia(), UsuarioDAO::contarPorTipo()	consultas com GROUP BY, sem uma query por linha
Gráficos em SVG	app/Views/partials/graficos.php	desenha linha, barras e funil sem biblioteca externa
Manutenção diária	manutencao_diaria() (app/Core/Upload.php)	roda no máximo 1x por dia, só quando o administrador abre o painel

Quadro 27 – Caminho no código do painel e dos gráficos
Fonte: Autoria própria (2026).

8.11.22 Manutenção de usuários (Administrador)

admin/pages/usuarios.php (AdminController::usuarios()) é o CRUD completo das contas: administrador, candidato e empresa, com busca por nome/e-mail, filtro por tipo e por situação (ativo/bloqueado), colunas que ordenam com um clique e paginação de 25 por página. O "Ver" (?ver=<id>) mostra a ficha da conta com os números dela (UsuarioDAO::resumo(): total de vagas, candidaturas e currículos, e o plano ativo); o "Editar" (?edit=<id>) abre o mesmo formulário do cadastro.

Duas travas valem sempre, tanto no formulário quanto nos botões de ativar/bloquear/excluir: o administrador nunca consegue excluir, bloquear ou remover o próprio acesso de administrador (as mensagens são específicas: "Você não pode excluir a própria conta.", "Você não pode bloquear a própria conta." e "Você não pode remover seu próprio acesso de administrador."), e o sistema nunca deixa a plataforma sem nenhum administrador ativo, UsuarioDAO::contarAdminsAtivos() é conferido antes de excluir, bloquear ou trocar o tipo/situação do último administrador ("É preciso manter pelo menos um administrador ativo."). Criar ou editar exige nome, e-mail válido e, para conta nova (ou troca de senha), 6 a 72 caracteres de senha; e-mail duplicado é recusado pela chave única da tabela.

Excluir um usuário (UsuarioDAO::excluir()) arrasta em cascata o perfil, os currículos, as vagas, as candidaturas e os matches dele, além dos arquivos enviados e das tentativas de login daquele e-mail, a mesma limpeza completa que o próprio candidato dispara ao excluir a própria conta. Ativar devolve o acesso ao login ("Conta ativada: o login volta a funcionar."); bloquear tira o acesso imediatamente, mesmo de uma sessão já aberta, porque revalidar_sessao() confere a conta ativa a cada requisição ("Conta bloqueada: o login e a sessão aberta perdem o acesso."). A tela é app/Views/admin/usuarios.php.

Etapa	Arquivo e método	O que acontece
Lista e ficha	public/index.php (rota admin/pages/usuarios.php) chama AdminController::usuarios()	UsuarioDAO::listar()/resumo(), com busca, filtro e paginação
Travas de segurança	UsuarioDAO::contarAdminsAtivos(), AdminController::usuarios()	nunca a própria conta, nunca o último administrador ativo
Criar e editar	UsuarioDAO::cadastrar()/atualizar() (tabela usuarios)	cria o perfil junto, quando o tipo é candidato ou empresa
Ativar e bloquear	UsuarioDAO::alterarAtivo()	bloquear tira o acesso da sessão aberta na próxima requisição
Exclusão	UsuarioDAO::excluir()	apaga a conta e tudo o que é dela, em cascata

Quadro 28 – Caminho no código da manutenção de usuários
Fonte: Autoria própria (2026).

8.11.23 Categorias (Administrador)

admin/pages/categorias.php (AdminController::categorias()) cadastra as áreas usadas para agrupar tanto vagas quanto cursos; o campo "tipo" de cada categoria (CategoriaDAO::TIPOS: vaga ou curso) decide onde ela aparece. A lista mostra quantos registros usam cada categoria (coluna em_uso, calculada na própria consulta de CategoriaDAO::listar()) e tem filtro por tipo e por nome.

Uma regra protege a consistência dos dados: uma categoria que já está em uso não pode trocar de tipo (de "Vagas" para "Cursos" ou o contrário), CategoriaDAO::salvar() confere isso antes de gravar e devolve "Esta categoria está em uso: não é possível trocar entre 'Vagas' e 'Cursos'. Crie uma categoria nova para o outro tipo."; o nome duplicado para o mesmo tipo é recusado pela chave única ("Já existe uma categoria com esse nome para esse tipo."). Excluir uma categoria (CategoriaDAO::excluir()) não apaga as vagas ou cursos que a usavam: a chave estrangeira está configurada como ON DELETE SET NULL, então eles simplesmente ficam sem categoria. Desativar (em vez de excluir) tira a categoria dos filtros da área pública, mas os itens que já a usam continuam com ela.

Etapa	Arquivo e método	O que acontece
Lista e CRUD	public/index.php (rota admin/pages/categorias.php) chama AdminController::categorias()	CategoriaDAO::listar()/salvar()/excluir() (tabela categorias)
Trava de tipo em uso	CategoriaDAO::salvar()	impede trocar entre "Vagas" e "Cursos" quando a categoria já está em uso
Exclusão sem quebrar vínculos	CategoriaDAO::excluir() (FK ON DELETE SET NULL)	vagas e cursos ficam sem categoria, em vez de serem apagados

Quadro 29 – Caminho no código das categorias
Fonte: Autoria própria (2026).

8.11.24 Cursos e e-books por ficha (uma e várias), imagem padrão e capa do PDF (Administrador)

`admin/pages/cursos.php (AdminController::cursos())` é o CRUD de cursos, e-books e vídeos, com uma única caixa de extração ("Extrair"), que aceita colar tanto uma ficha de pesquisa (os campos Título, Tipo, Instituição, Modalidade, Cidade, Nível, Carga horária, Gratuito, Preço, Área, Link, PDF, Imagem e Descrição, no formato de `PromptsPesquisa::formatoFicha()` quanto um texto de divulgação qualquer. Antes de interpretar o texto, o sistema tira as marcas de citação que as ferramentas de pesquisa (ChatGPT, Gemini, Perplexity e outras) costumam deixar na resposta, como [1] ou **【4†fonte】**.

Com UMA ficha, `ExtracaoCurso::fichas()` reconhece o formato (pelo "Título:" no começo do texto) e devolve um único item, que entra direto no formulário para revisão (`ImagemRemota::completar()` já confere o link da imagem). Com VÁRIAS fichas separadas por "---", a prévia entra em cena: `ImagemRemota::completar()` processa até 100 fichas de uma vez e cada item ganha uma lista de problemas (sem título, sem link válido, já cadastrado) e de alertas (sem imagem: entra com a padrão; sem imagem própria: entra com o banner da instituição; capa tirada da 1ª página do PDF; pago sem preço: entra como gratuito), guardados em `$_SESSION['import_cursos']`. O administrador marca os itens que quer e confirma (`acao=importar_salvar`), e cada um passa pelas mesmas regras do formulário individual (título, link `http/https`, sem repetir por url ou por título+instituição) antes de ser gravado.

A imagem nunca impede o cadastro, seja no formulário único ou no lote. `ImagemRemota::completar()/baixar()` abrem cada link da ficha (o da imagem, o do PDF e o do próprio conteúdo) e descobrem o que cada um é: imagem de verdade vira a imagem do conteúdo; PDF sem imagem dispara a `CapaPdf::gerar()`, que desenha a 1ª página inteira com o leitor de PDF do próprio Windows, chamado via PowerShell (`capa-pdf.ps1`), ou, sem isso, aproveita a maior imagem já desenhada na 1ª página (`PdfTexto::capa()`); link de página procura nela a imagem de divulgação (metatag `og:image`), descartando logotipos e ícones. Sem nenhuma imagem que sirva, o conteúdo entra com o banner da instituição (`ExtracaoCurso::capa()`) ou, na falta dele, com a imagem padrão da plataforma (`CursoDAO::imagemPadrao()`, em `public/assets/img/padrao/`), e a lista do painel marca esse item no filtro "Com imagem padrão" (`situacao=imagem_padrao`, `CursoDAO::ehImagemPadrao()`), para o administrador lembrar de completar o cadastro depois em Editar, enviando um arquivo, colando o link direto da imagem ou escolhendo um caminho já existente no site.

Etapa	Arquivo e método	O que acontece
Caixa única de extração	public/index.php (rota admin/pages/cursos.php) chama AdminController::cursos()	reconhece uma ficha, várias fichas ou um texto de divulgação qualquer
Uma ficha	ExtracaoCurso::fichas()/doTexto(), ImagemRemota::completar()	preenche o formulário único para revisão
Várias fichas	\$_SESSION['import_cursos'], acao=importar_salvar	monta a prévia em lote e cadastra só os itens marcados
Imagem e capa do PDF	ImagemRemota::completar()/baixar(), CapaPdf::gerar(), PdfTexto::capa()	baixa a imagem do link ou tira a capa da 1ª página do PDF
Imagem padrão	CursoDAO::imagemPadrao()/ehImagemPadrao()	entra com o banner da instituição ou a imagem padrão, nunca bloqueia o cadastro
Filtro de pendências	AdminController::cursos() (situacao=imagem_padrao)	lista os conteúdos que ainda precisam de uma imagem definitiva

Quadro 30 – Caminho no código do cadastro por ficha
Fonte: Autoria própria (2026).

8.11.25 Biblioteca de PDFs (Administrador)

A biblioteca é o conjunto de PDFs de e-books guardados no próprio site, em storage/uploads/biblioteca_*.pdf, reconhecidos pela função eh_pdf_biblioteca() (só aceita esse padrão de nome: um currículo de candidato, por exemplo, nunca entra nessa lista). Um PDF chega à biblioteca de três formas: a empresa... quer dizer, o administrador envia o arquivo direto no formulário do curso (salvar_pdf_biblioteca(), conferido pelo conteúdo, assinatura %PDF-, , até MAX_PDF_BIBLIOTECA, 25 MB); marca "Guardar o PDF na nossa biblioteca" e o sistema baixa da web o PDF do campo, do link oficial ou de dentro da página (ImagemRemota::pdfs(), até MAX_PDF_REMOTO, 150 MB, só de servidor público); ou, para todos os e-books de uma vez, pelo botão "Trazer os PDFs para a biblioteca" (acao=pdfs_biblioteca).

Esse botão roda ImagemRemota::pdfs() em todos os e-books que ainda abrem o PDF no site de origem (url_http_valida() mais o tipo ebook) e, para cada um que conseguir baixar, chama CursoDAO::guardarNaBiblioteca(): o link do conteúdo passa a apontar para o arquivo local e a descrição ganha a linha "Fonte original: <link>", preservando o crédito. A mensagem final resume quantos PDFs entraram na biblioteca e quantos continuaram com "Acessar" porque o link não levou a um PDF público. Um e-book com o PDF na biblioteca mostra o botão "Baixar" em vez de "Acessar" (pt_acesso_conteudo()) e o arquivo é entregue pelo ArquivoController::imagem(), a mesma rota que entrega fotos e logos, só que liberando esse um tipo de PDF para o público.

Etapa	Arquivo e método	O que acontece
Envio direto	salvar_pdf_biblioteca() (AdminController::cursos())	confere o conteúdo do arquivo e grava em storage/uploads/biblioteca_*.pdf
Download de um link	ImagemRemota::pdfs()	baixa o PDF do campo, do link oficial ou de dentro da página
Lote de todos os e-books	acao=pdfs_biblioteca, CursoDAO::guardarNaBiblioteca()	troca "Acessar" por "Baixar" em todos os e-books que ainda estão na web
Entrega pública	eh_pdf_biblioteca(), ArquivoController::imagem()	libera só esse padrão de PDF; currículos nunca saem por essa rota

Quadro 31 – Caminho no código da biblioteca de PDFs
Fonte: Autoria própria (2026).

8.11.26 Assinaturas (Administrador)

admin/pages/assinaturas.php (AdminController::assinaturas()) é a tela exclusiva do administrador para acompanhar e ajustar os planos: lista todas as assinaturas (AssinaturaDAO::listarTodas()) com filtro por plano, status e busca, mostra um resumo por plano (vigentes, canceladas, expiradas e a receita demonstrativa das vigentes) e permite conceder, editar, cancelar e excluir.

"Conceder" (acao=conceder) dá o plano certo para o tipo da conta, candidato recebe Candidato VIP, empresa recebe Empresa Premium (AssinaturaDAO::PLANO_DO_TIPO), por um número de dias escolhido (1 a 3660) e um valor (em branco, usa o preço padrão do plano, AssinaturaDAO::PRECOS); se a conta já tinha uma assinatura ativa, ela é cancelada automaticamente e o histórico é preservado, com um aviso na mensagem de sucesso. "Editar" muda valor, datas e situação diretamente, com as mesmas regras de consistência (fim não antes do início; para ficar "ativa" o fim precisa ser hoje ou depois) e, se a empresa perder o Premium nessa edição, o destaque das vagas dela sai na mesma hora. "Cancelar" mantém o registro no histórico; "Excluir" apaga a linha por completo. Cada conta tem no máximo uma assinatura ativa, em qualquer um desses caminhos.

Etapa	Arquivo e método	O que acontece
Lista e resumo	public/index.php (rota admin/pages/assinaturas.php) chama AdminController::assinaturas()	AssinaturaDAO::listarTodas()/resumoPorPlano()
Conceder	AssinaturaDAO::assinar()	dá o plano certo para o tipo da conta, por N dias, e cancela a anterior
Editar	AssinaturaDAO::atualizar()	valor, datas e situação, com as regras de consistência
Cancelar e excluir	AssinaturaDAO::cancelarPorId())excluir()	cancelar preserva o histórico; excluir apaga a linha

Quadro 32 – Caminho no código das assinaturas
Fonte: Autoria própria (2026).

8.11.27 Padrões automáticos das máquinas de extração (Sistema)

Os padrões automáticos (classe `PadroesExtracao`, `app/Services/Extracao/PadroesExtracao.php`) são a parte do Sistema que aprende sozinha com o que já está cadastrado, sem nenhuma tabela própria, sem tela de ajuste e sem nenhum treinamento: eles são contados de novo a cada requisição, direto das vagas e dos cursos que já passaram pela revisão de uma pessoa. As regras escritas no código (`ExtracaoVaga`, `ExtracaoCurso`) continuam mandando sempre; os padrões só decidem quando a regra não tem nenhuma pista, uma linha do anúncio sem palavra-chave conhecida, uma área que a regra não reconhece no título, ou uma empresa/instituição que a regra não acha no texto.

O funcionamento, por dentro: `PadroesExtracao::termos()` normaliza cada texto por caractere (sem acento, minúsculo, sem pontuação, todo número trocado por "#") e extrai palavras soltas e pares de palavras vizinhas, pulando as vazias do meio (por isso "café da manhã" vira o par "cafe manha"). `PadroesExtracao::montar()` conta, para cada termo, em que campo ele mais aparece nas vagas salvas (descrição, requisitos ou benefícios) e em que área ficam os títulos de vaga e de curso já cadastrados; um termo só se torna padrão de verdade quando aparece em pelo menos 2 cadastros diferentes (`MIN_CADASTROS`) e, em pelo menos 90% deles (`PUREZA`), cai no mesmo destino. Na hora da extração, `PadroesExtracao::decidir()` procura no texto da linha o padrão mais específico que casa com ele, um par de palavras vale mais que uma palavra solta, e empate é resolvido pelo padrão visto em mais cadastros, e só sobrepõe o palpite da regra quando o padrão aponta para outro destino.

Os padrões guardam ainda os nomes conhecidos (`PadroesExtracao::nomes()`): as empresas (o anunciante das vagas e o nome fantasia das contas de empresa) e as instituições dos cursos, sempre sem os nomes genéricos demais para identificar alguém, como "Loja" ou "Empresa" (`PadroesExtracao::nomeGenerico()`). Quando a regra não encontra o nome da empresa num cartaz novo, `PadroesExtracao::nomeConhecido()` procura no texto algum desses nomes já cadastrados, testando primeiro os mais longos. Cada decisão tomada por um padrão aparece no relatório da extração de vaga, no bloco "Padrões automáticos usados", com o termo que valeu, nada é decidido escondido. O currículo do candidato nunca entra nessa conta, porque tem dados pessoais (LGPD): a extração de currículo segue só as regras. Se o banco estiver fora do ar, a extração simplesmente segue só com as regras, sem quebrar.

Etapa	Arquivo e método	O que acontece
Normalização e termos	<code>PadroesExtracao::termos()</code>	tira acento, minúsculas, pontuação; número vira "#"; monta palavras e pares

Etapa	Arquivo e método	O que acontece
Montagem dos padrões	PadroesExtracao::montar()/padroes() (consulta vagas, cursos, categorias)	conta destino por termo; só vale com 2+ cadastros e 90% de pureza
Decisão na extração	PadroesExtracao::decidir(), ExtracaoVaga::doTexto()/relatorio()	só decide quando a regra não tem pista; aparece no relatório
Nomes conhecidos	PadroesExtracao::nomes()/nomeConhecido()/nomeGenerico()	reconhece empresas e instituições já cadastradas no texto novo

Quadro 33 – Caminho no código dos padrões automáticos
Fonte: Autoria própria (2026).

8.11.28 Limpeza diária de arquivos (Sistema)

Toda vez que o administrador abre a visão geral do painel (AdminController::painel()), o Sistema confere se já fez a manutenção do dia (manutencao_diaria(), em app/Core/Upload.php): um arquivo-marca em storage/logs/manutencao_diaria.txt guarda o horário da última vez, e a limpeza só roda de novo depois de 24 horas, nunca mais de uma vez por dia e nunca pela área pública do site.

A limpeza em si (limpar_uploads_orfaos()) varre storage/uploads e apaga todo arquivo que: (1) nenhuma linha do banco referencia mais, nem como imagem de vaga, nem como imagem ou PDF de curso, nem como foto de perfil, nem como arquivo de currículo (a mesma conferência de apagar_upload_sem_uso(), só que para a pasta inteira de uma vez), e (2) tem mais de 24 horas, para nunca apagar algo que está sendo usado àquela hora mesmo, como a foto de um currículo recém-lido esperando o candidato decidir se quer trocar. São sobras de contas excluídas, de cartazes lidos e não aproveitados em nenhuma vaga, ou de capas tiradas de fichas de curso que acabaram não sendo salvas. Qualquer erro durante a limpeza é só registrado no log de erros do PHP (error_log()) e nunca interrompe a página do painel.

Etapa	Arquivo e método	O que acontece
Disparo	AdminController::painel() chama manutencao_diaria()	só quando o administrador abre a visão geral do painel
Controle de frequência	storage/logs/manutencao_diaria.txt	no máximo uma vez por dia
Limpeza de verdade	limpar_uploads_orfaos() (app/Core/Upload.php)	apaga arquivo sem nenhum uso no banco e com mais de 24 horas

Quadro 34 – Caminho no código da limpeza diária
Fonte: Autoria própria (2026).

8.11.29 Doação por Pix (Sistema)

No rodapé de todas as páginas, e numa das mensagens do painel relâmpago da página inicial, existe um convite discreto para ajudar a manter o site no ar com uma doação por Pix

(app/Views/partials/doacao.php). Nada disso depende de nenhum serviço externo: o próprio Sistema monta o código "copia e cola" e o navegador desenha o QR Code sozinho.

Pix::doacao() lê a chave Pix de config/config.php (constante DOACAO_PIX_CHAVE, vazia por padrão, sem ela configurada, o espaço do QR mostra "Pix em breve" em vez de travar a página) e monta o payload no formato EMV do Banco Central (Pix::payload()): cada campo é um identificador de 2 dígitos mais o tamanho mais o valor, com o nome e a cidade do recebedor (DOACAO_NOME e DOACAO_CIDADE, sem acento, porque o BR Code só aceita ASCII, Pix::ascii() faz essa conversão) e, no final, o CRC16 (polinômio 0x1021, início 0xFFFF, calculado por Pix::crc16()) do texto inteiro, exatamente como o aplicativo do banco confere antes de aceitar o Pix. Como não existe valor fixo, quem doa escolhe o valor no próprio aplicativo do banco.

Esse mesmo texto alimenta duas coisas na tela: o QR Code, desenhado no navegador pela biblioteca assets/js/vendor/qrcode.js (licença MIT, atributo [data-qrcode] lido por app.js), e o botão "Copiar código Pix" (cv_doacao_copiar()), para quem prefere colar o código direto no aplicativo do banco em vez de escanear. Nenhum dado da doação passa pelo nosso servidor: o Sistema só entrega o texto pronto, e o pagamento inteiro acontece dentro do aplicativo bancário de quem doa.

Etapas	Arquivo e método	O que acontece
Exibição	app/Views/partials/doacao.php (rodapé de todas as páginas)	mostra o convite, o QR Code e o botão de copiar
Montagem do código	Pix::doacao()/payload() (config/config.php: DOACAO_PIX_CHAVE)	monta o BR Code no formato EMV do Banco Central
Checksum	Pix::crc16()	calcula o CRC16 exigido pelo app do banco para aceitar o código
QR no navegador	public/assets/js/vendor/ qrcode.js, app.js ([data-qrcode])	desenha o QR Code a partir do texto, sem nenhum serviço externo

Quadro 35 – Caminho no código da doação por Pix
Fonte: Autoria própria (2026).

09 CONCLUSÃO

O objetivo do projeto foi desenvolver uma plataforma que juntasse as vagas e os cursos do Distrito Federal e mostrasse a compatibilidade entre candidatos e vagas. Os objetivos específicos foram centralizar as vagas, preencher o perfil a partir do currículo, gerar o portfólio, calcular o match, organizar as candidaturas, permitir a publicação de vagas a partir de anúncios e cartazes, recomendar cursos e e-books, completar a leitura de documentos com padrões tirados dos próprios cadastros, garantir ao usuário o controle dos seus dados e oferecer o controle administrativo. Todas as especificações foram atendidas, como mostram os requisitos funcionais implementados e o plano de testes.

Respondendo ao problema da pesquisa, concluímos que uma plataforma web consegue, sim, reunir as vagas e os cursos da região e indicar de forma explicável o quanto cada candidato combina com cada vaga, sem que ninguém precise instalar programas. A hipótese básica se confirmou na prática: o candidato recebe o perfil preenchido a partir do currículo e entende a sua nota, e a empresa publica uma vaga a partir da foto do cartaz e recebe as candidaturas já ordenadas. As hipóteses secundárias também se confirmaram, já que a leitura do cartaz acontece no navegador e os padrões automáticos completam a extração onde as regras não tinham pista, sempre mostrando o padrão usado.

Acreditamos que alcançamos o que esperávamos, porque o nosso objetivo principal era facilitar o dia a dia de quem procura e de quem oferece trabalho. Com este projeto aprofundamos os nossos conhecimentos em programação orientada a objetos, banco de dados, segurança de aplicações web, extração de dados de textos e imagens, testes automatizados e modelagem de sistemas. Tivemos dificuldades, principalmente na leitura de currículos e cartazes em formatos muito diferentes, mas conseguimos concluir o trabalho e o sistema ajuda a organizar a divulgação de vagas e o processo de candidatura, levando para candidatos e empresas informações que antes estavam espalhadas e eram controladas à mão.

O estudo também tem limitações. Os planos ainda são demonstrativos, o envio de e-mails funciona apenas no modo de demonstração e a leitura automática depende da qualidade do arquivo ou da foto enviada, por isso o relatório da extração sempre pede a revisão antes de salvar. Também não fizemos uma pesquisa formal com candidatos e empresas usando o sistema por um período longo, o que seria importante para medir o impacto real da plataforma.

O projeto continua aberto a melhorias, como o envio real de e-mails, a integração com meios de pagamento para os planos, a ampliação do dicionário de competências usado pelo match e alguns ajustes de acessibilidade, como a ordem dos títulos. Os padrões automáticos já

reduzem parte da necessidade de ampliar as regras à mão, porque se atualizam a cada vaga e curso cadastrado. Continuando o trabalho, a plataforma pode ser implantada em escolas, instituições e órgãos da região que queiram aproximar a qualificação profissional das oportunidades de emprego.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 10520**: informação e documentação: citações em documentos: apresentação. Rio de Janeiro: ABNT, 2023.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.

BOOCH, Grady; RUMBAUGH, James; JACOBSON, Ivar. **UML**: guia do usuário. 2. ed. Rio de Janeiro: Elsevier, 2006.

BRASIL. Lei nº 8.078, de 11 de setembro de 1990. Dispõe sobre a proteção do consumidor e dá outras providências (Código de Defesa do Consumidor). Disponível em: https://www.planalto.gov.br/ccivil_03/leis/l8078compilado.htm. Acesso em: 1 out. 2026.

BRASIL. Lei nº 9.610, de 19 de fevereiro de 1998. Altera, atualiza e consolida a legislação sobre direitos autorais e dá outras providências. Disponível em: https://www.planalto.gov.br/ccivil_03/leis/l9610.htm. Acesso em: 1 out. 2026.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 27 set. 2026.

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados**. 7. ed. São Paulo: Pearson, 2018.

GIL, Antonio Carlos. **Como elaborar projetos de pesquisa**. 6. ed. São Paulo: Atlas, 2017.

GIT. **Git documentation**. 2026. Disponível em: <https://git-scm.com/doc>. Acesso em: 27 set. 2026.

GUEDES, Gilleanes T. A. **UML 2**: uma abordagem prática. 3. ed. São Paulo: Novatec, 2018.

HEUSER, Carlos Alberto. **Projeto de banco de dados**. 6. ed. Porto Alegre: Bookman, 2009.

MANNING, Christopher D.; RAGHAVAN, Prabhakar; SCHÜTZE, Hinrich. **Introduction to information retrieval**. Cambridge: Cambridge University Press, 2008.

MARCONI, Marina de Andrade; LAKATOS, Eva Maria. **Fundamentos de metodologia científica**. 8. ed. São Paulo: Atlas, 2017.

NAPTHA. **Tesseract.js**: pure JavaScript OCR for more than 100 languages. 2026. Disponível em: <https://github.com/naptha/tesseract.js>. Acesso em: 27 set. 2026.

ORACLE. **MySQL 8.0 Reference Manual**. 2026. Disponível em: <https://dev.mysql.com/doc/refman/8.0/en/>. Acesso em: 27 set. 2026.

OWASP FOUNDATION. **OWASP Top Ten**. 2026. Disponível em: <https://owasp.org/www-project-top-ten/>. Acesso em: 27 set. 2026.

PLANTUML. **PlantUML**: linguagem de diagramas UML. 2026. Disponível em: <https://plantuml.com/>. Acesso em: 27 set. 2026.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software**: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

SMITH, Ray. An overview of the Tesseract OCR engine. In: INTERNATIONAL CONFERENCE ON DOCUMENT ANALYSIS AND RECOGNITION, 9., 2007, Curitiba. **Proceedings** [...]. Los Alamitos: IEEE Computer Society, 2007. p. 629-633.

SOMMERVILLE, Ian. **Engenharia de software**. 10. ed. São Paulo: Pearson, 2018.

TESSERACT OCR. **Tesseract Open Source OCR Engine**. 2026. Disponível em: <https://github.com/tesseract-ocr/tesseract>. Acesso em: 27 set. 2026.

THE APACHE SOFTWARE FOUNDATION. **Apache module mod_rewrite**. 2026. Disponível em: https://httpd.apache.org/docs/current/mod/mod_rewrite.html. Acesso em: 27 set. 2026.

THE PHP GROUP. **Manual do PHP**. 2026. Disponível em: https://www.php.net/manual/pt_BR/. Acesso em: 27 set. 2026.

W3C. **WebAssembly Core Specification**. 2026. Disponível em: <https://www.w3.org/TR/wasm-core-2/>. Acesso em: 27 set. 2026.

APÊNDICE A – DOCUMENTAÇÃO DO CÓDIGO-FONTE

Neste apêndice documentamos o código-fonte do Conecta Vagas DF arquivo por arquivo. Para cada parte explicamos o que ela faz, por que ela existe, como funciona por dentro e com quais outras partes conversa, e reunimos em quadros os métodos e as funções de cada classe, com o que cada um faz. Os caminhos de arquivo são sempre relativos à pasta do projeto (por exemplo, `app/Core/Router.php`), e os nomes de métodos aparecem no formato `Classe::método()`.

A ordem das seções segue o caminho de um pedido dentro do sistema. Começamos pela organização das pastas e pelo núcleo da aplicação (seções A.1 e A.2), passamos pela porta de entrada, pelas rotas e pela configuração (A.3), pelos controladores (A.4), pelos modelos e pelo banco de dados (A.5 e A.6), pelos serviços de negócio e pelas máquinas de extração (A.7 a A.9), pelas telas e pelo código que roda no navegador (A.10 e A.11) e terminamos com os testes automáticos que protegem o código (A.12). A seção 8.11 do capítulo 08 complementa este apêndice, porque segue o caminho de cada funcionalidade do ponto de vista de quem usa o sistema.

A.1 Organização das pastas do projeto

O projeto mora em `C:\xampp\htdocs\tcc-final`, dentro do `htdocs` do XAMPP, e segue o padrão MVC (Model, View e Controller) em PHP puro, sem framework. Além das três camadas do MVC, criamos uma camada de Services, onde ficam as regras de negócio (match, competências, portfólio, Pix e as máquinas de extração), e uma pasta Core, onde fica a infraestrutura que o resto do sistema usa (rotas, sessão, login, CSRF, uploads, conexão com o banco e as telas). Nesta seção explicamos a árvore de pastas inteira, pasta por pasta, e o caminho que um pedido do navegador percorre até virar uma tela na frente do usuário. Deixamos de fora da árvore a pasta `.git` (controle de versão) e `storage/uploads` (arquivos enviados pelos usuários, que não vão para o repositório).

Pasta ou arquivo	O que é e para que serve
<code>.htaccess</code> (raiz)	Regra do Apache que manda todo pedido para a pasta <code>public/</code> , sem mudar o endereço que aparece no navegador (<code>/vagas.php</code> continua <code>/vagas.php</code>). É a primeira peça do caminho de qualquer requisição.
<code>index.php</code> (raiz)	Arquivo de reserva para quando o <code>mod_rewrite</code> do Apache está desligado: só redireciona (HTTP 302) para <code>public/</code> . Com o <code>mod_rewrite</code> ligado (o padrão do XAMPP), o <code>.htaccess</code> já desvia tudo antes e este arquivo nem chega a rodar.
<code>app/</code>	Todo o código da aplicação. O navegador nunca acessa esta pasta diretamente: o <code>app/.htaccess</code> nega qualquer pedido direto, e só o PHP, de dentro do próprio sistema, a lê.

Pasta ou arquivo	O que é e para que serve
app/Controllers/	Os 12 controllers do sistema (um por área: autenticação, vagas, currículo, candidaturas, perfil, administração etc.). Cada ação pública de um controller recebe a requisição, confere a permissão e o CSRF, chama os Models e os Services e, no fim, redireciona ou manda mostrar uma tela.
app/Core/	O núcleo do sistema: Autoloader, Router, Session, Auth, Csrf, Upload, Database, View, Controller (classe base) e helpers.php, além de bootstrap.php, que liga tudo isso. Detalhamos cada arquivo na seção A.2.
app/DTO/	UsuarioDTO e PerfilDTO: pequenos objetos que carregam os dados já tratados do formulário até o DAO que vai gravar no banco, sem misturar a leitura do \$_POST com o SQL.
app/Models/	As classes DAO (Data Access Object), uma por tabela ou por um pequeno grupo de tabelas relacionadas, com o SQL em PDO e consultas preparadas. São 9 DAOs para as 11 tabelas do banco tcc_final: UsuarioDAO, PerfilDAO, VagaDAO, CursoDAO, CategoriaDAO, CurriculoDAO, CandidaturaDAO, MatchDAO e AssinaturaDAO.
app/Services/	As regras de negócio maiores: MatchService (nota de compatibilidade), Competencias (dicionário de competências e sinônimos), Portfolio (validação do cadastro e montagem do portfólio) e Pix (código Pix de doação). A subpasta Extracao/ guarda as máquinas de leitura de currículo, vaga e curso.
app/Services/Extracao/	Os leitores de documento (PdfTexto, DocxTexto, LeitorDocumento) e as máquinas de extração propriamente ditas: ExtracaoCurriculo, AplicacaoCurriculo, ExtracaoVaga, ExtracaoCurso, PadroesExtracao (padrões automáticos), ImagemRemota, CapaPdf, OcrImagem, FontesCursos e PromptsPesquisa.
app/Views/	As telas, em HTML com PHP. A pasta layouts/ tem o cabeçalho, o rodapé e as abas do painel; partials/ guarda os pedaços reaproveitados (ícones, cartões, o relatório da extração); e o resto está dividido em uma pasta por área (home, vagas, cursos, planos, auth, perfil, admin, errors).
config/	config.php, com as constantes de configuração (banco, depuração, limites, pastas), e cacert.pem, a lista de certificados usada pelo cURL quando o sistema abre links externos (ImagemRemota). O config/.htaccess nega qualquer pedido direto.
database/	schema.sql (estrutura das 11 tabelas), seed.sql (dados de demonstração) e resetar_senhas.php (devolve as senhas das contas de teste). O database/.htaccess nega qualquer pedido direto.
docs/	Documentação do projeto: ARQUITETURA.md, PADROES_AUTOMATICOS.md, PESQUISA_CURSOS.md, PROMPTS_PESQUISA.md, o documento do TCC em .docx e os diagramas (casos de uso, classes, sequência, arquitetura, modelos conceitual e lógico). O docs/.htaccess nega qualquer pedido direto.
public/	A ÚNICA pasta que o Apache serve de verdade. Dentro dela ficam o index.php (porta de entrada, o front controller) e assets/ (CSS, JavaScript, imagens e o leitor de cartaz Tesseract.js, que roda no navegador). O public/.htaccess garante que nenhum outro .php dentro dela seja executado.
storage/	Arquivos gerados pelo próprio sistema, fora do alcance do navegador: uploads/ (fotos, logos, cartazes, currículos e PDFs da biblioteca), logs/ (manutenção diária e redefinições de senha) e backups/ (cópias do banco). O storage/.htaccess nega qualquer pedido direto.
tests/	Os testes automáticos: lint.php, smoke.php, jornadas.php, verificar.bat e a pasta amostras/, com leituras reais de OCR usadas pelos testes. Detalhamos cada um na seção A.12. O tests/.htaccess nega qualquer pedido direto.
.github/	Guarda o pre-commit, o gancho do Git que roda os testes antes de cada commit (também explicado na seção A.12).

Quadro 36 – Pastas e arquivos da raiz do projeto
Fonte: Autoria própria (2026).

O caminho de um pedido do navegador até a tela é sempre o mesmo, não importa a página pedida. Tomando como exemplo alguém que digita o endereço `/vagas.php`: primeiro o `.htaccess` da raiz reescreve o pedido para dentro de `public/` (se o arquivo pedido existisse de verdade ali, como um `.css`, o próprio Apache já entregaria e a história pararia aqui). Como "`vagas.php`" não é um arquivo real dentro de `public/`, o `public/.htaccess` manda tudo o que não existe fisicamente para `public/index.php`, que é o front controller, a porta de entrada única de todas as páginas do site. O `index.php` primeiro exige `app/Core/bootstrap.php`, que carrega `config/config.php`, ajusta a exibição de erros e o fuso horário, inclui as funções do núcleo (helpers, sessão, autenticação, CSRF, upload, view, banco), registra o Autoloader e calcula `BASE_URL` e `SITE_URL` a partir da pasta onde o projeto está instalado. De volta ao `index.php`, ele confere se o pedido é para uma imagem enviada (`assets/uploads/...`), que é entregue direto por `ArquivoController::imagem()` sem nem abrir sessão; senão, abre a sessão (`iniciar_sessao()`), revalida a conta logada no banco (`revalidar_sessao()`), envia os cabeçalhos de segurança e monta a tabela de rotas com Router. Por fim chama `$rotas->despachar("vagas.php")`: o `Router::despachar()` procura "`vagas.php`" na tabela, encontra `[VagaController::class, 'lista']`, cria um `VagaController` e chama o método `lista()`. O controller lê os filtros da busca, consulta as vagas no `VagaDAO` (e o match no `MatchDAO`, se houver um candidato logado) e termina chamando `$this->view('vagas/lista', get_defined_vars())`. Esse método, herdado de Controller, repassa para `View::render()`, que inclui em sequência o cabeçalho do layout, a tela `app/Views/vagas/lista.php` e o rodapé, montando a página completa que volta para o navegador.

Os endereços das rotas (`vagas.php`, `view/perfil/index.php`, `admin/pages/vagas.php` e assim por diante) são os mesmos das versões anteriores do projeto, de propósito: são só nomes de rota dentro da tabela do Router, não existem mais como arquivos de verdade no disco, mas links antigos e favoritos continuam funcionando. Detalhamos a tabela completa de rotas, os cabeçalhos de segurança e as constantes de configuração na seção A.3.

A.2 Núcleo da aplicação (app/Core)

A pasta `app/Core` reúne a infraestrutura que todo o resto do sistema usa: carregamento automático das classes, rotas, sessão, autenticação, proteção contra CSRF, upload de arquivos, conexão com o banco, montagem das telas e as funções de apoio gerais. Nenhuma regra de negócio mora aqui (isso fica em `app/Services`); o Core só resolve o "como o sistema funciona por baixo", a parte que, por ser um TCC sem framework, escrevemos nós mesmos. Boa parte

dos arquivos é um conjunto de funções soltas (não uma classe), porque são usadas em todo lugar: controllers, views, models e services.

A.2.1 Auth.php

Auth.php guarda as funções de autenticação e permissão: quem está logado, de que tipo é a conta e as travas que ficam no início das ações dos controllers. O tipo de conta é um dos três valores gravados em `usuarios.tipo`: "admin", "empresa" ou "candidato", e fica guardado na sessão junto com `usuario_id` e `usuario_nome` assim que o login acontece (em `iniciar_sessao_usuario()`, no `Session.php`). Quem navega pelo site sem conta simplesmente não tem `usuario_id` na sessão, e para o sistema é um candidato que ainda não se cadastrou.

As funções `exigirLogin()` e `exigirAdmin()` são a primeira linha de quase toda ação que mexe com dados: sem elas, qualquer pessoa poderia chamar diretamente `view/perfil/salvar.php` ou `admin/pages/usuarios.php`. `exigirLogin()` manda quem não está logado para `login.php`, mas sem sobrescrever um aviso que já esteja pendente (por exemplo, a mensagem de "sua sessão foi encerrada porque a senha mudou", que `revalidar_sessao()` já deixou guardada). `exigirAdmin()` soma a essa checagem a exigência de que o tipo seja "admin"; quem está logado mas não é administrador recebe a página de erro 403, e não um redirecionamento para o login (`negar_acesso()`). Vários controllers repetem essa mesma ideia na mão, chamando `exigirLogin()` e depois conferindo `isEmpresa()` ou `isCandidato()`, porque uma mesma tela às vezes aceita mais de um tipo de conta (por exemplo, `admin/pages/vagas.php` aceita empresa e administrador).

Função	O que faz
<code>usuarioLogado(): bool</code>	Diz se existe um <code>usuario_id</code> na sessão, ou seja, se alguém está autenticado nesta requisição.
<code>isAdmin(): bool</code>	Confere se o <code>usuario_tipo</code> da sessão é "admin".
<code>isEmpresa(): bool</code>	Confere se o <code>usuario_tipo</code> da sessão é "empresa".
<code>isCandidato(): bool</code>	Confere se o <code>usuario_tipo</code> da sessão é "candidato".
<code>exigirLogin(): void</code>	Se não há ninguém logado, grava o aviso "Faça login para continuar." (quando não existe outro aviso pendente) e redireciona para <code>login.php</code> , encerrando a ação atual.
<code>exigirAdmin(): void</code>	Chama <code>exigirLogin()</code> e, se a conta logada não for administrador, encerra a requisição com <code>negar_acesso()</code> (HTTP 403).
<code>negar_acesso(string \$mensagem, int \$codigo = 403): never</code>	Mostra a página de erro padrão do site (403 para acesso restrito, ou 404 quando o parâmetro pede) com a mensagem informada e encerra a requisição.
<code>destinoPainel(): string</code>	Devolve a tela inicial de cada tipo de conta depois do login: candidato vai para <code>view/perfil/index.php</code> ; empresa e administrador vão para <code>admin/index.php</code> .
<code>logout_url(): string</code>	Monta o link de sair (<code>view/usuario/logout.php</code>) já com o token CSRF na própria URL (<code>?t=...</code>), para o link "Sair" do menu funcionar mesmo sendo um GET.

Quadro 37 – Funções de app/Core/Auth.php
Fonte: Autoria própria (2026).

A.2.2 Autoloader.php

O Autoloader resolve um problema simples: sem ele, cada arquivo que usasse, por exemplo, a classe VagaDAO precisaria de um `require_once 'app/Models/VagaDAO.php'` escrito à mão, espalhado pelo projeto inteiro. Em vez disso, registramos uma única função com `spl_autoload_register()`, que o PHP chama sozinho sempre que o código tenta usar uma classe que ainda não foi carregada (por exemplo, ao executar `new VagaDAO()`). A função confere, com uma expressão regular, se o nome da classe é um identificador PHP válido (letras, números e sublinhado, sem começar com número) antes de tentar montar um caminho de arquivo a partir dele; isso evita que um nome de classe inventado ou manipulado sirva para sondar o sistema de arquivos. Em seguida procura, em ordem, um arquivo com esse nome mais ".php" dentro de `app/Controllers`, `app/Models`, `app/DTO`, `app/Services`, `app/Services/Extracao` e `app/Core`; a primeira pasta onde o arquivo existir é a que vale, e o arquivo é incluído com `require`. A regra é simples e única: o nome do arquivo é sempre igual ao nome da classe.

Função	O que faz
<code>spl_autoload_register</code> (função anônima)	Callback chamado automaticamente pelo PHP ao encontrar uma classe ainda não carregada. Confere se o nome é um identificador PHP válido e procura "<Classe>.php" nas pastas <code>Controllers</code> , <code>Models</code> , <code>DTO</code> , <code>Services</code> , <code>Services/Extracao</code> e <code>Core</code> , nessa ordem; a primeira que existir é incluída com <code>require</code> .

Quadro 38 – Função registrada em app/Core/Autoloader.php
Fonte: Autoria própria (2026).

A.2.3 Controller.php

`Controller.php` define a classe abstrata `Controller`, que é a base de todos os 12 controllers do sistema (a "C" do MVC). Ela só tem um método, `view()`, que os controllers chamam no fim de cada ação para mostrar uma tela. O truque está no segundo parâmetro: ao escrever `$this->view('vagas/lista', get_defined_vars())`, a função nativa `get_defined_vars()` do PHP devolve automaticamente todas as variáveis locais que o método já calculou até aquele ponto (`$vagas`, `$filtros`, `$title...`), sem que o programador precise listar uma por uma nem repetir o nome de cada variável. O método `view()` simplesmente repassa a tela e os dados para `View::render()` (detalhado a seguir).

Método	O que faz
protected function <code>view</code> (string \$tela, array \$dados = []): void	Mostra uma tela de <code>app/Views</code> dentro do layout do site, chamando <code>View::render(\$tela, \$dados)</code> . É assim que toda ação termina: <code>\$this->view('pasta/tela', get_defined_vars())</code> .

Quadro 39 – Método de app/Core/Controller.php
Fonte: Autoria própria (2026).

A.2.4 Csrff.php

Csrff.php protege os formulários contra CSRF (Cross-Site Request Forgery), o ataque em que um site malicioso tenta enviar, escondido, um formulário em nome de quem está logado no nosso sistema (por exemplo, forçar a exclusão de uma vaga sem a pessoa perceber). A defesa é um token aleatório: `csrf_token()` cria, na primeira vez em que é pedido, uma sequência de 32 bytes aleatórios (`random_bytes`) convertida em hexadecimal e a guarda na sessão; `csrf_campo()` devolve esse token já dentro de um `<input type="hidden" name="csrf">`, pronto para colar em qualquer `<form method="post">`. Quando o formulário é enviado, `validar_csrf()` compara o token recebido no POST com o que está guardado na sessão, usando `hash_equals()` (uma comparação em tempo constante, que não dá pista do valor certo por quanto tempo a comparação demorou). Se o token não bater ou não existir, a requisição é encerrada com uma página de erro HTTP 403 (escolhemos 403 e não o código semanticamente mais correto 419, porque o Apache não conhece 419 e o transformaria em 500). A cada login ou cadastro, `iniciar_sessao_usuario()` descarta o token antigo, então um token de uma sessão anterior nunca é reaproveitado.

Função	O que faz
<code>csrf_token(): string</code>	Devolve o token da sessão atual, criando um novo (32 bytes aleatórios em hexadecimal) na primeira vez em que é chamado.
<code>csrf_campo(): string</code>	Devolve o campo oculto <code><input type="hidden" name="csrf" value="..."></code> já escapado, pronto para um formulário POST.
<code>validar_csrf(): void</code>	Compara o token recebido no POST com o da sessão (<code>hash_equals</code>); se não bater ou estiver vazio, mostra a página de erro 403 e encerra a requisição.

Quadro 40 – Funções de app/Core/Csrf.php
Fonte: Autoria própria (2026).

A.2.5 Database.php

Database.php define a conexão PDO com o MySQL/MariaDB do XAMPP, pelo padrão de projeto Singleton: existe UMA só conexão por requisição, guardada na propriedade estática `$instance`, e o construtor é privado para ninguém criar uma segunda instância por engano. O método `Database::getConexao()` é chamado por todos os DAOs sempre que precisam falar com o banco; na primeira chamada ele monta a conexão, e nas chamadas seguintes (dentro da mesma requisição) devolve a mesma conexão já aberta. A conexão é configurada com `PDO::ATTR_ERRMODE => PDO::ATTR_ERRMODE_EXCEPTION` (qualquer erro do banco vira uma exceção em vez de um aviso silencioso), `PDO::ATTR_EMULATE_PREPARES => false` (consultas preparadas DE VERDADE, a

defesa central contra SQL injection, em vez de o próprio PDO simular a preparação colando os valores na string), PDO::ATTR_TIMEOUT => 5 (a página nunca fica "pendurada" esperando um MySQL que não responde) e a codificação utf8mb4 com o mesmo fuso horário do PHP (America/Sao_Paulo), para NOW() e CURDATE() do banco baterem com date() do PHP.

Para montar o DSN de conexão, o código tenta, em ordem, um socket Unix configurado (quando existe, comum no Linux), depois o host configurado em DB_HOST, depois um host alternativo (127.0.0.1 ↔ localhost, porque no Windows às vezes um resolve e o outro não) e, fora do Windows, alguns caminhos de socket comuns do XAMPP/LAMPP. Se todas as tentativas falharem, o método privado mensagemAmigavel() traduz os códigos de erro mais comuns do MySQL (1049: banco inexistente; 1044/1045: usuário ou senha recusados; 2002/2003/2006: serviço desligado) em uma orientação em português, e getConnection() lança uma DatabaseException com essa mensagem amigável e o erro técnico original guardado em getPrevious() (só mostrado na tela quando DEBUG é true). O método privado configurarSessao() ajusta o utf8mb4 e o fuso horário logo após abrir a conexão, antes de ela ser devolvida pela primeira vez.

Classe/Método	O que faz
final class DatabaseException extends RuntimeException	Exceção lançada quando a conexão com o banco falha. A mensagem (amigável, em português) pode ir direto para a tela; o erro técnico original fica em getPrevious() e só aparece com DEBUG=true.
Database::getConnection(): PDO	Devolve a conexão PDO única da requisição (cria na primeira chamada, reaproveita nas seguintes). Tenta vários DSNs em ordem (socket, host configurado, host alternativo, sockets comuns) e lança DatabaseException com uma mensagem amigável se nenhum funcionar.
private static mensagemAmigavel(? PDOException \$e): string	Traduz os códigos de erro mais comuns do MySQL (banco inexistente, usuário/senha recusados, serviço desligado) numa orientação clara em português.
private static configurarSessao(PDO \$pdo): void	Ajusta a conexão recém-aberta para utf8mb4 (SET NAMES) e o mesmo fuso horário do PHP (SET time_zone).

Quadro 41 – Classe e método públicos de app/Core/Database.php
Fonte: Autoria própria (2026).

A.2.6 Router.php

O Router é a classe que decide qual controller atende cada endereço. Ele guarda duas tabelas internas: \$rotas, que liga um caminho (em minúsculas, por exemplo "vaga.php") a um par [Controller::class, 'método'], e \$apelidos, que liga caminhos alternativos (como a raiz "" ou "admin", sem o "/index.php") ao caminho principal da mesma rota. O método rota() é chamado uma vez para cada endereço dentro de public/index.php, na montagem da tabela; despachar() recebe o caminho pedido, resolve um eventual apelido, procura na tabela e, se não encontrar,

mostra a página 404; se encontrar, guarda a rota atual em `self::$atual` (usada pelo menu do layout para marcar o item ativo) e instancia o controller com `(new $classe())->$metodo()`.

O trabalho mais delicado do Router é `caminhoPedido()`: a partir de `$_SERVER['REQUEST_URI']`, ele extrai só o caminho (sem a query string), decodifica caracteres de URL (`rawurldecode`), retira o prefixo `BASE_URL` (a pasta onde o projeto está instalado dentro do htdocs, por exemplo `"/tcc-final/"`), tira as barras das pontas e ainda trata quem abrir o endereço apontando direto para a pasta `public/` (por exemplo `.../public/vagas.php`), removendo esse prefixo também. A comparação com `BASE_URL` ignora maiúsculas e minúsculas de propósito, porque no Windows o Apache aceita a pasta do projeto com a caixa trocada (`TCC_Final` ou `tcc_final` abrem a mesma pasta no disco, mas a URL poderia vir diferente).

Método	O que faz
<code>rota(string \$caminho, array \$acao, string ...\$apelidos): void</code>	Registra uma rota: liga <code>\$caminho</code> (em minúsculas) ao par <code>[Controller, método]</code> de <code>\$acao</code> ; os <code>\$apelidos</code> extras (ex.: <code>""</code> para a raiz, <code>"admin"</code> sem o <code>"/index.php"</code>) passam a apontar para o mesmo <code>\$caminho</code> . Aceita GET e POST; a própria ação do controller decide o que fazer em cada verbo.
<code>despachar(string \$caminho): void</code>	Resolve um eventual apelido, procura o caminho na tabela de rotas; se não existir, mostra a página 404; se existir, guarda a rota atual e chama <code>(new Controller())->metodo()</code> .
<code>static atual(): string</code>	Devolve o caminho da rota em execução (ex.: <code>"vagas.php"</code>), usado pelo menu do layout para destacar o item ativo.
<code>static caminhoPedido(): string</code>	A partir de <code>\$_SERVER['REQUEST_URI']</code> , devolve o caminho pedido pelo navegador, relativo à pasta do projeto, decodificado, sem barras nas pontas e sem o prefixo <code>"public/"</code> quando alguém aponta direto para lá.

Quadro 42 – Métodos de `app/Core/Router.php`

Fonte: Autoria própria (2026).

A.2.7 Session.php

`Session.php` concentra tudo o que envolve a sessão do navegador: abertura segura do cookie, mensagens rápidas (`"flash"`), início e fim do login, e a revalidação da conta a cada requisição. A função `iniciar_sessao()` configura o cookie antes de chamar `session_start()`: usa um nome próprio (`CVDF_SESSAO`, em vez do `PHPSESSID` padrão, para não se misturar com outro sistema do mesmo localhost), restringe o cookie à pasta do projeto (`path = BASE_URL`), marca `HttpOnly` (JavaScript não consegue ler o cookie, o que atrapalha um ataque de XSS que tentasse roubar a sessão), `SameSite=Lax` (uma camada a mais contra CSRF) e `Secure` quando o site está em HTTPS. O modo estrito (`session.use_strict_mode`) faz o PHP recusar qualquer identificador de sessão "inventado" por quem não tem um cookie válido, e `session.use_only_cookies` impede que o ID da sessão seja aceito pela URL.

`revalidar_sessao()` é chamada em TODA requisição autenticada (a partir de `public/index.php`, logo depois de abrir a sessão) e é o que garante que uma ação do administrador tenha efeito imediato, e não só no próximo login: ela consulta no banco o nome, o tipo, se a conta está ativa e a senha atual do usuário da sessão. Em vez de guardar o hash da senha na sessão (o que duplicaria um dado sensível na memória do PHP), guardamos apenas uma "marca": o SHA-256 do hash (`marca_senha()`). Se a conta foi desativada/excluída, ou se a marca não bate mais com a senha atual do banco (ou seja, a senha mudou em outro lugar, por exemplo numa redefinição), a sessão é derrubada na hora, com `session_regenerate_id(true)`, e um aviso explica o motivo. Senão, o nome e o tipo da conta na sessão são atualizados com o que está no banco (para refletir, por exemplo, uma mudança de tipo feita pelo administrador). Se o banco estiver fora do ar, a função simplesmente não faz nada: a própria página vai mostrar o erro amigável ao tentar consultar o banco por conta própria.

`iniciar_sessao_usuario()` é chamada pelo `AuthController` depois de um login ou cadastro bem-sucedido: regenera o ID de sessão (contra fixação de sessão, o ataque em que alguém força a vítima a usar um ID de sessão já conhecido pelo atacante) e descarta o token CSRF antigo, grava `usuario_id`, `usuario_nome`, `usuario_tipo` e `login_em`, e já calcula a `marca_senha` da conta recém-logada. `renovar_marca_senha()` é usada quando o próprio usuário troca a própria senha (por exemplo, pelo link de redefinição): atualiza a marca só para a sessão atual, para ela continuar valendo, enquanto as outras sessões abertas com a senha antiga caem na próxima requisição delas.

Função	O que faz
<code>iniciar_sessao(): void</code>	Abre a sessão com cookie próprio (<code>CVDF_SESSAO</code>), restrito à pasta do projeto, <code>HttpOnly</code> , <code>SameSite=Lax</code> , <code>Secure</code> quando há <code>HTTPS</code> e modo estrito (recusa ID de sessão inventado). Não faz nada se já existe uma sessão aberta ou se o PHP roda em CLI.
<code>revalidar_sessao(): void</code>	A cada requisição, confere no banco se a conta logada continua ativa e se a senha não mudou (pela "marca" salva na sessão); se a conta foi desativada/excluída ou a senha trocou, encerra a sessão na hora e avisa o motivo.
<code>flash(string \$type, string \$message): void</code>	Guarda uma mensagem (tipo ok, erro ou info) para ser exibida uma única vez, na próxima página.
<code>getFlash(): ?array</code>	Retira e devolve a mensagem guardada por <code>flash()</code> (chamada pelo cabeçalho do layout, que a exibe e a descarta).
<code>iniciar_sessao_usuario(array \$u): void</code>	Inicia a sessão de um usuário autenticado: regenera o ID de sessão e o token CSRF (contra fixação de sessão) e grava <code>usuario_id</code> , <code>usuario_nome</code> , <code>usuario_tipo</code> , <code>login_em</code> e a marca da senha atual.
<code>marca_senha(string \$hashSenha): string</code>	Calcula o SHA-256 do hash de senha gravado no banco; é essa "marca", e não o hash em si, que fica guardada na sessão.
<code>renovar_marca_senha(int \$usuarioId, string \$hashSenha): void</code>	Quando o próprio usuário troca a própria senha, atualiza a marca desta sessão para ela continuar valendo (as demais sessões abertas caem na próxima requisição).

Função	O que faz
encerrar_sessao(): void	Encerra a sessão por completo: limpa os dados, apaga o cookie do navegador e destrói a sessão no servidor.

Quadro 43 – Funções de app/Core/Session.php
Fonte: Autoria própria (2026).

A.2.8 Upload.php

Upload.php cuida de todos os arquivos que os usuários enviam: fotos de perfil, logotipos de empresa, cartazes de vaga, currículos e os PDFs de e-book guardados na biblioteca da plataforma. Todos eles ficam fisicamente em storage/uploads/, fora da pasta public/, então o navegador nunca os acessa de forma direta; o que o banco guarda é sempre um caminho "lógico", no formato assets/uploads/<nome>, que é o mesmo endereço que aparece no HTML. A função caminho_upload() faz a ponte entre esse caminho lógico e o caminho real no disco (UPLOAD_DIR . \$nome), recusando qualquer coisa que não comece exatamente por "assets/uploads/" ou que tente embutir uma subpasta, uma barra invertida ou um "..": sem essa checagem, um caminho malicioso gravado por engano no banco poderia ler qualquer arquivo do servidor através do ArquivoController.

Quem entrega esses arquivos publicamente é sempre o próprio sistema, nunca o Apache diretamente: imagens e os PDFs da biblioteca saem por ArquivoController::imagem(), e os currículos (que têm dados pessoais) só saem por download.php, com checagem de permissão a cada pedido. O envio em si é validado pelo CONTEÚDO do arquivo, nunca pela extensão ou pelo nome original: salvar_imagem_enviada() usa finfo (a extensão fileinfo do PHP) para descobrir o tipo MIME real e getimagesize() para confirmar que é mesmo uma imagem decodificável, aceitando só JPG, PNG ou WEBP até um tamanho máximo (3 MB por padrão); salvar_pdf_biblioteca() confere o MIME application/pdf e ainda lê os 5 primeiros bytes do arquivo, exigindo a assinatura "%PDF-". Em ambos os casos o nome do arquivo salvo é inventado na hora (prefixo + data + bytes aleatórios em hexadecimal): o nome original escolhido pelo usuário nunca chega ao disco, o que evita truques como um nome de arquivo com caracteres de controle ou uma extensão disfarçada.

A limpeza é automática e conservadora. apagar_upload_sem_uso() só remove um arquivo se nenhuma linha de vagas, cursos, perfis ou currículos ainda apontar para ele (uma única consulta soma as quatro contagens); é chamada, por exemplo, quando uma vaga é apagada ou quando uma foto é substituída. limpar_uploads_orfaos() faz a mesma checagem de uso em lote, varrendo a pasta storage/uploads/ inteira, mas só apaga o que tem mais de \$horas (24 por padrão): isso poupa arquivos recém-gerados, como a foto de um currículo que o candidato ainda

está decidindo se aplica ao perfil. `manutencao_diaria()` é a função que efetivamente dispara essa limpeza, mas no máximo uma vez por dia: ela guarda um arquivo-marca (`storage/logs/manutencao_diaria.txt`) com o carimbo de hora da última execução e só roda de novo se já tiver passado 86400 segundos; é chamada por `AdminController::painel()` sempre que o administrador abre a visão geral, e nunca deixa uma falha interromper a página (qualquer exceção só vai para o log do PHP).

Função/constante	O que faz
<code>const MAX_PDF_BIBLIOTECA</code>	Tamanho máximo (25 MB) de um PDF enviado diretamente no cadastro de um e-book para a biblioteca da plataforma.
<code>const MAX_PDF_REMOTO</code>	Tamanho máximo (150 MB) de um PDF baixado da web para a biblioteca, ao abrir os links de uma ficha de curso.
<code>caminho_upload(string \$rel): ?string</code>	Converte um caminho lógico do banco (" <code>assets/uploads/nome</code> ") no caminho real do arquivo em <code>storage/uploads</code> ; devolve <code>null</code> se não for um upload válido (fora do padrão, com subpasta, <code>".."</code> , etc.).
<code>salvar_imagem_enviada(string \$campo, string \$prefixo, int \$maxBytes = 3MB): string null false</code>	Salva uma imagem enviada (JPG/PNG/WEBP, conferida pelo conteúdo) com nome aleatório; devolve o caminho gravável no banco, <code>null</code> se nada foi enviado, ou <code>false</code> se o arquivo for inválido.
<code>eh_pdf_biblioteca(string \$url): bool</code>	Diz se o caminho segue o padrão dos PDFs da biblioteca (" <code>assets/uploads/biblioteca_....pdf</code> ").
<code>salvar_pdf_biblioteca(string \$campo): string null false</code>	Salva o PDF de um e-book na biblioteca (conferido pelo MIME e pela assinatura <code>"%PDF-"</code>), com nome aleatório; mesmo padrão de retorno da função anterior.
<code>apagar_upload_sem_uso(string \$rel): void</code>	Apaga um arquivo enviado só se nenhum registro do banco (vagas, cursos, perfis, currículos) ainda o usa.
<code>safe_filename(string \$name): string</code>	Monta um nome de arquivo seguro para o cabeçalho de download (só letras, números, ponto, hífen e sublinhado).
<code>imagens_da_pasta(string \$pasta): array</code>	Lista as imagens (PNG/JPG/WEBP) de uma pasta dentro de <code>public/</code> , usada para sugerir imagens no cadastro de vagas/cursos e montar o carrossel da página inicial.
<code>caminho_imagem_valido(string \$caminho): string</code>	Confere se um caminho de imagem digitado no painel é um arquivo de imagem do próprio projeto (<code>assets/img</code> ou <code>assets/uploads</code> , sem <code>".."</code>); devolve vazio se for inválido.
<code>limpar_uploads_orfaos(int \$horas = 24): int</code>	Apaga, de <code>storage/uploads</code> , os arquivos com mais de <code>\$horas</code> que nenhum registro do banco usa; devolve quantos arquivos foram removidos.
<code>manutencao_diaria(?int \$usuarioId = null): void</code>	Dispara <code>limpar_uploads_orfaos()</code> no máximo uma vez por dia (controlado por um arquivo-marca em <code>storage/logs</code>), sem nunca interromper a página por causa de uma falha.

Quadro 44 – Constantes e funções de `app/Core/Upload.php`
Fonte: Autoria própria (2026).

A.2.9 View.php

`View.php` é a camada que monta o HTML final (a "V" do MVC) e também concentra o tratamento de erros do sistema. A classe `View` tem um único método, `render()`, chamado por `Controller::view()`: ele usa `extract()` para transformar cada posição do array de dados vindo do controller numa variável local (`$vagas`, `$filtros...`) e então inclui, em sequência, o cabeçalho do

layout (app/Views/layouts/header.php, que usa \$title, \$layoutLargo, \$descricaoPagina e \$ogImagem quando existirem), a tela pedida (app/Views/<tela>.php) e o rodapé (footer.php). Os nomes das variáveis internas do método levam "__" na frente (__tela, __dados) de propósito, para nunca colidir com uma variável de mesmo nome vinda da tela.

A função pagina_erro() mostra uma página de erro simples, SEM o layout do site (sem cabeçalho nem rodapé), o que é importante porque ela também é usada quando o próprio banco de dados está fora do ar, nessa hora, tentar montar o cabeçalho normal (que consulta categorias e outras coisas no banco) só pioraria a situação. Ela define o código HTTP, remove qualquer cache (Cache-Control: no-store) e inclui app/Views/errors/erro.php, que mostra \$detalhe (o texto técnico) somente quando DEBUG é true. A função tratar_excecao() é o tratador global de exceções do sistema, registrado em bootstrap.php com set_exception_handler(): qualquer exceção que escape de um controller sem ser tratada cai aqui, é gravada no log do PHP (error_log) e vira uma página amigável, nunca um "Fatal error" cru na tela. Ela distingue três casos: uma DatabaseException (o banco está fora do ar) mostra uma página 503 com orientação de como religar o MySQL e importar o schema, mas só com o detalhe técnico se DEBUG for true; sem DEBUG, a mensagem é genérica, para não revelar o nome do banco nem dicas de instalação a quem acessa de fora. Uma PDOException qualquer vira uma página 500 de "erro ao acessar o banco". Qualquer outra exceção vira uma página 500 genérica de "algo deu errado".

Elemento	O que faz
View::render(string \$tela, array \$dados = []): void	Extraí as variáveis de \$dados e inclui, em sequência, o cabeçalho do layout, a tela pedida (app/Views/<tela>.php) e o rodapé.
pagina_erro(int \$codigo, string \$titulo, string \$mensagem, string \$detalhe = ""): never	Mostra uma página de erro sem o layout do site (funciona mesmo com o banco fora do ar), define o código HTTP e encerra a requisição. \$detalhe só aparece com DEBUG=true.
tratar_excecao(Throwable \$ex): never	Tratador global de exceções (registrado em bootstrap.php): registra o erro no log do PHP e mostra uma página amigável (503 para falha do banco, 500 para os demais casos), nunca um erro cru na tela.

Quadro 45 – Classe, função global e tratador de erro de app/Core/View.php
Fonte: Autoria própria (2026).

A.2.10 bootstrap.php

bootstrap.php não define uma classe nem funções próprias: é o script de inicialização que prepara a aplicação inteira, chamado tanto por public/index.php (no site) quanto pelos testes de linha de comando (tests/smoke.php, tests/jornadas.php e database/resetar_senhas.php), o que garante que o site e os testes rodem exatamente sob a mesma configuração. Ele segue sempre a mesma ordem de quatro passos: primeiro exige config/config.php, que define as constantes de configuração (detalhadas na seção A.3); depois ajusta o comportamento do PHP conforme o

modo de depuração (exibir ou não os erros na tela, sempre registrar no log) e fixa o fuso horário America/Sao_Paulo; em seguida inclui, nesta ordem, os arquivos de função do núcleo (helpers, Session, Auth, Csrf, Upload, View, Database) e os dois arquivos de apresentação usados por controllers e telas (partials/icones.php e partials/componentes.php), registra o tratador global de exceções (quando não está em CLI) e, por fim, o Autoloader, que a partir daí cuida sozinho de carregar Controllers, Models, DTO, Services e Router sob demanda.

O último passo é uma função anônima que roda imediatamente (IIFE) e calcula três constantes essenciais: `BASE_URL` (a pasta onde o projeto está instalado dentro do DocumentRoot do Apache, por exemplo `"/tcc-final/"`), `HTTPS_ATIVO` (se a conexão atual é HTTPS) e `SITE_URL` (o endereço completo do site, já com o protocolo e o host). O cálculo de `BASE_URL` compara o `DOCUMENT_ROOT` com a pasta real do projeto (`ROOT_DIR`); no Windows isso precisa ignorar diferenças de maiúsculas/minúsculas na letra da unidade e lidar com espaços no nome da pasta (como `"TCC v2"`), por isso cada segmento do caminho é codificado para uso em URL com `rawurlencode`. Quando essa comparação não é possível (por exemplo, com um Alias do Apache apontando para fora do htdocs), o código cai num caminho alternativo que deduz a mesma informação a partir de `SCRIPT_NAME` e `SCRIPT_FILENAME`. `HTTP_HOST`, por vir direto do cliente, só é aceito se bater com um formato válido de host (e opcionalmente porta); qualquer coisa fora do padrão cai para `"localhost"`, para não deixar passar um valor malicioso no `SITE_URL` gerado, que é usado em toda chamada a `url()` e `redirect()`.

Etapas	O que faz
1. Configurações	Exige <code>config/config.php</code> , que define as constantes do projeto (banco, depuração, limites, pastas).
2. Ajustes do PHP	Liga <code>error_reporting(E_ALL)</code> , decide se os erros aparecem na tela conforme <code>DEBUG</code> , sempre grava no log (<code>log_errors</code>) e fixa o fuso horário America/Sao_Paulo.
3. Núcleo e carregamento automático	Inclui <code>helpers.php</code> , <code>Session.php</code> , <code>Auth.php</code> , <code>Csrf.php</code> , <code>Upload.php</code> , <code>View.php</code> e <code>Database.php</code> , os parciais de apresentação (ícones e componentes), registra o tratador global de exceções e, por fim, o Autoloader.
4. Endereço base do site	Calcula e define <code>BASE_URL</code> , <code>HTTPS_ATIVO</code> e <code>SITE_URL</code> a partir da pasta do projeto dentro do DocumentRoot do Apache (ou, como alternativa, a partir de <code>SCRIPT_NAME/SCRIPT_FILENAME</code>).

Quadro 46 – Passos de `app/Core/bootstrap.php`
Fonte: Autoria própria (2026).

A.2.11 helpers.php

`helpers.php` reúne as funções de apoio pequenas e sem estado usadas em todo o sistema: controllers, views, models e services. O próprio arquivo já se organiza em cinco grupos, que seguimos aqui: saída segura e endereços; leitura segura de `$_POST/$_GET`; conversão e

validação de valores; textos exibidos na tela; e informações da requisição e registro em log. A função mais repetida do projeto inteiro é provavelmente `e()`: toda vez que um valor vindo do banco ou do usuário aparece numa tela, ele passa por `e()`, que escapa o HTML (`htmlspecialchars` com `ENT_QUOTES` e UTF-8) e é a principal defesa contra XSS. `url()` e `redirect()` centralizam a montagem de endereços a partir de `SITE_URL`, calculada em `bootstrap.php`, então nenhuma tela escreve um endereço absoluto na mão.

O grupo de leitura de formulários (`post_str`, `post_int`, `get_str`) existe por causa do `declare(strict_types=1)` do projeto: com tipagem estrita, chamar `trim(null)` quando um campo não foi enviado gera um erro fatal (`TypeError`); essas funções tratam esse caso devolvendo o valor padrão em vez de deixar o PHP quebrar. `like()` escapa os caracteres `%` e `_` digitados pelo usuário numa busca, para eles serem tratados como texto literal e não como curinga do SQL LIKE. O grupo de ordenação e paginação (`lista_ordem`, `ordenar_linhas`, `paginar`, `painel_qs`, `registro_encontrado`) sustenta as tabelas do painel administrativo: só aceita campos de ordenação de uma lista permitida (nunca o nome de uma coluna vindo direto do `$_GET`), ordena sem diferenciar maiúsculas ou acentos e com desempate estável pelo id, corta a página pedida dentro do intervalo válido e remonta a URL da lista preservando os filtros atuais ao trocar só o que for pedido.

Função	O que faz
<code>e(mixed \$v): string</code>	Escapa um valor para exibir no HTML (<code>htmlspecialchars</code>), a defesa contra XSS usada em todo texto vindo do banco ou do usuário.
<code>url(string \$path = ""): string</code>	Monta o endereço completo de uma página ou arquivo do projeto a partir de <code>SITE_URL</code> .
<code>redirect(string \$path): never</code>	Redireciona (Location) para outra página do projeto e encerra a requisição.
<code>post_str(string \$key, string \$default = ""): string</code>	Lê um campo de <code>\$_POST</code> já aparado (<code>trim</code>); campo ausente ou enviado como lista devolve o valor padrão (evita erro com <code>strict_types</code>).
<code>post_int(string \$key, int \$default = 0): int</code>	Lê um campo numérico de <code>\$_POST</code> , convertendo para <code>int</code> ; valor não numérico devolve o padrão.
<code>get_str(string \$key, string \$default = ""): string</code>	Equivalente a <code>post_str()</code> , mas lendo de <code>\$_GET</code> .
<code>like(string \$termo): string</code>	Monta o termo de uma busca LIKE (" <code>%termo%</code> "), escapando <code>%</code> e <code>_</code> digitados para serem tratados como texto comum.
<code>volta_filtros(array \$chaves): string</code>	Monta a query string (<code>?tipo=ebook&q=excel</code>) a partir dos filtros reenviados pelo formulário de uma ação (campos <code>f_<chave></code>), para voltar à mesma lista filtrada.
<code>lista_ordem(array \$permitidos, string \$padrao, string \$dirPadrao = 'asc'): array</code>	Lê <code>?ordem=</code> e <code>?dir=</code> da URL, aceitando só campos de uma lista permitida; devolve [campo, direção].
<code>ordenar_linhas(array \$linhas, string \$campo, string \$dir = 'asc'): array</code>	Ordena uma lista de linhas pelo campo pedido (números como números, textos sem diferenciar maiúsculas/acentos), com vazios sempre no fim e desempate estável pelo id.

Função	O que faz
<code>paginar(array \$linhas, int \$porPagina = 25): array</code>	Corta a lista na página atual (?pagina=), mantendo o número dentro do intervalo válido; devolve [linhas da página, página, total de páginas].
<code>carregador_html(string \$texto = "", bool \$pronto = false): string</code>	Monta o carregador (barra de progresso) usado pelas máquinas de extração: amarelo enquanto lê, azul quando o resultado está pronto.
<code>painel_qs(array \$troca = []): string</code>	Monta a URL da lista do painel com os filtros atuais, trocando só o que vier em \$troca; nunca carrega "edit"/"ver"/"novo" (abrir um formulário não deve mudar a lista).
<code>registro_encontrado(?array \$registro, string \$param, string \$lista, string \$aviso = '...'): ?array</code>	Se o parâmetro de edição/visualização veio preenchido mas o registro não existe mais, avisa e volta para a lista em vez de abrir um formulário vazio.
<code>old(string \$key, string \$default = ""): string</code>	Devolve o valor digitado anteriormente num campo (para repreencher o formulário após um erro), já escapado com <code>e()</code> .
<code>enum_val(string \$valor, array \$permitidos, string \$padrao): string</code>	Devolve o valor se ele estiver numa lista de valores permitidos; caso contrário, devolve o padrão (usado para nunca aceitar um valor de enum fora do esperado).
<code>decimal_ou_null(string \$v): ? float</code>	Converte um valor digitado em formato brasileiro ou americano ("1.234,56" ou "1234.56") em float; vazio ou inválido vira null.
<code>data_ou_null(string \$v): ? string</code>	Confere se uma data está no formato AAAA-MM-DD (campo <code>type="date"</code>); inválida ou vazia vira null.
<code>trim_u(string \$s, string \$chars = '\t\n\r\0\x0B', string \$lado = 'ambos'): string</code>	Um <code>trim()</code> que entende acentos e símbolos (•, –, ,): o <code>trim()</code> nativo corta por byte e pode partir uma letra acentuada no fim da palavra.
<code>normalizar_email(string \$email): string</code>	Deixa um e-mail sempre em minúsculas e sem espaços, no formato gravado no banco.
<code>maskar_email(string \$email): string</code>	Mascara um e-mail para aparecer em log sem expor a pessoa (LGPD): "candidato@site.com" vira "ca*****@site.com".
<code>url_http_valida(string \$u): bool</code>	Confere se uma URL externa é segura para virar link (só http/https; bloqueia javascript:, data: etc.).
<code>salario_texto(mixed \$min, mixed \$max): string</code>	Monta o texto da faixa salarial ("R\$ 1.900,00 a R\$ 2.500,00", um valor único ou "A combinar").
<code>classe_match(float \$p): string</code>	Devolve a classe de cor do selo de match (excelente/alto/medio/baixo), delegando para <code>MatchService::classificar()</code> .
<code>rotulo(string \$v): string</code>	Traduz um valor gravado no banco para o rótulo em português mostrado na tela (ex.: "em_analise" vira "Em análise").
<code>ip_cliente(): string</code>	Devolve o IP do cliente (REMOTE_ADDR); cabeçalhos como X-Forwarded-For são ignorados por serem fáceis de falsificar.
<code>mensagem_erro_banco(Throwable \$e): string</code>	Monta o texto de uma falha do banco para mostrar na tela; o detalhe técnico só aparece com <code>DEBUG=true</code> ou quando já é uma <code>DatabaseException</code> (mensagem já amigável).
<code>registrar_log(string \$arquivo, string \$linha): void</code>	Grava uma linha, com data e hora, num arquivo de <code>storage/logs/</code> (pasta bloqueada ao navegador).

Quadro 47 – Funções de `app/Core/helpers.php`

Fonte: Autoria própria (2026).

A.3 Porta de entrada, rotas e configuração

`public/index.php` é o front controller do sistema: a única porta de entrada por onde passam todas as páginas. O arquivo começa exigindo `app/Core/bootstrap.php` (que prepara a configuração, o núcleo e o carregamento automático, como já explicamos na seção A.2) e segue em três passos. No segundo passo, antes mesmo de abrir a sessão, ele confere se o caminho pedido começa com `"assets/uploads/"`; se sim, entrega o arquivo direto por `ArquivoController::imagem()` e encerra ali mesmo, é assim que fotos, logos, cartazes e os PDFs da biblioteca são servidos sem exigir login e sem o peso de montar a sessão para uma simples imagem. Tecnicamente esse caso não é uma rota registrada no Router: é tratado à parte, de propósito, antes da tabela de rotas.

No terceiro passo, com qualquer outro caminho, o `index.php` abre a sessão (`iniciar_sessao()`), revalida a conta logada no banco (`revalidar_sessao()`) e envia um conjunto de cabeçalhos de segurança em toda resposta: `X-Content-Type-Options: nosniff` (o navegador não tenta "adivinhar" o tipo de um arquivo a partir do conteúdo, o que evita que um upload disfarçado seja interpretado como HTML/JS); `X-Frame-Options: SAMEORIGIN` e a diretiva `frame-ancestors 'self'` da CSP (o site não pode ser embutido num `<iframe>` de outro domínio, contra clickjacking); `Referrer-Policy: strict-origin-when-cross-origin` (não vaza o endereço completo da página atual para sites de fora); a remoção do cabeçalho `X-Powered-By` (não anuncia a versão do PHP para quem for procurar uma falha conhecida); uma `Content-Security-Policy` mínima, que restringe `base-uri` e `form-action` ao próprio site e bloqueia plugins (`object-src 'none'`), sem atrapalhar imagens nem scripts do próprio domínio; `Permissions-Policy`, desligando o acesso a câmera, microfone, geolocalização, pagamento e USB, que o site não usa; `Cross-Origin-Opener-Policy: same-origin-allow-popups`; e, quando a conexão está em HTTPS, `Strict-Transport-Security`, para o navegador sempre voltar por HTTPS da próxima vez. Só depois disso o `index.php` monta o Router, registra as 32 rotas do sistema e chama `$rotas->despachar($caminho)`.

Endereço	Controller e ação	Quem acessa	O que faz
<code>index.php</code> (ou <code>/</code>)	<code>HomeController::index</code>	qualquer pessoa	Página inicial: carrossel de Brasília, busca de vagas, vitrines rotativas de vagas/cursos/e-books e o aviso de doação Pix.
<code>contrato.php</code>	<code>HomeController::contrato</code>	qualquer pessoa	Mostra o termo de autorização e privacidade aceito no cadastro.
<code>vagas.php</code>	<code>VagaController::lista</code>	qualquer pessoa	Lista as vagas abertas, com busca e filtros (cidade, área, tipo, nível, modelo de trabalho); mostra o match quando há candidato logado.

Endereço	Controller e ação	Quem acessa	O que faz
vaga.php?id=	VagaController::detalhe	qualquer pessoa; vaga fechada: só a empresa dona e o administrador	Mostra a página de uma vaga (cartaz, descrição, requisitos, benefícios, contato) e o match do candidato logado.
cursos.php	CursoController::lista	qualquer pessoa	Lista cursos; com ?tipo=ebook ou ?tipo=video mostra e-books e vídeos no lugar de cursos.
curso.php?id=	CursoController::detalhe	qualquer pessoa; conteúdo oculto: só administrador	Mostra a página de um curso, e-book ou vídeo, com o botão Baixar (biblioteca) ou Acessar (link externo).
planos.php	PlanosController::index	qualquer pessoa; assinar exige login	Mostra os planos (Candidato VIP, Empresa Premium) e processa a assinatura, demonstrativa e sem cobrança real.
login.php	AuthController::login	quem ainda não está logado (logado é redirecionado)	Formulário (GET) e autenticação (POST), com pausa contra tentativas repetidas de senha errada.
cadastro.php	AuthController::cadastro	quem ainda não tem conta (logado é redirecionado)	Cria a conta (candidato ou empresa) e o perfil na mesma transação e já entra no sistema.
view/usuario/logout.php	AuthController::logout	quem está logado	Encerra a sessão: aceita POST com token, GET com token na URL ou GET vindo do próprio site; qualquer outro caso mostra uma confirmação.
esqueci_senha.php	PasswordController::esqueci	quem ainda não está logado	Gera o link de redefinição de senha (demonstrativo: vai para o log e aparece na tela só pelo próprio computador).
redefinir_senha.php?token=	PasswordController::redefinir	quem tem o link (válido 30 min, uso único)	Define a nova senha a partir do token e invalida o link em seguida.
view/perfil/index.php	PerfilController::index	candidato	"Meu perfil": máquina de extração do currículo e formulário do cadastro profissional.
view/perfil/salvar.php	PerfilController::salvar	candidato (POST)	Grava o formulário do perfil, a foto e o telefone, e recalcula o match.
view/perfil/portfolio.php[id=]	PerfilController::portfolio	o dono; demais só se o perfil for público, se a empresa recebeu a candidatura, ou administrador	Mostra o portfólio montado automaticamente; para o dono, mostra também o match com as vagas.
view/perfil/recalcular_match.php	PerfilController::recalcularMatch	candidato (POST)	Botão "Recalcular match" do portfólio.
view/perfil/conta_excluir.php	PerfilController::excluirConta	candidato (POST)	LGPD: exclui a própria conta e todos os dados dela, conferindo a senha antes.
view/perfil/curriculo_upload.php	CurriculoController::upload	candidato (POST)	Recebe o currículo (PDF/DOCX/DOC) e aciona a máquina de extração, montando o relatório.
view/perfil/aplicar_extracao.php	CurriculoController::aplicarExtracao	candidato (POST)	Aplica no perfil os dados que a extração encontrou, conforme o candidato confirmar no relatório.

Endereço	Controller e ação	Quem acessa	O que faz
view/perfil/curriculo_excluir.php	CurriculoController::excluir	candidato (POST)	Exclui um currículo enviado e o arquivo correspondente.
candidatar.php?vaga_id=	CandidaturaController::candidatar	candidato	Formulário (GET) e envio (POST) da candidatura; confere o limite de candidaturas ativas do plano gratuito.
view/perfil/candidatura_cancelar.php	CandidaturaController::cancelar	candidato (POST)	Cancela uma candidatura ainda não avançada no processo (enviada, em análise ou entrevista).
download.php?id=	ArquivoController::download	o dono, o administrador, a empresa que recebeu a candidatura, ou empresa Premium (perfil público)	Entrega o arquivo do currículo, conferindo a permissão a cada pedido.
admin/index.php	AdminController::painel	administrador e empresa	Painel: visão geral com números e gráficos (administrador) ou resumo das próprias vagas (empresa); dispara a manutenção diária.
admin/pages/usuarios.php	AdminController::usuarios	administrador	Lista, cria, edita, ativa/bloqueia e exclui contas de usuário.
admin/pages/categorias.php	AdminController::categorias	administrador	Mantém as categorias que agrupam vagas e cursos.
admin/pages/cursos.php	AdminController::cursos	administrador	Cadastro de cursos, e-books e vídeos, com a extração por ficha ou por texto.
admin/pages/assinaturas.php	AdminController::assinaturas	administrador	Concede, edita, cancela e exclui as assinaturas dos planos VIP e Premium.
admin/pages/vagas.php	EmpresaController::vagas	empresa (as próprias vagas) e administrador (todas)	Cadastro de vagas, com extração por cartaz (foto) ou pelo texto do anúncio.
admin/pages/candidaturas.php	EmpresaController::candidaturas	empresa e administrador	Lista as candidaturas recebidas, ordenadas pelo match; muda o status e o retorno ao candidato.
admin/pages/talentos.php	EmpresaController::talentos	empresa e administrador	Banco de talentos (perfis públicos); busca e download completos só para empresa Premium ou administrador.
admin/pages/empresa_perfil.php	EmpresaController::perfil	empresa	Dados da própria empresa: nome fantasia, CNPJ, setor, site, telefone, logotipo, cidade e descrição.

Quadro 48 – Tabela de rotas (public/index.php)

Fonte: Autoria própria (2026).

As rotas aceitam GET e POST pelo mesmo endereço: é a própria ação do controller que decide o que fazer em cada verbo (mostrar o formulário no GET, processar o envio no POST). Os endereços "" (raiz), "view/perfil" e "admin" são apelidos que o Router resolve para index.php, view/perfil/index.php e admin/index.php, respectivamente. Vale repetir que esses caminhos (vagas.php, view/perfil/index.php, admin/pages/vagas.php...) são só nomes dentro da

tabela do Router: não existem mais como arquivos físicos no disco, mas foram mantidos iguais aos das versões anteriores do projeto para não quebrar links e favoritos salvos.

Cada pasta do projeto que não deveria ser acessível pelo navegador tem um arquivo `.htaccess` próprio, como uma segunda barreira de segurança: mesmo que o `.htaccess` da raiz falhasse ou fosse contornado de alguma forma, cada uma dessas pastas se recusa sozinha a responder a um pedido.

Arquivo	O que bloqueia
<code>.htaccess</code> (raiz)	Reescreve todo pedido para dentro de <code>public/</code> (<code>RewriteRule</code>); como segunda camada, nega acesso direto a arquivos <code>.sql</code> , <code>.txt</code> , <code>.md</code> , <code>.log</code> , <code>.ini</code> , <code>.bak</code> , <code>.env</code> e a qualquer arquivo começado por ponto, mesmo que o <code>mod_rewrite</code> esteja desligado.
<code>public/.htaccess</code>	Deixa passar direto os arquivos que existem de verdade (CSS, JS, imagens); qualquer outro endereço vai para <code>index.php</code> . Impede que um <code>.php</code> colocado por engano dentro de <code>public/</code> (fora do <code>index.php</code>) seja executado, e nega acesso a arquivos ocultos e a cópias de segurança (<code>.bak</code> , <code>.old</code> , <code>.orig</code> , <code>.sql</code> , <code>.log</code> , <code>.ini</code> , <code>.env</code> , <code>.swp</code>).
<code>app/.htaccess</code>	Nega qualquer pedido (<code>Require all denied</code>): o código da aplicação nunca é servido pelo navegador.
<code>config/.htaccess</code>	Nega qualquer pedido: <code>config.php</code> (com os dados do banco) e <code>cacert.pem</code> nunca são servidos.
<code>database/.htaccess</code>	Nega qualquer pedido: <code>schema.sql</code> , <code>seed.sql</code> e <code>resetar_senhas.php</code> nunca são servidos.
<code>docs/.htaccess</code>	Nega qualquer pedido: a documentação e o <code>.docx</code> do TCC nunca são servidos diretamente pelo Apache.
<code>storage/.htaccess</code>	Nega qualquer pedido: uploads, logs e backups nunca são servidos diretamente (os uploads só saem pelo <code>ArquivoController</code> e por <code>download.php</code>).
<code>tests/.htaccess</code>	Nega qualquer pedido: os scripts de teste e as amostras de OCR nunca são servidos pelo navegador.

Quadro 49 – Arquivos `.htaccess` e o que bloqueiam

Fonte: Autoria própria (2026).

`config/config.php` guarda só os valores que mudam de um computador para outro (banco, modo de depuração, limites e pastas); o código que usa cada valor fica em `app/Core`. Quase todas as constantes também podem vir de variáveis de ambiente (`APP_DEBUG`, `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASS`, `DB_SOCKET`, `DOACAO_PIX_CHAVE`, `DOACAO_NOME`, `DOACAO_CIDADE`), o que permite publicar o mesmo código em outro servidor sem editar o arquivo. A constante mais delicada é `DEBUG`: quando ninguém a define por fora, ela só fica `true` na linha de comando (os testes) ou quando o pedido vem do próprio computador (`REMOTE_ADDR` `127.0.0.1` ou `::1`); qualquer acesso de outra máquina recebe sempre as mensagens amigáveis, nunca o detalhe técnico do erro. Isso é o que diferencia o ambiente de desenvolvimento do TCC (onde ver a pilha de erro ajuda a corrigir) de uma eventual publicação na internet, onde a recomendação do próprio projeto é fixar `APP_DEBUG=0` explicitamente.

Constante	Para que serve
DEBUG	Liga (true) ou desliga (false) a exibição do detalhe técnico dos erros. Sem APP_DEBUG definida, só é true na linha de comando ou pelo próprio computador (127.0.0.1:::1); de outra máquina é sempre false, mesmo em ambiente local.
SITE_NAME	Nome do site ("Conecta Vagas DF"), usado no título das páginas e em textos do layout.
FUSO_HORARIO	Fuso horário do PHP ("America/Sao_Paulo"), fixado em bootstrap.php e também aplicado à conexão do banco (Database.php), para datas baterem nos dois lados.
DB_HOST	Endereço do servidor MySQL/MariaDB (padrão 127.0.0.1, o do XAMPP).
DB_PORT	Porta do MySQL/MariaDB (padrão 3306).
DB_NAME	Nome do banco de dados (padrão "tcc_final").
DB_USER	Usuário do banco (padrão "root", o do XAMPP).
DB_PASS	Senha do banco (vazia por padrão, como o XAMPP vem configurado).
DB_SOCKET	Caminho do socket Unix do MySQL, usado só fora do Windows (no Windows a conexão é sempre por TCP).
ROOT_DIR	Caminho absoluto da pasta raiz do projeto no disco (um nível acima de config/).
APP_DIR	Caminho absoluto da pasta app/ (Controllers, Core, DTO, Models, Services, Views).
PUBLIC_DIR	Caminho absoluto da pasta public/ (a única servida pelo Apache).
UPLOAD_DIR	Caminho absoluto de storage/uploads/, onde ficam os arquivos enviados pelos usuários, fora do alcance direto do navegador.
LOG_DIR	Caminho absoluto de storage/logs/, onde ficam os registros internos (redefinições de senha, manutenção diária).
DOACAO_QR_IMAGEM	Caminho da imagem do QR Code de doação Pix exibido no rodapé, usado quando DOACAO_PIX_CHAVE está vazia.
DOACAO_PIX_CHAVE	Chave Pix (opcional) a partir da qual o próprio sistema gera o QR Code de doação (Pix::doacao()), no lugar da imagem fixa.
DOACAO_NOME	Nome do beneficiário mostrado no código Pix "copia e cola" (até 25 letras, sem acento).
DOACAO_CIDADE	Cidade do beneficiário mostrada no código Pix "copia e cola" (até 15 letras, sem acento).
MAX_FILE_SIZE	Tamanho máximo de um currículo enviado (10 MB).
LOGIN_MAX_TENTATIVAS	Quantidade de senhas erradas seguidas, para o mesmo e-mail e a partir do mesmo IP, que pausa o login (8).
LOGIN_MAX_TENTATIVAS_CONTA	Quantidade de senhas erradas para o mesmo e-mail, somando todos os IPs, que pausa o login (20), protege contra um ataque distribuído ao mesmo e-mail.
LOGIN_MAX_TENTATIVAS_IP	Quantidade de senhas erradas de um mesmo IP, somando todos os e-mails tentados, que pausa o login daquele IP (60).
LOGIN_JANELA_MINUTOS	Janela de tempo, em minutos, usada tanto para contar as tentativas quanto para a duração da pausa do login (5).

Quadro 50 – Constantes de config/config.php

Fonte: Autoria própria (2026).

A.4 Controladores (app/Controllers)

No Conecta Vagas DF, o controller é a camada que fica entre o navegador e o restante do sistema. Toda requisição entra por `public/index.php`, o front controller, que prepara a aplicação, abre a sessão, confere a conta logada com `revalidar_sessao()`, envia os cabeçalhos de segurança e, por último, entrega o caminho pedido ao Router (`app/Core/Router.php`). O Router procura o endereço na tabela de rotas, cria o objeto do controller com `new` e chama o método correspondente, sem parâmetros. Cada método público de controller é, portanto, uma ação ligada a um endereço. O controller não escreve SQL nem HTML: ele confere quem está pedindo, valida o token CSRF dos formulários, lê e limpa os campos, chama os Models (os DAOs, que falam com o banco) e os Services (as regras de negócio, como o cálculo do match e as máquinas de extração) e, no fim, redireciona para outra página ou mostra uma tela da pasta `app/Views`.

A classe base `app/Core/Controller.php` é abstrata e bem pequena. Ela tem um único método, protegido, `view(string $tela, array $dados = [])`, que repassa a tela e os dados para `View::render()`, responsável por montar a página dentro do layout do site (cabeçalho e rodapé). Os doze controllers estendem essa classe e são declarados `final`, porque nenhum deles é herdado por outro. A convenção que adotamos é terminar as ações com `$this->view('pasta/tela', get_defined_vars())`: a função `get_defined_vars()` do PHP entrega à tela todas as variáveis calculadas dentro da ação (`$vagas`, `$filtros`, `$title` e as demais), sem precisar montar o array à mão. Por isso, nas subseções a seguir, quando dizemos que uma variável vai para a tela, é por esse mecanismo.

Quase todas as ações seguem o mesmo roteiro. Primeiro vem a trava de acesso: `exigirLogin()` manda para `login.php` quem não está logado, com o aviso “Faça login para continuar.”; `exigirAdmin()` exige uma conta de administrador; e as funções `isCandidato()`, `isEmpresa()` e `isAdmin()` conferem o tipo de conta guardado na sessão. Quem não tem permissão recebe `negar_acesso()`, que mostra a página de erro padrão com o código 403 (ou 404). Depois, a mesma ação atende o GET e o POST: no GET mostra o formulário ou a lista; no POST chama `validar_csrf()`, que compara com `hash_equals()` o campo oculto `csrf` do formulário com o token da sessão e, se não bater, encerra com a página “Sessão expirada” (HTTP 403). Os campos são lidos por `post_str()`, `post_int()` e `get_str()`, que só aceitam valores simples e tiram os espaços das pontas, e os campos de lista fechada (tipo de conta, status, nível) passam por `enum_val()`, que troca qualquer valor fora da lista por um valor padrão.

Depois de gravar, as ações usam o padrão Post/Redirect/Get: guardam uma mensagem com flash() (tipos ok, erro ou info), que o cabeçalho do layout exibe uma única vez, e chamam redirect(), que envia o cabeçalho Location e encerra o script. Assim, apertar F5 depois de salvar não reenvia o formulário. Nas telas de lista do painel, os controllers usam ainda painel_qs(), que remonta o endereço da lista mantendo os filtros, a ordem e a página atuais (o usuário volta exatamente para onde estava), registro_encontrado(), que avisa e volta para a lista quando alguém tenta editar um registro já excluído, lista_ordem() e ordenar_linhas(), para a ordenação por coluna, e paginar(), que corta a lista na página pedida.

Para não repetir o capítulo 08, as regras de negócio (limites, prazos, mensagens e validações) ficam explicadas uma única vez, na seção 8.11, do ponto de vista de quem usa. Aqui mostramos o caminho dentro do código: qual método atende o pedido, quais travas ele aplica e quais classes chama. Quando uma regra é citada, indicamos entre parênteses a subseção de 8.11 onde ela está detalhada.

Controlador	Área do sistema	Quem usa
AdminController	Visão geral do painel e cadastros exclusivos do administrador: usuários, assinaturas, categorias e o catálogo de cursos, e-books e vídeos, com a extração de cursos	Administrador; a visão geral também é usada pela Empresa
ArquivoController	Entrega das imagens enviadas e dos PDFs da biblioteca; download protegido do currículo	Todos (imagens); Candidato dono, Empresa autorizada e Administrador (currículo)
AuthController	Login, cadastro e saída	Qualquer pessoa (sem conta); qualquer conta para sair
CandidaturaController	Envio e cancelamento de candidatura, com o limite do plano gratuito	Candidato
CurriculoController	Envio do currículo com a máquina de extração, aplicação do relatório e exclusão	Candidato
CursoController	Lista e página dos cursos, e-books e vídeos	Todos
EmpresaController	Vagas com a extração pelo cartaz ou anúncio, candidaturas recebidas, Banco de Talentos e dados da empresa	Empresa e Administrador
HomeController	Página inicial e termo de privacidade	Todos
PasswordController	Recuperação de senha por link	Qualquer pessoa (sem conta)
PerfilController	Meu perfil, portfólio, recálculo do match e exclusão da própria conta	Candidato; o portfólio também é visto por Empresa, Administrador e pessoas sem conta
PlanosController	Planos, assinatura e cancelamento	Todos veem; assinar exige conta de Candidato ou Empresa
VagaController	Lista e página das vagas	Todos

Quadro 51 – Os doze controladores e as áreas que atendem
Fonte: Autoria própria (2026).

A.4.1 AdminController

O AdminController (app/Controllers/AdminController.php) cuida do painel administrativo. Ele reúne a visão geral do painel, que o Administrador e a Empresa usam (cada um vendo só o que lhe cabe), e os cadastros exclusivos do Administrador: usuários, assinaturas, categorias e o catálogo de cursos, e-books e vídeos. As telas do painel que a Empresa usa no dia a dia (vagas, candidaturas, Banco de Talentos e dados da empresa) ficam no EmpresaController. Todas as telas deste controller estão em app/Views/admin e recebem a variável \$abaAtiva, que o menu lateral do painel usa para marcar o item aberto.

Método	Rota (public/index.php)	Quem acessa	O que faz
painel()	admin/index.php (também admin)	Administrador e Empresa; o Candidato é levado ao perfil	Monta o painel de indicadores, gráficos e listas recentes; para a Empresa, só os números das vagas e candidaturas dela.
usuarios()	admin/pages/usuarios.php	Administrador	Lista, filtra, cria, edita, ativa, bloqueia e exclui contas, com travas que protegem o próprio administrador e o último administrador ativo.
assinaturas()	admin/pages/assinaturas.php	Administrador	Lista as assinaturas, concede plano a uma conta por um número de dias, edita, cancela e exclui assinaturas.
categorias()	admin/pages/categorias.php	Administrador	Cadastro das áreas de vagas e de cursos: criar, editar, ativar, desativar e excluir.
cursos()	admin/pages/cursos.php	Administrador	Cadastro de cursos, e-books e vídeos, com a caixa de extração (uma ou várias fichas), a importação em lote e a biblioteca de PDFs.
comFonte() (privado e estático)	sem rota	uso interno	Acrescenta à descrição a linha “Fonte original:” com o link de onde veio o PDF guardado na biblioteca, sem repetir se o link já estiver no texto.

Quadro 52 – Ações e métodos do AdminController

Fonte: Autoria própria (2026).

A ação painel() começa com exigirLogin() e, se a conta for de candidato, redireciona para view/perfil/index.php, porque o candidato não tem painel. Em seguida define a janela de 30 dias dos gráficos de atividade e decide o escopo dos números pela variável \$pid: para a Empresa, é o id do perfil dela, obtido por PerfilDAO::obterOuCriar(); para o Administrador, fica null, valor que nos DAOs significa “o sistema todo”. Se uma empresa ainda não tiver perfil, \$pid vira -1, para que ela veja zero vagas em vez de ver tudo. Com esse escopo, a ação busca o resumo das vagas (VagaDAO::resumoPainel()), as candidaturas por status, por dia e o resumo do match (CandidaturaDAO::contarPorStatus(), contarPorDia() e resumoMatch()) e as oito

candidaturas mais recentes (`listarRecentes()`). Os números já chegam agrupados do banco, por consultas com GROUP BY nos DAOs, sem uma consulta por linha.

Com os totais por status, a própria ação calcula o funil de seleção. “Recebidas” é a soma de todas as candidaturas menos as canceladas; “Analisadas” são as recebidas que já saíram do status enviada; “Entrevista” soma quem está em entrevista e quem já foi aprovado; e “Aprovadas” são as de status aprovado. Cada etapa conta quem chegou nela ou passou dela. Quando a conta é de administrador, a ação dispara `manutencao_diaria()`, que no máximo uma vez por dia apaga de `storage/uploads` os arquivos que nenhum registro usa, e carrega os indicadores do sistema inteiro: usuários por tipo e os cinco mais recentes (`UsuarioDAO`), vagas por área e por cidade (`VagaDAO`), o resumo do match das vagas abertas (`MatchDAO::resumoVagasAbertas()`), as assinaturas por plano (`AssinaturaDAO::resumoPorPlano()`), o total de conteúdos publicados e de e-books (`CursoDAO`) e as seis vagas mais recentes. Para a Empresa, carrega o desempenho de cada vaga (`VagaDAO::desempenhoPorEmpresa()`) e se ela tem o plano Premium. A tela exibida é `admin/painel`.

A ação `usuarios()` começa com `exigirAdmin()`. No POST, depois de `validar_csrf()`, o campo oculto `acao` decide o que fazer. Na exclusão, o sistema recusa se o id for o do próprio administrador logado e também se o alvo for o último administrador ativo (`UsuarioDAO::contarAdminsAtivos()` menor ou igual a 1). Passando pelas travas, `UsuarioDAO::excluir()` apaga a conta, e o perfil, os currículos, as vagas e as candidaturas saem junto em cascata, pelas chaves estrangeiras. As ações ativar e desativar chamam `UsuarioDAO::alterarAtivo()`, que aplica as mesmas travas (não bloquear a si mesmo nem o último administrador ativo) e devolve a mensagem de erro ou um texto vazio. Quando uma conta é bloqueada, a sessão aberta dela perde o acesso já na requisição seguinte, porque `revalidar_sessao()` confere o campo ativo no banco a cada página.

Sem ação especial, o POST é a gravação do formulário de usuário. A ação monta o array com nome (até 255 caracteres), e-mail normalizado (minúsculo e sem espaços), tipo (só admin, candidato ou empresa, por `enum_val()` com `UsuarioDAO::TIPOS`), telefone, situação e senha, e valida: nome obrigatório, e-mail válido por `filter_var()`, senha com pelo menos 6 caracteres (obrigatória na criação e, na edição, só quando for preenchida) e no máximo 72 caracteres, que é o limite do `bcrypt`. Também impede que o administrador tire de si mesmo o tipo admin ou se deixe inativo. Com erro, a mensagem vai por flash e a página volta ao formulário de edição. Sem erro, `UsuarioDAO::atualizar()` grava a edição, ou `UsuarioDAO::cadastrar()` cria a conta, já com o perfil quando o tipo é candidato ou empresa. No GET, a ação prepara o registro em

edição (\$edit), o resumo de um usuário para o botão Ver (\$ver, por UsuarioDAO::resumo()), os filtros por tipo, busca e situação (ativo ou bloqueado), a ordenação por coluna e a paginação de 25 por página, e mostra admin/usuarios.

A ação assinaturas() também exige administrador e trata quatro operações no POST. Cancelar chama AssinaturaDAO::cancelarPorId(), que só funciona com assinatura ativa e mantém o registro no histórico. Excluir apaga a assinatura. Conceder busca a conta escolhida e descobre o plano pelo tipo dela, pela constante AssinaturaDAO::PLANO_DO_TIPO: a conta de candidato recebe o plano assinante (Candidato VIP) e a de empresa recebe o plano empresa (Empresa Premium). Se o campo de valor ficar vazio, entra o preço da tabela AssinaturaDAO::PRECOS (R\$ 9,90 e R\$ 49,90). Antes de gravar, um match (true) do PHP confere, em ordem, se a conta existe, se o tipo tem plano, se a conta está ativa, se a duração está entre 1 e 3660 dias e se o valor está entre 0 e 99.999,99. Passando, AssinaturaDAO::assinar() grava numa transação que cancela a assinatura ativa anterior da conta e insere a nova, e a mensagem informa qual assinatura anterior foi cancelada. Qualquer outro POST é a edição de valor, datas e situação, validada dentro de AssinaturaDAO::atualizar(). No GET, a tela admin/assinaturas recebe a lista filtrada por plano, situação, busca e conta, ordenada e paginada de 25 em 25, o resumo por plano e as contas ativas de candidato e de empresa, em ordem alfabética, para o campo de conceder.

A ação categorias() exige administrador e cuida das áreas usadas para classificar vagas e cursos, na tabela categorias, cujo campo tipo vale vaga ou curso. Excluir remove a categoria, e as vagas e os cursos que a usavam ficam sem categoria. Ativar e desativar mudam só a visibilidade: a categoria desativada some dos filtros, mas os itens continuam ligados a ela. Ao salvar, o nome é obrigatório (até 100 caracteres), o tipo é limitado a CategoriaDAO::TIPOS e, na edição, a ação confere antes se a categoria ainda existe. As mensagens de erro do próprio DAO, guardadas em CategoriaDAO::\$erro, são repassadas no flash, como “Já existe uma categoria com esse nome para esse tipo.” ou o aviso de que uma categoria em uso não pode trocar entre vagas e cursos. No GET, a lista em admin/categorias pode ser filtrada por tipo e por busca sem diferenciar acentos (Competencias::normalizar()), ordenada por nome, tipo, situação ou uso, e paginada de 30 em 30.

A ação cursos() é a mais longa do controller. Ela exige administrador, carrega as categorias do tipo curso e, no POST, depois de validar_csrf(), escolhe o caminho pelo campo acao. Excluir chama CursoDAO::excluir(); ativar e desativar publicam ou ocultam o conteúdo da área pública por CursoDAO::alterarAtivo(). A ação pdfs_biblioteca traz para a plataforma o PDF de todos os e-books que ainda abrem no site de origem: ela aumenta o tempo limite do

PHP para 900 segundos, separa os e-books cujo link é um endereço http ou https e pede a `ImagemRemota::pdfs()` que baixe os PDFs, do link direto ou achados dentro da página. Cada PDF baixado é gravado por `CursoDAO::guardarNaBiblioteca()`, que troca o link do e-book pelo arquivo em `storage/uploads` (nome `biblioteca_*.pdf`) e acrescenta à descrição a fonte original, pelo método privado `comFonte()`. Se a gravação falhar, o arquivo baixado é apagado com `apagar_upload_sem_uso()`. A mensagem informa quantos e-books passaram a ter o botão “Baixar” e quantos continuam com “Acessar”, porque o link não levou a um PDF público.

A extração de cursos começa pela ação extrair, com o texto colado na caixa única da tela (campo `texto_anuncio`), e nada é salvo nesta etapa. Primeiro, uma expressão regular verifica se o texto tem linhas no formato de ficha, isto é, começando por “Título:”, com ou sem numeração e negrito. As fichas são a resposta da pesquisa guiada que o administrador faz numa IA de pesquisa externa a partir do prompt montado pela plataforma. Se houver fichas, `ExtracaoCurso::fichas()` separa cada uma e lê os campos rotulados (título, link, PDF, área, descrição, imagem e os demais), usando os nomes das categorias cadastradas. Se vier mais de uma ficha, é uma importação em lote: a ação aumenta o tempo limite para 300 segundos, limita o lote a 100 fichas e chama `ImagemRemota::completar()`, que abre cada link da ficha e descobre se ele é imagem, PDF ou página, para ficar com uma imagem que abre de verdade e, no caso dos e-books, com o PDF achado. Em seguida, cada item recebe a lista de problemas (sem título, sem link válido ou já cadastrado, conferido por `CursoDAO::buscarRepetido()` pelo mesmo link ou pelo mesmo título e instituição) e a lista de alertas (sem imagem, capa tirada da primeira página do PDF, link que não abriu como deveria, conteúdo pago sem preço). Os itens ficam guardados na sessão em `import_cursos` e a página volta para a âncora `#extrair`, onde a tela mostra a prévia com caixas de marcar.

Se veio uma única ficha, a ação completa só ela com `ImagemRemota::completar()`, com tempo limite de 120 segundos. Se o texto não tem formato de ficha, é um texto de divulgação comum, e `ExtracaoCurso::doTexto()` tira dele os campos do cadastro por regras de palavras-chave; quando a regra não reconhece a área ou a instituição, entram os padrões automáticos tirados do catálogo já cadastrado (`PadroesExtracao`). Nos dois casos, a categoria sugerida é procurada pelo nome em `CategoriaDAO::buscarPorNome()` e o resultado vira o array `$form`, que preenche o formulário para o administrador revisar. Se a ficha trouxe um link de imagem, ele fica em `imagem_url`, para ser baixado só ao salvar; se não, o campo `imagem` recebe o caminho que veio na ficha ou o banner da instituição, escolhido por `ExtracaoCurso::capa()`.

A ação `importar_salvar` cadastra as fichas marcadas na prévia. Ela lê os itens guardados na sessão e os índices marcados (campo `itens[]`) e aplica as mesmas regras do formulário: só

passam itens com título, com link http ou https válido e que não estejam repetidos. As imagens dos itens válidos são baixadas de uma vez por `ImagemRemota::baixar()`, que reduz cada imagem para no máximo 800 pixels de largura e grava em `storage/uploads`; se a opção de guardar os PDFs ficou marcada, os PDFs dos e-books são baixados por `ImagemRemota::pdfs()`. No laço de gravação, cada item é conferido de novo contra repetição dentro do próprio lote, recebe a imagem baixada, ou o caminho válido da ficha, ou a imagem padrão do formato (`CursoDAO::imagemPadrao()`), tem tipo, modalidade e nível limitados por `enum_val()` e é gravado por `CursoDAO::salvar()` já publicado. Conteúdo pago sem preço entra como gratuito, para ser revisado depois, e o e-book com PDF baixado passa a apontar para o arquivo da biblioteca. No fim, todo arquivo baixado que não foi usado é apagado com `apagar_upload_sem_uso()`, a prévia sai da sessão e a mensagem resume quantos foram cadastrados, quantos entraram com a imagem padrão e quantos foram pulados. A ação `importar_cancelar` só descarta a prévia.

Sem nenhuma dessas ações, o POST é a gravação do formulário de um conteúdo. A ação monta o array com os campos (título até 255 caracteres; tipo, modalidade e nível por `enum_val()`; duração; gratuito; preço; link; caminho da imagem conferido por `caminho_imagem_valido()`; situação) e padroniza o nome da instituição pelo link oficial, com `FontesCursos::nomeOficial()`. As validações são: título obrigatório; link só com http ou https, ou um PDF da própria biblioteca, o que impede links “javascript:” no botão do curso; categoria somente do tipo curso; conteúdo pago com preço maior que zero; e preço até 99.999.999,99. Os arquivos chegam por três caminhos: o PDF enviado para a biblioteca (`salvar_pdf_biblioteca()`, até 25 MB e conferido pelo conteúdo), que vira o link do conteúdo; a imagem enviada (`salvar_imagem_enviada()`, JPG, PNG ou WEBP até 3 MB); e, se a opção de guardar o PDF na biblioteca estiver marcada e nenhum arquivo tiver sido enviado, o PDF baixado da web pelo link direto ou pelo link oficial. Uma imagem informada por link só é baixada quando não veio arquivo nem caminho, e um link que não baixa não impede o cadastro. Se no fim ainda não houver imagem, entra o banner da instituição ou a imagem padrão da plataforma, com aviso para trocar depois em Editar.

Se houver erros, nenhum arquivo fica órfão: a imagem enviada, o PDF e a imagem baixada são apagados com `apagar_upload_sem_uso()`, o formulário volta preenchido em `$form` e a mensagem lista os erros. Sem erros, `CursoDAO::salvar()` grava o conteúdo; se a imagem ou o PDF mudaram, os arquivos antigos são apagados, e o redirecionamento volta para a mesma aba, filtros e ordem por `painel_qs()`. No GET, a tela `admin/cursos` recebe o formulário (vazio, de edição ou já no formato pedido em `?novo=ebook`), a lista com abas por formato, filtros por

busca, área e situação (publicado, oculto ou com imagem padrão), a ordenação por coluna, a paginação de 25, a contagem de conteúdos com imagem padrão e de e-books ainda na web, as imagens disponíveis nas pastas do projeto e a prévia da importação, se houver uma na sessão.

A.4.2 ArquivoController

O ArquivoController entrega os arquivos enviados pelos usuários. Esses arquivos ficam em storage/uploads, fora da pasta public/, então o navegador nunca chega a eles diretamente: tudo passa por este controller, que decide o que pode sair e para quem. No banco, os caminhos são gravados como assets/uploads/nome-do-arquivo, e a função caminho_upload() traduz esse caminho para a pasta real, aceitando só nomes simples, sem subpastas, sem “..” e sem barras invertidas.

Método	Rota (public/index.php)	Quem acessa	O que faz
imagem(string \$caminho)	assets/uploads/<arquivo>, chamada direta em public/index.php antes de abrir a sessão	Qualquer pessoa	Entrega fotos, logos, cartazes e imagens de vagas e cursos, além dos PDFs da biblioteca de e-books.
download()	download.php?id=	Candidato dono, Administrador e Empresa autorizada	Abre ou baixa o currículo depois de conferir a permissão e o caminho do arquivo.

Quadro 53 – Ações do ArquivoController

Fonte: Autoria própria (2026).

A ação imagem() é a única que não passa pelo Router: public/index.php a chama assim que percebe que o caminho começa com assets/uploads/, antes mesmo de abrir a sessão, porque imagens públicas não precisam dela. O tipo do arquivo é conferido pelo conteúdo, com a classe finfo, e não pela extensão. Só saem imagens JPEG, PNG, WEBP e GIF (o SVG fica de fora, porque pode conter script) e PDFs cujo nome seja de um e-book da biblioteca, conferido por eh_pdf_biblioteca() (nome biblioteca_*). Qualquer outro arquivo, inclusive um currículo, recebe a página 404. Para o que pode sair, a ação envia o tipo correto, o cabeçalho X-Content-Type-Options: nosniff e um cache público de sete dias, já que o nome do arquivo muda a cada envio e o cache nunca mostra uma imagem velha. O PDF da biblioteca abre no próprio navegador (inline) e, para ele, o limite de tempo do PHP é desligado, para que um e-book grande não seja cortado numa internet lenta. O envio é feito por readfile(), que não carrega o arquivo inteiro na memória.

A ação download() começa com exigirLogin(), busca o currículo pelo id com CurrículoDAO::buscar() e mostra 404 se ele não existir. A permissão é montada em etapas: podem abrir o Administrador e o próprio candidato dono do currículo; se não for nenhum dos

dois e a conta for de empresa, pode abrir a empresa que recebeu uma candidatura com aquele currículo (CurriculoDAO::enviadoParaEmpresa()) ou a empresa com plano Premium, desde que o perfil do candidato seja público e esteja ativo, que é o caso do Banco de Talentos. Qualquer outro caso recebe a página 403. Mesmo com o caminho vindo do banco, a ação confere com `realpath()` se o arquivo está de fato dentro de `storage/uploads`, o que impede a leitura de qualquer arquivo fora dessa pasta. Quando quem baixa não é o dono, o download é contado por `CurriculoDAO::registrarDownload()`. Na entrega, o PDF abre no navegador e os arquivos do Word (DOC e DOCX) sempre baixam como anexo, com o nome limpo por `safe_filename()` e o cabeçalho `Cache-Control: private, no-store`, para o currículo não ficar guardado em cache compartilhado.

A.4.3 AuthController

O AuthController cuida do acesso à conta: entrar (login), criar a conta (cadastro) e sair (logout). Ele é usado por quem ainda não entrou no sistema e por qualquer conta logada na hora de sair. A recuperação de senha ficou separada no PasswordController.

Método	Rota (public/index.php)	Quem acessa	O que faz
login()	login.php	Qualquer pessoa sem conta; quem já está logado é redirecionado	Mostra o formulário e autentica, com proteção contra tentativas repetidas de senha.
cadastro()	cadastro.php	Qualquer pessoa (sem conta)	Cria a conta de Candidato ou de Empresa junto com o perfil, já faz o login e leva à próxima etapa.
logout()	view/usuario/logout.php	Conta logada	Encerra a sessão, aceitando só pedidos que vêm do próprio site ou trazem o token.

Quadro 54 – Ações do AuthController
Fonte: Autoria própria (2026).

A ação `login()` trata primeiro o retorno: a tela de planos manda `?voltar=planos` para que, depois de entrar, a pessoa volte para lá. O destino só é aceito se estiver na lista fixa `$voltas` (hoje, só `planos.php`), nunca um endereço digitado pelo usuário, o que evita o redirecionamento para outro site. Quem já está logado vai direto para esse destino ou para a página inicial do seu tipo de conta, dada por `destinoPainel()`: o candidato vai para `view/perfil/index.php`; a empresa e o administrador, para `admin/index.php`. No GET, a ação mostra a tela `auth/login`.

No POST, depois de `validar_csrf()`, o e-mail é normalizado e a senha é lida sem cortar espaços, porque eles podem fazer parte dela. Antes de qualquer conferência, `UsuarioDAO::minutosBloqueioLogin()` consulta a tabela `tentativas_login` e diz se aquele e-mail

ou aquele IP está em pausa. Os três limites e a janela de tempo ficam em `config/config.php` e estão explicados na seção 8.11.4. Se não houver pausa, `UsuarioDAO::autenticar()` confere a senha com `password_verify()`, com a tolerância a erros de digitação descrita na mesma seção.

Na falha, a tentativa é registrada por `registrarFalhaLogin()` e a mensagem é a mesma em todos os casos, para não revelar se o e-mail existe (as mensagens estão na seção 8.11.4). Só no ambiente local, com a constante `DEBUG`, ela diz o motivo exato, para facilitar os testes. No sucesso, as falhas daquele IP e e-mail são apagadas e `iniciar_sessao_usuario()` gera um novo ID de sessão, contra a fixação de sessão, descarta o token CSRF antigo e guarda na sessão o id, o nome, o tipo e a marca da senha, que `revalidar_sessao()` confere a cada requisição.

A ação `cadastro()` redireciona quem já está logado e, no GET, mostra `auth/cadastro`. No POST, valida o CSRF e monta os dados. O tipo passa por `enum_val()` e só pode ser candidato ou empresa, então ninguém se cadastra como administrador pelo site. As regras de validação e o aceite exigido pela LGPD estão na seção 8.11.3. Sem erro, `UsuarioDAO::cadastrar()` insere o usuário com a senha transformada em hash por `password_hash()` e cria o perfil na mesma transação. Em seguida a pessoa já entra no sistema e é levada à tela de dados da empresa (`admin/pages/empresa_perfil.php`) ou ao perfil do candidato com `?novo=1`.

A ação `logout()` existe para impedir que outro site consiga deslogar o usuário com um simples link ou imagem. Ela aceita sair em três casos: POST com o token CSRF válido; GET com o token na URL, no parâmetro `t`, que é o link gerado por `logout_url()`; ou GET vindo de uma página do próprio site, reconhecido pelo cabeçalho `Sec-Fetch-Site: same-origin`, que os navegadores atuais enviam. Nesses casos, `encerrar_sessao()` apaga os dados, o cookie e o ID da sessão, e a pessoa volta para a página inicial. Em qualquer outro caso, a tela `auth/logout` pede a confirmação. Quem nem está logado é mandado direto para `index.php`.

A.4.4 CandidaturaController

O `CandidaturaController` cuida das candidaturas do lado do Candidato: enviar uma candidatura para uma vaga e cancelar uma candidatura. O lado da Empresa, que muda o status e dá o retorno ao candidato, fica em `EmpresaController::candidaturas()`. É aqui que vale a regra do plano gratuito do candidato: até 3 candidaturas ativas ao mesmo tempo, contando as de status enviada, em `_analise` e `entrevista`. Com o plano Candidato VIP, não há limite.

Método	Rota (<code>public/index.php</code>)	Quem acessa	O que faz
<code>candidatar()</code>	<code>candidatar.php?vaga_id=</code>	Candidato	Mostra o formulário (GET) e envia a candidatura (POST), respeitando o limite do plano.

Método	Rota (public/index.php)	Quem acessa	O que faz
cancelar()	view/perfil/ candidatura_cancelar.php (POST)	Candidato	Cancela uma candidatura própria que ainda não passou da etapa de análise.

Quadro 55 – Ações do CandidaturaController

Fonte: Autoria própria (2026).

A ação `candidatar()` exige login e conta de candidato; os outros tipos recebem 403. Ela obtém o perfil do candidato com `PerfilDAO::obterOuCriar()`, lê o id da vaga pela URL ou pelo formulário e confere com `VagaDAO::estaAberta()` se a vaga está aberta, ou seja, ativa, dentro do prazo e de uma empresa com a conta ativa; se não estiver, volta para `vagas.php` com o aviso de que a vaga não recebe mais candidaturas. Também procura candidatura anterior para a mesma vaga: se já existe uma que não foi cancelada, o candidato é levado de volta à vaga com o aviso “Você já se candidatou a esta vaga.”. Em seguida, `AssinaturaDAO::podeCandidatar()` diz se ele ainda pode se candidatar: o VIP sempre pode; no plano gratuito, a resposta traz quantas candidaturas ativas existem e o limite de 3.

No POST, depois de `validar_csrf()`, quem passou do limite do plano (seção 8.11.11) é mandado para `planos.php`. O currículo escolhido precisa ser do próprio perfil (`CurriculoDAO::buscarDoPerfil()`), o que impede enviar o currículo de outra pessoa trocando o id no formulário. A gravação é feita por `CandidaturaDAO::enviarComLimite()`, que conta as candidaturas ativas e grava dentro da mesma transação, com a linha do perfil travada por `SELECT ... FOR UPDATE`: assim, dois envios ao mesmo tempo não conseguem passar do limite. Uma candidatura cancelada para a mesma vaga é reaberta com o status enviada; se não houver, uma nova linha é inserida na tabela candidaturas. No GET, a tela `vagas/candidatar` recebe a lista de currículos do candidato e a informação se ele é VIP. Essa informação sai da própria resposta de `AssinaturaDAO::podeCandidatar()`, que só traz o limite no plano gratuito, então a assinatura é consultada uma única vez por pedido.

A ação `cancelar()` aceita só POST de candidato com o CSRF válido. Ela localiza o perfil do candidato logado e chama `CandidaturaDAO::cancelarDoCandidato()`, cujo UPDATE só muda para cancelada a candidatura desse perfil que ainda estiver nos status enviada ou em_analise. O que acontece quando a candidatura já está numa etapa mais avançada está na seção 8.11.12. O redirecionamento volta para `view/perfil/index.php#candidaturas`.

A.4.5 CurriculoController

O `CurriculoController` cuida do currículo do Candidato: o envio do arquivo com a máquina de extração, a aplicação dos dados escolhidos no relatório da extração e a exclusão do

currículo. A abertura do arquivo para leitura fica em `ArquivoController::download()`. É neste controller que o currículo vira dados do perfil, e é do perfil que saem o portfólio, o match com as vagas, o Banco de Talentos e as candidaturas.

Método	Rota (public/index.php)	Quem acessa	O que faz
<code>upload()</code>	<code>view/perfil/curriculo_upload.php</code> (POST)	Candidato	Recebe o arquivo, extrai os dados, aplica ao perfil, grava o currículo, recalcula o match e abre o relatório da extração.
<code>aplicarExtracao()</code>	<code>view/perfil/aplicar_extracao.php</code> (POST)	Candidato	Aplica ao perfil os dados que a extração encontrou mas não aplicou, quando o candidato os marca no relatório.
<code>excluir()</code>	<code>view/perfil/curriculo_excluir.php</code> (POST)	Candidato	Apaga o currículo (registro e arquivo) e recalcula o match.

Quadro 56 – Ações do `CurriculoController`
Fonte: Autoria própria (2026).

A ação `upload()` exige candidato e só aceita POST. Antes do CSRF, ela trata um caso especial: quando o arquivo passa do limite `post_max_size` do PHP, o formulário chega vazio, sem o token; para não mostrar uma mensagem enganosa de sessão expirada, a ação percebe o POST vazio com `CONTENT_LENGTH` maior que zero e avisa que o arquivo é grande demais. Só então chama `validar_csrf()`. O arquivo do campo `curriculo` passa por uma série de conferências, cada uma com sua mensagem: erro do envio (limite do servidor, arquivo grande, nenhum arquivo), tamanho máximo de 10 MB (constante `MAX_FILE_SIZE`), extensão PDF, DOC ou DOCX e, por fim, a assinatura real do arquivo, lida nos primeiros 8 bytes: %PDF no PDF, PK no DOCX (que é um arquivo ZIP) e a assinatura do formato binário antigo do Word no DOC. Assim, um arquivo apenas renomeado não passa. O arquivo é gravado em `storage/uploads` com nome aleatório (`cv_`, o id do usuário, a data e quatro bytes aleatórios), e uma função registrada com `register_shutdown_function()` apaga o arquivo se a requisição parar antes de ele ser gravado no banco, para não sobrar arquivo órfão na pasta.

Em seguida vem a extração. `ExtracaoCurriculo::extrair()` aciona o `LeitorDocumento`, que tira o texto do arquivo sem API externa (leitor de PDF em PHP puro, leitura do XML do DOCX e leitura do DOC binário), e `ExtracaoCurriculo::metodo()` informa qual método funcionou, para aparecer no relatório. `ExtracaoCurriculo::extrairCampos()` separa o texto nos campos do perfil, aproveitando também as linhas escritas com a maior fonte (`LeitorDocumento::$destaques`), que normalmente são o nome do candidato: nome, e-mail, telefone, data de nascimento ou idade, título profissional, resumo, objetivo, experiências, formação, cursos, habilidades, competências, idiomas, cidade, UF, nível, informações

adicionais, links, CNH, disponibilidade, pretensão salarial e PCD. A extração do currículo funciona só por regras; por ter dados pessoais (LGPD), o currículo não alimenta os padrões automáticos usados na extração de vagas e de cursos. Se a leitura der erro, o arquivo é apagado e o candidato é avisado. Se o `LeitorDocumento` indicar que o conteúdo descompactado passou do limite (`LeitorDocumento::$grandeDemais`, proteção contra a chamada bomba de descompressão), o arquivo também é recusado, com a orientação de salvá-lo de novo como PDF.

Com os campos extraídos, `AplicacaoCurriculo::calcular()` decide, campo a campo, o que vai para o perfil, devolvendo os novos dados, os itens do relatório e os valores pendentes. Campo vazio no perfil recebe o valor do currículo (status aplicado); campo já preenchido mantém o valor do candidato (status mantido) e o valor do currículo fica guardado como pendente, a não ser que ele tenha marcado a opção substituir; listas como habilidades, competências e informações adicionais são unidas sem repetir (status mesclado); e valores iguais ficam com o status igual. A foto embutida no arquivo é procurada por `LeitorDocumento::extrairFoto()` e gravada por `AplicacaoCurriculo::salvarFoto()`, que confere se é JPG ou PNG de verdade e de até 3 MB: se o perfil não tem foto, ela passa a ser a foto do perfil; se já tem, a nova fica pendente para o candidato trocar, se quiser. Uma foto pendente de um relatório anterior que não foi usada é apagada nessa hora. Depois, `AplicacaoCurriculo::salvar()` grava o perfil; se falhar, o arquivo e a foto são apagados e o candidato é avisado.

Os dados que pertencem à conta (tabela `usuarios`) recebem tratamento à parte. O telefone do currículo é gravado se a conta não tem telefone ou se o candidato pediu para substituir; se for diferente do atual, fica pendente. O nome nunca é trocado sozinho: se for diferente do nome da conta, aparece no relatório como sugerido. O e-mail lido é só informativo, porque o e-mail da conta é o login. A idade, quando não há a data de nascimento, e a informação de PCD também entram como informativas. Só então o currículo é gravado na tabela `curriculos` por `CurriculoDAO::salvar()`, com o título, o caminho, o tipo e o texto extraído, e a variável `$gravado` passa a ser verdadeira, o que desarma a limpeza do arquivo. Em seguida, `MatchService::recalcular()` refaz o match do candidato com todas as vagas ativas; se o cálculo falhar, a mensagem avisa, mas o envio não é desfeito.

Se nenhum texto foi lido, como num PDF que é só uma imagem escaneada, o currículo fica salvo, o relatório anterior é descartado e o candidato é orientado a preencher o perfil à mão. Caso contrário, a ação guarda na sessão, em `relatorio_extracao`, o relatório completo: o nome do arquivo, o id do currículo, o método de leitura, a data e a hora, se foi pedido substituir, todos

os itens (conta, foto, campos do perfil e informativos) com rótulo, valor resumido por `AplicacaoCurriculo::resumo()`, valor atual e status, os pendentes, o nome sugerido e o resultado do match. A mensagem conta quantos dados foram aplicados ou mesclados, e o redirecionamento leva a `view/perfil/portfolio.php?relatorio=1`, onde o relatório (`app/Views/partials/relatorio_extracao.php`) aparece para o dono, com a lista do que foi encontrado, aplicado, mantido e do que falta.

A ação `aplicarExtracao()` atende o formulário do relatório em que o candidato marca os dados do currículo que quer usar. Ela exige candidato, POST e CSRF e lê o relatório da sessão; se ele não existir mais, avisa que expirou. Os campos marcados chegam na lista `campos[]`. O nome da conta só é trocado se o candidato marcou o nome e havia um nome sugerido; nesse caso, `UsuarioDAO::atualizarNome()` grava e o nome guardado na sessão é atualizado. O telefone pendente vai para a conta por `UsuarioDAO::atualizarTelefone()`. Para os campos do perfil, a ação parte dos dados atuais (`AplicacaoCurriculo::dtoDe()`), troca a foto pela pendente, se o arquivo ainda existir, e copia cada campo marcado que esteja ao mesmo tempo em `AplicacaoCurriculo::CAMPOS` e na lista de pendentes, o que impede gravar um campo qualquer enviado pelo formulário. Se algo do perfil mudou, `AplicacaoCurriculo::salvar()` grava, e só depois disso a foto antiga é apagada. O relatório é atualizado na sessão (os itens aplicados passam ao status aplicado e saem dos pendentes), o match é recalculado quando o perfil mudou e a página volta ao relatório no portfólio.

A ação `excluir()` exige candidato, POST e CSRF. Ela busca o currículo pelo id e pelo perfil do candidato logado (`CurriculoDAO::buscarDoPerfil()`), de modo que ninguém apaga o currículo de outra pessoa. Se `CurriculoDAO::excluir()` remover o registro, o arquivo é apagado do disco e o match é recalculado. As candidaturas já enviadas com esse currículo continuam registradas, só que sem o arquivo, e a mensagem explica isso ao candidato.

A.4.6 CursoController

O `CursoController` mostra, na área pública, o catálogo de cursos, e-books e vídeos: a lista (`cursos.php`) e a página de cada conteúdo (`curso.php`). Qualquer pessoa pode ver; para o Candidato logado, as páginas acrescentam recomendações ligadas ao match. O cadastro e a edição ficam no painel, em `AdminController::cursos()`.

Método	Rota (public/index.php)	Quem acessa	O que faz
<code>lista()</code>	<code>cursos.php</code> (com ? tipo=ebook ou ? tipo=video)	Todos	Lista um formato por vez, com abas, filtros, atalhos de área, ordenação, recomendações para o candidato e paginação.

Método	Rota (public/index.php)	Quem acessa	O que faz
detalhe()	curso.php?id=	Todos; conteúdo oculto só para o Administrador	Mostra o conteúdo, as competências que ele desenvolve, as vagas que pedem essas competências e outros conteúdos do mesmo formato.

Quadro 57 – Ações do CursoController

Fonte: Autoria própria (2026).

A ação `lista()` mostra um formato por página, sem misturar: cursos (padrão), e-books (?tipo=ebook) ou vídeos (?tipo=video), com o tipo conferido por `enum_val()` e `CursoDAO::TIPOS`. Os filtros são a busca, a área (`categoria_id`) e o preço (gratuito), e a ordem pode ser a padrão (mais recentes), por título ou por instituição. A busca e o preço são aplicados no banco por `CursoDAO::listar(true, ...)`, que só traz conteúdos ativos; o formato e a área são separados no próprio controller, para contar quantos conteúdos cada aba tem e montar os atalhos de área, que mostram só as áreas com conteúdo naquele formato, da maior para a menor. Se o banco estiver fora do ar, a mensagem amigável de `mensagem_erro_banco()` vai para a tela no lugar da lista.

Na aba de cursos, quando quem acessa é um candidato logado, a ação monta a seção “Recomendados para você”. Ela percorre os cinco melhores matches do candidato (`MatchDAO::listarPorCandidato()`), conta as competências que mais aparecem como faltantes nos detalhes desses matches, pega as seis mais frequentes e pede a `Competencias::cursosPara()` os três cursos que melhor cobrem essas competências. A ação também prepara o título e o subtítulo de cada formato, os links das abas e da paginação mantendo os filtros (por funções anônimas, como `$linkLista`) e a paginação de 20 cartões por página, e mostra a tela `cursos/lista`.

A ação `detalhe()` busca o conteúdo pelo id. Se ele não existir, ou estiver inativo e quem acessa não for o Administrador, a resposta recebe o código HTTP 404 e a tela `cursos/nao_encontrado`. Encontrado o conteúdo, a ação extrai as competências que ele desenvolve (`Competencias::doCurso()`) e procura até quatro vagas abertas que pedem essas competências (`Competencias::vagasParaCurso()`). Para o candidato logado, carrega o mapa de match de cada vaga e marca as competências que faltam a ele nos cinco melhores matches, para a tela destacar o que o curso ensina e ainda falta ao candidato. Os outros conteúdos sugeridos são só do mesmo formato, até quatro. A ação ainda prepara os textos da página com as funções de apresentação de `app/Views/partials/componentes.php`, como `pt_secao_formato()` e `pt_acesso_conteudo()`, que decide se o botão é “Baixar”, para um PDF da biblioteca, ou “Acessar”, para um link da web, e mostra `cursos/detalhe`.

A.4.7 EmpresaController

O EmpresaController reúne as telas do painel que a Empresa usa no dia a dia: as vagas, com a máquina de extração de vagas, as candidaturas recebidas, o Banco de Talentos e os dados da empresa. O Administrador também usa as três primeiras, vendo as vagas e as candidaturas de todas as empresas. É aqui que valem as regras do plano da empresa: no Plano Básico gratuito, até 2 vagas abertas ao mesmo tempo, sem destaque e com o Banco de Talentos em modo de prévia; no Empresa Premium, vagas ilimitadas, destaque e Banco de Talentos completo.

Método	Rota (public/index.php)	Quem acessa	O que faz
vagas()	admin/pages/vagas.php	Empresa (só as próprias vagas) e Administrador (todas)	Lista, cria, edita, ativa, pausa, encerra e exclui vagas; extrai a vaga do cartaz (imagem, com OCR) ou do texto de um anúncio.
candidaturas()	admin/pages/candidaturas.php	Empresa e Administrador	Lista as candidaturas recebidas, muda o status e grava o retorno ao candidato; só o Administrador exclui.
talentos()	admin/pages/talentos.php	Empresa e Administrador	Banco de Talentos: candidatos com perfil público, completo no Premium e em prévia no plano básico.
perfil()	admin/pages/empresa_perfil.php	Empresa	Grava os dados da empresa (nome fantasia, CNPJ, setor, site, cidade, UF e apresentação), a logo e o telefone.
imagemDoFormulario() (privado)	sem rota	uso interno	Aceita como imagem da vaga só um arquivo de assets/img, a imagem atual da vaga ou um cartaz lido que ainda existe.
descartarCartazesLidos() (privado)	sem rota	uso interno	Apaga os cartazes lidos nesta sessão que não viraram vaga.

Quadro 58 – Ações e métodos do EmpresaController

Fonte: Autoria própria (2026).

A ação `vagas()` exige login e conta de empresa ou de administrador. No começo, carrega as categorias de vaga, o perfil da empresa ou, para o Administrador, a lista de empresas (para ele escolher em nome de quem publica), e se a conta tem os recursos Premium; o Administrador sempre tem. Também define a função anônima `$vagaPermitida`, usada em toda a ação: dado um `id`, ela devolve a vaga só se o Administrador estiver logado ou se a vaga pertencer ao perfil da empresa logada. Assim, uma empresa nunca altera a vaga de outra, nem trocando o `id` no formulário.

No POST, depois de `validar_csrf()`, a ação `excluir` apaga a vaga permitida, e as candidaturas e os matches dela saem junto. As ações `ativar`, `pausar` e `encerrar` mudam o status com um clique. Ao reativar uma vaga que não estava aberta, se a conta não for de administrador, `AssinaturaDAO::podePublicarVaga()` confere o limite do plano básico, e quem passou do limite é mandado para `planos.php`. A troca é feita por `VagaDAO::alterarStatus()`, que, quando recebe

um limite, conta as vagas ativas e grava o novo status na mesma transação, com a linha do perfil da empresa travada, de modo que dois cliques simultâneos não ultrapassem o limite. Ao ativar, `MatchService::recalcularVaga()` calcula o match da vaga com todos os candidatos, e a mensagem avisa se a data “Inscrições até” já passou, caso em que a vaga só volta à busca depois que a data for renovada.

A máquina de extração de vagas atende duas ações, `ler_cartaz` e `extrair`, e nada é salvo nelas: o resultado só preenche o formulário para revisão. Na ação `ler_cartaz`, a imagem do cartaz chega pelo campo `cartaz` e é gravada por `salvar_imagem_enviada()`, que aceita JPG, PNG ou WEBP de até 8 MB, conferidos pelo conteúdo. Os cartazes lidos antes nesta sessão e não usados são apagados por `descartarCartazesLidos()`, e o novo cartaz é anotado em `$_SESSION['cartazes_lidos']`. A leitura do texto da imagem (OCR) é feita, por padrão, no navegador de quem envia, pelo `Tesseract.js` que acompanha o site (`public/assets/js/leitor-cartaz.js`): o navegador manda, junto com o cartaz, até quatro leituras no formato TSV, no campo `ocr_tsv[]`. Como isso é dado vindo do usuário, `OcrImagem::leiturasDoNavegador()` só aceita uma lista de 1 a 4 textos UTF-8 de tamanho limitado e com o formato de uma leitura do `Tesseract`. `ExtracaoVaga::doImagem()` monta o texto a partir dessas leituras; sem elas, usa o `Tesseract` instalado no servidor, se existir. O cartaz passa a ser a imagem da vaga no formulário. Na ação `extrair`, a empresa colou o texto do anúncio (WhatsApp, Instagram ou site) no campo `texto_anuncio`; se a caixa estiver vazia, a ação volta com aviso; se não, chama `ExtracaoVaga::doTexto()`.

Nos dois casos, a `ExtracaoVaga` separa do texto o cargo, a empresa anunciante, o local, a faixa salarial, o tipo de contratação, o nível, o modelo de trabalho (presencial, remoto ou híbrido), a descrição, os requisitos, os benefícios, o contato, a quantidade de vagas, as competências e a área sugerida. Ela trabalha por regras (palavras-chave e títulos de seção) e, onde a regra não tem pista, usa os padrões automáticos tirados das vagas já cadastradas (`PadroesExtracao`), que se atualizam a cada vaga salva. Também gera avisos sobre o que não conseguiu confirmar, como cargo não encontrado, salário não informado ou leitura de imagem com pouca nitidez (confiança abaixo de 75%).

De volta ao controller, a área sugerida é procurada pelo nome entre as categorias de vaga (`CategoriaDAO::buscarPorNome()`) e descartada se estiver desativada. `ExtracaoVaga::relatorio()` monta o relatório campo a campo, com três situações: lido do anúncio; valor padrão, quando o anúncio não diz e ficou o padrão, que merece conferência; e não encontrado, com a contagem de cada uma. O texto lido do cartaz volta para a caixa de texto, para a empresa corrigir um erro do OCR e extrair de novo sem perder o cartaz, que continua

como imagem graças ao campo `imagem_atual`, conferido por `imagemDoFormulario()`. A ação ainda procura uma vaga aberta parecida (`VagaDAO::buscarParecida()`, com mesmo título, anunciante e cidade) para alertar sobre duplicidade e, por fim, monta `$form` unindo os campos extraídos ao id, à categoria, à empresa, à imagem e aos dados da vaga em edição. A tela `admin/vagas` mostra o relatório e o formulário já preenchido.

Sem uma dessas ações, o POST é a gravação do formulário. Se houver id, a vaga precisa passar por `$vagaPermitida`, ou a resposta é 403. A empresa dona é a do perfil logado; para o Administrador, é a empresa escolhida, que precisa ser um perfil de empresa ativo da lista. Os textos são cortados no tamanho das colunas; o tipo de contratação, o nível, o modelo e o status passam por `enum_val()` com as listas de `VagaDAO`; os salários passam por `decimal_ou_null()` e a data de expiração por `data_ou_null()`. O destaque é recurso Premium: sem o plano, a vaga mantém o destaque que já tinha. As validações são: empresa definida (ou o aviso para completar o perfil da empresa), caminho de imagem válido, título obrigatório, categoria só do tipo vaga, salários entre 0 e 99.999.999,99 e o máximo não menor que o mínimo, UF com duas letras e data de expiração não vencida para vaga ativa. Para não publicar o mesmo anúncio duas vezes, uma vaga ativa igual a outra já aberta gera erro, a não ser que a empresa marque a opção de publicar mesmo assim (campo `publicar_duplicada`).

O limite do plano básico (seção 8.11.18) vale ao publicar uma vaga ativa e ao reativar uma vaga que não estava aberta, inclusive a vaga ativa que tinha expirado e ganhou uma nova data. Nesses casos, se a conta não é de administrador, `AssinaturaDAO::podePublicarVaga()` é consultado e o limite é passado para `VagaDAO::salvarComLimite()`, que conta as vagas ativas da empresa e grava a vaga na mesma transação, com a linha do perfil travada por `SELECT ... FOR UPDATE`; sem limite (Administrador, Premium ou vaga que já estava aberta), a gravação é direta. Quando a transação encontra o limite já atingido, a mensagem é a mesma de `podePublicarVaga()`, a constante `AssinaturaDAO::AVISO_LIMITE_VAGAS`. A imagem enviada no campo `imagem_arquivo` é salva por `salvar_imagem_enviada()`, até 3 MB. Havendo erros, a imagem recém-enviada é apagada, o formulário volta com a imagem escolhida antes (o cartaz lido continua nele) e os erros vão no flash. Gravando, a imagem antiga da vaga é apagada se mudou, os outros cartazes lidos e não usados são descartados, `MatchService::recalcularVaga()` calcula o match e a mensagem informa com quantos candidatos. O redirecionamento volta para a mesma lista por `painel_qs()`.

No GET, a ação prepara a vaga em edição, somente se for permitida (caso contrário, 403), o formulário vazio com valores padrão (Brasília, DF, CLT, júnior, presencial, ativa e a imagem `assets/img/vagas/vaga1.jpg`) e a lista: todas as vagas para o Administrador, ou só as da

empresa, por `VagaDAO::listarPorEmpresa()`. Cada vaga recebe a situação calculada (ativa, pausada, encerrada ou expirada, quando ativa com a data vencida), com a contagem por situação, filtros por situação, busca sem acento por título, empresa, quem publicou e cidade, filtro por empresa para o Administrador, ordenação por coluna e paginação de 25. `OcrImagem::disponivel()` informa à tela se o servidor tem o Tesseract, que serve de reserva para a leitura do cartaz.

A ação `candidaturas()` exige empresa ou administrador. No POST, com o CSRF validado, a lista de status aceitos depende de quem está logado: o Administrador pode usar todos `(CandidaturaDAO::STATUS)`, e a Empresa usa `CandidaturaDAO::STATUS_EMPRESA`, que não tem o status cancelada, porque cancelar é ação exclusiva do candidato. A observação, que é o retorno ao candidato, é limitada a 2.000 caracteres. A exclusão só é aceita do Administrador. Para a Empresa, `CandidaturaDAO::atualizarStatusPorEmpresa()` faz o UPDATE ligado à vaga, exigindo que a vaga seja da empresa e que a candidatura não esteja cancelada; o Administrador usa `CandidaturaDAO::atualizarStatus()`. O candidato vê o novo status e o retorno no próprio perfil. No GET, a tela `admin/candidaturas` recebe a lista filtrada por vaga e por status, na ordem padrão de relevância (candidato VIP primeiro e depois o maior match, como vem do banco) ou ordenada por match, data ou nome, com paginação de 20, e a lista de vagas para o filtro.

A ação `talentos()` exige empresa ou administrador e mostra o Banco de Talentos, com os candidatos de perfil público (`PerfilDAO::listarTalentos()`, com busca por termo e por cidade). O acesso completo, com nome, contato e currículo, é do plano Premium e do Administrador. No plano básico o nome fica oculto, então a busca não pode servir para confirmar se uma pessoa está cadastrada: a ação refiltra o resultado e só mantém os candidatos encontrados pelo título profissional, habilidades, competências, experiências ou cursos, e não pelo nome nem pelos links. Pelo mesmo motivo, a ordenação por nome só aparece para quem vê o nome. A lista é paginada de 24 em 24 e vai para a tela `admin/talentos`.

A ação `perfil()` é exclusiva da Empresa e grava os dados da empresa no perfil (tabela `perfis`): nome fantasia, CNPJ, setor, site, cidade, UF e apresentação, mantendo os demais campos do perfil. As regras são: nome fantasia obrigatório, porque é ele que aparece nas vagas; CNPJ com 14 dígitos; site só com http ou https, conferido por `url_http_valida()`, o que bloqueia endereços “javascript:”; e UF com duas letras. O CNPJ é guardado já formatado (00.000.000/0000-00). A logo é enviada por `salvar_imagem_enviada()` e, se a gravação der certo, a logo antiga é apagada; se falhar, a nova é que é descartada. O perfil da empresa fica

sempre público e com o aceite da LGPD, e o telefone vai para a conta, por `UsuarioDAO::atualizarTelefone()`. A tela é `admin/empresa_perfil`.

Os dois métodos privados protegem os arquivos da vaga. `imagemDoFormulario()` recebe o caminho de imagem que veio do formulário e o passa por `caminho_imagem_valido()`, que só aceita arquivos de imagem de `assets/img` ou `assets/uploads`, sem “..”. Se o caminho for de `assets/uploads` e não for a imagem atual da vaga, ele só é aceito quando o nome do arquivo começa com `cartaz_` e o arquivo existe; qualquer outro upload é recusado, para que uma empresa não aponte a sua vaga para a foto ou a logo de outra conta. `descartarCartazesLidos()` percorre a lista `$_SESSION['cartazes_lidos']` e apaga cada arquivo com `apagar_upload_sem_uso()`, que só apaga o que nenhum registro do banco usa. Ele nunca usa um caminho vindo do formulário, apenas os arquivos enviados pela própria sessão.

A.4.8 HomeController

O HomeController cuida das páginas institucionais, abertas a qualquer pessoa: a página inicial (`index.php`, também atendida pela raiz do site) e o termo de autorização e privacidade (`contrato.php`), que a pessoa precisa aceitar no cadastro, conforme a LGPD.

Método	Rota (public/index.php)	Quem acessa	O que faz
<code>index()</code>	<code>index.php</code> e a raiz do site	Todos	Monta a página inicial: carrossel de boas-vindas, quem somos, vagas, faixa de assinaturas e as vitrines de cursos, e-books e vídeos.
<code>contrato()</code>	<code>contrato.php</code>	Todos	Mostra o termo de autorização e privacidade (LGPD).

Quadro 59 – Ações do HomeController
Fonte: Autoria própria (2026).

A ação `index()` busca as vagas abertas por `VagaDAO::listar(true)`, que traz as vagas em destaque primeiro e depois as mais novas, e os conteúdos ativos por `CursoDAO::listar(true)`. Se o banco falhar, a mensagem amigável vai para a tela em `$dbErro` e a página continua sendo exibida. Com alguém logado, a ação busca a assinatura ativa; se for um candidato, busca também o mapa de match de cada vaga (`MatchDAO::mapaPorCandidato()`) e as candidaturas já enviadas, menos as canceladas, para que os cartões mostrem a porcentagem de match e a marca de candidatura enviada. As vagas são divididas para a vitrine rotativa: cinco cartões na tela e o restante numa fila, que o JavaScript do site (`app.js`) faz entrar um a um. O mesmo é feito para cada formato de conteúdo (cursos, e-books e vídeos), cada um com a sua vitrine. As imagens do carrossel são todas as da pasta `public/assets/img/brasil`, bastando trocar os arquivos, e os créditos das fotos de licença livre ficam no array `$creditos`; se a pasta estiver

vazia, entra a imagem `assets/img/header-bg.jpg`. A tela é `home/index`, com layout largo e a descrição da página para os buscadores.

A ação `contrato()` apenas define o título e mostra a tela `home/contrato`.

A.4.9 PasswordController

O `PasswordController` cuida da recuperação de senha. Como explicado na seção 8.11.5, nesta versão do TCC a recuperação é demonstrativa, sem envio de e-mail real. Do lado do código, o banco guarda só o hash SHA-256 do token, a validade do link fica na constante `UsuarioDAO::REDEFINICAO_MINUTOS` e, no modo de demonstração (constante `DEBUG`), o link é gravado em `storage/logs/redefinicoes_senha.log` com o e-mail mascarado.

Método	Rota (<code>public/index.php</code>)	Quem acessa	O que faz
<code>esqueci()</code>	<code>esqueci_senha.php</code>	Qualquer pessoa (sem conta); quem está logado vai para o painel	Pede o e-mail e gera o link de redefinição, sem revelar se o e-mail existe.
<code>redefinir()</code>	<code>redefinir_senha.php?token=</code>	Quem tem o link	Confere o token e grava a nova senha.

Quadro 60 – Ações do `PasswordController`

Fonte: Autoria própria (2026).

A ação `esqueci()` manda quem já está logado para o painel. No POST, depois do CSRF, o e-mail é normalizado e validado. Para que a tela não vire uma ferramenta de descobrir e-mails cadastrados, ela usa a mesma trava de tentativas do login (`UsuarioDAO::minutosBloqueioLogin()`), mas com o argumento `somarIps: false`, que ignora o limite por conta: assim, o dono ainda consegue pedir o link enquanto a conta sofre tentativas vindas de outros IPs. `UsuarioDAO::criarTokenRedefinicao()` gera o token, ou devolve null se o e-mail não existir, se a conta estiver inativa ou se houver pedidos demais. Só no modo de demonstração o link é registrado no log por `registrar_log()` e, no acesso pelo próprio computador (127.0.0.1 ou ::1), mostrado na tela em `$linkDemo`. Fora desse modo, nada vai para o log, porque o link funciona como uma senha provisória e o e-mail é dado pessoal. A resposta na tela é sempre a mesma, exista ou não o e-mail, e a tela é `auth/esqueci_senha`.

A ação `redefinir()` lê o token pela URL (GET) ou pelo formulário (POST) e procura o usuário dono dele com `UsuarioDAO::buscarPorTokenRedefinicao()`; um token inválido, já usado ou vencido leva de volta a `esqueci_senha.php`. `UsuarioDAO::redefinirSenha()` grava tudo numa transação: marca o token como usado (se outro pedido já o usou, nada é alterado), grava o novo hash da senha, invalida os outros tokens da conta e limpa as tentativas de login daquele e-mail. As sessões abertas com a senha antiga caem na requisição seguinte, porque

revalidar_sessao() compara a marca da senha. As regras da nova senha e as mensagens estão na seção 8.11.5.

A.4.10 PerfilController

O PerfilController é a área do Candidato: o cadastro do perfil (Meu perfil), o portfólio profissional com a máquina de match, o recálculo do match e a exclusão da própria conta. O fluxo que desenhamos é o seguinte: o candidato envia o currículo (CurriculoController::upload()) ou preenche o formulário; quando os campos obrigatórios estão completos, conforme Portfolio::validarCadastro(), a aba Portfólio é liberada no topo; e lá ficam o portfólio montado automaticamente e o match com as vagas. Os campos obrigatórios são telefone, título profissional, cidade, resumo profissional, objetivo, formação e habilidades. O portfólio também pode ser visto por Empresas, pelo Administrador e por pessoas sem conta, conforme a privacidade escolhida pelo candidato.

Método	Rota (public/index.php)	Quem acessa	O que faz
index()	view/perfil/index.php (também view/perfil)	Candidato; empresa e administrador vão para o painel	Mostra o formulário do perfil, os currículos, as candidaturas, o plano e, quando pedido, o relatório da última extração.
salvar()	view/perfil/salvar.php (POST)	Candidato	Grava o perfil, a foto e o telefone, recalcula o match e informa o que falta para liberar o portfólio.
portfolio()	view/perfil/portfolio.php (com ?id= opcional)	Candidato dono; Empresa, Administrador e pessoas sem conta conforme a privacidade	Mostra o portfólio montado com os dados do perfil e, para o dono, a máquina de match.
recalcularMatch()	view/perfil/ recalcular_match.php (POST)	Candidato	Atende o botão “Recalcular match” do portfólio.
excluirConta()	view/perfil/ conta_excluir.php (POST)	Candidato	Apaga a própria conta e todos os dados, com confirmação pela senha (LGPD).

Quadro 61 – Ações do PerfilController

Fonte: Autoria própria (2026).

A ação index() exige login; se a conta não for de candidato, redireciona para o painel. Ela obtém o perfil do candidato com PerfilDAO::obterOuCriar(), que cria um perfil vazio se ainda não existir, e reúne o que a tela precisa: se ele é VIP, a assinatura ativa e quantas candidaturas ativas ele tem (para mostrar o uso do limite do plano gratuito), a lista de currículos, as candidaturas com o status e o retorno das empresas, a validação do cadastro

(Portfolio::validarCadastro(), com o percentual de completude) e, quando a URL traz ?relatorio, o relatório da última extração guardado na sessão. A tela é perfil/index.

A ação salvar() aceita só POST de candidato com o CSRF válido. Ela lê os dezesseis campos de texto do perfil (título profissional, resumo, cidade, UF, habilidades, experiências, formação, cursos complementares, informações adicionais, idiomas, competências, disponibilidade, objetivo, links, CNH e pretensão salarial), além da opção de perfil público, do nível de experiência (enum_val() com PerfilDAO::NIVEIS) e da data de nascimento (data_ou_null(), que só aceita uma data válida no formato ano-mês-dia). A UF precisa ter duas letras e a CNH precisa ser uma categoria válida (A, B, C, D, E, AB, AC, AD ou AE). Os links são separados por espaço, vírgula ou ponto e vírgula, e só ficam os que têm formato de endereço web, um por linha. A foto é opcional (JPG, PNG ou WEBP até 3 MB) e só é salva depois de validado o resto do formulário; a foto antiga só é apagada quando o perfil já foi gravado com a nova e, se a gravação falhar, quem é apagada é a nova.

Depois de gravar o perfil por PerfilDAO::salvar(), a ação grava o telefone, que pertence à conta, na tabela usuarios, e chama MatchService::recalcular() para refazer o match. Por fim, compara a validação do cadastro antes e depois da gravação: se agora está completo, avisa que a aba Portfólio foi liberada (ou, se já estava, que o portfólio e o match foram atualizados); se não, lista os campos que ainda faltam. O redirecionamento volta para view/perfil/index.php.

A ação portfolio() atende dois casos. Sem id na URL, é o portfólio do próprio candidato logado, e a ação exige login de candidato. Com ?id=, é o portfólio de um candidato qualquer, buscado por PerfilDAO::buscarPorId(). Nos dois casos, o perfil precisa ser de candidato e estar ativo, senão a resposta é 404. Para o dono, o portfólio só abre depois que o cadastro estiver validado; enquanto faltar algo, ele volta ao formulário com a lista do que falta, ou ao relatório da extração, se veio dele. Ainda para o dono, a ação carrega a máquina de match: os matches do candidato (se não houver nenhum, recalcula na hora), os cursos ativos para as recomendações e se ele é VIP. O candidato do plano gratuito vê os três melhores matches, e o VIP vê todos.

A parte de privacidade define quem vê o quê. Pode ver o portfólio o dono, o Administrador, a empresa que recebeu candidatura desse candidato (CandidaturaDAO::empresaRecebeuDoCandidato()) ou qualquer pessoa, se o perfil for público; nos outros casos, a resposta é 403, com a mensagem de que o candidato deixou o perfil privado. Os dados de contato aparecem para o dono, o Administrador, a empresa que recebeu a candidatura e a empresa Premium. A empresa sem Premium e sem candidatura desse candidato vê só a prévia, igual à do Banco de Talentos do plano básico: o nome cortado nas quatro

primeiras letras e seguido de asteriscos, só a inicial no lugar da foto e um trecho curto do resumo. Pessoas sem conta e outros candidatos veem o portfólio público completo, mas sem o contato.

Em seguida, a ação organiza os dados do perfil para a tela com os métodos da classe `Portfolio`: experiências e formação em itens padronizados, listas de habilidades, cursos e idiomas, e links. Ela junta às informações adicionais a CNH e a disponibilidade, quando ainda não aparecem no texto, detecta menção a PCD, monta o local no formato “Ceilândia, Brasília - DF” para as regiões administrativas do DF e calcula as iniciais do nome. A tela é perfil/portfólio, que mostra também o relatório da extração ao dono quando a URL traz ?relatorio=1.

A ação `recalcularMatch()` aceita só POST de candidato com o CSRF válido, chama `MatchService::recalcular()` para o perfil do candidato e informa quantas vagas foram analisadas. Se o cálculo falhar, uma mensagem de erro é mostrada sem derrubar a página. O redirecionamento volta para a seção de match do portfólio (`#match`).

A ação `excluirConta()` atende o direito do titular de apagar os próprios dados, previsto na LGPD; as confirmações pedidas e as mensagens estão na seção 8.11.14. Ela exige candidato, POST e CSRF, confere a senha com `UsuarioDAO::senhaConfere()` e aplica a mesma trava de tentativas do login, registrando cada senha errada em `tentativas_login`. `UsuarioDAO::excluir()` apaga a conta, e o perfil, os currículos, as candidaturas, os matches e as assinaturas saem em cascata pelas chaves estrangeiras. Os arquivos (currículos e foto) e as tentativas de login guardadas pelo e-mail são apagados à parte. Por fim, a sessão é encerrada e uma nova é aberta só para levar a mensagem até a página inicial.

A.4.11 PlanosController

O `PlanosController` cuida da tela de planos e assinaturas (`planos.php`). Os dois planos pagos, Candidato VIP e Empresa Premium, com preços e benefícios, estão descritos na seção 8.11.13, e a cobrança é demonstrativa. No código, as regras de cada plano ficam em `AssinaturaDAO`; o controller só recebe o pedido e chama os métodos certos.

Método	Rota (public/index.php)	Quem acessa	O que faz
<code>index()</code>	<code>planos.php</code>	Todos veem; assinar e cancelar exigem login	Mostra os planos, a assinatura ativa e o histórico (GET) e processa assinar e cancelar (POST, campo <code>acao</code>).

Quadro 62 – Ação do `PlanosController`
Fonte: Autoria própria (2026).

A ação `index()` carrega, para quem está logado, a assinatura ativa e o histórico de assinaturas da conta. No POST, depois de `validar_csrf()`, quem não está logado vai para o login. O campo `acao` tem três valores: `assinar_candidato` e `assinar_empresa` conferem o tipo da conta e chamam `AssinaturaDAO::assinar()`, e `cancelar` chama `AssinaturaDAO::cancelar()`, que mantém o histórico. O método `assinar()` confere de novo, dentro de uma transação, se a conta está ativa e se o tipo combina com o plano, cancela qualquer assinatura ativa anterior e insere a nova com início na data de hoje e fim 30 dias depois. Qualquer outro valor recebe “Ação inválida.”. Todas as respostas usam flash e redirecionam de volta para `planos.php`; no GET, a tela é `planos/index`.

A.4.12 VagaController

O `VagaController` mostra as vagas na área pública: a lista com filtros (`vagas.php`) e a página de cada vaga (`vaga.php`). Qualquer pessoa pode ver; o Candidato logado vê também a porcentagem de match e as candidaturas que já enviou. O cadastro e a edição das vagas ficam no painel, em `EmpresaController::vagas()`.

Método	Rota (public/index.php)	Quem acessa	O que faz
<code>lista()</code>	<code>vagas.php</code>	Todos	Lista as vagas abertas com busca, filtros, abas por área, ordenação por match e paginação.
<code>detalhe()</code>	<code>vaga.php?id=</code>	Todos; vaga pausada, encerrada ou vencida só para a Empresa dona e o Administrador	Mostra a vaga no formato de anúncio, com o match, os cursos relacionados e outras vagas.

Quadro 63 – Ações do `VagaController`
Fonte: Autoria própria (2026).

A ação `lista()` lê os filtros da URL: busca (`q`), cidade, área (`categoria_id`), nível, modelo de trabalho (`remoto`) e tipo de contratação. A área só vale se for uma categoria de vaga ativa; um `id` de categoria de curso é ignorado. A ação busca todas as vagas abertas e, se houver algum filtro, faz a consulta filtrada (`VagaDAO::listar(true, $filtros)`); sem filtro, reaproveita a mesma lista. Para o candidato logado, carrega o mapa de match e as candidaturas enviadas e, se ele pedir `?ordem=match`, ordena as vagas da maior para a menor porcentagem. A paginação é de 20 cartões por página. A ação também conta as vagas por área para as abas, monta as etiquetas dos filtros em uso, cada uma com o link que a remove (por uma função anônima, `$qs`, que mantém os outros filtros), e a trilha de navegação. O título da página muda conforme a área escolhida, e a tela é `vagas/lista`.

A ação `detalhe()` busca a vaga e confere se ela está aberta. Uma vaga fechada só aparece para a empresa dona e para o Administrador; para os demais, e para um id inexistente, a resposta é 404 com a tela `vagas/nao_encontrada`. A visualização é contada uma vez por sessão, guardando o id em `$_SESSION['vagas_vistas']`, e a visita da empresa dona não conta. A ação extrai as competências da vaga (`Competencias::daVaga()`) e, para o candidato logado, busca o match (`MatchDAO::buscar()`) e a candidatura existente, para recomendar os cursos que ensinam o que falta; sem match, usa `Competencias::cursosParaVaga()`. O que o candidato vê nessa página está na seção 8.11.1.

As outras vagas sugeridas são até cinco, primeiro da mesma área e depois as mais recentes. Por fim, a ação prepara o anúncio para a tela: a faixa salarial (`salario_texto()`), os requisitos e os benefícios em itens, os contatos clicáveis (o WhatsApp abre com uma mensagem pronta sobre a vaga), a data de expiração, a trilha de navegação, o texto para compartilhar e a imagem para a pré-visualização do link em redes sociais (`$ogImagem`). A tela é `vagas/detalhe`.

A.5 Modelos e objetos de dados (app/Models e app/DTO)

Nesta seção documentamos a camada de acesso a dados do sistema, formada pelas classes DAO (Data Access Object) da pasta `app/Models` e pelos DTOs (Data Transfer Object) da pasta `app/DTO`. Cada DAO corresponde a uma tabela do banco `tcc_final` e concentra todo o SQL daquela tabela: criar, buscar, listar, atualizar e excluir registros, além das contagens usadas pelos painéis administrativos. Dessa forma, os controllers (`app/Controllers`) nunca escrevem SQL diretamente; eles chamam os métodos do DAO correspondente e recebem de volta arrays associativos ou valores já prontos para a tela. Os DTOs, por sua vez, não têm nenhuma regra de negócio: servem apenas para levar os dados de um formulário, ou da extração automática de um currículo, de um cartaz ou de uma ficha de curso, até o método `salvar()` do DAO, com um getter para cada coluna e, no caso do `PerfilDTO`, o método `tem()` que informa se um campo chegou a ser enviado, para não sobrescrever com NULL um dado que a tela atual não trata. Por trás de todos os DAOs está a classe `app/Core/Database.php`, que implementa o padrão Singleton: a primeira chamada a `Database::getConexao()` abre uma única conexão PDO com o MySQL/MariaDB para toda a requisição, e as chamadas seguintes reaproveitam essa mesma conexão. A conexão é configurada com `PDO::ERRMODE_EXCEPTION`, para erro de banco virar exceção PHP, `PDO::ATTR_EMULATE_PREPARES=false`, para os prepared statements serem de verdade, executados pelo próprio MySQL, o que impede injeção de SQL, e `charset utf8mb4`, com o fuso horário do PHP aplicado à sessão do banco por `SET time_zone`, para que

NOW() e CURDATE() batam com a hora do servidor. Por isso, do primeiro ao último DAO desta seção, toda consulta é montada com marcadores de posição (?) e os valores entram só na chamada de execute(), nunca concatenados na string do SQL. Vale registrar também que as listagens completas, como VagaDAO::listar() e CursoDAO::listar(), devolvem todas as linhas que atendem aos filtros, sem paginação por LIMIT/OFFSET no SQL; o único recorte por quantidade fixa que aparece nos DAOs é o das listagens curtas dos painéis, como CandidaturaDAO::listarRecentes() e UsuarioDAO::listarRecentes(), que usam LIMIT para mostrar só os últimos registros nos widgets da visão geral.

A.5.1 AssinaturaDAO

A classe AssinaturaDAO (app/Models/AssinaturaDAO.php) é responsável pela tabela assinaturas e também concentra as regras dos dois planos pagos da plataforma. As constantes PLANOS guardam os dois planos possíveis, assinante (Candidato VIP) e empresa (Empresa Premium); STATUS guarda as três situações de uma assinatura, ativa, cancelada e expirada; PRECOS traz o valor mensal demonstrativo de cada plano, R\$ 9,90 para assinante e R\$ 49,90 para empresa; e PLANO_DO_TIPO liga o tipo de conta, candidato ou empresa, ao plano que ela pode assinar. Os limites do plano gratuito também são constantes: LIMITE_CANDIDATURAS_GRATIS (3 candidaturas ativas) e LIMITE_VAGAS_GRATIS (2 vagas ativas). As telas de planos, a página inicial, o painel e as mensagens leem esses números daqui, e AVISO_LIMITE_CANDIDATURAS e AVISO_LIMITE_VAGAS guardam a mensagem de limite atingido. Assim, mudar um limite é mudar uma linha. Sempre que uma instância do DAO é criada, o construtor chama o método privado expirarAssinaturas(), que atualiza para expirada toda assinatura vencida (data_fim menor que a data de hoje) e, para cada empresa que perdeu o Premium, chama removerDestaqueSemPremium() para tirar o destaque das vagas dela; uma variável estática garante que essa checagem rode só uma vez por requisição, mesmo com vários DAOs sendo instanciados na mesma página.

Método	O que faz	Tabelas que lê ou grava
__construct()	Ao instanciar o DAO, chama expirarAssinaturas() para atualizar o status das assinaturas vencidas (roda só uma vez por requisição).	Lê e grava em assinaturas; grava em vagas ao remover destaque
buscarAtivaPorUsuario(usuarioId)	Busca a assinatura ativa e ainda dentro do prazo (data_fim maior ou igual a hoje) do usuário, a mais recente.	Lê assinaturas
isCandidatoVip(usuarioId)	Diz se o usuário tem assinatura ativa do plano assinante (Candidato VIP).	Lê assinaturas
isEmpresaPremium(usuarioId)	Diz se o usuário tem assinatura ativa do plano empresa (Empresa Premium).	Lê assinaturas

Método	O que faz	Tabelas que lê ou grava
assinar(usuarioId, plano, valor, dias=30)	Cria uma assinatura nova dentro de uma transação: confere se o tipo de conta pode assinar aquele plano, cancela qualquer assinatura ativa anterior da conta e grava a nova com data_fim igual a hoje mais os dias informados.	Lê usuarios e assinaturas; grava em assinaturas
cancelar(usuarioId)	Cancela a assinatura ativa da conta, mantendo o histórico, e, se for empresa, remove o destaque das vagas dela.	Grava em assinaturas; aciona atualização em vagas
removerDestaqueSemPremium(usuarioId)	Tira o destaque (destaque=0) de todas as vagas da empresa quando ela não tem mais Premium vigente.	Lê assinaturas e perfis; grava em vagas
listarPorUsuario(usuarioId)	Lista o histórico de assinaturas da conta, da mais recente para a mais antiga.	Lê assinaturas
contarCandidaturasAtivas(perfilCandidatoId)	Conta as candidaturas do candidato nos status enviada, em análise ou entrevista (a condição CandidaturaDAO::ATIVAS).	Lê candidaturas
contarVagasAtivas(perfilEmpresaId)	Conta as vagas da empresa com status ativa e ainda dentro do prazo (a condição VagaDAO::ATIVA).	Lê vagas
podeCandidatar(usuarioId, perfilCandidatoId)	Confere o limite do plano gratuito, 3 candidaturas ativas (LIMITE_CANDIDATURAS_GRATIS); candidato VIP não tem limite. Devolve se é permitido e o motivo; no plano gratuito, devolve também quantas candidaturas ativas há e o limite.	Lê assinaturas e candidaturas
podePublicarVaga(usuarioId, perfilEmpresaId)	Confere o limite do plano básico, 2 vagas ativas (LIMITE_VAGAS_GRATIS); empresa Premium não tem limite. Mesmo formato de retorno de podeCandidatar().	Lê assinaturas e vagas
resumoPorPlano()	Painel do administrador: para cada plano, soma quantas assinaturas estão vigentes, canceladas, expiradas e a receita mensal das vigentes.	Lê assinaturas
listarTodas(filtros)	Lista todas as assinaturas com os dados da conta (nome, e-mail, tipo) e se estão vigentes; aceita filtros por plano, status, usuário e um termo de busca por nome ou e-mail.	Lê assinaturas e usuarios
buscar(id)	Busca uma assinatura pelo id, já com os dados da conta.	Lê assinaturas e usuarios
atualizar(id, dados)	Edição pelo administrador: valida datas, valor e situação, cancela outras assinaturas ativas da mesma conta se esta for marcada como ativa, grava as mudanças numa transação e ajusta o destaque das vagas se a empresa perdeu o Premium.	Lê e grava em assinaturas; aciona atualização em vagas
cancelarPorId(id)	Cancela uma assinatura específica pelo id, sem apagar o histórico.	Grava em assinaturas
excluir(id)	Apaga o registro da assinatura do histórico.	Grava em assinaturas

Quadro 64 – Métodos públicos de AssinaturaDAO
Fonte: Autoria própria (2026).

A consulta mais importante da classe é a de `resumoPorPlano()`, usada na visão geral do administrador: ela agrupa a tabela assinaturas por plano e usa `SUM()` sobre expressões booleanas, como `status igual a ativa` e `data_fim maior ou igual a hoje`, para contar, numa única consulta, quantas assinaturas estão vigentes, canceladas ou expiradas, e soma o valor das

vigentes como receita. O método assinar() é a transação mais sensível: abre uma transação, confere que o tipo de conta, lido de usuarios, combina com o plano pedido, cancela qualquer assinatura ativa anterior da mesma conta, para nunca existirem duas assinaturas pagas ativas ao mesmo tempo, e só então insere a nova, com data_fim calculada pelo próprio MySQL (DATE_ADD(CURDATE(), INTERVAL ? DAY)); qualquer falha no meio do caminho desfaz tudo com rollBack(). Já atualizar() é a tela de edição do administrador: antes de gravar, confere que a data de fim não é anterior à de início e que uma assinatura marcada como ativa não tem data de fim no passado, e, assim como em assinar(), cancela as demais assinaturas ativas da conta para preservar a regra de uma só assinatura vigente por vez.

A.5.2 CandidaturaDAO

CandidaturaDAO (app/Models/CandidaturaDAO.php) cuida da tabela candidaturas, que representa o vínculo entre um candidato (perfil) e uma vaga. A constante STATUS lista os seis status possíveis, enviada, em_analise, entrevista, aprovado, rejeitado e cancelada; STATUS_EMPRESA repete a lista sem o cancelada, porque cancelar uma candidatura é uma decisão exclusiva do candidato. ATIVAS guarda a condição SQL de candidatura em andamento (status enviada, em_analise ou entrevista); ela é usada na contagem do limite em enviarComLimite() e em AssinaturaDAO::contarCandidaturasAtivas(), para a tela e a gravação contarem da mesma forma. A classe guarda numa constante privada, SELECT_EMPRESA, o SELECT completo usado nas listagens da empresa e do administrador: ele junta candidaturas com vagas, perfis e usuarios dos dois lados, candidato e empresa, com matches, para trazer a nota e o nível do match, e com curriculos, para o título e o tipo do arquivo enviado, além de uma subconsulta EXISTS que marca is_vip quando o candidato tem o plano assinante ativo.

Método	O que faz	Tabelas que lê ou grava
enviarComLimite(pid, vid, curriculoId, carta, limite)	Envia ou reenvia, se a candidatura anterior estava cancelada, uma candidatura dentro de uma transação que trava a linha do perfil (FOR UPDATE) e confere o limite de candidaturas ativas antes de gravar, para dois envios simultâneos não ultrapassarem o limite do plano gratuito. limite igual a null significa candidato VIP, sem limite.	Lê e trava perfis; lê e grava candidaturas
buscarDoCandidato(pid, vid)	Busca a candidatura de um candidato a uma vaga específica.	Lê candidaturas
empresaRecebeuDoCandidato(perfilCandidatoId, perfilEmpresaId)	Diz se a empresa já recebeu alguma candidatura, não cancelada, desse candidato; libera o contato no portfólio mesmo sem plano Premium.	Lê candidaturas e vagas
listarPorCandidato(pid)	Lista as candidaturas do candidato, da mais recente para a mais antiga, com o título, cidade e nome da empresa da vaga.	Lê candidaturas, vagas, perfis e usuarios

Método	O que faz	Tabelas que lê ou grava
listarPorEmpresa(perfil EmpresaId, vagaId=0, status="")	Lista as candidaturas recebidas pela empresa, com filtro opcional por vaga e por status.	Lê candidaturas, vagas, perfis, usuarios, matches, curriculos e assinaturas
listarTodas(vagaId=0, status="")	Mesma listagem, sem restringir por empresa; usada na visão do administrador.	As mesmas tabelas de listarPorEmpresa
atualizarStatus(id, status, obs="")	Troca o status e a observação de uma candidatura; uso do administrador, sem restringir por empresa.	Grava em candidaturas
atualizarStatusPorEmpresa(id, perfilEmpresaId, status, obs="")	Mesma troca, mas só se a candidatura for de uma vaga da empresa informada e ainda não tiver sido cancelada pelo candidato.	Lê e grava em candidaturas e vagas
cancelarDoCandidato(id, pid)	O candidato cancela a própria candidatura, só quando ela ainda está enviada ou em _analise.	Grava em candidaturas
excluir(id)	Apaga a candidatura.	Grava em candidaturas
contarPorStatus(perfilEmpresaId=null)	Painel: conta as candidaturas agrupadas por status, com todos os status presentes mesmo quando zerados.	Lê candidaturas, e vagas quando filtra por empresa
contarPorDia(dias=30, perfilEmpresaId=null)	Painel: candidaturas recebidas por dia nos últimos N dias, sem as canceladas, com os dias sem envio aparecendo como zero.	Lê candidaturas, e vagas quando filtra por empresa
resumoMatch(perfilEmpresaId=null)	Painel: total de candidaturas, quantas têm match calculado, a média da pontuação e a quantidade em cada faixa, baixo, médio, alto e excelente.	Lê candidaturas, vagas e matches
listarRecentes(limite=8, perfilEmpresaId=null)	As candidaturas mais recentes, com as mesmas colunas da listagem completa, limitadas à quantidade informada.	As mesmas tabelas de listarPorEmpresa

Quadro 65 – Métodos públicos de CandidaturaDAO

Fonte: Autoria própria (2026).

O método privado `listar()`, chamado por `listarPorEmpresa()` e `listarTodas()`, monta o filtro dinamicamente, por vaga e por status, e ordena o resultado por `is_vip` decrescente, depois por `c.status` igual a cancelada, expressão que vale 0 ou 1 e joga as canceladas para o fim, depois pela pontuação do match decrescente e por último pela data de candidatura; essa ordenação é o que garante que a empresa veja primeiro os candidatos VIP e, dentro de cada grupo, os de maior compatibilidade. Ele também decodifica, em PHP, o JSON guardado em `match_detalhes` antes de devolver as linhas. A transação mais sensível da classe é `enviarComLimite()`: ela usa `SELECT` com `FOR UPDATE` na linha do perfil do candidato para travá-la antes de contar quantas candidaturas ativas já existem, de forma que duas candidaturas enviadas ao mesmo tempo pelo mesmo candidato não furem o limite de três do plano gratuito; a chave única (`perfil_candidato_id`, `vaga_id`) da tabela permite reaproveitar uma candidatura cancelada em vez de criar uma linha nova. Para o painel, `contarPorStatus()` e `contarPorDia()` evitam uma consulta por status ou por dia: `contarPorStatus()` agrupa com `GROUP BY` status e completa com zero os status sem nenhuma candidatura, e `contarPorDia()` gera os últimos N dias em PHP e preenche com o total de cada dia que vier do SQL, mostrando zero nos dias sem candidatura.

O método privado `existe(id)` serve de reforço em `atualizarStatus()`: quando o `UPDATE` não muda nenhuma linha porque os valores já eram iguais, ele confirma que a candidatura realmente existe antes de devolver sucesso.

A.5.3 CategoriaDAO

`CategoriaDAO` (`app/Models/CategoriaDAO.php`) atende a tabela `categorias`, que guarda as áreas usadas para agrupar vagas e cursos. A constante `TIPOS` define os dois usos possíveis de uma categoria, vaga ou curso, e a propriedade pública `erro` guarda a mensagem do último problema ao salvar ou excluir, para a tela exibir ao usuário.

Método	O que faz	Tabelas que lê ou grava
<code>listar(tipo=null, apenasAtivas=false)</code>	Lista as categorias, com a contagem de quantas vagas ou cursos, conforme o tipo, usam cada uma; filtra por tipo e, se pedido, só as ativas.	Lê categorias, vagas e cursos
<code>salvar(dados, id=0)</code>	Cria ou atualiza uma categoria; ao editar, impede trocar o tipo, de vaga para curso ou o contrário, se a categoria já estiver em uso, para não deixar vagas presas a uma categoria de curso.	Lê vagas, cursos e categorias; grava em categorias
<code>buscar(id)</code>	Busca uma categoria pelo id.	Lê categorias
<code>buscarPorNome(nome, tipo)</code>	Busca uma categoria pelo nome e tipo; usada para evitar duplicidade antes de cadastrar vagas ou cursos extraídos automaticamente.	Lê categorias
<code>alterarAtivo(id, ativo)</code>	Ativa ou desativa a categoria com um clique.	Grava em categorias
<code>excluir(id)</code>	Exclui a categoria; vagas e cursos que a usavam ficam sem categoria, pois a chave estrangeira é <code>ON DELETE SET NULL</code> .	Grava em categorias

Quadro 66 – Métodos públicos de `CategoriaDAO`
Fonte: Autoria própria (2026).

A consulta de `listar()` é a que mais chama atenção: em vez de uma subconsulta fixa, ela usa um `IF(c.tipo igual a vaga, ..., ...)` para decidir, linha a linha, se o `em_uso` de cada categoria vem da contagem de vagas ou da contagem de cursos, já que uma categoria de vaga nunca é usada por um curso e vice-versa. Essa mesma lógica é reaproveitada por `salvar()`, numa consulta separada que soma as duas tabelas, para bloquear a troca de tipo de uma categoria que já está em uso; sem essa trava, mudar uma categoria de vaga para curso deixaria vagas antigas apontando para uma categoria do tipo errado.

A.5.4 CurriculoDAO

`CurriculoDAO` (`app/Models/CurriculoDAO.php`) atende a tabela `curriculos`, onde ficam os arquivos de currículo, PDF, DOCX ou DOC, enviados pelo candidato e o texto que a extração conseguiu ler de cada um, usado pelo `MatchService` para calcular a compatibilidade com as vagas.

Método	O que faz	Tabelas que lê ou grava
listarPorPerfil(pid)	Lista os currículos do perfil, do mais recente para o mais antigo.	Lê currículos
buscar(id)	Busca um currículo pelo id, sem conferir o dono.	Lê currículos
buscarDoPerfil(id, pid)	Busca um currículo pelo id, só se pertencer ao perfil informado.	Lê currículos
ultimoTexto(pid)	Devolve o texto extraído do currículo mais recente do perfil que tenha texto; vazio quando nenhum currículo tem texto extraído.	Lê currículos
ultimosTextos()	Devolve, numa única consulta, o texto do currículo mais recente de cada perfil que tem currículo com texto; usado para recalculando o match de uma vaga contra todos os candidatos de uma vez.	Lê currículos
salvar(dados)	Grava um novo currículo, com título cortado em 255 caracteres, arquivo, tipo, texto extraído e template.	Grava em currículos
excluir(id, pid)	Exclui o currículo, só se pertencer ao perfil informado.	Grava em currículos
enviadoParaEmpresa(c urriculoId, perfilEmpresaId)	Diz se aquele currículo foi enviado numa candidatura, não cancelada, para uma vaga da empresa; controla quem pode abrir o arquivo.	Lê currículos, candidaturas e vagas
registrarDownload(id)	Soma 1 ao contador de downloads do currículo.	Grava em currículos

Quadro 67 – Métodos públicos de CurrículoDAO
Fonte: Autoria própria (2026).

O destaque da classe é `ultimosTextos()`: em vez de uma consulta por perfil, ela usa uma subconsulta correlacionada, filtrando pelo id do currículo mais recente com texto de cada perfil, para trazer numa única execução o texto mais recente de cada perfil que já enviou currículo com texto extraído; essa é a consulta usada quando a empresa ou o administrador altera uma vaga e o sistema recalcula o match dela contra todos os candidatos de uma vez, evitando repetir a mesma busca centenas de vezes. A verificação de permissão fica em `enviadoParaEmpresa()`, que cruza currículos com candidaturas e vagas para só liberar a abertura do arquivo para a empresa que de fato recebeu aquela candidatura.

A.5.5 CursoDAO

CursoDAO (`app/Models/CursoDAO.php`) atende a tabela cursos, que guarda os três tipos de conteúdo do catálogo de capacitação, curso, ebook e video. As constantes TIPOS, MODALIDADES, ead, presencial e hibrido, e NIVEIS, iniciante, intermediario e avancado, espelham os ENUMs da tabela, e IMAGENS_PADRAO guarda o caminho da imagem padrão de cada tipo, usada quando a ficha colada pelo administrador não tem imagem ou o link não baixa.

Método	O que faz	Tabelas que lê ou grava
imagemPadrao(tipo)	Método estático que devolve o caminho da imagem padrão daquele tipo de conteúdo.	Não acessa o banco

Método	O que faz	Tabelas que lê ou grava
ehImagemPadrao(imagem)	Método estático que diz se o conteúdo ainda está com a imagem padrão, ou sem imagem, para o painel sinalizar que falta trocar.	Não acessa o banco
listar(ativos=true, filtros)	Lista os conteúdos com o nome da categoria; filtra por termo de busca no título, descrição ou instituição, por categoria e por gratuidade; só os ativos aparecem fora do painel.	Lê cursos e categorias
buscar(id)	Busca um conteúdo pelo id, com o nome da categoria.	Lê cursos e categorias
salvar(dados, id=0)	Cria ou atualiza um conteúdo.	Grava em cursos
buscarRepetido(url, titulo, instituicao)	Procura um conteúdo já cadastrado com o mesmo link, ignorando a barra final, ou com o mesmo título e instituição; evita duplicar ao importar várias fichas de uma vez.	Lê cursos
guardarNaBiblioteca(id, pdf, descricao)	Troca a url do e-book para o PDF salvo na biblioteca do próprio site, o botão passa de Acessar para Baixar, e atualiza a descrição com o crédito da fonte.	Grava em cursos
alterarAtivo(id, ativo)	Publica ou oculta o conteúdo com um clique.	Grava em cursos
excluir(id)	Exclui o conteúdo e apaga a imagem e o PDF enviados, caso nenhum outro registro ainda os use.	Lê e grava em cursos

Quadro 68 – Métodos públicos de CursoDAO
Fonte: Autoria própria (2026).

A consulta de buscarRepetido() usa TRIM(TRAILING '/' FROM ...) dos dois lados da comparação para que um link cadastrado com ou sem a barra final no fim do endereço seja reconhecido como o mesmo; ela também aceita um segundo critério, título e instituição iguais, para pegar o caso de o mesmo curso ser colado duas vezes com links ligeiramente diferentes. Em excluir(), antes de apagar a linha a classe guarda o caminho da imagem e da url, quando é um PDF da biblioteca, e, depois do DELETE, chama a função apagar_upload_sem_uso() para cada um, que só remove o arquivo do disco se nenhum outro curso ainda apontar para ele.

A.5.6 MatchDAO

MatchDAO (app/Models/MatchDAO.php) atende a tabela matches, onde fica a nota de compatibilidade entre um candidato e uma vaga calculada pelo MatchService, junto com o nível, baixo, medio, alto ou excelente, e os detalhes da nota, gravados em JSON na coluna detalhes.

Método	O que faz	Tabelas que lê ou grava
salvar(pid, vid, score, nivel, detalhes=[])	Grava o match com INSERT e ON DUPLICATE KEY UPDATE, aproveitando a chave única (perfil_candidato_id, vaga_id): se já existe, atualiza a pontuação, o nível e os detalhes e zera a marca de visualizado.	Grava em matches
limparPorCandidato(pid)	Apaga todos os matches de um candidato; usado, por exemplo, quando ele exclui o currículo.	Grava em matches

Método	O que faz	Tabelas que lê ou grava
limparPorVaga(vid)	Apaga todos os matches de uma vaga.	Grava em matches
listarPorCandidato(pid)	Lista os matches do candidato só com vagas ainda abertas, da maior para a menor pontuação, com dados da vaga e da empresa.	Lê matches, vagas, perfis e usuarios
buscar(pid, vid)	Busca o match de um candidato com uma vaga específica.	Lê matches
mapaPorCandidato(pid)	Devolve um mapa de vaga_id para pontuação com todos os matches do candidato, usado para marcar a nota na lista de vagas sem precisar de uma consulta por vaga.	Lê matches
decodificar(linha)	Método estático que decodifica o JSON da coluna detalhes para um array PHP.	Não acessa o banco
resumoVagasAbertas(perfilEmpresaId=null)	Painel: total de pares candidato-vaga calculados entre vagas abertas, a média da pontuação e a quantidade em cada faixa de nível.	Lê matches e vagas

Quadro 69 – Métodos públicos de MatchDAO

Fonte: Autoria própria (2026).

O uso de ON DUPLICATE KEY UPDATE em salvar() depende da chave única uniq_match(perfil_candidato_id, vaga_id) da tabela: como o match é recalculado toda vez que o candidato troca de currículo, o perfil é salvo ou a vaga é criada ou alterada, essa cláusula evita ficar conferindo se já existe uma linha antes de decidir entre INSERT e UPDATE, numa só ida ao banco. mapaPorCandidato() existe para a lista de vagas não fazer uma consulta de match por vaga mostrada na tela: ela traz todos os matches do candidato de uma vez e o controller consulta o array em memória.

A.5.7 PerfilDAO

PerfilDAO (app/Models/PerfilDAO.php) atende a tabela perfis, que guarda tanto os dados profissionais do candidato quanto os dados da empresa, sempre um perfil por usuário, uma relação de 1 para 1, com a coluna usuario_id marcada UNIQUE. A constante NIVEIS lista os quatro níveis de experiência usados tanto no perfil do candidato quanto nas vagas.

Método	O que faz	Tabelas que lê ou grava
criar(uid, seNaoExistir=false)	Cria o perfil padrão do usuário, Brasília, DF, público e com aceite de LGPD registrado, único lugar do sistema com esses valores; com seNaoExistir igual a true usa INSERT IGNORE e não faz nada se o usuário já tiver perfil.	Grava em perfis
obterOuCriar(uid)	Busca o perfil do usuário e, se ainda não existir, cria um vazio antes de buscar de novo.	Lê e, se preciso, grava em perfis
buscarPorUsuarioId(uid)	Busca o perfil pelo id do usuário, já com nome, e-mail, telefone, tipo e situação da conta.	Lê perfis e usuarios
buscarPorId(id)	Mesma busca, pelo id do próprio perfil.	Lê perfis e usuarios

Método	O que faz	Tabelas que lê ou grava
salvar(PerfilDTO)	Atualiza o perfil a partir de um PerfilDTO; grava os campos da extração de currículo, links, cnh e pretensão salarial, só quando eles vierem no DTO, para não apagar um dado já salvo quando a tela atual não trata aquele campo.	Grava em perfis
listarEmpresas()	Lista os perfis de empresas com conta ativa, ordenados pelo nome fantasia ou, se não houver, pelo nome da conta.	Lê perfis e usuarios
listarTalentos(termo="", cidade="")	Banco de Talentos: lista candidatos ativos com perfil público, com o id do último currículo e a marca de VIP; filtra por um termo de busca, nome, título profissional, habilidades, competências, experiências, cursos complementares ou links, e por cidade, mostrando os VIP primeiro.	Lê perfis, usuarios, curriculos e assinaturas
listarCandidatos()	Lista todos os candidatos ativos; usado para recalculando o match de uma vaga contra todos eles.	Lê perfis e usuarios

Quadro 70 – Métodos públicos de PerfilDAO
Fonte: Autoria própria (2026).

O método salvar() é o que mais mexe em texto: antes de montar o UPDATE, normaliza o nível de experiência, só aceita um dos valores de NIVEIS, senão usa junior, a UF, duas letras maiúsculas, e a data de nascimento, só grava se estiver no formato AAAA-MM-DD; o método privado txt() apara espaços e transforma string vazia em NULL, cortando no tamanho da coluna quando é informado um limite. A parte mais específica do método é como ele trata links, cnh e pretensao_salarial: como esses três campos só existem em algumas telas, vêm, por exemplo, da extração do currículo, o SQL só inclui essas colunas no UPDATE quando o PerfilDTO::tem() confirma que o campo foi enviado; assim, salvar o perfil pela tela da empresa, que não manda esses campos, nunca apaga o que já estava gravado ali. listarTalentos() é a consulta de filtro mais ampla da classe: o termo de busca é testado contra sete colunas diferentes com LIKE, e uma subconsulta EXISTS contra assinaturas marca is_vip para ordenar os candidatos VIP no topo da lista.

A.5.8 UsuarioDAO

UsuarioDAO (app/Models/UsuarioDAO.php) atende a tabela usuarios, as contas de login, e também as duas tabelas de segurança da conta, tentativas_login, para o bloqueio por força bruta, e redefinicoes_senha, para a recuperação de senha. A constante TIPOS lista os três tipos de conta, admin, candidato e empresa; REDEFINICAO_MINUTOS define que um link de redefinição de senha vale por 30 minutos; e as propriedades públicas erro e motivoFalha guardam, respectivamente, a mensagem do último erro de cadastro ou atualização e o motivo

da última falha de login, `sem_conta`, desativada ou senha, usado para decidir a mensagem mostrada na tela.

Método	O que faz	Tabelas que lê ou grava
<code>cadastrar(UsuarioDTO)</code>	Cria o usuário e, se for candidato ou empresa, o perfil, dentro da mesma transação; confere antes se o e-mail já está em uso e guarda a senha com <code>password_hash()</code> .	Lê e grava em <code>usuarios</code> ; grava em <code>perfis</code>
<code>atualizar(id, dados)</code>	Atualiza nome, e-mail, tipo, telefone, situação e, se informada, a senha; impede deixar o sistema sem nenhum administrador ativo e garante que contas candidato ou empresa tenham perfil.	Lê e grava em <code>usuarios</code> ; grava em <code>perfis</code>
<code>atualizarTelefone(id, telefone)</code>	Atualiza só o telefone da conta.	Grava em <code>usuarios</code>
<code>atualizarNome(id, nome)</code>	Atualiza só o nome da conta; usado quando o candidato aceita o nome encontrado no currículo.	Grava em <code>usuarios</code>
<code>emailEmUso(email, excetoId=0)</code>	Diz se o e-mail já pertence a outra conta.	Lê <code>usuarios</code>
<code>listar(tipo=null, q="")</code>	Lista as contas, com o id do perfil, para o link Ver portfólio dos candidatos; filtra por tipo e por um termo no nome ou e-mail.	Lê <code>usuarios</code> e <code>perfis</code>
<code>resumo(id)</code>	Ficha da conta para a tela Ver do painel: dados do usuário e do perfil, mais o total de vagas, candidaturas e currículos e o plano ativo.	Lê <code>usuarios</code> , <code>perfis</code> , <code>vagas</code> , <code>candidaturas</code> , <code>curriculos</code> e <code>assinaturas</code>
<code>alterarAtivo(id, ativo, meuId)</code>	Ativa ou bloqueia a conta com um clique; impede bloquear a própria conta e impede desativar o último administrador ativo.	Lê e grava em <code>usuarios</code>
<code>contarAdminsAtivos()</code>	Conta quantos administradores ativos existem.	Lê <code>usuarios</code>
<code>buscarPorId(id)</code>	Busca a conta pelo id.	Lê <code>usuarios</code>
<code>buscarPorEmail(email)</code>	Busca a conta pelo e-mail, normalizado antes da busca.	Lê <code>usuarios</code>
<code>variantesSenha(senha)</code>	Método estático que gera até 4 variações da senha digitada, a original, sem espaços nas pontas, com a primeira letra em caixa trocada e com Caps Lock, para o login tolerar erros comuns de digitação.	Não acessa o banco
<code>autenticar(email, senha)</code>	Confere a senha, testando as variantes, contra o hash salvo, atualiza o último acesso e, se o algoritmo do hash estiver desatualizado, gera um hash novo; devolve false e grava o motivo em <code>motivoFalha</code> quando a conta não existe, a senha não confere ou a conta está desativada.	Lê <code>usuarios</code> ; grava <code>ultimo_acesso</code> e, às vezes, a senha em <code>usuarios</code>
<code>minutosBloqueioLogin(ip, email, somarIps=true)</code>	Calcula quantos minutos faltam para o login deste e-mail e IP ser liberado, combinando três limites: erros deste e-mail neste IP, erros deste IP em qualquer e-mail e erros deste e-mail vindos de qualquer IP; devolve 0 quando não há bloqueio.	Lê <code>tentativas_login</code>
<code>tentativasRestantes(ip, email)</code>	Quantos erros ainda cabem para este IP e e-mail antes da pausa, para avisar na tela.	Lê <code>tentativas_login</code>
<code>registrarFalhaLogin(ip, email)</code>	Grava uma tentativa de senha errada; de vez em quando, 1 em 20, aproveita para apagar tentativas com mais de 1 dia.	Grava e, ocasionalmente, exclui linhas de <code>tentativas_login</code>

Método	O que faz	Tabelas que lê ou grava
limparFalhasLogin(ip, email)	Apaga as tentativas erradas deste IP e e-mail, após um login bem-sucedido.	Grava em tentativas_login
criarTokenRedefinicao(email, ip)	Gera o token de redefinição de senha, 32 bytes aleatórios, invalida qualquer link anterior ainda não usado e grava só o hash SHA-256 do token, com validade de 30 minutos; recusa se a conta não existir, estiver inativa ou já tiver 3 pedidos na última hora.	Lê usuarios e redefinicoes_senha; grava em redefinicoes_senha
buscarPorTokenRedefinicao(token)	Busca a conta dona de um token válido, não usado e dentro do prazo, conferindo o hash SHA-256.	Lê redefinicoes_senha e usuarios
redefinirSenha(token, novaSenha)	Troca a senha usando o token, de uso único: marca o token como usado antes de gravar a senha nova, numa transação, e limpa as tentativas de login daquele e-mail.	Lê e grava em redefinicoes_senha; grava em usuarios; grava em tentativas_login
senhaConfere(id, senha)	Confere se a senha digitada é a da conta, testando as mesmas variantes de autenticar().	Lê usuarios
excluir(id)	Exclui a conta; perfil, currículos, candidaturas, vagas, matches, assinaturas e pedidos de redefinição saem em cascata pelas chaves estrangeiras; a classe ainda apaga os arquivos enviados, currículos, foto ou logo, imagens de vaga, e as tentativas de login daquele e-mail.	Lê currículos, perfis e vagas; grava em usuarios e tentativas_login
contarPorTipo(dias=30)	Painel: para cada tipo de conta, o total, quantos ativos e quantos novos nos últimos N dias.	Lê usuarios
listarRecentes(limite=5)	As últimas contas cadastradas, sem a senha.	Lê usuarios

Quadro 71 – Métodos públicos de UsuarioDAO
Fonte: Autoria própria (2026).

O bloqueio de força bruta é o trecho de segurança mais elaborado da classe. `minutosBloqueioLogin()` soma três contagens diferentes sobre a tabela `tentativas_login` na mesma janela de tempo, `LOGIN_JANELA_MINUTOS`, definida em `config/config.php`: erros para o mesmo e-mail vindos deste IP, `LOGIN_MAX_TENTATIVAS`, erros deste IP somando todos os e-mails, `LOGIN_MAX_TENTATIVAS_IP`, pensado contra quem tenta várias contas do mesmo lugar, e erros do mesmo e-mail vindos de qualquer IP, `LOGIN_MAX_TENTATIVAS_CONTA`, pensado contra um ataque distribuído; o parâmetro `somarIps` igual a `false` ignora esse terceiro limite, usado em `Esqueci minha senha` para o dono da conta, entrando de outro IP, ainda conseguir pedir o link mesmo com a conta sob ataque. O tempo de liberação é calculado a partir do horário da tentativa mais antiga dentro da janela mais o tamanho da janela, então o bloqueio nunca é permanente: ele vai se esvaziando sozinho à medida que as tentativas antigas saem da janela. Já a recuperação de senha gira em torno de `criarTokenRedefinicao()` e `redefinirSenha()`: o token em si, 32 bytes aleatórios em hexadecimal, só existe na memória do PHP no momento da geração, pois o banco guarda somente o hash SHA-256 dele na coluna `token_hash`; isso significa que mesmo um acesso de leitura ao banco não permite montar um link de redefinição válido. `redefinirSenha()` marca o token como usado

antes de trocar a senha, dentro de uma transação, e confere o número de linhas afetadas logo em seguida: se outra requisição já tiver usado o mesmo token entre a leitura e a gravação, a segunda chamada encontra zero linhas afetadas e desiste, o que evita que o mesmo link sirva duas vezes. Por fim, `autenticar()` nunca revela, pelo tempo de resposta, se um e-mail tem conta ou não: quando o e-mail não existe, o método roda `password_verify()` do mesmo jeito, contra um hash fixo que nunca confere, só para gastar um tempo parecido com o de uma conferência real.

A.5.9 VagaDAO

VagaDAO (`app/Models/VagaDAO.php`) atende a tabela `vagas`, a principal do sistema. As constantes `TIPOS`, `clt`, `pj`, `estagio` e `temporario`, `NIVEIS`, os mesmos quatro níveis do perfil, `MODELOS`, `presencial`, `remoto` e `hibrido`, e `STATUS`, `ativa`, `pausada` e `encerrada`, espelham os `ENUMs` da tabela. A classe guarda dois trechos de SQL como constantes reaproveitadas por quase todos os métodos: `SELECT`, `privada`, que já junta a vaga com a categoria, com a empresa dona, ou o anunciante, quando a vaga foi publicada a partir da leitura de um cartaz, e com uma subconsulta que diz se a empresa tem o plano Premium vigente; e `ATIVA`, a condição que define uma vaga aberta, status igual a `ativa` e, se tiver data de expiração, ainda não ter passado. `ATIVA` é pública porque `AssinaturaDAO::contarVagasAtivas()` usa a mesma condição para contar o limite do plano básico; assim, o número mostrado na tela e o limite conferido na gravação nunca discordam.

Método	O que faz	Tabelas que lê ou grava
<code>salvar(dados, id=0)</code>	Cria ou atualiza a vaga chamando o método privado <code>gravar()</code> ; sem controle de limite de plano.	Grava em <code>vagas</code>
<code>salvarComLimite(dados, id, limite)</code>	Salva a vaga dentro de uma transação que trava a linha do perfil da empresa (<code>FOR UPDATE</code>) e confere o limite de vagas abertas antes de gravar; limite igual a <code>null</code> , administrador, empresa Premium ou vaga que já estava aberta, pula a checagem.	Lê e trava perfis; lê e grava <code>vagas</code>
<code>listar(ativas=true, filtros)</code>	Lista vagas com busca por palavra no título, descrição, requisitos, categoria, nome da empresa ou anunciante, e filtros por cidade, categoria, nível, modelo de trabalho e tipo de contratação; na área pública, só mostra vagas abertas de empresas com a conta ativa; ordena por destaque e depois pelas mais recentes.	Lê <code>vagas</code> , categorias, perfis e <code>usuarios</code>
<code>buscar(id)</code>	Busca uma vaga pelo id, com os mesmos dados agregados do <code>SELECT</code> padrão.	Lê <code>vagas</code> , categorias, perfis e <code>usuarios</code>
<code>buscarParecida(titulo, anunciante, cidade, excetoId=0)</code>	Procura uma vaga já aberta com o mesmo título, anunciante e cidade, para avisar a empresa antes de publicar um anúncio repetido.	Lê <code>vagas</code> , categorias, perfis e <code>usuarios</code>

Método	O que faz	Tabelas que lê ou grava
estaAberta(vaga)	Confere, em PHP, se um array de vaga já carregado representa uma vaga aberta, status ativa, prazo não vencido e empresa ativa.	Não acessa o banco
listarPorEmpresa(pid)	Lista as vagas da empresa com o total de candidaturas, sem as canceladas, de cada uma, destaque primeiro e depois as mais recentes.	Lê vagas, categorias, perfis, usuarios e candidaturas
excluir(id)	Exclui a vaga; candidaturas e matches dela saem em cascata pelas chaves estrangeiras, e a imagem enviada é apagada do disco se não for mais usada em outro lugar.	Lê e grava em vagas
alterarStatus(id, status, limite=null)	Ativa, pausa ou encerra a vaga; ao reativar com limite, plano básico, conta as vagas abertas da empresa e grava numa mesma transação, como salvarComLimite().	Lê e trava perfis; lê e grava vagas
incrementarVisualizacao(id)	Soma 1 ao contador de visualizações da vaga.	Grava em vagas
resumoPainel(perfilEmpresaId=null)	Painel: total de vagas cadastradas, abertas, pausadas, encerradas, vencidas, status ativa com prazo já passado, em destaque e a soma das visualizações.	Lê vagas
contarPorCategoria(perfilEmpresaId=null)	Painel: vagas abertas por categoria, da maior para a menor quantidade.	Lê vagas e categorias
contarPorCidade(perfilEmpresaId=null)	Painel: vagas abertas por cidade e UF, da maior para a menor quantidade.	Lê vagas
desempenhoPorEmpresa(perfilEmpresaId)	Desempenho de cada vaga da empresa: candidaturas, sem as canceladas, quantas aguardam análise, visualizações e a média do match de quem se candidatou.	Lê vagas, candidaturas e matches

Quadro 72 – Métodos públicos de VagaDAO
Fonte: Autoria própria (2026).

A classe tem duas transações que seguem o mesmo padrão de proteção contra condição de corrida: `salvarComLimite()` e `alterarStatus()`, quando reativa uma vaga. As duas travam a linha da tabela perfis correspondente à empresa com `SELECT` e `FOR UPDATE` antes de contar quantas vagas dela já estão abertas, usando a condição `ATIVA`, de forma que duas publicações ou duas reativações simultâneas não consigam, juntas, ultrapassar o limite de duas vagas abertas do plano básico; se o limite já foi atingido, a transação é desfeita e o método devolve a string `limite`, que o controller usa para mostrar o aviso de upgrade para o plano Premium. O método privado `gravar()` concentra o SQL de `INSERT` e `UPDATE` da tabela, chamado tanto por `salvar()` quanto de dentro da transação de `salvarComLimite()`, para o `INSERT` e o `UPDATE` nunca ficarem desalinhados entre os dois caminhos. Para os painéis, `resumoPainel()` é a consulta mais densa: soma, numa linha só, sete `SUM()` diferentes sobre expressões booleanas, inclusive reaproveitando a constante `ATIVA` dentro do SQL, para trazer todos os indicadores de vagas de uma vez, sem uma consulta por indicador; `contarPorCategoria()` e `contarPorCidade()` seguem o mesmo princípio, agora com `GROUP BY`, para alimentar os gráficos de vagas por área e por

cidade do painel. Em todas essas consultas de painel, perfilEmpresaId igual a null soma os dados da plataforma inteira, visão do administrador, e um id soma só os dados daquela empresa, sem precisar de dois métodos diferentes.

A.5.10 PerfilDTO

PerfilDTO (app/DTO/PerfilDTO.php) é o objeto que carrega os dados de um formulário de perfil, ou das máquinas de extração de currículo, até PerfilDAO::salvar(). Ele não atende tabela nenhuma diretamente: existe só para dar um nome e um tipo a cada campo antes de ele virar uma coluna da tabela perfis. O construtor recebe um array associativo qualquer, por exemplo os dados de \$_POST ou o resultado da extração do currículo, e guarda esse array internamente; cada método get devolve o valor daquele campo, ou um padrão quando ele não veio, nivel_experiencia vira junior, publico vira 1, aceite_lgpd vira 1. O método tem(campo) devolve verdadeiro só quando a chave existe no array original, mesmo que o valor seja nulo ou vazio, e é a forma de PerfilDAO::salvar() diferenciar um campo que o formulário atual não trata, que não deve apagar o que já estava salvo, de um campo que o formulário enviou vazio de propósito, que deve limpar o que estava salvo.

Método	Campo correspondente em perfis	Valor padrão quando ausente
getId()	id	null
getUsuarioId()	usuario_id	null
getTituloProfissional()	titulo_profissional	null
getBio()	bio	null
getDataNascimento()	data_nascimento	null
getCidade()	cidade	null
getUf()	uf	null
getNomeFantasia()	nome_fantasia	null
getCnpj()	cnpj	null
getSetor()	setor	null
getSite()	site	null
getHabilidades()	habilidades	null
getExperiencias()	experiencias	null
getFormacao()	formacao	null
getIdiomas()	idiomas	null
getCursosComplementares()	cursos_complementares	null
getInformacoesAdicionais()	informacoes_adicionais	null
getCompetencias()	competencias	null
getNivelExperiencia()	nivel_experiencia	'junior'

Método	Campo correspondente em perfis	Valor padrão quando ausente
getDisponibilidade()	disponibilidade	null
getObjetivo()	objetivo	null
getFoto()	foto	null
getPublico()	publico	1
getAceiteLgpd()	aceite_lgpd	1
getLinks()	links	null
getCnh()	cnh	null
getPretensaoSalarial()	pretensao_salarial	null
tem(campo)	Não corresponde a uma coluna; método de apoio	Devolve se o campo foi informado no array recebido pelo construtor

Quadro 73 – Métodos públicos de PerfilDTO
Fonte: Autoria própria (2026).

A.5.11 UsuarioDTO

UsuarioDTO (app/DTO/UsuarioDTO.php) é o mesmo tipo de objeto, só que mais simples, usado para levar os dados do formulário de cadastro até UsuarioDAO::cadastrar(). Também guarda um array associativo e devolve cada campo com um getter, com dois padrões definidos: tipo vira candidato e ativo vira 1 quando não informados.

Método	Campo correspondente em usuarios	Valor padrão quando ausente
getId()	id	null
getNome()	nome	null
getEmail()	email	null
getSenha()	senha	null
getTipo()	tipo	'candidato'
getTelefone()	telefone	null
getAtivo()	ativo	1

Quadro 74 – Métodos públicos de UsuarioDTO
Fonte: Autoria própria (2026).

A.6 Banco de dados: scripts e dados de demonstração (database/)

O arquivo database/schema.sql cria o banco tcc_final do zero: ele começa com DROP DATABASE IF EXISTS tcc_final seguido de CREATE DATABASE tcc_final CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci, de forma que todo o banco use o conjunto de caracteres utf8mb4, acentos, emojis e qualquer caractere Unicode, desde a criação, o mesmo charset configurado na conexão PDO de app/Core/Database.php. As 11 tabelas são criadas nesta ordem, que segue a ordem de dependência das chaves estrangeiras: usuarios, perfis, categorias, vagas, cursos, curriculos, matches, assinaturas, candidaturas, tentativas_login e

redefinicoes_senha. usuarios é a primeira porque perfis e assinaturas dependem dela; perfis vem antes de vagas e curriculos, que dependem de um perfil; categorias vem antes de vagas e cursos, que podem apontar para uma categoria; e matches e candidaturas, que dependem de duas ou três tabelas ao mesmo tempo, perfis, vagas e, no caso de candidaturas, também curriculos, só são criadas depois de todas elas existirem. tentativas_login não referencia nenhuma outra tabela, e redefinicoes_senha fecha o arquivo porque depende de usuarios. Todas as tabelas usam o motor InnoDB, que é o que suporta chaves estrangeiras no MySQL.

Quase todas as chaves estrangeiras do arquivo usam ON DELETE CASCADE, com duas exceções que usam ON DELETE SET NULL: vagas.categoria_id e cursos.categoria_id, porque excluir uma categoria não deve apagar as vagas ou os cursos que a usavam, só deixá-los sem categoria. É esse ON DELETE CASCADE que sustenta a exclusão de conta pedida pela LGPD: ao excluir uma linha de usuarios, o banco apaga sozinho o perfil, pela chave perfis.usuario_id, e a partir do perfil excluído apaga em cascata os currículos, curriculos.perfil_id, as vagas, se for empresa, vagas.perfil_empresa_id, os matches, matches.perfil_candidato_id e matches.vaga_id, e as candidaturas, candidaturas.perfil_candidato_id e candidaturas.vaga_id; as assinaturas, assinaturas.usuario_id, e os pedidos de redefinição de senha, redefinicoes_senha.usuario_id, saem direto pela chave que aponta para usuarios. O arquivo também define restrições UNIQUE importantes: usuarios.email, um e-mail por conta, perfis.usuario_id, um perfil por usuário, a relação de 1 para 1, a combinação nome e tipo de categorias, não repete nome dentro do mesmo tipo, a combinação perfil_candidato_id e vaga_id de matches e de candidaturas, um match e uma candidatura por par candidato-vaga, e redefinicoes_senha.token_hash. Os ENUMs aparecem em sete colunas, usuarios.tipo, perfis.nivel_experiencia, vagas.tipo_vaga, nivel_experiencia, remoto e status, cursos.tipo, modalidade e nivel, matches.nivel, assinaturas.plano e status, e candidaturas.status, sempre com um valor padrão coerente com o cadastro mais comum, por exemplo, vagas.status começa como ativa e vagas.tipo_vaga como clt. Por fim, o arquivo cria índices nas colunas mais usadas em filtro, ordenação e junção, como vagas.status, vagas.destaque, vagas.cidade e uf juntas, categorias com tipo, ativo e nome, tentativas_login com ip, email e created_at, e assinaturas com usuario_id, status e data_fim, para as consultas das listagens e dos painéis não precisarem varrer a tabela inteira.

O arquivo database/seed.sql é importado depois do schema.sql e povoa o banco com os dados de demonstração usados na apresentação do TCC. Ele cria as três contas de teste, administrador, empresa e candidato, com as senhas do README já em hash bcrypt, o perfil de cada uma delas, 20 categorias, 13 de vagas e 7 de cursos, 62 vagas, cada uma com a imagem

do cartaz correspondente em `public/assets/img/vagas`, 64 conteúdos de capacitação, 43 cursos e 21 e-books gratuitos de fontes oficiais como Fundação Bradesco, Escola Virtual de Governo (Enap), SEBRAE, SENAC, FGV, Banco Central, Ministério do Trabalho e Microsoft Learn, cada um com a própria imagem, e a assinatura VIP, plano assinante, do candidato de teste, válida por 30 dias a partir da importação. O arquivo usa uma variável de sessão do MySQL, `@perfil_empresa_teste`, e subconsultas, como `SELECT id FROM categorias WHERE nome igual a tal e tipo igual a tal`, para ligar as vagas e os cursos às categorias certas sem depender de saber de antemão o id que cada INSERT anterior gerou.

`database/resetar_senhas.php` é um script de linha de comando, recusa rodar fora do CLI, para quando alguém não consegue mais entrar no ambiente de testes: ele devolve as três contas de teste às senhas do README, reativa as contas, ativo igual a 1, mesmo que estivessem bloqueadas, e apaga todas as linhas de `tentativas_login`, liberando qualquer pausa de login em andamento. Como ele muda a senha das contas, as sessões abertas delas em outros navegadores são encerradas, o sistema reconhece a troca de hash e pede login de novo, e por isso o script também avisa isso na saída do terminal. Assim como a pasta `database/` inteira, o acesso direto pelo navegador é bloqueado por um `.htaccess` com `Require all denied`: o script só roda mesmo pelo terminal, com o PHP de linha de comando do XAMPP.

Tabela de origem	Chave estrangeira	Tabela de destino	Cardinalidade	O que acontece ao excluir
perfis	usuario_id	usuarios	1 para 1	Exclui o perfil (ON DELETE CASCADE)
vagas	perfil_empresa_id	perfis	N para 1	Exclui as vagas da empresa (ON DELETE CASCADE)
vagas	categoria_id	categorias	N para 1	Deixa a vaga sem categoria (ON DELETE SET NULL)
cursos	categoria_id	categorias	N para 1	Deixa o curso sem categoria (ON DELETE SET NULL)
curriculos	perfil_id	perfis	N para 1	Exclui os currículos do perfil (ON DELETE CASCADE)
matches	perfil_candidato_id	perfis	N para 1	Exclui os matches do candidato (ON DELETE CASCADE)
matches	vaga_id	vagas	N para 1	Exclui os matches da vaga (ON DELETE CASCADE)
assinaturas	usuario_id	usuarios	N para 1	Exclui o histórico de assinaturas da conta (ON DELETE CASCADE)
candidaturas	perfil_candidato_id	perfis	N para 1	Exclui as candidaturas do candidato (ON DELETE CASCADE)
candidaturas	vaga_id	vagas	N para 1	Exclui as candidaturas da vaga (ON DELETE CASCADE)

Tabela de origem	Chave estrangeira	Tabela de destino	Cardinalidade	O que acontece ao excluir
candidaturas	curriculo_id	curriculos	N para 1	Deixa a candidatura sem currículo indicado (ON DELETE SET NULL)
redefinicoes_senha	usuario_id	usuarios	N para 1	Exclui os pedidos de redefinição da conta (ON DELETE CASCADE)

Quadro 75 – Relacionamentos
Fonte: Autoria própria (2026).

A.7 Serviços de negócio (app/Services)

A.7.1 MatchService.php

O MatchService é a máquina de compatibilidade entre candidato e vaga. Para cada par candidato-vaga ela calcula uma pontuação de 0 a 100, sempre explicável: a pontuação é a soma de quatro partes com pesos fixos, definidos na constante PESOS: competências 50 pontos, cargo 20 pontos, localização 15 pontos e nível de experiência 15 pontos. Em versões anteriores do sistema esta classe se chamava MatchController, como o próprio comentário do arquivo registra.

A parte de competências (50 pontos) começa em Competencias::daVaga(), que extrai do título, da descrição e dos requisitos da vaga quais competências do dicionário ela exige. Essas competências exigidas são comparadas com as competências do candidato (Competencias::doPerfil(), que já herda dados vindos da extração do currículo, como cursos complementares, informações adicionais, disponibilidade e CNH). A pontuação é 50 vezes a proporção de competências exigidas que o candidato atende (count(atendidas)/count(exigidas)). Quando a vaga não tem nenhuma competência reconhecida pelo dicionário, o MatchService usa um critério alternativo: compara as palavras relevantes (tokens) da descrição e dos requisitos da vaga com as palavras do histórico e do título do candidato, e pontua pela proporção de palavras em comum, com um mínimo de 3 palavras na vaga para não inflar o resultado de anúncios muito curtos.

A parte de cargo (20 pontos) compara as palavras do título da vaga com as palavras do título profissional/objetivo do candidato (peso cheio) e com as palavras do histórico dele - experiências, habilidades, formação, cursos complementares e bio (peso de 60%). Vale o maior dos dois resultados multiplicado por 20. Palavras muito comuns (artigos, preposições e termos genéricos como vaga, empresa, profissional) ficam fora da comparação pela lista STOP, e o método privado tokens() também normaliza plural simples (terminações em s) e gênero (terminações em a/o) para que vendedora e vendedor contem como a mesma palavra.

A parte de localização (15 pontos) segue uma ordem de prioridade: vaga remota vale os 15 pontos cheios; mesma cidade do perfil também vale 15; mesmo estado (UF) vale 9; quando o candidato está fora da região da vaga mas marcou disponibilidade para mudança (um texto detectado por expressão regular nos campos disponibilidade e informações adicionais, herdados da extração do currículo) a vaga ainda rende 6 pontos; sem nenhuma coincidência, a localização vale 0.

A parte de nível de experiência (15 pontos) usa a constante NIVEIS (estagiário=0, júnior=1, pleno=2, sênior=3) para medir a distância entre o nível do candidato e o nível pedido pela vaga. Nível exatamente igual rende os 15 pontos; candidato um nível acima do pedido rende 12; dois ou mais níveis acima rende 8; candidato um nível abaixo do pedido rende 7; dois ou mais níveis abaixo rende 0.

Além da pontuação, o método calcular() monta uma lista de observações que não entram na nota mas explicam a vaga ao candidato: se a vaga pede uma categoria de CNH (localizada por expressão regular, ex. CNH B), o sistema confere se o candidato informou uma categoria compatível; se a vaga é destinada a PCD e o candidato se declarou PCD, isso é destacado; se a vaga envolve viagens, o sistema avisa se o candidato tem disponibilidade. Essas observações só aparecem quando a vaga menciona o assunto, e ficam em detalhes['observacoes']. Todo o detalhamento (pontos e máximo de cada parte, competências exigidas/atendidas/faltantes, motivo do cargo, motivo da localização, motivo do nível e as observações) é devolvido como um array e gravado em JSON na coluna matches.detalhes, para ser exibido tanto ao candidato quanto à empresa.

A pontuação final é limitada a 100 (min(100, soma das quatro partes)) e arredondada em duas casas decimais; classificar() a converte num rótulo: 75 ou mais é excelente, de 55 a 74 é alto, de 35 a 54 é médio, e abaixo disso é baixo. Por exemplo, no teste automatizado do sistema um candidato de Taguatinga/DF, nível júnior, com as competências Vendas e Excel e título profissional vendedor, é comparado a uma vaga de Vendedor Interno em Taguatinga/DF, nível júnior, pedindo vendas e Excel: o resultado é 100 pontos, classificado como excelente, porque as quatro partes batem perfeitamente (competências 50/50, cargo 20/20 pelo título, localização 15/15 mesma cidade, nível 15/15 igual).

O recálculo do match não é automático a cada visita: ele acontece em momentos específicos. recalcular(perfilId) é chamado quando o candidato envia ou exclui um currículo, salva o perfil, ou aciona o botão Recalcular match do portfólio - ele apaga os matches antigos daquele candidato (MatchDAO::limparPorCandidato) e calcula de novo contra todas as vagas ativas (VagaDAO::listar(true)). recalcularVaga(vagaId) é chamado ao criar ou editar uma vaga

- se a vaga não está ativa ou já expirou, apenas limpa os matches dela (MatchDAO::limparPorVaga) e devolve 0; senão, busca os textos dos currículos de todos os candidatos numa única consulta (CurriculoDAO::ultimosTextos()), para não fazer uma consulta por candidato) e recalcula contra cada perfil de candidato cadastrado.

Método	O que faz
recalcular(int \$perfilId): int	Recalcula o match de um candidato contra todas as vagas ativas; devolve a quantidade de vagas comparadas.
recalcularVaga(int \$vagaId): int	Recalcula o match de uma vaga contra todos os candidatos; limpa os matches se a vaga não estiver ativa ou já tiver expirado.
calcular(array \$cand, array \$v): array	Calcula a pontuação (0 a 100), o nível (excelente/alto/medio/baixo) e o detalhamento em JSON de um par candidato-vaga já preparado.
classificar(float \$pontos): string	Converte uma pontuação numérica no rótulo textual do nível de compatibilidade (excelente, alto, medio ou baixo).

Quadro 76 – Métodos públicos do MatchService

Fonte: Autoria própria (2026).

A.7.2 Competencias.php

A classe Competencias é o dicionário único de competências usado por toda a plataforma: a mesma lista decide o que o currículo do candidato sabe, o que a vaga exige e o que o curso ensina, permitindo comparar os três lados com as mesmas palavras. A constante DICCIONARIO tem cerca de 50 competências canônicas (por exemplo Atendimento ao cliente, Vendas, Excel, PHP, Elétrica, Enfermagem, Inglês), cada uma associada a uma lista de sinônimos já normalizados (sem acento, minúsculos) que costumam aparecer em anúncios, currículos e ementas de curso, como corretor, vendedora e porta a porta para a competência Vendas.

O método normalizar() é a base de tudo: converte o texto para minúsculas, troca letras acentuadas e apóstrofes por suas versões simples (á para a, ç para c, e assim por diante), substitui qualquer caractere que não seja letra ou número por um espaço e corta espaços nas pontas. É essa normalização que faz Café da Manhã e café da manha serem comparados como a mesma coisa.

O método extrair() faz a busca propriamente dita. Ele cerca o texto normalizado com espaços nas pontas (para poder procurar palavra ou frase inteira, nunca um pedaço de palavra), remove primeiro as expressões da lista RUIDO - frases que pareceriam uma competência mas não são, como proteção de dados (não é Análise de dados) ou retenção de talentos (não é Telemarketing, mas operador de cobrança continua valendo) - e só então percorre o dicionário procurando, para cada competência, se algum dos sinônimos aparece como palavra ou frase inteira (cercada de espaços) no texto. Por exemplo, extrair('Experiência com vendas,

atendimento ao cliente e Excel avançado') encontra Vendas, Atendimento ao cliente e Excel; já extrair('a construção de um ambiente positivo, proteção de dados e cobrança por consumo') não encontra nenhuma competência, porque as três expressões do texto estão na lista de ruído; e extrair('Eletricista de obra') encontra Construção civil, mostrando que a palavra obra sozinha é válida fora do contexto de ruído.

A partir de extrair(), três métodos buscam competências em contextos diferentes: daVaga() usa título, descrição e requisitos da vaga (os benefícios não contam, porque não descrevem a função); doPerfil() usa praticamente todos os campos do perfil do candidato, incluindo os que vieram da extração automática do currículo (cursos complementares, informações adicionais, disponibilidade e CNH, prefixada com a palavra CNH) mais o texto do currículo mais recente; e doCurso() usa o título e a descrição do curso e, só quando os dois não revelam nenhuma competência, cai para o nome da categoria do curso. Essa ordem de prioridade existe porque a categoria agrupa vários temas - o teste automatizado mostra que um curso de Power BI dentro da categoria Marketing, Dados e UX não herda a competência UX e design (já que o título e a descrição falam de dados, não de design), mas um curso chamado apenas Módulo 1, sem descrição, na categoria Informática e Excel, herda Excel e Informática da categoria por falta de outra pista.

Os três métodos finais cruzam competências para gerar recomendações, todos ordenando pela quantidade de competências em comum e limitando a quantidade de resultados: cursosPara() recebe as competências que faltam ao candidato (geradas pelo MatchService) e devolve os cursos do catálogo que ensinam alguma delas, cada curso marcado com a chave cobre; cursosParaVaga() faz o mesmo a partir das competências exigidas por uma vaga específica (usado na página da vaga quando o candidato não tem recomendação própria), marcando cada curso com em_comum; e vagasParaCurso() faz o caminho inverso, mostrando na página de um curso as vagas ativas que pedem as competências que aquele curso ensina, também marcadas com em_comum.

Método	O que faz
normalizar(string \$s): string	Coloca o texto em minúsculas, remove acentos e pontuação, deixando só letras, números e espaços simples.
extrair(string \$texto): array	Procura no texto, já sem as expressões de ruído, os sinônimos de cada competência do dicionário e devolve os nomes canônicos encontrados.
daVaga(array \$v): array	Competências exigidas por uma vaga, a partir do título, da descrição e dos requisitos.
doCurso(array \$c): array	Competências ensinadas por um curso, a partir do título e da descrição; usa a categoria só se os dois não indicarem nada.
doPerfil(array \$p, string \$textoCurriculo = "): array	Competências do candidato, somando todos os campos do perfil (inclusive os herdados do currículo) com o texto do currículo mais recente.

Método	O que faz
<code>cursosPara(array \$faltantes, array \$cursos, int \$limite = 3): array</code>	Cursos do catálogo que ensinam competências que faltam ao candidato, ordenados pela quantidade coberta.
<code>cursosParaVaga(array \$competenciasVaga, array \$cursos, int \$limite = 3): array</code>	Cursos que ensinam competências exigidas por uma vaga específica.
<code>vagasParaCurso(array \$competenciasCurso, array \$vagas, int \$limite = 6): array</code>	Vagas ativas que pedem competências ensinadas por um curso.

Quadro 77 – Métodos públicos da classe Competencias

Fonte: Autoria própria (2026).

A.7.3 Portfolio.php

A classe Portfolio monta o portfólio profissional do candidato a partir dos dados do perfil, preenchidos manualmente ou pela extração automática do currículo: organiza experiências numa linha do tempo, formação por curso/instituição/situação e listas de habilidades, idiomas e cursos complementares. Ela também decide se o cadastro do candidato está completo o bastante para liberar o portfólio público. As constantes OBRIGATORIOS (telefone, título profissional, cidade, resumo profissional, objetivo profissional, formação acadêmica e habilidades) e RECOMENDADOS (experiências, foto, cursos complementares e idiomas) listam os campos usados por `validarCadastro()`: o portfólio só é considerado completo quando nenhum campo obrigatório está vazio; os recomendados não bloqueiam, mas entram no cálculo do percentual de preenchimento, que é a proporção de campos (obrigatórios mais recomendados) já preenchidos.

A parte mais elaborada da classe é o reconhecimento de períodos de tempo e de blocos de experiência dentro de um texto solto, como o que vem do currículo ou que o próprio candidato digita. O método `periodo()` usa uma expressão regular construída em `rxPeriodo()` que reconhece datas em qualquer formato comum em currículos brasileiros: mês abreviado ou extenso (jan, janeiro), número de mês, ano de 4 dígitos, e palavras que indicam o presente (atual, atualmente, presente, hoje, em andamento). Só é considerado um período de fato quando há um intervalo (início e fim), um início com atual no lugar do fim, ou a palavra desde antes do início - um ano solto, sem nenhum desses sinais, não é tratado como período. Por exemplo, `periodo()` reconhece Jan/2020 – Atual, 2019 - 2021, 03/2018 a 12/2019, Desde 2022 e janeiro de 2019 a março de 2021, todos convertidos internamente em meses absolutos (ano vezes 12 mais o mês) para permitir ordenação e comparação.

O método `experiencias()` recebe o texto completo (de um campo do perfil ou extraído do currículo) e devolve uma lista de blocos com empresa, cargo, período e descrição. Ele

reconhece três formatos distintos de como as pessoas escrevem suas experiências: quando há linhas com rótulos explícitos (Empresa:, Cargo:, Período:), usa o caminho `expRotulada()`; quando o cargo e a empresa aparecem numa linha só (Cargo – Empresa | período, ou Empresa | Cargo | período), usa `partes()` e `empresaCargo()` para separar quem é cargo (reconhecido pela lista de palavras-chave CARGOS, como auxiliar, vendedor, desenvolvedor) e quem é empresa; e quando empresa, cargo e período vêm em linhas separadas, o método detecta se o período aparece antes ou depois do cabeçalho (empresa/cargo) e junta as linhas de descrição ao bloco correto, inclusive linhas sem período evidente lidas antes do primeiro bloco. Os blocos finais são sempre ordenados do mais recente para o mais antigo (`ordenarExp()`), com o item ainda em andamento (Atual) aparecendo no topo. No teste automatizado, o texto ATACADÃO DIA A DIA\nAuxiliar Administrativo\n2021 – 2024 é corretamente separado em empresa ATACADÃO DIA A DIA e cargo Auxiliar Administrativo.

O método `formacao()` segue uma lógica parecida para a educação: cada linha é quebrada em curso, instituição e situação (`linhaFormacao()`) reconhecendo rótulos (Instituição:, Conclusão:), anos (usados como ano de conclusão ou previsão), palavras de situação (cursando, concluído, incompleto, trancado) e nomes de instituições conhecidas (`ehInstituicao()`), que reconhece tanto nomes como UnB, Senai, Estácio, Fiap quanto siglas de duas letras maiúsculas ou mais). Quando uma linha contém só a situação ou só o nome da instituição (sem um curso junto), ela é anexada ao item anterior em vez de criar um novo bloco.

As demais funções da classe cuidam da normalização final: `lista()` transforma um texto corrido (habilidades, idiomas) numa lista de itens sem repetição, separando por vírgula, ponto e vírgula ou linha, mas sem quebrar vírgulas dentro de parênteses; `links()` reconhece endereços escritos no perfil e identifica o tipo pela URL (LinkedIn, GitHub, GitLab, Behance, Instagram, Lattes, Dribbble ou Site genérico); e `serializarExperiencias()/serializarFormacao()` fazem o caminho inverso, transformando a lista de blocos estruturados de volta no texto canônico que é gravado no perfil e relido depois pelo próprio portfólio.

Método	O que faz
<code>validarCadastro(?array \$perfil): array</code>	Confere quais campos obrigatórios e recomendados do perfil estão vazios e calcula o percentual de preenchimento do cadastro.
<code>periodo(string \$linha): ?array</code>	Procura um período de tempo numa linha de texto (início, fim ou atual) em qualquer formato comum de currículo.
<code>experiencias(string \$texto): array</code>	Separa um texto de experiências profissionais em blocos de empresa, cargo, período e descrição, ordenados do mais recente ao mais antigo.
<code>formacao(string \$texto): array</code>	Separa um texto de formação acadêmica em blocos de curso, instituição e situação.
<code>lista(string ...\$textos): array</code>	Transforma um ou mais textos corridos numa lista de itens únicos, sem vírgulas dentro de parênteses.

Método	O que faz
links(string \$texto): array	Reconhece endereços de internet escritos no perfil e identifica o tipo de cada um (LinkedIn, GitHub etc.).
serializarExperiencias(array \$itens): string	Converte uma lista de blocos de experiência de volta no texto canônico gravado no perfil.
serializarFormacao(array \$itens): string	Converte uma lista de blocos de formação de volta no texto canônico Curso – Instituição (situação).

Quadro 78 – Métodos públicos da classe Portfolio
Fonte: Autoria própria (2026).

A.7.4 Pix.php

A classe Pix monta o código Pix copia e cola (o BR Code definido pelo Banco Central) usado no QR Code de doação do rodapé do site. Ela só monta o texto: nenhum dado sai do servidor e nenhuma API externa é chamada - o mesmo texto vira tanto o QR Code, desenhado no navegador pela biblioteca `assets/js/vendor/qrcode.js`, quanto o botão Copiar código Pix. O método `doacao()` lê a chave Pix, o nome do beneficiário e a cidade das constantes de configuração (`DOACAO_PIX_CHAVE`, `DOACAO_NOME`, `DOACAO_CIDADE`, em `config/config.php`) e devolve o código pronto com a descrição Apoio ao projeto; se a chave Pix ainda não foi configurada, devolve uma string vazia e o QR Code simplesmente não aparece.

O formato EMV do Pix é montado campo a campo pelo método `payload()`: cada campo tem um identificador de 2 dígitos, um tamanho de 2 dígitos e o valor (função privada `campo()`). O payload tem o indicador de formato (campo 00 = 01), o bloco do recebedor (campo 26, que por dentro contém o GUI `br.gov.bcb.pix` no subcampo 00 e a chave Pix no subcampo 01, mais uma descrição opcional no subcampo 02 quando ela couber dentro do limite de 99 caracteres do campo 26 inteiro), a categoria do recebedor (campo 52 = 0000, não informada), a moeda (campo 53 = 986, Real), o valor da doação quando informado (campo 54, só entra se for maior que zero - sem valor, quem doa escolhe o valor no próprio aplicativo do banco), o país (campo 58 = BR), o nome do recebedor (campo 59, até 25 caracteres), a cidade (campo 60, até 15 caracteres) e um identificador de transação (campo 62, subcampo 05, limitado a letras, números e asterisco, por padrão ***). No final, o campo 63 (CRC16) fecha o payload. Como o padrão Pix só aceita caracteres ASCII, o método privado `ascii()` remove acentos do nome e da cidade antes de montá-los no payload - por isso Brasília se transforma em BRASILIA dentro do código, como confirma o teste automatizado ao conferir que o payload contém a sequência 6008BRASILIA (campo 60, tamanho 08, valor BRASILIA).

O método `crc16()` implementa o algoritmo CRC-16/CCITT-FALSE (polinômio 0x1021, valor inicial 0xFFFF), exatamente como os aplicativos de banco conferem a integridade do código Pix: processa byte a byte do texto, deslocando e aplicando XOR com o polinômio

quando o bit mais significativo está ligado, e devolve o resultado em hexadecimal maiúsculo com 4 dígitos. O teste automatizado confere esse algoritmo contra um exemplo oficial do próprio Banco Central, obtendo o CRC 1D3D, e também confere que o payload gerado por `payload()` termina exatamente com o CRC16 calculado sobre o restante do próprio texto.

Método	O que faz
<code>doacao(): string</code>	Monta o código Pix de doação a partir da chave, do nome e da cidade configurados; devolve vazio se a chave não estiver configurada.
<code>payload(string \$chave, string \$nome, string \$cidade, ?float \$valor = null, string \$descricao = "", string \$txid = "***"): string</code>	Monta um código Pix completo (BR Code) no formato EMV, com todos os campos e o CRC16 final.
<code>crc16(string \$s): string</code>	Calcula o CRC16 (CCITT-FALSE) de um texto, em hexadecimal maiúsculo de 4 dígitos, como o Banco Central exige no fim do código Pix.

Quadro 79 – Métodos públicos da classe Pix
Fonte: Autoria própria (2026).

A.8 Extração de vagas e cursos e padrões automáticos

A.8.1 ExtracaoVaga.php

A classe `ExtracaoVaga` transforma o texto de um anúncio de vaga (colado do WhatsApp, do Instagram ou de um site) ou a leitura de um cartaz em imagem nos campos do cadastro de vaga: título, empresa anunciante, local, faixa salarial, tipo de contratação, nível, modelo de trabalho, descrição, requisitos, benefícios, contato do anúncio, quantidade de vagas, competências e categoria. O resultado só preenche o formulário - quem publica a vaga revisa e confirma cada campo antes de salvar, e os avisos (`array avisos`) dizem o que a máquina não conseguiu confirmar e merece atenção na revisão.

Quando a entrada é uma imagem, `doImagem()` decide como obter o texto: se o navegador já enviou leituras feitas pelo leitor de OCR da própria plataforma (`Tesseract.js` rodando no navegador, via `OcrImagem::leiturasDoNavegador`), usa `OcrImagem::montar()` para juntá-las; senão, tenta o Tesseract do servidor, se estiver disponível (`OcrImagem::disponivel()`); sem nenhum dos dois, devolve os campos vazios com um aviso para preencher manualmente. O resultado inclui também o texto bruto lido (`texto_ocr`), a confiança da leitura (`confianca`) e um aviso extra quando a confiança fica abaixo de 75%, pedindo para conferir cada campo com mais cuidado.

O método `doTexto()` é o coração da extração e passa por muitas etapas. Primeiro vem a limpeza do texto do OCR: a função privada `vocabularioOcr()` reúne todas as palavras de 4 ou mais letras que aparecem em qualquer uma das leituras (texto principal, destaques,

complemento, todas), e `juntarPartidas()` usa esse vocabulário para reconstruir palavras que o leitor de imagem partiu em duas (por exemplo MÁQUI NA se torna MÁQUINA quando maquina aparece inteira em outra leitura) e também para separar palavras que ficaram coladas (CONSULTORDE se torna CONSULTOR DE quando a mesma leitura aparece separada em outra passada do OCR). Em seguida, linhas que são só slogan da empresa (`ehSlogan()`), como frases do tipo fazem a diferença ou soluções que geram valor) ou restos de logotipo mal lidos (`ehLixoOcr()`) são descartadas de todos os campos.

Depois vem a etapa de herança do cartaz: as leituras extras (array complemento, texto solto que o OCR capturou fora da ordem normal, como letra clara sobre fundo escuro) são quebradas em pedaços quando duas colunas do cartaz foram lidas juntas, separadas por um símbolo que o OCR confundiu com um ícone (por exemplo VT (DF ou GO) © Plano de Saúde se separa em duas informações pelo símbolo ©), e cada pedaço que ainda não apareceu e tem cara de informação de vaga (`linhaUtil()`) é incorporado à lista de linhas. Só então o contato do anúncio (telefone, WhatsApp, e-mail, assunto) é extraído pelo método privado `contato()`, antes de descartar as linhas que só convidam a enviar currículo (`ehLinhaContato()`), e as linhas restantes são separadas em seções com cabeçalho explícito (SECOES: descricao, requisitos, beneficios, local, horario, contato) e linhas soltas, sem cabeçalho (`separar()`).

A identificação da empresa anunciante (método privado `anunciante()`) tenta, em ordem: marcas conhecidas do DF (constante `MARCAS`, com nomes como McDonald's, Atacadão, Leroy Merlin); frases como O Giraffas está contratando ou Somos a Cinco Estrelas; reconstrução do logotipo a partir de pedaços de texto em maiúsculas cujas letras aparecem encostadas dentro do nome de usuário ou do domínio de um e-mail do próprio cartaz (por exemplo SMILE e & FACE formam Smile & Face, confirmado porque batem com `admsmileface@gmail.com`); nomes repetidos no cartaz (lidos no topo e no rodapé, como DOM CASERO duas vezes); linhas com palavras de tipo de negócio (constante `NEGOCIOS`: restaurante, panificadora, construtora etc.); e por fim o domínio do e-mail escrito no cartaz. Quando nenhuma dessas regras encontra a empresa, `ExtracaoVaga` recorre aos padrões automáticos: `PadroesExtracao::nomeConhecido('empresa', $tudo)` procura se o texto cita uma empresa já cadastrada no sistema, e essa decisão fica registrada em `$r['padroes']`.

O título da vaga (método privado `titulo()`) também segue uma sequência de tentativas: rótulo explícito (Vaga:, Cargo:, Função:), inclusive quando o cargo vem em linhas separadas abaixo do rótulo; frase do tipo contratando/precisa de/procura seguida do cargo; cartaz com o cargo escrito em várias linhas de letra grande (`tituloEmLinhas()`), que junta linhas em maiúsculas enquanto parecerem continuação do cargo, até 8 palavras); dois ou mais cargos destacados em

letra grande, juntados com uma barra (por exemplo Vendedora / Gerente); lista de mais de 5 cargos, que se torna Vagas abertas, Empresa; o melhor candidato entre as linhas e os destaques, pontuado por tamanho e posição; e, por fim, a primeira linha com cara de título num anúncio de texto corrido. O método privado `limparTitulo()` remove prefixos genéricos (vagas de emprego para, contrata-se), código da vaga entre parênteses, localidade colada no título e normaliza a caixa das letras (`caixa()`), preservando siglas conhecidas como RH, CLT, PHP e CNH.

As demais informações são extraídas por expressões regulares dedicadas: `salario()` busca valores em reais associados a salário, remuneração ou bolsa-auxílio, descartando valores de benefícios, diárias, comissões e adicionais que apareçam perto (por meio de palavras-chave como vale, vr, vt, comissão, adicional) e aceitando uma faixa quando há mais de um valor plausível; `local()` reaproveita `ExtracaoCurriculo::local()` e a tabela de pontos conhecidos do DF (constante `PONTOS`, como Conjunto Nacional, Taguatinga Shopping, Park Shopping) para mapear o shopping citado para a cidade correta, caindo em Brasília/DF como padrão quando a vaga não é remota e nada foi encontrado; e `tipo_vaga`, `nivel_experiencia` e `remoto` são decididos por palavras-chave simples (estágio, CLT, PJ, temporário; sênior, pleno, júnior; híbrido, remoto, presencial), caindo nos valores padrão (`clt`, `junior`, `presencial`) quando o anúncio não diz nada, o que é registrado no array `padrao` do resultado.

As linhas soltas que não entraram em nenhuma seção com cabeçalho são classificadas uma a uma em descrição, requisitos ou benefícios por três dicionários de palavras-chave (`PALAVRAS_BENEFICIO`, `PALAVRAS_REQUISITO`, `PALAVRAS_HORARIO`). Quando uma linha não bate com nenhuma palavra-chave (sem pista), a decisão passa para os padrões automáticos: `PadroesExtracao::decidir('vaga_linha', $l, 'descricao')` verifica se o texto da linha bate com um padrão aprendido nas vagas já cadastradas e revisadas, e só nesse caso a decisão troca de descrição para outro campo, sempre registrada em `$r['padroes']`. Depois, `casarValores()` junta um rótulo de benefício solto no cartaz com o valor correspondente lido em outra linha (por exemplo VALE REFEIÇÃO mais R\$ 48,00 por dia se tornam Vale Refeição: R\$ 48,00 por dia), e `juntar()` remove linhas repetidas ou quase idênticas lidas duas vezes pelo OCR, usando uma comparação de semelhança de pelo menos 78% entre os textos sem espaços.

Por fim, `Competencias::daVaga()` extrai as competências do anúncio e `categoria()` escolhe a área da vaga comparando essas competências (e as do título, que contam em dobro) com a constante `CATEGORIAS`; quando nenhuma categoria bate, `PadroesExtracao::decidir('vaga_categoria', ...)` tenta decidir pelo padrão dos títulos das vagas já cadastradas. O método `relatorio()` monta a tabela de revisão mostrada ao usuário, campo por

campo, classificando cada um como lido (veio do anúncio), padrão (o anúncio não dizia e ficou o valor padrão) ou falta (não foi possível preencher), contando quantos campos caem em cada situação.

Alguns exemplos reais do arquivo `tests/smoke.php` mostram essas etapas funcionando juntas: o texto VAGA: Vendedor Interno - Taguatinga com Salário: R\$ 3.000 a R\$ 5.500 + comissões e Benefícios: VT + VR R\$ 33,40 gera título Vendedor Interno, faixa salarial de R\$ 3.000 a R\$ 5.500 (ignorando o VR) e cidade Taguatinga; um cartaz real de roçadeira, com o logotipo ilegível (REQ.(ÓOM), o cargo em quatro linhas de letra grande (OPERADOR DE / MÁQUI NA / COSTAL / (ROÇADEIRA)) e um slogan (SOLUÇÕES QUE GERAM VALOR...), resulta no título Operador de Máquina Costal (Roçadeira), sem empresa (o logotipo não deu para confirmar), categoria Serviços Gerais e Limpeza, benefício Vale Refeição: R\$ 48,00 por dia (corrigindo o ror DIA do OCR para por dia) e contato WhatsApp (61) 97402-3121; e um cartaz real lido pelo navegador (Mimória) tem o cargo colado pelo OCR (CONSULTORDE) corrigido para Consultor de Vendas, com a empresa Mimória Business confirmada porque as letras do cartaz aparecem encostadas dentro do e-mail `mimoriabusiness@gmail.com`.

Método	O que faz
<code>doImagem(string \$path, ? array \$leituras = null): array</code>	Extraí os campos da vaga a partir da leitura de uma imagem de cartaz (leituras do navegador ou do Tesseract do servidor).
<code>doTexto(string \$texto, array \$ocr = []): array</code>	Extraí todos os campos da vaga a partir de um texto de anúncio (e, opcionalmente, de dados extras vindos de uma leitura de imagem).
<code>relatorio(array \$r, string \$categoriaNome = ""): array</code>	Monta a tabela de revisão campo a campo (lido, padrão ou faltando) do resultado de uma extração.
<code>categoria(array \$competencias, array \$doTitulo = []): string</code>	Sugere a categoria (área) da vaga a partir das competências encontradas, dando peso dobrado às competências do título.

Quadro 80 – Métodos públicos da classe `ExtracaoVaga`
Fonte: Autoria própria (2026).

A.8.2 `ExtracaoCurso.php`

A classe `ExtracaoCurso` transforma o texto de divulgação de um curso, e-book ou vídeo nos campos do cadastro: título, descrição, tipo, modalidade, nível, duração, se é gratuito, preço, link, instituição, categoria e competências ensinadas. Como na extração de vagas, o resultado só preenche o formulário - o administrador revisa antes de salvar.

O método `doTexto()` procura o título primeiro por um rótulo explícito (Curso:, Título:, Nome do curso:) e, na falta dele, usa a primeira linha curta que não pareça um link, e-mail ou preço; um título todo em maiúsculas é convertido para Título Com Iniciais Maiúsculas. A instituição é procurada primeiro por rótulo (Instituição:, Oferecido por:, Realização:), depois

pela constante `INSTITUICOES` (um dicionário de cerca de 20 instituições conhecidas, como Fundação Bradesco, SEBRAE, Senai, Coursera, Alura, cada uma com suas variações de nome e domínio) comparada tanto ao texto quanto à URL encontrada; quando nenhuma das duas bate, entra o padrão automático `PadroesExtracao::nomeConhecido('instituicao', $texto)`, que procura se o texto cita uma instituição já cadastrada no catálogo.

Os demais campos são decididos por palavras-chave simples: preço e gratuito a partir de um valor em reais no texto (`gratuito=0` só quando há valor e a palavra gratuito não aparece); duração a partir de um número seguido de horas, semanas, meses, dias, aulas ou módulos; modalidade (híbrido, presencial ou EAD, que é o padrão); nível (avançado, intermediário ou iniciante, que é o padrão); e tipo (e-book, vídeo ou curso, que é o padrão). As competências vêm de `Competencias::doCurso()` e a categoria é decidida por `categoriaSugerida()`: primeiro tenta a regra interna `categoria()`, que compara as competências encontradas com a constante `CATEGORIAS` (5 grandes áreas, como Informática e Excel ou Marketing, Dados e UX); quando a regra não reconhece nenhuma área, o padrão automático `PadroesExtracao::decidir('curso_categoria', $titulo, "")` tenta decidir pelos títulos de cursos já cadastrados, registrando a decisão em `$r['padroes']`.

O método `capa()` escolhe o banner de divulgação de um curso pela instituição, nunca pela área: a constante `CAPAS_INSTITUICAO` associa cada instituição (Fundação Bradesco, SEBRAE, Escola Virtual do Governo, Google, FGV) a uma imagem de banner que já tem a marca dela estampada, então um e-book do Banco Central não pode sair com o banner da FGV só porque está na mesma área de Finanças. A capa só vale para o tipo curso: e-books e vídeos nunca recebem o banner institucional (ficam com o ícone do formato na tela de cadastro), e instituições sem banner próprio (como SENAC ou Banco Central) também ficam sem capa - o teste automatizado confirma que `ExtracaoCurso::capa('Fundação Bradesco – Escola Virtual')` devolve a imagem `curso1.png`, `ExtracaoCurso::capa('Banco Central do Brasil')` devolve vazio, e `ExtracaoCurso::capa('Sebrae', '', 'ebook')` também devolve vazio por ser um e-book.

O método `primeiroLink()` extrai o primeiro endereço `http(s)` do texto, cuidando de dois detalhes: corta pontuação de fim de frase colada no link, mas mantém parênteses que fazem parte do próprio endereço quando estão em par (por exemplo `.../Cartilha%20(2)%20(1).pdf` continua inteiro), e remove apenas o parêntese de fechamento que sobra quando o link foi escrito entre parênteses no texto (veja `https://site.com/a)` devolve só `https://site.com/a`.

O método `fichas()` é a importação em lote: ele lê de uma vez várias fichas de curso coladas pelo administrador, normalmente a resposta de uma IA de pesquisa (Perplexity, ChatGPT etc.). Antes de separar as fichas, o texto passa por uma limpeza da formatação que as

IAs costumam deixar: negrito e títulos de markdown, links no formato [texto](endereço), números de citação como [1], e as marcas de citação específicas do ChatGPT (:contentReference[oaicite:1]{index=1}) e de outras ferramentas (【4†fonte】) - o teste automatizado confirma que um texto como Bom guia [2]. :contentReference[oaicite:1]{index=1} 【4†fonte】 se transforma simplesmente em Bom guia. As fichas são separadas por uma linha contendo só --- (ou, na falta dela, por um novo Título: repetido), e cada bloco passa primeiro pela extração normal (doTexto()) e depois tem seus campos sobrescritos pelos rótulos explícitos da ficha (camposRotulados(), que reconhece os mesmos rótulos usados no formulário de cadastro, listados na constante ROTULOS).

Dentro de daFicha(), a instituição informada na ficha é padronizada pelo nome oficial da fonte do link (FontesCursos::nomeOficial()); a imagem segue uma ordem de preferência: um link de imagem informado na ficha (imagem_url, conferido na prévia e baixado só ao cadastrar, por ImagemRemota), um caminho de imagem do próprio site que já existe (imagem_propria, entra direto), e na falta dos dois o banner da instituição (capa()) fica de reserva; e a área usa o nome informado na ficha se ele bater com uma categoria já cadastrada, senão cai de novo em categoriaSugerida(). No teste automatizado, um lote com duas fichas (uma em markdown, com e-book e link da Escola Virtual; outra com curso presencial em Taguatinga/DF, preço R\$ 120,00) é interpretado corretamente: a primeira fica com tipo ebook e a URL exata do link markdown, e a segunda fica com modalidade presencial, gratuito igual a 0, preço 120,0 e a cidade Taguatinga/DF incorporada à descrição.

Método	O que faz
doTexto(string \$texto): array	Extraí os campos do cadastro de um curso, e-book ou vídeo a partir de um texto de divulgação único.
capa(string \$instituicao, string \$url = "", string \$tipo = 'curso'): string	Escolhe o banner de divulgação com a marca da instituição (só para cursos; e-books e vídeos ficam sem capa institucional).
primeiroLink(string \$texto): string	Extraí o primeiro endereço http(s) do texto, preservando parênteses que fazem parte do próprio link.
fichas(string \$texto, array \$categorias = []): array	Separa e interpreta várias fichas de curso coladas de uma vez (resposta de uma IA de pesquisa), limpando marcas de formatação e de citação.
promptPesquisa(array \$categorias, int \$quantidade = 20): string	Atalho que monta o prompt de pesquisa guiada (via FontesCursos::prompt) sem direcionar formato, área ou fonte.
categoria(array \$competencias): string	Sugere a categoria (área) de um curso a partir das competências encontradas nele.

Quadro 81 – Métodos públicos da classe ExtracaoCurso
Fonte: Autoria própria (2026).

A.8.3 PadroesExtracao.php

A classe `PadroesExtracao` é a peça que chamamos de padrões automáticos. Ela preenche, de forma determinística e explicável, as lacunas que as regras de `ExtracaoVaga` e `ExtracaoCurso` deixam quando não têm nenhuma pista no texto: uma linha solta sem palavra-chave, uma área que o título não indica com clareza, ou uma empresa ou instituição que não está em nenhuma lista fixa do código. Para isso, a classe conta o que já aconteceu nos cadastros que uma pessoa revisou e salvou no banco (vagas e cursos), e só vira padrão o que se repete de forma consistente.

O critério de decisão está nas duas constantes no topo da classe: `MIN_CADASTROS` (2) exige que um termo apareça em pelo menos 2 cadastros diferentes, e `PUREZA` (0,9) exige que, nesses cadastros, pelo menos 90% das vezes o termo tenha caído no mesmo destino. Os dois juntos evitam que uma coincidência isolada (por exemplo, uma palavra que apareceu uma vez na descrição de uma vaga por acaso) vire regra; e evitam que uma palavra ambígua (que às vezes vai para um campo, às vezes para outro) decida qualquer coisa. O método `filtrar()` aplica exatamente esse corte sobre uma contagem de ocorrências por termo e por destino.

A normalização dos termos (método `termos()`) segue o mesmo princípio de normalizar por caractere usado em toda a plataforma: o texto passa por `Competencias::normalizar()` (sem acento, minúsculo, só letras e números), todo número é trocado por um caractere `#` (assim `R$ 30,00` e `R$ 31,50` geram o mesmo termo), palavras de 1 ou 2 letras e as palavras vazias da lista `VAZIAS` (artigos, preposições, conectivos como `a`, `de`, `para`, `com`, `vaga`, `empresa`) são descartadas isoladamente mas continuam servindo de ponte entre palavras vizinhas. Por isso, `café da manhã` e `Café Da Manhã` - `R$ 30,00` geram exatamente os mesmos termos, incluindo o par `cafe manha` (a preposição `da` no meio é pulada, não quebra o par), como confirma o teste automatizado comparando `PadroesExtracao::termos('CAFÉ DA MANHÃ, R$ 30,00')` com `PadroesExtracao::termos('café da manhã - r$ 31,50')`.

Os padrões são organizados em três contextos independentes, todos montados por `montar()` a partir dos registros do banco (ou de dados em memória, nos testes): `vaga_linha` guarda, para cada termo (palavra ou par de palavras) encontrado nas descrições, requisitos e benefícios das vagas já cadastradas, qual desses três campos ele mais costuma indicar - é esse contexto que `ExtracaoVaga` consulta (`decidir('vaga_linha', ...)`) para decidir uma linha solta do anúncio sem nenhuma palavra-chave de regra; `vaga_categoria` guarda, para cada termo visto nos títulos das vagas, a categoria mais comum - usado quando a regra de categoria não reconhece nenhuma competência no título; e `curso_categoria` faz o mesmo com os títulos do

catálogo de cursos. No teste automatizado com quatro vagas de exemplo, a palavra uniforme aparece nos benefícios de duas vagas diferentes (categoria no mesmo destino) e se torna um padrão, enquanto vitrine aparece numa descrição só e não se torna; churrasqueiro aparece no título de duas vagas, as duas na categoria Alimentação, e se torna padrão vaga_categoria, mas atendente aparece em vagas de categorias diferentes (Atendimento ao Público e Alimentação) e, por não atingir 90% de pureza no mesmo destino, não se torna padrão.

O quarto contexto, nomes(), é diferente: em vez de aprender um destino a partir de contagens, ele apenas lista nomes próprios já cadastrados - anunciante das vagas e nome_fantasia das empresas (tipo empresa), ou instituicao dos cursos (tipo instituicao) - e nomeConhecido() procura se algum desses nomes aparece literalmente no texto do anúncio ou do curso, testando sempre do nome mais longo para o mais curto. Esse é o caminho que ExtracaoVaga e ExtracaoCurso usam como último recurso, quando nenhuma regra de texto (marcas conhecidas, frases, logotipo) encontra a empresa ou a instituição. Para que um nome genérico como Loja ou Empresa Teste nunca seja reconhecido como se fosse uma empresa de verdade, nomeGenerico() rejeita nomes curtos demais ou formados só por palavras da lista NOMES_GENERICOS (vaga, empresa, local, horario, contato, entre outras). No teste automatizado, depois de cadastrar os nomes Padaria Pão Quente, Loja e Empresa Teste como já conhecidos, o anúncio Venha trabalhar na Padaria Pão Quente reconhece a Padaria Pão Quente como anunciante, mas os outros dois nomes nunca seriam aceitos por serem genéricos.

Os padrões automáticos não têm treinamento, modelo estatístico, ajuste de pesos nem tela de configuração; o próprio comentário do arquivo resume isso como sem tabela, sem tela de ajuste e sem treino. O que existe é uma contagem simples e auditável de quantas vezes um termo, já visto em cadastros revisados por uma pessoa, caiu em cada destino possível. O método decidir() sempre informa qual foi o termo exato que valeu (a chave termo no resultado), o que torna cada decisão rastreável e reproduzível enquanto os mesmos dados estiverem no banco. Essa escolha também respeita a LGPD de um jeito direto: o currículo do candidato, que contém dados pessoais, nunca entra na contagem. Só as tabelas vagas e cursos (e os campos nome_fantasia dos perfis de empresa e instituicao dos cursos) alimentam os padrões, e a extração do currículo segue apenas as regras fixas do código, como o comentário da classe deixa claro.

Por fim, padroes() e nomes() consultam o banco uma única vez por requisição (o resultado fica em cache nas propriedades estáticas \$padroes e \$nomes) e, se o banco estiver indisponível, o erro é capturado e registrado em log, e a extração simplesmente segue usando só as regras fixas, sem padrões automáticos - o sistema nunca quebra por falta de banco. Os

métodos `ligar()`, `usarPadroes()`, `usarNomes()` e `limpar()` existem para os testes automatizados: `ligar(false)` desliga os padrões (usado para conferir as regras isoladamente, sem interferência do que já está cadastrado), `usarPadroes()/usarNomes()` alimentam a classe com dados em memória no lugar de uma consulta ao banco, e `limpar()` descarta o que foi montado para forçar uma nova leitura.

Método	O que faz
<code>decidir(string \$contexto, string \$texto, string \$palpiteRegra): array</code>	Decide uma classificação: usa o padrão aprendido quando ele existir e discordar do palpite da regra; senão, mantém o palpite da regra.
<code>padraoQueCasa(string \$contexto, string \$texto): ? array</code>	Procura, entre os termos do texto, o padrão mais específico já aprendido para aquele contexto.
<code>nomeConhecido(string \$tipo, string \$texto): string</code>	Procura no texto uma empresa ou instituição já cadastrada no sistema, do nome mais longo para o mais curto.
<code>nomeGenerico(string \$nome): bool</code>	Diz se um nome é curto demais ou feito só de palavras genéricas, para nunca virar um nome conhecido.
<code>termos(string \$texto): array</code>	Normaliza um texto em palavras e pares de palavras vizinhas (sem acento, minúsculas, número vira #), descartando palavras vazias e muito curtas.
<code>padroes(): array</code>	Monta (ou devolve do cache da requisição) os padrões dos três contextos a partir dos cadastros do banco.
<code>montar(array \$vagas, array \$cursos): array</code>	Monta os três mapas de padrões a partir de registros de vagas e cursos (usado também para montar padrões em memória nos testes).
<code>nomes(string \$tipo): array</code>	Lista os nomes conhecidos de empresas ou instituições já cadastrados, sem os genéricos, do mais longo para o mais curto.
<code>ligar(bool \$ligado): void</code>	Liga ou desliga o uso dos padrões automáticos (os testes das regras rodam com os padrões desligados).
<code>usarPadroes(array \$padroes): void</code>	Substitui os padrões por um conjunto fornecido em memória, no lugar de consultar o banco.
<code>usarNomes(string \$tipo, array \$nomes): void</code>	Substitui os nomes conhecidos de um tipo por uma lista fornecida em memória.
<code>limpar(): void</code>	Descarta os padrões e nomes já montados, para serem lidos de novo do banco na próxima consulta.

Quadro 82 – Métodos públicos da classe `PadroesExtracao`
Fonte: Autoria própria (2026).

A.8.4 PromptsPesquisa.php

A classe `PromptsPesquisa` monta o texto padrão que é entregue a uma IA de pesquisa na internet (Perplexity, ChatGPT, Google Gemini, Microsoft Copilot ou Claude) para encontrar e descrever cursos, e-books e vídeos gratuitos. Esse prompt não aparece dentro do painel administrativo: ele é publicado em `docs/PROMPTS_PESQUISA.md`, gerado pelo script `docs/gerar_prompts.php`, e o administrador copia a resposta da IA (uma ou mais fichas) para a caixa `Extrair` ou `Importar` vários da tela `admin/pages/cursos.php`, de onde `ExtracaoCurso::fichas()` faz a leitura.

A constante IAS guarda, para cada uma das 5 IAs suportadas, o nome, onde configurar o prompt mestre (um Space no Perplexity, um Projeto ou GPT personalizado no ChatGPT, um Gem no Gemini, a primeira mensagem de uma conversa no Copilot, um Projeto no Claude) e uma instrução extra reforçando que a IA deve pesquisar de fato na web, nunca responder de memória. O método `formatoFicha()` define os rótulos exatos de uma ficha, na mesma ordem do formulário de cadastro e exatamente os rótulos que `ExtracaoCurso::fichas()` sabe interpretar: Título, Tipo, Instituição, Modalidade, Cidade, Nível, Carga horária, Gratuito, Preço, Área (lista as categorias informadas, ou as áreas padrão de `ExtracaoCurso::AREAS` quando nenhuma é passada), Link, PDF (só para e-book gratuito, o link direto do arquivo) e Imagem (o link direto do arquivo de imagem), terminando com Descrição.

O método privado `regras()` concentra as instruções de pesquisa comuns a todos os prompts: abrir a página oficial de cada item antes de responder, nunca inventar link, carga horária, preço ou imagem (escrevendo Não informado ou Não encontrada quando não achar), conferir de fato cada link antes de responder (um endereço montado de cabeça, com erro de digitação, ou que cai numa página de erro ou numa lista genérica não vale), que a imagem deve ser um arquivo direto (terminando em .jpg, .jpeg, .png ou .webp, nunca a página do curso, um logotipo genérico ou uma miniatura de busca), a lista de fontes oficiais preferidas (montada dinamicamente a partir de `FontesCursos::FONTES`), responder tudo em português, indicar se o curso tem certificado, marcar item fora do ar como NÃO ENCONTRADO: <item>, <motivo>, e responder só com as fichas, sem introdução, conclusão, tabela, negrito ou marcas de citação, separadas por uma linha contendo apenas ---.

`mestre()` monta o prompt configurado uma única vez em cada IA: explica que, a partir daquele momento, cada mensagem do administrador será um ou mais links ou títulos (um por linha) e que a IA deve responder uma ficha por item, mantendo a mesma ordem; no caso do Copilot, acrescenta uma instrução para a IA responder apenas Pronto na primeira mensagem, já que o prompt mestre precisa ser a primeira mensagem da conversa. `avulso()` monta o equivalente para uma conversa única, já com os links ou títulos (limpos por entradas(), numerados e sem repetição, no máximo 20) embutidos dentro do próprio prompt, identificando cada item como LINK ou TÍTULO pela função `url_http_valida()`, e opcionalmente fixando o formato (todos cursos, todos e-books ou todos vídeos) quando o administrador já sabe o que quer.

O teste automatizado confirma que o prompt mestre das 5 IAs sempre contém a seção **FORMATO DE CADA FICHA**, a lista de áreas informada (Área: uma destas: Área X) e a instrução **NÃO ENCONTRADO**; e faz uma ida e volta completa: uma ficha escrita exatamente

no formato do prompt mestre, com negrito de markdown e uma marca de citação [1] colada no link, é interpretada de volta por `ExtracaoCurso::fichas()` recuperando corretamente todos os campos (título Guia Y, tipo ebook, modalidade ead, nível intermediário, duração 3 horas, gratuito igual a 0, preço 19,9, categoria Área X, link sem a marca de citação, instituição padronizada como Banco Central do Brasil e descrição mencionando certificado). Já `avulso()` é testado com uma lista de entradas misturando links repetidos, um título com instituição entre parênteses e uma linha vazia: o prompt gerado numera e rotula corretamente cada item (1. <https://cartilha.cert.br/>, LINK, 2. Guia Z (SEBRAE), TÍTULO), sem repetir o link que apareceu duas vezes.

Método	O que faz
<code>formatoFicha(array \$areas): string</code>	Monta o texto com os rótulos exatos de uma ficha de curso, na mesma ordem do formulário de cadastro.
<code>mestre(string \$ia, array \$areas): string</code>	Monta o prompt mestre de uma IA de pesquisa específica, configurado uma única vez e reutilizado em conversas futuras.
<code>avulso(array \$entradas, array \$areas, string \$formato = "): string</code>	Monta um prompt pronto para uma conversa única, já com os links ou títulos informados embutidos.
<code>entradas(array \$linhas): array</code>	Limpa uma lista de linhas de entrada (números, marcadores, vazias e repetidas), limitando a 20 itens.

Quadro 83 – Métodos públicos da classe `PromptsPesquisa`
Fonte: Autoria própria (2026).

A.8.5 FontesCursos.php

A classe `FontesCursos` organiza a pesquisa guiada de cursos e e-books, usada na tela `admin/pages/cursos.php` em Importar vários de uma vez. A constante `FONTES` lista cerca de 17 instituições oficiais de ensino gratuito e certificado (Fundação Bradesco – Escola Virtual, Escola Virtual.Gov/Enap, SEBRAE, Google Grow, Microsoft Learn, FGV Online, IFB, SENAI, SENAC, Banco Central, CERT.br/NIC.br, Febraban, Ministério do Trabalho e Emprego, eduCAPES, CVM, Cisco Networking Academy e Fundação Estudar), cada uma com o nome oficial, os domínios que identificam seus links, a página de catálogo onde a pesquisa deve começar, os formatos que costuma oferecer (curso, e-book, vídeo), as áreas típicas e uma dica curta sobre o que esperar dela. A fonte `eduCAPES` é marcada como repositório porque republica material de outras instituições - por isso o nome dela nunca deve substituir a instituição real informada na ficha.

O método `fonteDoLink()` reconhece a qual fonte um link pertence comparando o domínio do link (sem `www`) com os domínios cadastrados, aceitando tanto o domínio exato quanto um subdomínio dele (`sp.senai.br` é reconhecido como `senai.br`), e quando o domínio cadastrado inclui um caminho (como `gov.br/investidor`, para diferenciar a CVM de outras

páginas do governo), exige que o caminho do link também bata; quando mais de uma fonte poderia combinar, vence a correspondência mais específica (o domínio mais longo). O teste automatizado confirma, por exemplo, que <https://www.gov.br/investidor/pt-br/a.pdf> é reconhecido como a fonte cvm, mas <https://www.gov.br/outra> não é reconhecido por nenhuma fonte (falta o caminho /investidor), e que um domínio parecido mas diferente, como <https://senai.brasil.com>, não é confundido com o SENAI oficial.

O método `nomeOficial()` padroniza o nome da instituição de uma ficha pelo nome oficial da fonte reconhecida no link, preservando o nome informado quando o link não é de nenhuma fonte conhecida ou quando a fonte é um repositório (eduCAPES), e mantém uma eventual parceria escrita entre parênteses no nome original (por exemplo Enap - Escola Virtual.Gov (conteúdo Microsoft) se torna Escola Virtual.Gov (Enap) (conteúdo Microsoft)).

O método `cobertura()` calcula, a partir de todos os cursos já cadastrados, quantos itens de cada tipo (curso, e-book, vídeo) existem em cada área e quantos vieram de cada fonte oficial, e identifica as lacunas: a metade das áreas com menos conteúdo cadastrado, da menor para a maior, usada para direcionar a pesquisa para onde a plataforma está mais fraca. No teste automatizado, com três itens cadastrados distribuídos em três áreas (A com 2 itens, B com 1, C com 0), as lacunas calculadas são C e B, nessa ordem.

O método `prompt()` monta o pedido final para a IA de pesquisa, reaproveitando o mesmo formato de ficha de `PromptsPesquisa::formatoFicha()`. Ele ajusta a quantidade pedida entre 5 e 40 itens; decide o que pesquisar - um formato específico (cursos, e-books ou vídeos) ou uma mistura com pelo menos 5 e-books quando nenhum formato é escolhido; decide onde focar - uma área específica, uma fonte específica (nesse caso pesquisa somente ali) ou, na falta das duas, as áreas de lacuna calculadas por `cobertura()`; monta a lista de fontes oficiais a pesquisar (o catálogo de cada uma, a dica de busca `site:dominio` e a observação de cada fonte), filtrando só as fontes que oferecem o formato pedido; e acrescenta a lista dos links já cadastrados (até 150), para a IA não repetir o mesmo conteúdo com outro endereço. No teste automatizado, um pedido de 12 e-books, direcionado pelas lacunas C e B, gera um prompt que pede liste 12 e-books, cita MENOS conteúdo na plataforma: C, B, inclui a dica `site:cartilha.cert.br` (que aceita e-book) mas não `site:learn.microsoft.com` (que só aceita curso), avisa para NÃO repetir o link já cadastrado, e lista a fonte CERT.br / NIC.br exatamente uma vez; e um segundo pedido, restrito à fonte sebrae e à área B com quantidade pedida de 99, gera um prompt que fala em pesquisar SOMENTE nesta fonte, cita só SEBRAE , , não cita Microsoft Learn, fala em Todos da área "B" e tem a quantidade corrigida para o limite de 40.

Método	O que faz
fonteDoLink(string \$url): string	Reconhece a qual fonte oficial cadastrada um link pertence, pelo domínio (ou subdomínio) e, quando houver, pelo caminho da URL.
nomeOficial(string \$instituicao, string \$url): string	Padroniza o nome da instituição de uma ficha pelo nome oficial da fonte reconhecida no link, preservando parcerias e repositórios.
cobertura(array \$cursos, array \$areas): array	Calcula quantos itens de cada tipo existem por área e por fonte, e aponta as áreas com menos conteúdo (lacunas).
prompt(array \$op, array \$areas, array \$lacunas = [], array \$jaCadastrados = []): string	Monta o prompt de pesquisa guiada, direcionado por formato, área ou fonte, evitando repetir os links já cadastrados.

Quadro 84 – Métodos públicos da classe FontesCursos
Fonte: Autoria própria (2026).

A.9 Leitura de documentos, currículo, imagens e capa do PDF

Nesta seção documentamos as classes da pasta `app/Services/Extracao` que transformam arquivos em texto e em dados: o currículo que o candidato envia (PDF, DOCX ou DOC), a foto que vem dentro dele, o cartaz da vaga que a empresa envia como imagem, as imagens e os PDFs dos cursos e e-books que o Administrador importa a partir das fichas da pesquisa e a capa do e-book, tirada da primeira página do PDF. Nenhuma dessas peças usa API externa nem biblioteca instalada por gerenciador de pacotes. Tudo foi escrito em PHP puro, com as extensões que já vêm no XAMPP (mbstring, zlib, GD, cURL, DOM e, quando estiver ligada, zip). Programas externos como `pdftotext`, `antiword` e `Tesseract` são usados só como reserva, quando existem no computador, e o sistema funciona sem eles.

Tomamos essa decisão por três motivos. O primeiro é a instalação: o sistema precisa rodar no XAMPP de qualquer laboratório da escola, sem instalar nada além dele. O segundo é a LGPD: o currículo tem dados pessoais do candidato, e mandar o arquivo para um serviço de terceiros só para ler o texto seria expor esses dados sem necessidade. O terceiro é o controle: lendo o arquivo nós mesmos, sabemos a posição e o tamanho de cada trecho de texto, o que permite resolver currículos em duas colunas e achar o nome escrito com a maior letra, coisas que uma conversão simples para texto não entrega.

O caminho do currículo passa por quase todas as classes desta seção. O `CurriculoController::upload()` confere a extensão e a assinatura dos primeiros bytes do arquivo (%PDF para PDF, PK para o ZIP do DOCX e D0 CF 11 E0 para o arquivo do Word antigo), grava o arquivo em `storage/uploads` e chama `ExtracaoCurriculo::extrair()`, que entrega o trabalho ao `LeitorDocumento`. Este escolhe o `PdfTexto` ou o `DocxTexto`. Com o texto pronto, `ExtracaoCurriculo::extrairCampos()` separa os campos, `AplicacaoCurriculo::calcular()` decide

o que entra no perfil (tabela perfis), `LeitorDocumento::extrairFoto()` procura a foto e o texto fica gravado na tabela currículos, na coluna `curriculo_texto`. O cartaz da vaga usa a `OcrImagem`, e os cursos e e-books usam a `ImagemRemota` e a `CapaPdf`.

Arquivo	Papel no sistema
<code>LeitorDocumento.php</code>	Porta de entrada: escolhe o leitor pelo tipo do arquivo, aplica os limites de tamanho, limpa o texto e escolhe a foto do currículo.
<code>PdfTexto.php</code>	Leitor de PDF em PHP puro: objetos, object streams, <code>FlateDecode</code> , fontes, posição de cada trecho, ordem de leitura em duas colunas, imagens e capa.
<code>DocxTexto.php</code>	Converte o XML do Word (<code>WordprocessingML</code>) em texto: parágrafos, listas, tabelas, caixas de texto, hyperlinks e linhas em destaque.
<code>OcrImagem.php</code>	Leitura do cartaz da vaga por OCR: valida as leituras feitas no navegador (<code>Tesseract.js</code>) e, como reserva, lê com o Tesseract do servidor.
<code>ExtracaoCurriculo.php</code>	Separa os dados do currículo: linhas de dados, nome, título, seções, contato, local, datas, experiências, formação e os demais campos do perfil.
<code>AplicacaoCurriculo.php</code>	Aplica os campos extraídos ao perfil: preenche os vazios, mantém os preenchidos, mescla as listas e monta o relatório da extração.
<code>ImagemRemota.php</code>	Abre cada link da ficha de curso ou e-book, descobre se é imagem, PDF ou página e baixa o que serve, só de servidor público.
<code>CapaPdf.php</code> e <code>capa-pdf.ps1</code>	Gera a capa do e-book a partir da primeira página do PDF, pelo leitor de PDF do Windows ou, na falta dele, pela maior imagem da página.

Quadro 85 – Arquivos da leitura de documentos e o papel de cada um

Fonte: Autoria própria (2026).

Como essas classes recebem arquivos enviados por usuários e conteúdo baixado da internet, cada uma tem limites de tamanho e de tempo. O quadro a seguir reúne esses limites, que explicamos em detalhe em cada subseção.

Limite	Valor	Onde e por quê
Tamanho do currículo enviado	10 MB	<code>MAX_FILE_SIZE</code> em <code>config/config.php</code> , conferido pelo <code>CurriculoController::upload()</code> .
Conteúdo descompactado por arquivo	30 MB	<code>LeitorDocumento::LIMITE_DESCOMPACTADO</code> , usado também pelo <code>PdfTexto</code> . Protege contra a bomba de descompressão.
Trechos de texto por PDF	200.000	<code>PdfTexto::MAX_TRECHOS</code> . Um currículo tem poucos milhares; passar disso indica arquivo anormal.
Texto final do currículo	100.000 caracteres	<code>LeitorDocumento::limpar()</code> corta o texto nesse tamanho.
Imagem candidata a foto	2 KB a 12 MB, 80 a 6000 px	<code>LeitorDocumento::extrairFoto()</code> : fora disso é ícone, lixo ou imagem grande demais.
Foto gravada no perfil	até 800 px e 3 MB	<code>fotoLeve()</code> reduz; <code>AplicacaoCurriculo::salvarFoto()</code> recusa acima de 3 MB.
Imagem do cartaz para OCR	40 milhões de pixels	<code>OcrImagem::MAX_PIXELS</code> , conferido no cabeçalho antes de abrir a imagem.
Leitura (TSV) vinda do navegador	1,5 MB cada, até 4	<code>OcrImagem::MAX_TSV</code> e <code>leiturasDoNavegador()</code> .
Tempo do Tesseract no servidor	90 segundos	<code>OcrImagem::tsvParalelo()</code> encerra o que não terminou.
Imagem baixada da internet	5 MB	<code>ImagemRemota::LIMITE_IMAGEM</code> .

Limite	Valor	Onde e por quê
Página HTML aberta para descobrir o link	512 KB	ImagemRemota::LIMITE_PAGINA: as metatags ficam no começo da página.
PDF baixado só para tirar a capa	80 MB	ImagemRemota::LIMITE_PDF_CAPA.
PDF baixado para a biblioteca	150 MB	MAX_PDF_REMOTO em app/Core/Upload.php.
Imagem gravada de curso ou e-book	até 800 px de largura	ImagemRemota::gravar() reduz e salva em JPEG.
Cada operação do leitor de PDF do Windows	60 segundos	capa-pdf.ps1, funções Esperar e EsperarAcao.

Quadro 86 – Limites de segurança da leitura de arquivos e imagens

Fonte: Autoria própria (2026).

A.9.1 LeitorDocumento (app/Services/Extracao/LeitorDocumento.php)

O LeitorDocumento é a porta de entrada da leitura de arquivos. É uma classe final, só com métodos estáticos, que recebe o caminho de um arquivo já gravado no servidor e devolve o texto puro, em UTF-8, pronto para a ExtracaoCurriculo. Junto com o texto, ela deixa disponíveis quatro propriedades estáticas, preenchidas de novo a cada leitura: \$metodo, com o nome do leitor que funcionou (por exemplo "leitor PDF interno", "pdftotext", "ZipArchive", "leitor DOCX interno", "antiword" ou "leitor DOC interno (aproximado)"), que aparece no relatório da extração; \$destaques, com as linhas escritas na maior fonte, que quase sempre são o nome do candidato; \$links, com os endereços dos hyperlinks do arquivo; e \$grandeDemais, que fica verdadeira quando a leitura parou por passar do limite de tamanho.

Quem chama a classe é a ExtracaoCurriculo (métodos extrair() e extrairCampos()) e o CurriculoController::upload(), que lê as propriedades \$destaques e \$grandeDemais e chama extrairFoto(). Outras classes reaproveitam seus utilitários: a OcrImagem usa encontrarComando(), q() e linhaComando() para achar e chamar o Tesseract; a ExtracaoCurriculo usa limpar() e chaveLink(); e o PdfTexto usa a constante LIMITE_DESCOMPACTADO como orçamento de leitura. O trabalho pesado fica nas classes vizinhas PdfTexto e DocxTexto.

Método	O que faz
extrair(string \$path): string	Zera o estado, escolhe o leitor pela extensão (pdf, docx, doc ou txt), trata o estouro de tamanho e os erros, converte para UTF-8, limpa o texto e acrescenta no fim a linha "Links:" com os endereços que só existiam nos hyperlinks.
chaveLink(string \$u): string	Devolve o endereço em minúsculas, sem http:// ou https://, sem www. e sem a barra final. Serve para comparar links: "https://www.linkedin.com/in/ana/" e "linkedin.com/in/ana" têm a mesma chave.
listarZip(string \$path): array	Lista os nomes das entradas de um arquivo ZIP, sempre com "/" (mesmo quando o compactador gravou "\"), pelo ZipArchive ou pelo leitor interno.

Método	O que faz
lerEntradaZip(string \$path, string \$nome, int \$limite): ?string	Leitor mínimo de ZIP em PHP puro: acha a entrada no diretório central e devolve o conteúdo guardado ou descompactado, sem nunca passar de \$limite bytes.
extrairFoto(string \$path): ?array	Procura a foto do candidato entre as imagens do DOCX (pasta word/media) ou do PDF (PdfTexto::imagens()), dá uma nota a cada uma e devolve a melhor como JPEG de até 800 px, com os dados, a extensão, a largura e a altura.
pontuacaoFoto(string \$dados, int \$w, int \$h): float	Calcula a nota de "parece foto de currículo" (0 significa que não é) pela forma, pelo tamanho, pelos tons de pele e pela variedade de cores da imagem.
encontrarComando(string \$nome, array \$candidatos): ?string	Procura um programa externo (pdftotext, antiword, tesseract) nos caminhos conhecidos e, se não achar, pergunta ao sistema operacional (where no Windows, command -v no Linux).
q(string \$v): string	Coloca um argumento de linha de comando entre aspas duplas, escapando as aspas internas.
linhaComando(string \$c): string	Acrescenta ao comando o descarte da saída de erro (2>NUL no Windows e 2>/dev/null no Linux).
textoUtil(string \$t): bool	Considera útil o texto que tem pelo menos 20 letras.
limpar(string \$t): string	Padroniza as quebras de linha, desfaz ligaduras tipográficas, remove caracteres de controle, de largura zero e de uso privado, junta espaços repetidos e corta o texto em 100.000 caracteres.

Quadro 87 – Métodos públicos do LeitorDocumento
Fonte: Autoria própria (2026).

Escolha do leitor por tipo de arquivo. O método extrair() olha a extensão e usa um match do PHP 8 para chamar pdf(), docx() ou doc(); arquivos .txt são lidos diretamente (o envio de currículo, porém, só aceita PDF, DOC e DOCX). Para o PDF, a primeira tentativa é sempre o leitor interno PdfTexto, porque ele preserva as linhas e a ordem de leitura das colunas e ainda fornece os destaques e os links. O resultado é aceito quando é útil (textoUtil(), pelo menos 20 letras) e legível: o método privado legivel() exige que mais de 85% dos caracteres sejam letras, números, espaços ou pontuação comum. Essa conferência existe porque há PDFs com fontes sem mapa de caracteres, e neles o leitor devolve símbolos sem sentido em vez de palavras. Nesse caso, a segunda tentativa é o pdftotext (Poppler ou Xpdf), procurado em caminhos conhecidos (o Git para Windows, por exemplo, já traz um) e chamado com as opções -layout e -enc UTF-8; a saída vai para um arquivo temporário ao lado do currículo, é lida até o limite de 30 MB e apagada em seguida. Se o pdftotext não existir ou não ajudar, fica o texto do leitor interno, mesmo imperfeito.

Para o DOCX, o método docx() abre o ZIP, lê word/document.xml e o arquivo de relações word/_rels/document.xml.rels (que diz para onde aponta cada hyperlink e quais são os cabeçalhos e rodapés) e entrega os dois ao DocxTexto. O cabeçalho do documento costuma trazer o nome e o contato, por isso entra antes do corpo. Do rodapé, aproveitamos só as linhas de contato (com @, http, www ou um número de telefone) e descartamos as que parecem

numeração de página, como "Página 2". Para o DOC antigo (Word 97 a 2003), o método `doc()` usa o `antiword` quando ele existe. Sem ele, fazemos uma leitura aproximada: o Word antigo guarda o texto dentro do arquivo binário em UTF-16LE ou em Windows-1252, então procuramos sequências de pelo menos 6 caracteres UTF-16 e de pelo menos 12 caracteres de 8 bits, ficamos com a versão UTF-16 quando ela tem pelo menos 60% do tamanho da outra, retiramos os nomes internos do Word que aparecem no binário (Normal, Heading 1, Times New Roman, Root Entry, WordDocument e outros) e as linhas com menos de 3 letras, e recuperamos os campos HYPERLINK.

Limites de tamanho e a bomba de descompressão. O DOCX é um ZIP e o PDF guarda quase tudo compactado. Um arquivo malicioso de poucos megabytes pode se expandir para gigabytes ao ser descompactado (a chamada bomba de descompressão) e derrubar o PHP por falta de memória. Por isso definimos `LIMITE_DESCOMPACTADO = 30 MB` por arquivo; um currículo normal fica bem abaixo de 1 MB. No DOCX, o método privado `abrirZip()` devolve uma função de leitura que divide um orçamento único entre todas as entradas lidas. Com o `ZipArchive`, o tamanho declarado de cada entrada é conferido antes de descompactar e a leitura pede no máximo esse tamanho, então mesmo um tamanho declarado falso não faz o PHP ler mais do que o permitido. No leitor interno, o `gzinflate()` recebe o teto como tamanho máximo de saída, e o aviso "insufficient memory" do `zlib` também é tratado como estouro. Passar do limite lança uma `LengthException`; `extrair()` a captura, marca `$grandeDemais`, devolve texto vazio, e o `CurriculoController` apaga o arquivo e pede ao candidato que salve o currículo de novo (por exemplo, com "Salvar como PDF" no Word) antes de reenviar. Qualquer outro erro de leitura também resulta em texto vazio, sem derrubar a página.

Leitor de ZIP em PHP puro. A extensão `zip` do PHP vem desligada em algumas instalações do XAMPP, e sem ela o DOCX não abriria. Por isso escrevemos um leitor mínimo de ZIP. O método privado `diretorioZip()` procura, de trás para frente, a assinatura do fim do diretório central (os bytes PK 05 06), lê dali quantas entradas existem e onde o diretório começa, e percorre cada registro (assinatura PK 01 02) pegando o método de compressão, os tamanhos compactado e original, o nome e a posição do cabeçalho local. O método `lerEntradaZip()` pula o cabeçalho local (30 bytes mais o nome e o campo extra) e devolve os dados: guardados sem compressão (método 0) ou descompactados com `deflate` (método 8); outros métodos são ignorados. O arquivo inteiro fica num cache estático, chaveado pelo caminho e pela data de modificação, para não ser relido a cada entrada. Os nomes com barra invertida são convertidos para barra normal porque o `Compress-Archive` do Windows grava "word\document.xml" em vez de "word/document.xml".

Limpeza do texto. Antes de devolver, `extrair()` converte para UTF-8 o texto que não estiver nessa codificação (supondo Windows-1252) e passa pelo `limpar()`. Ele padroniza as quebras de linha, desfaz as ligaduras tipográficas que os PDFs trazem ("Certificação", com o caractere único fi, vira "Certificação"), remove o hífen invisível, os espaços de largura zero e a marca BOM, troca os espaços finos por espaços comuns, apaga os caracteres de controle e os caracteres da área de uso privado do Unicode (U+E000 a U+F8FF), onde ficam os ícones de fontes como a Font Awesome usados nos modelos de currículo para desenhar telefone e e-mail. Depois junta espaços repetidos, tira os espaços em volta das quebras, deixa no máximo uma linha em branco seguida e corta em 100.000 caracteres. Por fim, os links que só existiam no hyperlink entram no texto: se o currículo tem a frase "Meu LinkedIn" ligada a <https://linkedin.com/in/ana> e esse endereço não aparece escrito, o texto termina com "Links: <https://linkedin.com/in/ana>". Assim o endereço fica gravado junto com o currículo e é encontrado pela extração.

Foto do currículo. O `extrairFoto()` junta as imagens candidatas (no DOCX, os JPG e PNG da pasta word/media; no PDF, as imagens devolvidas por `PdfTexto::imagens()`) e aplica filtros eliminatórios: tamanho do arquivo entre 2 KB e 12 MB, tipo JPEG ou PNG conferido pelo conteúdo, cada lado entre 80 e 6000 pixels e proporção entre 0,5 e 1,7, o que tira faixas, banners e colunas decorativas. Também descartamos a página A4 escaneada inteira, reconhecida pela proporção entre 0,66 e 0,76 (a da folha A4 é cerca de 0,707) com o lado maior de 1200 pixels ou mais. As que sobram recebem a nota de `pontuacaoFoto()` e vence a maior.

A nota começa por dois fatores: forma vale 1,0 para proporção de retrato 3x4 ou quadrada (entre 0,65 e 1,05) e 0,6 para as outras; tamanho vale 1,0 quando o lado menor tem pelo menos 150 pixels e o maior no máximo 2500, e 0,7 nos outros casos. Sem a extensão GD, a nota é só forma vezes tamanho. Com a GD, a imagem é reduzida para 64 pixels de largura e cada pixel é analisado: contamos os pixels com tom de pele pela regra RGB clássica de detecção de pele (vermelho acima de 95, verde acima de 40, azul acima de 20, diferença entre o maior e o menor canal acima de 15, diferença entre vermelho e verde acima de 15 e vermelho maior que os outros dois), a variedade de tons (cada canal reduzido a 16 níveis, o que dá até 4096 tons), os níveis de cinza e os pixels coloridos. As decisões são estas: com menos de 5% de pixels coloridos, a imagem é tratada como foto em preto e branco e vale 0,8 vezes forma vezes tamanho, a não ser que tenha menos de 30 níveis de cinza (desenho ou texto), quando vale 0; com menos de 40 tons diferentes, é logotipo, ícone ou fundo liso e vale 0; com menos de 2% de pele, é uma foto sem pessoa e vale só 0,15 vezes forma vezes tamanho (serve apenas se não houver outra). Nos demais casos, a nota é $(1 + 2 \times \text{notaPele} + \text{mínimo entre 1 e variedade}/300) \times$

forma x tamanho, em que notaPele chega a 1 com 8% de pele e cai para 0,3 quando a pele passa de 85% da imagem, porque aí costuma ser textura ou fundo cor de pele. A nota máxima, portanto, é 4. O teste automático `tests/smoke.php` confirma o comportamento: um retrato sintético de 120 x 160 pixels com um rosto oval ganha nota acima de 1, e um logotipo azul sobre fundo branco ganha 0.

A foto escolhida passa pelo método privado `fotoLeve()`: se já é um JPEG de até 800 pixels, fica como está; senão é redimensionada para no máximo 800 pixels no lado maior, sobre fundo branco (para PNG com transparência), e salva em JPEG com qualidade 86. Quem grava o arquivo é a `AplicacaoCurriculo::salvarFoto()`, descrita na subseção A.9.6. O controlador só procura a foto quando conseguiu ler o texto do currículo.

A.9.2 PdfTexto (`app/Services/Extracao/PdfTexto.php`)

O PdfTexto é o leitor de PDF que escrevemos em PHP puro. Ele é criado com os bytes do arquivo (`new PdfTexto(file_get_contents($caminho))`) e, no construtor, já indexa todos os objetos do PDF. Depois disso, oferece o texto da página na ordem de leitura, os links, as imagens candidatas a foto e a capa. Três classes o usam: o `LeitorDocumento`, para o texto, os destaques e os links do currículo; o `LeitorDocumento::extrairFoto()`, para as imagens; e a `CapaPdf`, para a capa dos e-books. A propriedade pública `$destaques` guarda as linhas escritas com a maior fonte da primeira página.

Escolhemos escrever o leitor em vez de depender de um programa externo porque o `pdftotext` não vem no Windows nem no XAMPP, e porque precisávamos de informações que a conversão comum perde: a posição de cada trecho e o tamanho da letra. Com elas, o leitor consegue separar as duas colunas dos modelos de currículo do Canva e do Word e reconhecer o nome do candidato pela maior fonte. O leitor foi pensado para extrair texto e imagens; ele não desenha a página.

Método	O que faz
<code>__construct(string \$raw)</code>	Recebe os bytes do PDF e chama <code>indexar()</code> , que separa todos os objetos e streams, inclusive os que estão dentro de object streams.
<code>texto(): string</code>	Interpreta o conteúdo de cada página na ordem da árvore de páginas e devolve o texto em linhas, na ordem de leitura (coluna por coluna quando há duas). Preenche <code>\$destaques</code> com a maior fonte da primeira página.
<code>links(): array</code>	Devolve as URLs das anotações de link (/URI) do arquivo, só http, https, mailto e www, sem repetição.
<code>imagensJpeg(): array</code>	Devolve os bytes das imagens JPEG (filtro DCTDecode) do arquivo, que no PDF ficam guardadas como um JPEG completo.
<code>imagens(int \$maxPixels = 3000000): array</code>	Devolve os JPEG e também as imagens compactadas com FlateDecode (RGB ou cinza, 8 bits) remontadas em PNG, sem as máscaras de transparência. É a lista de candidatas a foto do currículo.

Método	O que faz
<code>capa(int \$maxPixels = 25000000): ?string</code>	Devolve a maior imagem desenhada na primeira página (inclusive dentro de Form XObjects), em JPEG ou PNG, ou null quando a página não tem imagem aproveitável.

Quadro 88 – Métodos públicos do PdfTexto

Fonte: Autoria própria (2026).

Para explicar o leitor, vale lembrar como um PDF é organizado. Ele é um conjunto de objetos numerados ("12 0 obj ... endobj"). Os valores podem ser dicionários (<< /Chave valor >>), listas ([...]), nomes (/Font), textos entre parênteses ou em hexadecimal (<0041>), números e referências a outros objetos ("12 0 R"). Dados binários, como o conteúdo das páginas, as fontes e as imagens, ficam em streams, quase sempre compactados. As páginas formam uma árvore: o objeto /Catalog aponta para /Pages, que tem filhos (/Kids) até chegar a cada /Page, e cada página tem um /Contents (o fluxo de conteúdo, com os comandos de desenho) e um /Resources (as fontes e imagens que ela usa).

Objetos e object streams. O método privado `indexar()` procura no arquivo cada ocorrência de "número geração obj" e guarda o corpo até o "endobj". Quando o corpo tem um stream, separa o dicionário dos dados: usa o /Length quando ele é um número direto e cabe no arquivo; caso contrário, corta no último "endstream". Não dependemos da tabela xref (o índice de posições que fica no fim do arquivo), porque ela vem quebrada em muitos PDFs gerados por sites, e a varredura direta é mais tolerante. A partir da versão 1.5 do formato, vários objetos podem vir compactados juntos dentro de um stream do tipo /ObjStm (object stream). Para esses, o leitor descompacta o stream, lê no cabeçalho os pares "número do objeto e posição" (o dicionário informa quantos objetos há, em /N, e onde começam os dados, em /First) e recorta cada objeto, sem sobrescrever um objeto que já tenha sido encontrado solto no arquivo.

Leitura dos valores. O método privado `ler()` é um analisador recursivo que lê um valor a partir de uma posição: dicionário, lista, texto literal, texto hexadecimal, nome, referência, número ou operador. O texto literal trata os escapes do formato (\n, \t, \) e os códigos octais como \351) e os parênteses aninhados. A referência "12 0 R" é testada antes do número, para não ser lida como dois números soltos. No PHP, os textos ficam como ['_str' => ...], as referências como ['_ref' => 12] e os operadores como ['_op' => 'Tj']. O método `resolver()` segue as referências até o valor final, com no máximo 10 saltos, para não entrar em laço num arquivo malformatado.

FlateDecode e orçamento de leitura. O método privado `decodificar()` devolve o conteúdo de um stream. Se ele usa o filtro FlateDecode (a compressão zlib, a mais comum), o método `inflar()` tenta três formas, porque os geradores de PDF variam: `gzuncompress()` para o formato

zlib com cabeçalho, `gzinflate()` para o deflate puro e `gzinflate()` pulando os 2 bytes do cabeçalho, para streams com cabeçalho defeituoso. Cada tentativa recebe como tamanho máximo o que resta do orçamento mais um byte; o orçamento começa em 30 MB (o mesmo `LeitorDocumento::LIMITE_DESCOMPACTADO`) e todo stream lido é descontado dele, mesmo quando o mesmo stream é pedido mais de uma vez. Se o zlib avisa "insufficient memory" (a saída passaria do teto) ou o orçamento fica negativo, o leitor lança uma `LengthException` e a leitura para. Quando o dicionário pede um preditor PNG (`/DecodeParms` com `/Predictor 10` ou mais, comum nos object streams), o método `despredizer()` desfaz os filtros por linha (nenhum, Sub, Up e Average) usando a largura informada em `/Columns`. Streams de imagem (`DCTDecode`, `JPXDecode`, `CCITTFaxDecode` e `JBIG2Decode`) não interessam ao texto e voltam vazios. Além do orçamento em bytes, contamos os trechos de texto: um currículo tem poucos milhares e, acima de `MAX_TRECHOS = 200.000`, a leitura também é interrompida.

Fontes, `ToUnicode` e `Differences`. Num PDF, os bytes de um texto não são letras: são códigos de glifos da fonte usada naquele trecho. Para transformar códigos em letras, o método privado `infoFonte()` monta, uma vez por fonte (há um cache), as informações necessárias. Se a fonte tem um mapa `/ToUnicode`, o método `lerCMap()` lê esse mapa: o bloco `codespacerange` diz quantos bytes tem cada código (1 ou 2), o bloco `bfchar` liga um código a um texto Unicode e o bloco `bfrange` liga uma faixa inteira de códigos, seja a uma lista de destinos, seja a um valor inicial que vai sendo incrementado. Faixas com mais de 5000 códigos são ignoradas, para um mapa malicioso não ocupar a memória. Nas fontes compostas (`/Type0`, comuns no Canva e no Word), cada código tem 2 bytes e as larguras vêm da fonte descendente, no array `/W`, que aceita dois formatos ("`c [w1 w2 ...]`" e "`c1 c2 w`"), com `/DW` como largura padrão. Nas fontes simples, as larguras vêm de `/FirstChar` e `/Widths`, com `/MissingWidth` na falta; as fontes padrão sem tabela de larguras, como a Helvetica, usam a média aproximada de 520 milésimos. Se a fonte redefine códigos em `/Encoding` `/Differences`, cada código recebe a letra do nome do glifo: por exemplo, "`[ 225 /acute /acircumflex ]`" quer dizer que o código 225 é "á" e o 226 é "â". A constante `GLIFOS` traz os nomes da Adobe Glyph List usados em português (acentos, cedilha, aspas, travessões, números) e nomes no formato `uniXXXX` também são aceitos.

O método privado `decodificarTexto()` aplica essas informações: para cada código, soma a largura, conta os glifos e os espaços e escolhe o texto pela ordem de prioridade `ToUnicode`, depois `Differences` e, por último, a tabela `Windows-1252`. Um exemplo comum em currículos do Canva é o nome "Ana" escrito como `<0024004E0044>` numa fonte composta: o mapa `ToUnicode` diz que 0024 é "A", 004E é "n" e 0044 é "a".

Interpretação do conteúdo e matriz de transformação. O método privado interpretar() percorre o fluxo de conteúdo da página. Os operandos vão para uma pilha até aparecer um operador, que os consome. O leitor mantém o estado gráfico: a matriz de transformação atual (CTM), a fonte e seu tamanho, o espaçamento entre caracteres (Tc), o espaçamento entre palavras (Tw), a escala horizontal (Tz), a distância entre linhas (TL) e o deslocamento vertical (Ts). Os operadores tratados são: q e Q, que guardam e restauram o estado numa pilha; cm, que multiplica a CTM; BT, que começa um bloco de texto; Tf, que escolhe a fonte; Td, TD, Tm e T*, que posicionam a linha; Tj, aspas simples e aspas duplas, que desenham um texto; e TJ, que desenha uma lista de textos intercalada com números de ajuste fino (o kerning, em milésimos do tamanho da fonte). O operador Do, quando aponta para um Form XObject (um bloco de conteúdo reaproveitável), faz o leitor interpretar o conteúdo desse bloco com os recursos dele e com a matriz /Matrix dele multiplicada pela CTM, até 6 níveis de profundidade. Imagens embutidas no meio do conteúdo (entre BI e EI) são puladas.

As matrizes do PDF têm seis números [a b c d e f] e representam transformações do plano (escala, rotação e translação); o método mul() calcula o produto de duas delas. A posição de cada trecho é obtida multiplicando o deslocamento vertical, a matriz de texto (Tm) e a CTM: os dois últimos números do resultado são o x e o y do início do trecho na página. Depois de desenhar, a matriz de texto avança pela largura do trecho, calculada como ((soma das larguras / 1000) x tamanho da fonte + Tc x glifos + Tw x espaços) x escala horizontal, e assim temos também o x do fim (x1). O tamanho real da letra é o tamanho da fonte multiplicado pelo comprimento do vetor vertical da matriz (hypot(c, d)), porque muitos geradores escrevem "l Tf" e fazem a escala pela matriz. Cada trecho vira um registro com x, y, x1, tamanho e texto. Por exemplo, "BT /F1 12 Tf 72 700 Td (Experiência) Tj ET" gera o trecho "Experiência" em x = 72, y = 700 e tamanho 12.

Recursos por página. As fontes e imagens de uma página ficam em /Resources, mas o PDF permite que esse dicionário seja herdado do nó pai na árvore de páginas. O método privado recursosDaPagina() sobe pelos /Parent (até 20 níveis) até achar os recursos. O método paginas() devolve os números dos objetos de página na ordem da árvore, começando no /Catalog e descendo pelos /Kids (até 30 níveis); se a árvore não for reconhecida, usa todos os objetos /Type /Page em ordem numérica, e se nem isso existir, texto() interpreta todos os streams do arquivo. As páginas são separadas por uma linha em branco.

Ordem de leitura em duas colunas. Com os trechos posicionados, o método privado montarPagina() remove os repetidos no mesmo lugar (alguns geradores simulam negrito desenhando o mesmo texto duas vezes) e chama segmentos(). Esse método ordena os trechos

de cima para baixo e da esquerda para a direita, junta numa mesma linha os que têm altura parecida (diferença de até 0,45 vez o tamanho da letra) e, dentro da linha, junta em segmentos os trechos próximos (espaço de até 1,6 vez o tamanho da letra), colocando um espaço entre eles quando a distância passa de 0,18 vez o tamanho. Isso é necessário porque muitos PDFs não têm o caractere de espaço: o espaço é só a distância entre os trechos. Trechos distantes na mesma altura ficam em segmentos separados.

Em seguida, o método `corredor()` procura um corredor vertical sem texto que separe duas colunas. Ele percorre a largura ocupada pelo texto, de 12% a 88%, em passos de 2 pontos, e conta quantos segmentos atravessam cada posição. A melhor posição é a de menor cruzamento, que precisa ser no máximo 20% dos segmentos; dentro dela, escolhe a faixa contínua mais larga (pelo menos 4 pontos) e corta no meio. O corte só é aceito se os dois lados tiverem pelo menos 3 segmentos, pelo menos 12% dos caracteres, média de pelo menos 8 caracteres por segmento (para não confundir com uma coluna de datas alinhadas à direita) e se estiverem de fato lado a lado, com sobreposição vertical de pelo menos 40% da coluna menor. Com o corte definido, o método `ordenar()` lê a página em faixas: um segmento que atravessa o corredor, como um cabeçalho de largura total com o nome, sai sozinho e fecha a faixa; dentro de cada faixa, sai a coluna da esquerda inteira e depois a da direita, a não ser que a da direita tenha letra mais de 1,3 vez maior (é nela que está o nome em destaque), quando a ordem se inverte. O método é recursivo por mais um nível, para colunas que se dividem de novo.

Por fim, o método `linhasDe()` transforma os segmentos de uma coluna em linhas de texto: de cima para baixo, segmentos na mesma altura viram linhas separadas, exceto o par "Rótulo:" e valor, que fica na mesma linha ("Telefone: (61) 99999-0000"); um espaço vertical maior que 2,2 vezes o tamanho da letra vira uma linha em branco, separando blocos; e um item de lista quebrado em duas linhas, cuja primeira parte termina num conector ("Banco de Dados –", "Técnico em", "Curso de"), é juntado com o segmento logo abaixo na mesma posição horizontal. Um exemplo mostra o ganho: num modelo do Canva com barra lateral à esquerda (Contato, Habilidades, Idiomas) e coluna principal à direita (Resumo, Experiências), uma leitura linha a linha misturaria "Contato Resumo" e "(61) 99999-0000 Profissional com 3 anos de experiência"; com o corredor, o leitor entrega a barra lateral inteira e depois a coluna principal, e cada título de seção fica junto do seu conteúdo.

O método `calcularDestakes()` trabalha só na primeira página: se a maior fonte for pelo menos 1,25 vez maior que a mediana dos tamanhos, devolve até 3 segmentos com tamanho a partir de 97% do maior, na ordem da página. É assim que a `ExtracaoCurriculo` recebe o nome do candidato quando ele está escrito em letras grandes no topo. O método `links()` procura /URI

nos dicionários das anotações, aceita o endereço em texto literal ou hexadecimal, converte para UTF-8 quando vem em UTF-16 (marca FE FF) e só aceita http, https, mailto e www, o que deixa de fora endereços perigosos como javascript:.

Extração de imagens. No PDF, uma foto em JPEG fica guardada como um arquivo JPEG completo dentro do stream (filtro DCTDecode), e o `imagensJpeg()` só precisa conferir que os bytes começam pela assinatura FF D8; JPEG compactado de novo com `FlateDecode` é ignorado. Mas o Word e o Canva costumam gravar uma foto que era PNG como pixels crus compactados com `FlateDecode`, e por isso o `imagens()` também remonta essas imagens. O método privado `flateParaPng()` confere a largura e a altura (pelo menos 80 pixels e no máximo \$maxPixels, 3 milhões por padrão), 8 bits por componente e o espaço de cor (o `componentesCor()` aceita `DeviceRGB` e `CalRGB`, com 3 canais, `DeviceGray` e `CalGray`, com 1, e `ICCBased` com 1 ou 3 canais; CMYK e cores indexadas ficam de fora). Depois descompacta dentro do orçamento, desfaz os filtros PNG por linha (o `desfiltrarPng()` trata os cinco: nenhum, Sub, Up, Average e Paeth) e o `montarPng()` escreve um arquivo PNG do zero: a assinatura de 8 bytes, o bloco IHDR (largura, altura, 8 bits e o tipo de cor 2 para RGB ou 0 para cinza), o bloco IDAT com as linhas compactadas por `gzcompress()` e o bloco IEND, cada um com seu tamanho e seu CRC32. As máscaras de transparência (/SMask) não são fotos e ficam de fora. Tudo isso sem biblioteca de imagem.

Capa da primeira página. O método `capa()` pega a primeira página e coleta as imagens dos seus recursos, inclusive as que estão dentro de Form XObjects (até 3 níveis), com a área de cada uma (/Width x /Height). Em ordem da maior para a menor, devolve a primeira que consegue: um JPEG como está, desde que não seja CMYK (4 canais, comum em material de gráfica, que sairia com as cores trocadas), ou uma imagem `FlateDecode` remontada em PNG, com limite de 25 milhões de pixels. Se a primeira página só tem texto e desenhos vetoriais, devolve null. A limitação é que o texto desenhado por cima da imagem não aparece, porque o leitor não desenha a página; por isso a `CapaPdf` tenta antes o leitor de PDF do Windows e usa o `capa()` como reserva.

A.9.3 DocxTexto (app/Services/Extracao/DocxTexto.php)

O arquivo DOCX do Word é, na verdade, um arquivo ZIP com vários XML dentro, no formato WordprocessingML: `word/document.xml` tem o corpo do documento, `word/_rels/document.xml.rels` tem as relações (para onde aponta cada hyperlink, quais são os cabeçalhos e rodapés), `word/header1.xml` e `word/footer1.xml` têm o cabeçalho e o rodapé, e a pasta `word/media` guarda as imagens. A abertura do ZIP fica no `LeitorDocumento` (pelo

ZipArchive ou pelo leitor de ZIP em PHP puro, descrito na subseção A.9.1, usado quando a extensão zip está desligada). O DocxTexto recebe o XML de uma parte e as relações dela e devolve o texto. O LeitorDocumento::docx() cria uma única instância e a usa para o corpo, os cabeçalhos e os rodapés, de modo que os destaques e os links vão se acumulando nas propriedades públicas.

Método ou propriedade	O que faz
converter(string \$xml, array \$rels): string	Converte o XML de uma parte do documento em texto, uma linha por parágrafo ou linha de tabela, guardando antes as relações para resolver os hyperlinks. É o único método público da classe.
\$destaques (propriedade)	Até 3 parágrafos escritos com a maior fonte do documento, quando ela se destaca do texto comum (normalmente o nome do candidato).
\$links (propriedade)	Endereços externos dos hyperlinks encontrados (http, https, mailto e www), lidos das relações ou dos campos HYPERLINK.

Quadro 89 – Métodos e propriedades públicas do DocxTexto
Fonte: Autoria própria (2026).

Leitura com DOM e segurança do XML. O converter() carrega o XML com a classe DOMDocument do PHP e trabalha com os namespaces do Word (o do texto, <http://schemas.openxmlformats.org/wordprocessingml/2006/main>, e o das relações). A opção LIBXML_NONET proíbe a biblioteca de XML de acessar a rede durante a leitura; como o arquivo vem do usuário, isso impede que um XML malicioso faça o servidor buscar entidades externas (o ataque conhecido como XXE). A opção LIBXML_COMPACT economiza memória. Se o XML estiver quebrado e o DOM não conseguir carregar, usamos uma extração simples: trocamos as marcas de tabulação, de quebra e de fim de parágrafo por tabulação e quebras de linha, retiramos as demais marcas e decodificamos as entidades.

Antes de carregar, o converter() remove por expressão regular os blocos mc:Fallback. O Word grava cada caixa de texto duas vezes: no formato moderno (dentro de mc:Choice) e no formato antigo VML (dentro de mc:Fallback), para programas mais velhos. Sem essa remoção, o texto da barra lateral dos modelos de currículo, que costuma ficar em caixas de texto, apareceria duplicado. Depois, o método privado blocos() percorre o corpo: w:p é parágrafo, w:tbl é tabela, e os contêineres w:sdt (controles de conteúdo), w:sdtContent, w:customXml e w:txbxContent (conteúdo de caixa de texto) são percorridos recursivamente, assim como elementos de outros namespaces.

Parágrafos. O método privado paragrafo() lê primeiro as caixas de texto de primeiro nível que estão dentro do parágrafo (o ancestral() confere o aninhamento, para uma caixa interna não ser lida duas vezes) e depois o texto do próprio parágrafo, pelo textoRuns(). Esse método junta o conteúdo de w:t; transforma w:tab e w:ptab em tabulação, w:br e w:cr em quebra

de linha, `w:noBreakHyphen` em hífen e `w:sym` (ícone de fonte de símbolos) em espaço; lê `w:sz`, o tamanho da fonte em meios pontos (28 quer dizer 14 pontos), para guardar o maior tamanho do parágrafo; e registra os hyperlinks, tanto pelo atributo `r:id` resolvido nas relações quanto pelos campos `HYPERLINK "endereço"` (`w:instrText`). O texto apagado no controle de alterações (`w:delText`) é ignorado, porque traria de volta o que o candidato já tinha apagado, e as propriedades do parágrafo (`w:pPr`) também, porque não são texto. Parágrafos com o estilo `Title` ou `Título` contam como fonte 40, já que nesses o tamanho vem do estilo e não do trecho. Parágrafos de lista numerada ou com marcadores (`w:numPr`) ganham o prefixo "`•`", para a `ExtracaoCurriculo` reconhecer os itens.

Tabelas. Muitos modelos de currículo usam tabelas para montar o layout. O método privado `tabela()` lê cada linha e suas células. Quando a linha tem duas ou mais células e todas são curtas (uma linha só, com até 120 caracteres), ela vira uma linha de texto: com duas células em que a primeira é um rótulo curto (até 30 caracteres, sem números nem `@`), sai "`Rótulo: valor`", e com três ou mais sai "`A | B | C`". Por exemplo, a linha de tabela "`Telefone`" e "`(61) 99999-0000`" vira "`Telefone: (61) 99999-0000`", que a `ExtracaoCurriculo` reconhece como linha de dados. Quando as células são longas (uma tabela de layout com duas colunas de conteúdo), as linhas de cada célula saem uma depois da outra, coluna por coluna, sem misturar.

Destaques. Ao terminar, o método privado `calcularDestaques()` compara os tamanhos de fonte dos parágrafos. Se o maior tamanho for menor que 16 pontos e também menor que 1,3 vez a mediana, não há destaque. Caso contrário, os parágrafos com pelo menos 98% do maior tamanho entram em `$destaques`, até 3. Num currículo com o nome em 26 pontos e o texto em 11 pontos, o destaque é o nome.

A.9.4 `OcrImagem` (`app/Services/Extracao/OcrImagem.php`)

A `OcrImagem` faz a leitura de texto em imagens (OCR, reconhecimento óptico de caracteres) para o cartaz da vaga que a empresa envia no painel. O motor de leitura é o Tesseract, um programa de código aberto, e a leitura pode acontecer em dois lugares. O padrão é o navegador de quem envia o cartaz: o script `public/assets/js/leitor-cartaz.js` usa o `Tesseract.js`, uma versão do Tesseract compilada para `WebAssembly`, com todos os arquivos servidos pelo próprio site em `public/assets/js/vendor/tesseract` (o worker, o núcleo em `WebAssembly` e o modelo de português `por.traineddata.gz`). A reserva é o Tesseract instalado no servidor, se houver. Nos dois casos são feitas as mesmas quatro leituras e o resultado é montado no servidor com a mesma calibragem, pelo `montar()`.

O fluxo começa no `EmpresaController`, na ação `ler_cartaz`: a imagem é gravada por `salvar_imagem_enviada()` (JPG, PNG ou WEBP de até 8 MB) e o controlador chama `ExtracaoVaga::doImagem()` com as leituras recebidas do formulário, já conferidas por `OcrImagem::leiturasDoNavegador($_POST['ocr_tsv'])`. O `doImagem()` usa o `montar()` quando as leituras do navegador são válidas, o `ler()` quando elas não vieram e o servidor tem o Tesseract (`disponivel()`), e devolve o formulário vazio, com o aviso de preencher à mão, quando não há leitor nenhum. Com confiança abaixo de 75%, o relatório avisa para conferir cada campo. Escolhemos o navegador como padrão porque o XAMPP da escola não tem o Tesseract e instalar programas no servidor seria uma barreira; assim o trabalho pesado fica no computador de quem envia, o servidor recebe só texto e a imagem não vai para nenhum serviço de terceiros.

Método	O que faz
<code>comando(): ?string</code>	Devolve o caminho do executável do Tesseract no servidor (ou null), procurando nas pastas de instalação comuns do Windows, inclusive a instalação só para o usuário, e do Linux. O resultado fica em cache.
<code>disponivel(): bool</code>	Diz se o servidor pode ler cartazes sozinho: precisa do Tesseract e da extensão GD.
<code>ler(string \$path): array</code>	Faz as quatro leituras no servidor e monta o resultado (texto, complemento, destaques, todas as linhas e confiança); sem leitura possível, devolve o resultado vazio.
<code>leituras(string \$path): ?array</code>	Prepara a imagem e roda o Tesseract quatro vezes (normal e negativo, modos 3 e 11), em paralelo quando possível, devolvendo os quatro textos TSV; null se não foi possível ler.
<code>leiturasDoNavegador(mixed \$tsvs): ?array</code>	Valida as leituras que chegaram do navegador: de 1 a 4 textos UTF-8 de até 1,5 MB, sendo o primeiro no formato TSV do Tesseract. Devolve os quatro textos ou null.
<code>montar(string \$tsv3, string \$tsv11, string \$tsvNeg3 = "", string \$tsvNeg11 = ""): array</code>	Junta as leituras em linhas, descarta as de baixa confiança e monta o texto na ordem do cartaz, o complemento, os destaques, todas as linhas boas e a confiança média.

Quadro 90 – Métodos públicos da `OcrImagem`
Fonte: Autoria própria (2026).

Preparação da imagem. O método privado `preparar()` (e a função `preparar` do leitor-`cartaz.js`, que faz o mesmo no navegador) ajusta a imagem para o Tesseract ler melhor. Antes de abrir a imagem, as dimensões são lidas só do cabeçalho com `getimagesize()`: um PNG de poucos kilobytes pode declarar 50.000 x 50.000 pixels e, como na GD cada pixel ocupa 4 bytes, esgotaria a memória ao ser aberto. Por isso a imagem precisa ter entre 20 pixels de lado e 40 milhões de pixels no total (`MAX_PIXELS`). Depois a imagem ganha fundo branco (para PNG transparente), vira tons de cinza e recebe correção de gama 2,2, que escurece os tons médios: letra amarela ou laranja sobre fundo claro, comum em cartazes, deixa de ficar quase branca no cinza. Em seguida é ampliada para cerca de 2200 pixels de largura, porque cartazes de rede social chegam com uns 1080 pixels e letra pequena; o fator fica entre 1 e 3 e também é limitado

para o resultado não passar de 40 milhões de pixels. A imagem é salva num PNG temporário com nome aleatório (`random_bytes`) e sem compressão, já que será apagada. Da mesma imagem ampliada sai o negativo, com contraste ajustado: letra branca sobre faixa verde ou azul vira letra escura sobre fundo claro, que é o que o Tesseract lê bem.

As quatro leituras. Cada imagem (normal e negativo) é lida em dois modos de segmentação do Tesseract: o modo 3 (página automática), que mantém as linhas na ordem, como "Salário: R\$ 1.900,00", e o modo 11 (texto esperso), que acha textos soltos e letreiros decorados. A saída é pedida em TSV, uma tabela em que cada palavra vem com posição, altura e confiança. No servidor, o `idioma()` usa o português (por) quando ele está instalado, consultando `tesseract --list-langs`, e o inglês nos outros casos. O `tsvParalelo()` abre os quatro processos ao mesmo tempo com `proc_open()`, cerca de quatro vezes mais rápido, cada um limitado a uma thread (variável `OMP_THREAD_LIMIT=1`) para não disputarem o processador e gravando o resultado num arquivo temporário, o que evita travar com o canal de saída cheio. O limite é de 90 segundos; o que não terminou é encerrado. Se o modo paralelo falhar, as leituras são feitas uma por vez pelo `tsv()`, e se as duas leituras da imagem normal vierem vazias (por exemplo, arquivo ainda preso pelo antivírus), tentamos mais uma vez. Os arquivos temporários são apagados num bloco `finally`, aconteça o que acontecer.

No navegador, o `leitor-cartaz.js` carrega o `Tesseract.js` assim que a caixa de extração é aberta, com um ou dois workers (dois quando o computador tem pelo menos 4 núcleos e 4 GB de memória), cada um fixo num modo de leitura. Ao escolher o arquivo, ele prepara as duas imagens num canvas (cinza pela luminância $0,299R + 0,587G + 0,114B$, tabela de gama, ampliação suave e negativo com contraste), faz as quatro leituras, acrescenta o cabeçalho do TSV e envia os quatro textos em campos ocultos `ocr_tsv[]` junto com o cartaz. Um carregador amarelo mostra a porcentagem enquanto carrega e lê, e fica azul quando está pronto. Se algo falhar no navegador (falta de memória, imagem estranha, navegador sem WebAssembly), o cartaz é enviado sem as leituras e o servidor tenta ler.

Validação das leituras do navegador. O que chega do formulário é dado do usuário e não pode ser tratado como confiável. O `leiturasDoNavegador()` só aceita uma lista de 1 a 4 textos, cada um com até 1,5 MB (`MAX_TSV`; um cartaz cheio de texto gera uns 100 KB) e em UTF-8 válido; padroniza as quebras de linha e exige que o primeiro texto tenha cara de leitura do Tesseract, com linhas que começam por um dígito seguido de 10 colunas numéricas separadas por tabulação. Um texto comum, como "Vaga: Vendedor" seguido de "Salário: R\$ 2.000", é recusado, e as listas faltantes são completadas com textos vazios. Mesmo que alguém forje uma leitura, o efeito é o mesmo de colar o texto do anúncio na caixa de extração: o

montar() só aproveita linhas no formato do Tesseract e a empresa revisa o formulário antes de publicar. O `tests/smoke.php` confere esses casos com leituras reais guardadas em `tests/amostras`.

Montagem do resultado. O método privado `linhas()` lê o TSV, fica só com as palavras (nível 5 da tabela) e as agrupa por bloco, parágrafo e linha. Nas pontas de cada linha, palavras com confiança abaixo de 40 são retiradas, porque costumam ser ícones ou pedaços de foto. Cada linha guarda o texto, a confiança média, a altura mediana das letras, a posição do topo e a lista de confianças. O método `boa()` decide se a linha é aproveitável: confiança média de pelo menos 55 (`CONFIANCA_LINHA`) na leitura estruturada e de pelo menos 62 nas leituras esparsas e do negativo, que trazem mais ruído; pelo menos uma palavra com 3 ou mais letras ou dígitos, ou uma sigla típica de vaga (`VT`, `VR`, `VA`, `PLR`, `CLT`, `PJ`); e pelo menos 60% dos caracteres, sem contar espaços, sendo letras ou dígitos, porque muitos símbolos soltos indicam ruído de fundo.

O `montar()` devolve cinco informações. O texto são as linhas boas da leitura estruturada da imagem normal, na ordem do cartaz. O complemento são as linhas das outras três leituras que não estão contidas nesse texto, comparadas sem acento e sem diferença de maiúsculas pelo `Competencias::normalizar()`, como títulos decorados e valores soltos. Os destaques são até 10 linhas com pelo menos 4 letras, ordenadas pela altura das letras, da maior para a menor, sem repetição; em geral a primeira é o título do cartaz, como "AUXILIAR DE COZINHA". Todas são as linhas boas de todas as leituras, com as repetições, porque o que se repete no cartaz (o nome da empresa no topo e no rodapé, por exemplo) é uma pista forte. A confiança é a média das confianças das palavras aproveitadas, de 0 a 100. A `ExtracaoVaga` usa esse resultado para preencher os campos da vaga.

A.9.5 `ExtracaoCurriculo` (`app/Services/Extracao/ExtracaoCurriculo.php`)

A `ExtracaoCurriculo` é a máquina de extração do currículo. Ela recebe o texto devolvido pelo `LeitorDocumento`, junto com as linhas em destaque, e devolve um array com 23 campos, sempre presentes e vazios quando não encontrados (a constante `CAMPOS`): `nome`, `email`, `telefone`, `data_nascimento`, `idade`, `titulo_profissional`, `bio`, `objetivo`, `experiencias`, `formacao`, `curso`, `habilidades`, `competencias`, `idiomas`, `cidade`, `uf`, `nivel_experiencia`, `informacoes_adicionais`, `links`, `cnh`, `disponibilidade`, `pretensao_salarial` e `pcd`. A regra principal é não inventar informação: a classe só organiza o texto que existe no arquivo. Ela funciona só com regras escritas no código. Diferente das extrações de vaga e de curso, o currículo não usa os padrões automáticos da `PadroesExtracao`, porque tem dados pessoais do candidato (LGPD) e não deve alimentar estatísticas de outros cadastros.

A classe é chamada pelo `CurriculoController::upload()` e conversa com três vizinhas: o `LeitorDocumento` (leitura do arquivo, `limpar()` e `chaveLink()`), a `Competencias` (o dicionário de competências e o `normalizar()`, que tira acentos, deixa em minúsculas e troca símbolos por espaço, usado em todas as comparações) e o `Portfolio` (que reconhece períodos, experiências e formação e os escreve no formato padronizado). A `ExtracaoVaga` também reaproveita o método público `local()` para achar a cidade de uma vaga.

Método	O que faz
<code>extrair(string \$path): string</code>	Lê o arquivo (PDF, DOCX ou DOC) e devolve o texto, chamando <code>LeitorDocumento::extrair()</code> .
<code>metodo(): string</code>	Devolve o nome do leitor usado na última extração (<code>LeitorDocumento::\$metodo</code>), mostrado no relatório.
<code>extrairCampos(string \$texto, array \$destaques = []): array</code>	Executa todas as etapas da extração e devolve os 23 campos da constante <code>CAMPOS</code> . Os destaques (linhas em fonte grande) ajudam a achar o nome.
<code>local(string \$t): array</code>	Devolve [cidade, UF] encontrados no texto, dando prioridade às regiões administrativas do DF e às cidades do entorno; devolve dois textos vazios quando não acha.
<code>juntarLista(array ...\$listas): string</code>	Une listas de itens sem repetir, comparando sem acento e sem diferença de maiúsculas, e devolve os itens separados por vírgula.

Quadro 91 – Métodos públicos da `ExtracaoCurriculo`
Fonte: Autoria própria (2026).

O método `extrairCampos()` trabalha em sete etapas, resumidas no quadro a seguir e explicadas nos parágrafos seguintes.

Etapas	O que acontece (métodos privados envolvidos)
1. Linhas e linhas de dados	O texto é limpo e dividido em linhas (<code>linhas()</code>); as linhas "Rótulo: valor" de dados pessoais são lidas e retiradas do fluxo (<code>linhaDeDados()</code>).
2. Nome e título	O nome sai do rótulo "Nome:", da maior fonte ou da primeira linha com cara de nome (<code>nome()</code> , <code>ehNome()</code>); a linha logo abaixo pode ser o título profissional (<code>ehTitulo()</code>).
3. Seções	O texto é dividido pelos títulos de seção, com ou sem acento, em maiúsculas, numerados ou espaçados (<code>tituloSecao()</code> , <code>separarSecoes()</code>).
4. Contato, local e datas	E-mail, telefone, cidade e UF, data de nascimento, idade e links (<code>email()</code> , <code>telefone()</code> , <code>localPreferido()</code> , <code>local()</code> , <code>nascimento()</code> , <code>idade()</code> , <code>links()</code>).
5. Texto das seções	Resumo, objetivo, experiências e formação padronizadas, cursos e idiomas (<code>paragrafo()</code> , <code>bioDoCabecalho()</code> , <code>itens()</code> , <code>idiomas()</code> , <code>idiomasSoltos()</code>).
6. Dados complementares	CNH, pretensão salarial, disponibilidade, PCD e informações adicionais (<code>cnh()</code> , <code>pretensao()</code> , <code>disponibilidade()</code> , <code>pcd()</code> , <code>informacoes()</code>).
7. Título, nível, habilidades e competências	Título alternativo, nível pelo tempo de experiência e as listas de habilidades e competências (<code>tituloAlternativo()</code> , <code>nivel()</code> , <code>juntarLista()</code>).

Quadro 92 – Etapas do `ExtracaoCurriculo::extrairCampos()`
Fonte: Autoria própria (2026).

Etapas 1: linhas e linhas de dados. O texto passa de novo pelo `LeitorDocumento::limpar()` e é dividido em linhas não vazias. Títulos espaçados letra por letra, um enfeite comum nos modelos ("E X P E R I Ê N C I A"), são reconhecidos e viram a palavra junta. Em seguida, cada

linha é testada pelo `linhaDeDados()`: primeiro o `semMarcador()` retira marcadores do começo da linha (pontos, traços, setas, sinais de visto, emojis e ícones de fontes), depois a linha é dividida em pares separados por " · ", " | " ou " • " (como em "LinkedIn: x · GitHub: y"), e cada par no formato "Rótulo: valor", com rótulo de 2 a 30 caracteres, é procurado na constante `ROTULOS`. Esse mapa liga rótulos normalizados aos campos nome, endereço, telefone, e-mail, nascimento, idade, CNH, pretensão, disponibilidade, PCD e link; aceita variações como "Celular", "WhatsApp", "Data de nascimento", "Habilitação" e "Expectativa salarial", e aceita rótulos duplos como "Telefone/WhatsApp:". A linha "PCD – descrição", sem dois-pontos, também é aceita. As linhas de dados reconhecidas saem do fluxo para não se misturarem às seções.

Uma decisão importante de LGPD está nesse mapa: os rótulos estado civil, CPF, RG, nacionalidade, naturalidade, filhos, sexo, gênero e religião levam ao campo "sensível", que é lido apenas para ser descartado. Esses dados não servem para o match com as vagas e poderiam abrir espaço para discriminação, então o sistema não os grava no perfil.

Etapas 2: nome e título. O `nome()` tenta, nesta ordem: o valor do rótulo "Nome:", quando tem cara de nome; os destaques de maior fonte, primeiro as duas primeiras linhas juntas (nome quebrado em duas linhas) e depois só a primeira, localizando no texto as linhas correspondentes para retirá-las; e por último a primeira linha com cara de nome entre as 10 primeiras, antes do primeiro título de seção, pulando linhas como "Currículo" ou "CV". Um nome em maiúsculas quebrado em duas linhas ("RAFAEL MENDES" e "OLIVEIRA") é juntado. O `ehNome()` exige de 2 a 7 palavras só com letras, espaços, ponto, apóstrofo ou hífen, sem @ e sem números, e que a linha não seja título de seção, cargo (palavras típicas como auxiliar, analista, técnico, desenvolvedor, atendente, jovem aprendiz) nem local. Depois do nome, até 3 linhas são examinadas: linhas de contato são puladas, e a primeira linha com cara de título profissional (`ehTitulo()`: de 3 a 80 caracteres, sem @, sem link, sem três dígitos seguidos, sem dois-pontos, sem terminar em ponto e sem ser local) vira o `titulo_profissional`. O `capitalizar()` só reescreve maiúsculas e minúsculas quando a linha está toda em maiúsculas ou toda em minúsculas: "MARIA DA SILVA" vira "Maria da Silva", com os conectores (da, de, do, e, em, com, para, na, no) em minúsculas e siglas conhecidas (TI, RH, UX, UI, PHP, SQL, DF, GO, PCD, CNH) em maiúsculas.

Etapas 3: seções e seus títulos. O `tituloSecao()` decide se uma linha é título de seção. Linhas com mais de 60 caracteres e sem dois-pontos são descartadas; a linha é dividida no primeiro dois-pontos, a numeração inicial ("1.", "II") é retirada e o resultado normalizado é comparado com a constante `SECOES`. Ela tem 15 seções, cada uma com suas variações de

título, e o quadro a seguir mostra exemplos. Quando a comparação exata falha, aceitamos também um começo conhecido, desde que a linha esteja toda em maiúsculas ou termine em dois-pontos, tenha até 45 caracteres, nenhum número e até 5 palavras: assim "EXPERIÊNCIAS NA ÁREA ADMINISTRATIVA" cai em experiências e "Formação acadêmica e técnica:" cai em formação. O conteúdo escrito na mesma linha do título ("Idiomas: Inglês intermediário") entra na seção. O `separarSecoes()` percorre as linhas: as que vêm antes do primeiro título formam o cabeçalho (onde costumam ficar o contato, o local e um resumo curto), e as demais vão para a seção atual. As linhas da seção de contato são somadas ao cabeçalho.

Seção	Exemplos de títulos (comparados sem acento e sem maiúsculas)
resumo	resumo, resumo profissional, perfil, sobre mim, apresentação, qualificações, quem sou eu
objetivo	objetivo, objetivo profissional, cargo pretendido, área de interesse, vaga pretendida
experiencias	experiência, experiência profissional, histórico profissional, trajetória profissional, estágios
formacao	formação, formação acadêmica, escolaridade, educação, graduação, formação técnica
cursos	cursos, cursos complementares, certificações, capacitações, treinamentos, formação complementar
habilidades	habilidades, conhecimentos, ferramentas, tecnologias, hard skills, informática, skills, stack
competencias	competências, competências comportamentais, soft skills, pontos fortes, qualidades
idiomas	idiomas, idioma, línguas, língua estrangeira
contato	contato, dados pessoais, informações de contato, endereço
links	links, redes sociais, portfólio, perfis
pcd	PCD, pessoa com deficiência, acessibilidade, laudo
disponibilidade	disponibilidade, disponibilidade de horário
pretensao	pretensão, pretensão salarial
referencias	referências, referências profissionais, referências pessoais
outros	informações adicionais, voluntariado, observações, hobbies, projetos, prêmios, diferenciais

Quadro 93 – Seções reconhecidas no currículo e exemplos de títulos aceitos
Fonte: Autoria própria (2026).

Etapa 4: contato, local e datas. O e-mail e o telefone são procurados primeiro no cabeçalho e nas linhas de dados e só depois no texto inteiro sem a seção de referências, para não pegar o e-mail ou o telefone de quem dá a referência. O `email()` pega o primeiro endereço e o deixa em minúsculas. O `telefone()` aceita +55 opcional, DDD entre 11 e 99 e separadores variados; prefere o primeiro celular (9 seguido de 4 dígitos) e, na falta dele, o primeiro fixo, sempre no formato "(61) 98765-4321". A expressão recusa números colados a outros dígitos ou a barras, para não confundir datas e documentos com telefone. Assim, "+55 61 9 8765-4321" vira "(61) 98765-4321".

O local é procurado pelo `localPreferido()` primeiro nas linhas de endereço e do cabeçalho que parecem local, e só depois no texto todo. O `local()` usa duas listas: mais de 35 regiões administrativas e bairros do DF (Ceilândia, Taguatinga, Samambaia, Águas Claras, Sol Nascente, Arniqueira, entre outras) e cidades do entorno e de Goiás (Valparaíso de Goiás, Águas Lindas de Goiás, Luziânia, Novo Gama, Formosa, Goiânia, entre outras), estas com UF GO. A região específica tem prioridade sobre "Brasília" e, entre várias, vence a que aparece primeiro; "Valparaíso" é completado para "Valparaíso de Goiás". Sem nenhuma delas, o método procura o padrão "Cidade - UF" com as 27 siglas. Por exemplo, "QNM 12, Ceilândia - Brasília/DF" resulta em Ceilândia, DF. O `ehLocal()`, usado em várias etapas para saber se uma linha é endereço, também reconhece palavras de endereço do DF como quadra, QNM, conjunto, lote, SQS, SHIS e setor.

O `nascimento()` reconhece "Nascimento: 15/03/2004", "nascida em 15/03/2004" e "D.N. 15/03/2004", confere se a data existe (`checkdate`) e se o ano está entre 1930 e 13 anos antes do ano atual, e grava no formato do banco, "2004-03-15". A `idade()` é calculada pela data de nascimento; sem ela, usa o rótulo "Idade:" ou a expressão "22 anos" no cabeçalho, entre 14 e 80. O `links()` procura endereços de sites profissionais conhecidos, mesmo escritos sem http (`linkedin.com`, `github.com`, `gitlab.com`, `behance.net`, `lattes.cnpq.br`, `wa.me`, `github.io`, `vercel.app` e outros), e qualquer endereço com http ou www; acrescenta `https://` quando falta, retira a pontuação final e elimina as repetições pela `chaveLink()`, ficando com a versão mais completa do endereço.

Etapas 5: texto das seções e experiências e formação padronizadas. O resumo vira o campo bio pelo `paragrafo()`, que junta as linhas quebradas pelo layout do PDF e só mantém a quebra depois de ponto, exclamação, interrogação, dois-pontos ou ponto e vírgula. Sem seção de resumo, o `bioDoCabecalho()` aproveita as linhas longas do cabeçalho (60 caracteres ou mais) que não são contato, desde que o parágrafo final tenha pelo menos 80 caracteres. O objetivo segue a mesma regra. A seção de experiências é entregue ao `Portfolio::experiencias()`, que reconhece os períodos ("03/2021 – Atual", "Jan/2020 a Dez/2022", "desde 2019"), a empresa, o cargo e os itens de descrição, e o `Portfolio::serializarExperiencias()` escreve o formato padronizado: um bloco por experiência, com empresa, cargo, período e os itens começando por "• ", separados por linha em branco. A formação passa pelo `Portfolio::formacao()` e pelo `Portfolio::serializarFormacao()`, que escreve uma linha por curso no formato "Curso – Instituição (situação)", como "Técnico em Informática – Escola Técnica de Ceilândia (Cursando)". Quando o Portfolio não reconhece nenhum item, o texto da seção é mantido como veio, para não perder informação. Os cursos viram um item por linha pelo `itens()`. Os idiomas

são padronizados pelo idiomas(): "Inglês – Avançado" e "Inglês: avançado" viram "Inglês (avançado)", com os níveis básico, intermediário, avançado, fluente, nativo, técnico, instrumental e A1 a C2, entre outros; sem seção de idiomas, o idiomasSoltos() procura inglês, espanhol, francês, alemão, italiano e Libras no texto todo.

Etapa 6: dados complementares. O cnh() procura "CNH", "Habilitação" ou "Carteira de motorista" seguidos da categoria e só aceita as categorias válidas (A, B, C, D, E, AB, AC, AD e AE). O pretensao() aceita um valor ("R\$ 2.500,00"), uma faixa ("R\$ 2.000 a R\$ 2.500") ou as expressões "a combinar", "negociável" e "conforme mercado". O disponibilidade() junta o valor do rótulo ou da seção com frases da seção de outras informações que falam de disponibilidade, viagens, mudança de cidade, horário integral, comercial ou flexível, home office e trabalho remoto ou híbrido, limitando o resultado a 100 caracteres, que é também o tamanho da coluna disponibilidade da tabela perfis. O pcd() usa o rótulo, a seção PCD ou, por último, uma linha que fale de PCD, laudo médico, deficiência auditiva, visual, física, intelectual ou motora, ou de um código CID. O informacoes() monta as informações adicionais com o PCD primeiro ("PCD: ...") e depois a seção de outras informações, sem repetir o que já tem campo próprio (CNH, disponibilidade, pretensão) e sem dados sensíveis (estado civil, solteiro, casado, CPF, RG, nacionalidade, naturalidade).

Etapa 7: título, nível, habilidades e competências. Se não houve título logo abaixo do nome, o tituloAlternativo() procura no objetivo frases como "atuar como X", "vaga de X" ou "na área de X" (cortando em "ou" e "e"), aceita um objetivo curto de uma linha só, sem ponto final, ou usa o cargo da experiência mais recente. O nivel() decide entre estagiário, junior, pleno e senior: o título com "sênior" ou "sr" já define sênior, "pleno" define pleno, e estagiário, estágio, jovem aprendiz ou trainee definem estagiário. Nos outros casos, somamos os meses de todas as experiências, juntando os períodos que se sobrepõem para não contar duas vezes (dois empregos ao mesmo tempo em 2022 contam uma vez só); sem períodos, vale a frase "N anos de experiência". Com 96 meses (8 anos) ou mais, o nível é sênior, a não ser que o cargo seja operacional ou de apoio (auxiliar, assistente, atendente, operador, motorista, caixa, porteiro e outros), que fica no máximo em pleno; com 42 meses (3 anos e meio) ou mais, pleno; com algum tempo, júnior; com experiências listadas sem datas, também júnior; e sem experiência nenhuma, estagiário.

As habilidades vêm da seção de habilidades pelo itens(), que separa por linha, vírgula, ponto e vírgula ou " • ", sem separar vírgulas dentro de parênteses ("Excel (tabelas, gráficos)" é um item só). O Competencias::extrair() procura no texto inteiro os termos do dicionário de competências, mas eles só completam a lista quando o candidato listou menos de 3 itens, e sem

repetir o que ele já escreveu de outro jeito (por exemplo, não acrescenta "Banco de dados SQL" se ele escreveu "MySQL"). A mesma regra vale para as competências comportamentais (Competencias::COMPORTAMENTAIS). O `juntarLista()` faz a união final, sem repetição. Respeitamos assim o que o candidato escreveu, e o match continua lendo o perfil e o currículo inteiros de qualquer forma.

A.9.6 AplicacaoCurriculo (app/Services/Extracao/AplicacaoCurriculo.php)

A `AplicacaoCurriculo` decide como os campos extraídos entram no perfil do candidato (tabela perfis) e monta os itens do relatório da extração. Do perfil, os dados seguem para o portfólio, o match, o Banco de Talentos e as candidaturas, que leem o perfil. A classe é usada pelo `CurriculoController` em dois momentos: no `upload()`, que chama `calcular()`, `salvarFoto()`, `salvar()` e `resumo()`; e no `aplicarExtracao()`, quando o candidato escolhe no relatório usar um dado do currículo, que chama `dtoDe()`, consulta CAMPOS e chama `salvar()`.

Método	O que faz
<code>calcular(array \$p, array \$c, bool \$substituir): array</code>	Compara, campo a campo, o perfil atual (\$p) com os campos extraídos (\$c) e devolve os dados a salvar, os itens do relatório (com o status de cada campo) e os valores pendentes.
<code>dtoDe(array \$p): array</code>	Monta o array de dados do perfil com todas as colunas gravadas pelo PerfilDAO, a partir do perfil atual, para salvar sem perder nada.
<code>salvar(array \$dados): bool</code>	Grava o perfil com <code>PerfilDAO::salvar(new PerfilDTO(\$dados))</code> .
<code>salvarFoto(array \$foto, int \$usuarioId): ?string</code>	Confere a foto encontrada no arquivo pelo conteúdo, grava em <code>storage/uploads</code> com nome aleatório e devolve o caminho lógico <code>"assets/uploads/..."</code> , ou null se ela for recusada.
<code>resumo(string \$v, int \$max = 140): string</code>	Gera o texto curto exibido no relatório: as quebras de linha viram " · " e o texto é cortado no tamanho máximo com reticências.

Quadro 94 – Métodos públicos da `AplicacaoCurriculo`
Fonte: Autoria própria (2026).

A constante CAMPOS liga 18 colunas da tabela perfis a um rótulo para o relatório, à chave correspondente da extração e a um tipo. O tipo texto vale para um valor único (título, resumo, objetivo, experiências, formação, cidade, UF, CNH e outros); o tipo lista vale para itens separados por vírgula (habilidades e competências); e o tipo linhas vale para itens um por linha (informações adicionais e links). Os nomes nem sempre coincidem: a coluna `cursos_complementares` recebe a chave `cursos` da extração.

O `calcular()` segue estas regras para cada campo, nesta ordem. Se o currículo não trouxe valor, o item fica com o status `nao_encontrado`, e o relatório mostra o que faltou. Se o campo é do tipo lista ou linhas, o perfil já tem valor e o candidato não marcou "substituir", os dois são mesclados sem repetição (listas pelo `ExtracaoCurriculo::juntarLista()`, linhas pelo `unirLinhas()`, sempre comparando sem acento e sem maiúsculas), com o status mesclado, ou igual se nada

mudou. Se o valor novo é igual ao atual, o status é igual. Se o campo do perfil está vazio, tem o valor padrão de perfil novo ou o candidato marcou "substituir", o valor do currículo é aplicado (status aplicado). Nos demais casos, o valor do candidato é mantido (status mantido) e o do currículo fica guardado em pendentes, para ele aplicar depois pelo relatório, pelo `aplicarExtracao()`. Os valores padrão existem porque a tabela perfis cria todo perfil com cidade "Brasília" e UF "DF"; por isso esses valores contam como vazios (a UF só quando o currículo trouxe outra), assim como o nível de experiência quando o perfil ainda não tem experiências.

Dois exemplos mostram as regras. Um perfil com as habilidades "Excel, Atendimento" que recebe um currículo com "Excel, Vendas, Power BI" fica com "Excel, Atendimento, Vendas, Power BI" (mesclado). Um perfil com o título "Vendedor" que recebe um currículo com o título "Assistente Administrativo" mantém "Vendedor" (mantido), e o relatório oferece o botão para trocar. O `dtoDe()` parte do perfil atual e copia todas as colunas gravadas pelo `PerfilDAO` (a constante privada `COLUNAS_DTO`, que inclui também os campos de empresa, a foto e a opção de perfil público), para que salvar o resultado da extração não apague nenhum campo que ela não tocou; ele também define `aceite_lgp` como 1, mantém o valor de `publico` (1 quando não existe) e usa `junior` como nível quando o perfil não tem um.

O `salvarFoto()` não confia na extensão: confere o tipo pelo conteúdo com a classe `finfo` (só `image/jpeg` ou `image/png`), recusa arquivos acima de 3 MB e exige que o PHP consiga ler as dimensões da imagem. O nome do arquivo é `foto_` seguido do id do usuário e de 10 caracteres aleatórios, gravado em `storage/uploads`; o banco guarda o caminho lógico `"assets/uploads/..."`. O controlador completa as regras que envolvem a conta do usuário: sem foto no perfil, a foto do currículo é aplicada; com foto, ela fica pendente para o candidato escolher. O nome da conta nunca é trocado automaticamente, só sugerido no relatório; o telefone vai para `usuarios.telefone` quando a conta ainda não tem (ou quando o candidato marcou "substituir"); e o e-mail encontrado é só informativo.

A.9.7 ImagemRemota (`app/Services/Extracao/ImagemRemota.php`)

A `ImagemRemota` entra em ação quando o Administrador importa cursos e e-books a partir das fichas da pesquisa, no painel Cursos e e-books. Cada ficha chega com até três links (Imagem, PDF e Link do conteúdo) e a classe abre cada um deles para descobrir o que realmente é e trazer para a plataforma a imagem e o PDF que servem, sem precisar que o Administrador confira endereço por endereço. É uma classe final, só com métodos estáticos, chamada pelo `AdminController` na prévia da importação e no cadastro. Como aqui não há dado pessoal de candidato envolvido, e sim conteúdo público de divulgação de cursos e e-books, a preocupação

central não é a LGPD, e sim a segurança do download: o sistema está buscando arquivos escolhidos por quem pesquisou os cursos, em servidores que não controlamos, então cada download tem limite de tamanho, de tempo e de destino.

O método `completar()` é o coração da classe e organiza a prévia em cinco passos. No primeiro, confere se o link do campo Imagem de cada ficha abre de verdade uma imagem (`validas()`); quando o item já tinha um caminho de imagem do próprio site (`imagem_propria`, por exemplo um arquivo em `assets/img/padrao`), esse caminho fica como está e recebe a origem 'caminho', porque já é a imagem certa. No segundo passo, a classe junta num só conjunto todos os links que ainda precisam ser abertos (o da Imagem quando não validou, o do PDF e o do Link do conteúdo) e busca todos ao mesmo tempo, pedindo só o começo de cada resposta: o `content-type` e os primeiros bytes já dizem se é um PDF (assinatura `%PDF` ou tipo `application/pdf`), uma página HTML (tipo `text/html`) ou uma imagem (tipo `image/*`); o que não respondeu 200 de um servidor público entra como 'falhou'. Quando o link é uma página, a classe já aproveita essa mesma resposta para separar até 4 imagens de divulgação candidatas (`candidatas()`) e o PDF que a página oferece (`pdfDaPagina()`).

No terceiro passo, para os itens do tipo e-book que ainda não têm imagem, a capa vem da primeira página do PDF: a classe baixa (com limite de 80 MB, a `LIMITE_PDF_CAPA`) o PDF achado nos links do item ou na página de um deles, confere se é mesmo um PDF completo e pede a capa à `CapaPdf::gerar()`, descrita na próxima subseção; a imagem que volta é gravada em `storage/uploads` e o item passa a ter `imagem_origem` igual a 'capa_pdf'. No quarto passo, os itens que continuam sem imagem recebem a primeira das candidatas da página que realmente abre como imagem, com origem 'pagina'; é a imagem de divulgação do curso ou do e-book, a mesma que aparece quando o link é compartilhado nas redes sociais. No quinto e último passo, a classe monta os `aviso_links`: o Link que não abriu, o Link terminado em `.pdf` que devolveu uma página, o link do PDF que não abre um PDF (quando nem a página dele oferece um PDF para servir de reserva) e o e-book sem nenhum PDF achado em lugar nenhum, caso em que o conteúdo fica só com o botão Acessar em vez de Baixar. Esses avisos aparecem na prévia para o Administrador confirmar antes de salvar; a imagem nunca impede o cadastro, porque na falta dela o `CursoDAO` usa o banner da instituição ou a imagem padrão do formato.

O método `candidatas()` procura a imagem de divulgação de uma página HTML em três lugares, na ordem de confiança. Primeiro, as metatags `og:image`, `og:image:secure_url`, `twitter:image` e `twitter:image:src` e o link `rel="image_src"`: são as mesmas que o Facebook e o WhatsApp leem para montar a prévia de um link compartilhado. Segundo, as imagens `<img>` cujo texto alternativo diz "imagem do curso" ou "capa de", ou cujo endereço traz palavras como

imagem_curso, capa ou cover, caso de páginas como a da Escola Virtual.Gov. Terceiro, nas lojas virtuais que usam a plataforma Magento (como a loja do Sebrae), a foto da galeria do produto, que vem escondida dentro de um bloco de JavaScript com o endereço escrito em JSON ("full": "https://..."). De todas as candidatas achadas, a classe descarta qualquer endereço com placeholder, no_selection, logo, favicon, sprite ou avatar no caminho, porque são imagens de reserva do próprio site ou logotipos, não a imagem do conteúdo; o daPagina() devolve só a primeira candidata que sobrou.

Método	O que faz
completar(array \$itens): array	Prévia da importação: abre cada link da ficha (Imagem, PDF e Link), descobre se é imagem, PDF ou página, escolhe a imagem e o PDF do e-book que servem e devolve os itens completados, com os avisos do que não abriu como deveria.
baixar(array \$urls, string \$prefixo): array	Baixa as imagens escolhidas no cadastro, confere se são mesmo JPG, PNG ou WEBP, reduz cada uma para até 800 px de largura e grava em storage/uploads; devolve só as que deram certo (link igual ao caminho salvo).
pdfs(array \$urls): array	Baixa o PDF dos e-books para a biblioteca da plataforma, do link direto ou achado na página (pdfDaPagina()); confere se é um PDF de verdade e completo antes de gravar em storage/uploads.
pdfDaPagina(string \$html, string \$base): string	Acha o PDF que a página do e-book oferece: a metatag citation_pdf_url dos repositórios acadêmicos (como o eduCAPES) ou, na falta dela, o melhor link para um .pdf da própria página, dando preferência ao que tem palavras como baixar, download, e-book, cartilha, guia, manual, livro ou apostila no texto do link.
daPagina(string \$html, string \$base): string	Devolve a imagem de divulgação mais provável da página: a primeira da lista de candidatas().
candidatas(string \$html, string \$base): array	Lista as imagens de divulgação de uma página HTML, da mais provável para a menos provável (metatags og:image e twitter:image, imagem de curso ou capa pelo texto alternativo, foto de produto de loja Magento), já com o endereço completo e sem logotipo, ícone, sprite, avatar ou imagem provisória.
linkPublico(string \$url): bool	Confere se o endereço é http ou https e se o nome do servidor aponta só para IPs públicos, sem cair em localhost, 127.0.0.1 ou qualquer faixa de rede interna.

Quadro 95 – Métodos públicos da ImagemRemota
Fonte: Autoria própria (2026).

Segurança do download. Como a classe busca endereços escolhidos por uma pesquisa (às vezes feita com ajuda de IA), ela nunca confia no link de cara. O url_http_valida() (função utilitária de app/Core/helpers.php) exige um endereço válido que comece por http:// ou https://, o que já descarta javascript:, data: e outros esquemas perigosos. Em seguida, o método privado ipConferido(), o mesmo usado pelo método público linkPublico(), resolve o nome do servidor (gethostbyname()) e confere se todos os IPs encontrados são públicos, pela função nativa filter_var() com as marcas FILTER_FLAG_NO_PRIV_RANGE e FILTER_FLAG_NO_RES_RANGE, que rejeitam as faixas reservadas e privadas (como

127.0.0.1, 192.168.0.0/16 e o IPv6 ::1): sem essa conferência, uma ficha maliciosa poderia apontar para um endereço da rede interna da escola e usar o servidor como ponte para acessar o que não devia (um ataque conhecido como SSRF). Um redirecionamento poderia levar para outro endereço depois dessa primeira conferência, e o nome do servidor poderia até passar a apontar para outro IP entre a conferência e o download (a técnica chamada DNS rebinding). Por isso o cURL não segue redirecionamentos sozinho (CURLOPT_FOLLOWLOCATION desligado): o método privado `transferir()` lê o destino de cada resposta 3xx (CURLINFO_REDIRECT_URL), confere esse endereço novo da mesma forma e só então faz o pedido seguinte, até 5 saltos (a constante REDIRECIONAMENTOS). Em cada pedido, a conexão vai direto ao IP que acabou de ser conferido, fixado com CURLOPT_RESOLVE, sem o cURL consultar o DNS de novo. No fim, a classe ainda confere o IP ao qual o cURL realmente se conectou (CURLINFO_PRIMARY_IP), pelo mesmo `ipPublico()`, e descarta uma resposta vinda de IP privado. Os protocolos aceitos ficam travados em http e https (CURLOPT_PROTOCOLS).

O HTTPS é sempre verificado contra uma lista de autoridades certificadoras, apontada pela constante CERTIFICADOS para `config/cacert.pem` (a lista da Mozilla, baixada de `curl.se/ca`, no formato que o cURL entende). Fizemos isso porque a lista que vem junto do cURL do XAMPP é antiga e passou a recusar certificados de sites que renovaram o certificado com uma autoridade mais nova, como o `ev.org.br` e o site da Fundação Bradesco, usados como fonte de cursos; sem o arquivo próprio, o download falharia por erro de certificado mesmo o site estando correto. As duas requisições também mandam um User-Agent de navegador comum (Chrome no Windows) e o cabeçalho Accept-Language em português, porque alguns sites recusam ou respondem diferente para um cliente sem esses dados.

Downloads simultâneos e limites de tamanho e tempo. Há dois métodos privados de download, `buscar()` e `baixarArquivos()`, e os dois passam pelo `transferir()`, que usa `curl_multi` (abre vários pedidos de uma vez e fica alternando entre eles, em vez de esperar um de cada vez) e segue os redirecionamentos como descrito acima. O `buscar()` é usado para descobrir o que um link é e para validar imagens, pede até 16 links ao mesmo tempo (SIMULTANEAS), com 6 segundos para conectar e 15 segundos no total, e só guarda o conteúdo até o limite (5 MB para imagem, a LIMITE_IMAGEM, ou 512 KB para página, a LIMITE_PAGINA, que já basta porque as metatags ficam no <head>); quando o parâmetro \$parcial é verdadeiro, o começo do conteúdo já serve, mesmo sem a resposta ter terminado. Em ambos os métodos, a função de escrita (CURLOPT_WRITEFUNCTION) conta os bytes recebidos a cada pedaço e devolve 0 assim que passa do limite, o que faz o cURL interromper o download na hora, sem gastar tempo

nem memória com o resto de um arquivo grande demais. A cada redirecionamento, essa função recomeça do zero, porque o conteúdo de uma resposta de redirecionamento não interessa. Já o `baixarArquivos()`, usado para o PDF da capa (até 80 MB) e para o PDF da biblioteca (até 150 MB, a constante `MAX_PDF_REMOTO` de `app/Core/Upload.php`), baixa 6 arquivos por vez direto para um arquivo temporário em disco (`tempnam()`), com 8 segundos para conectar e 180 segundos no total, porque um PDF de livro pode ser grande e lento de baixar; gravar direto no disco evita manter um arquivo de até 150 MB inteiro na memória do PHP. Se o arquivo temporário não puder ser criado (pasta cheia ou sem permissão), só aquele link fica de fora e os outros do lote seguem. Enquanto espera as respostas, o laço chama `curl_multi_select()`; quando essa espera falha na hora (devolve -1, o que acontece em algumas combinações de Windows e cURL), o laço faz uma pausa de 10 milissegundos, em vez de repetir sem parar e ocupar o processador.

Redução e gravação da imagem. O método privado `gravar()` recebe os bytes já baixados, abre com `imagecreatefromstring()` (a GD do PHP, que lê JPG, PNG e WEBP) e, se a largura passar de 800 pixels (a constante `LARGURA`), reduz proporcionalmente com `imagecopyresampled()`; imagens já menores que isso não são ampliadas. Antes de copiar, a classe pinta o fundo da imagem final de branco, para um PNG com transparência não virar uma imagem com fundo preto quando salva em JPEG. O resultado é sempre gravado como JPEG, com qualidade 85, em `storage/uploads`, com um nome aleatório que leva o prefixo recebido (por exemplo `capa` ou `curso`), a data e um trecho aleatório (`random_bytes()`), e a classe devolve o caminho relativo `assets/uploads/....`. Antes de chegar a esse ponto, o `ehImagem()` já confirma, pelo próprio conteúdo do arquivo (`getimagesizefromstring()`, não pela extensão do link), que é de fato um JPG, PNG ou WEBP, com pelo menos 120 por 90 pixels (descarta ícone) e no máximo 25 milhões de pixels (descarta imagem grande demais para reduzir sem gastar memória demais).

PDFs da biblioteca: `pdfs()` e `pdfDaPagina()`. Quando o Administrador marca para guardar o PDF de um e-book na biblioteca da plataforma (o que muda o botão do conteúdo de `Acessar para Baixar`), ou quando roda o botão de trazer os PDFs de todos os e-books de uma vez, o `pdfs()` recebe a lista de links candidatos (o campo PDF, o link oficial ou a página do e-book) e baixa todos com `baixarArquivos()`. Para cada um, o `ehPdf()` confere o conteúdo de verdade: usa o `finfo` (a extensão `fileinfo` do PHP) para exigir o tipo `application/pdf` e lê os últimos bytes do arquivo para achar o marcador `%%EOF`, que todo PDF completo tem no final; um PDF cortado pela metade, por exemplo por ter passado do limite de tamanho, não tem esse marcador e é descartado. Quando o link baixado não é um PDF, mas é uma página HTML de

até 3 MB, a classe tenta mais uma vez, agora com o PDF que o `pdfDaPagina()` acha dentro dela. O PDF aprovado é entregue ao `guardarPdf()`, que o move (ou copia, se mover entre discos diferentes falhar) para `storage/uploads/biblioteca_*.pdf`, com um nome aleatório; é esse prefixo `biblioteca_` que o `ArquivoController::imagem` confere antes de liberar o arquivo ao público, para nunca entregar por engano um currículo em vez de um e-book. O `pdfDaPagina()`, já descrito acima, procura primeiro a metatag `citation_pdf_url`, comum em repositórios acadêmicos como o eduCAPES, e só na falta dela escolhe entre os links que terminam em `.pdf` da página, dando pontuação maior ao que parece ser o botão de baixar o conteúdo.

A.9.8 CapaPdf (app/Services/Extracao/CapaPdf.php e capa-pdf.ps1)

A `CapaPdf` resolve um problema específico: quando a ficha de um e-book não traz uma imagem de capa (ou traz o próprio PDF no lugar da imagem), a capa precisa vir de dentro do PDF. A forma mais fiel de fazer isso é desenhar a primeira página exatamente como ela aparece num leitor de PDF, com o título, as cores de fundo e qualquer texto escrito por cima da ilustração; por isso a primeira tentativa usa o leitor de PDF que já vem instalado no Windows. Só quando essa primeira forma não está disponível, ou falha para aquele PDF específico, a classe recorre a uma segunda forma, mais simples: aproveitar a maior imagem que está desenhada na primeira página, pelo `PdfTexto::capa()` (documentado na subseção A.9.2), que é o nosso leitor de PDF em PHP puro. A diferença entre as duas é que o `PdfTexto` não desenha a página, só lista as imagens dela; então a capa de reserva costuma ser a foto de fundo da capa, sem o título por cima, enquanto a capa desenhada no Windows traz a página completa. Quem chama a `CapaPdf` é a `ImagemRemota`, no terceiro passo do `completar()`, e quem usa o resultado é o próprio chamador, que confere se voltaram bytes de imagem de verdade antes de gravar com `gravar()`.

Método ou arquivo	O que faz
<code>gerar(array \$pdfs): array</code>	Único método público da classe. Recebe um conjunto de PDFs (chave escolhida por quem chama, ligada ao arquivo em disco), manda desenhar a 1ª página de todos de uma vez pelo leitor do Windows (<code>desenharNoWindows()</code>) e, só para os que não deram certo, tenta o <code>PdfTexto::capa()</code> como reserva, dentro de um <code>try/catch</code> ; devolve, para cada chave que conseguiu capa, os bytes da imagem (PNG ou JPEG).
<code>capa-pdf.ps1</code> (script PowerShell)	Recebe, pelo parâmetro <code>-Lista</code> , um arquivo de texto com uma linha por PDF (o caminho do PDF e o caminho do PNG de saída, separados por tabulação) e, por <code>-Largura</code> , a largura de saída (800 por padrão); desenha a 1ª página de cada PDF nessa largura, usando o leitor de PDF do próprio Windows, e grava o resultado como PNG. Um PDF protegido por senha ou com defeito fica sem PNG, e o script segue para o próximo, sem interromper os demais.

Quadro 96 – Métodos da `CapaPdf` e papel do `capa-pdf.ps1`
Fonte: Autoria própria (2026).

O método privado `desenharNoWindows()` só é tentado quando o sistema operacional é Windows, a função `exec()` está liberada no PHP e o script `capa-pdf.ps1` existe. Em vez de chamar o PowerShell uma vez para cada PDF, o que seria lento, porque abrir o PowerShell tem um custo fixo considerável, a classe grava um único arquivo de lista (por `tempnam()`), com uma linha por PDF no formato caminho do PDF, tabulação, caminho do PNG de saída, e chama o script uma única vez, passando essa lista e a largura desejada (800 pixels, a mesma de `ImagemRemota::gravar()`). O comando roda com `powershell.exe -NoProfile -NonInteractive -ExecutionPolicy Bypass -File`, o que evita que o perfil do usuário ou a política de execução do PowerShell do computador interrompam o script com uma pergunta ou um bloqueio. Depois que o script termina, a classe lê cada PNG que foi criado, guarda os bytes e apaga os arquivos temporários (a lista e os PNGs), um por um, em qualquer caso.

Por dentro, o `capa-pdf.ps1` usa o `Windows.Data.Pdf`, a mesma biblioteca de leitura de PDF que o navegador Edge usa por baixo dos panos, e que já vem pronta em qualquer Windows 10 ou 11, sem precisar instalar nada. O problema é que essa biblioteca pertence ao WinRT (Windows Runtime), cujos métodos são assíncronos por natureza (devolvem um `IAsyncOperation` ou um `IAsyncAction`, não o resultado direto), e o PowerShell 5.1 não sabe esperar por eles do jeito comum. Por isso o script usa reflexão (`GetMethods()`) para achar o método `AsTask` do .NET, que transforma essas operações do WinRT em Task comuns, e define duas funções auxiliares: `Esperar()`, para operações que devolvem um resultado (abrir o arquivo, carregar o PDF), e `EsperarAcao()`, para a que só executa uma ação (desenhar a página); as duas chamam `Wait(60000)`, ou seja, esperam no máximo 60 segundos por cada etapa antes de seguir, o tempo que está no quadro de limites de segurança da seção A.9 como "cada operação do leitor de PDF do Windows".

Para cada linha da lista, o script abre o arquivo (`StorageFile.GetFileFromPathAsync`), carrega o PDF a partir dele (`PdfDocument.LoadFromFileAsync`), monta as opções de desenho (`PdfPageRenderOptions`, com `DestinationWidth` igual à largura pedida, o que mantém a proporção da página e define só a largura final), pega a primeira página do documento (`GetPage(0)`, página de número 0, que é a primeira) e manda desenhá-la num fluxo de memória (`RenderToStreamAsync`); depois copia esse fluxo para um arquivo PNG no caminho de saída da lista. Tudo dentro de um try/catch por PDF: se o arquivo estiver protegido por senha, corrompido ou com algum recurso que o `Windows.Data.Pdf` não suporta, aquele PDF específico fica sem PNG e o laço passa para o próximo, sem interromper a geração dos demais; é esse PDF sem PNG que, de volta no PHP, vai cair na reserva do `PdfText::capa()`.

A reserva PdfText::capa() (detalhada na subseção A.9.2) entra quando o computador não é Windows, quando a função exec() está desligada no PHP (comum em hospedagens compartilhadas, por segurança), ou quando o desenho pelo Windows falhou para aquele PDF. Esse método olha só a primeira página do PDF e devolve a maior imagem desenhada nela, inclusive dentro de Form XObjects, aceitando um JPEG (filtro DCTDecode) direto ou uma imagem comprimida com FlateDecode remontada em PNG. Um JPEG em CMYK, porém, é descartado mesmo sendo a maior imagem da página: o CMYK é o espaço de cor de quatro canais (ciano, magenta, amarelo e preto) usado por material pensado para gráfica, e a biblioteca GD do PHP, usada depois para reduzir a imagem a 800 pixels em ImagemRemota::gravar(), interpreta esses quatro canais como se fossem RGB, o que troca as cores e deixa a capa com um tom esverdeado ou invertido; como não há no PHP puro uma forma simples e confiável de converter CMYK para RGB corretamente, preferimos descartar essa imagem e deixar o conteúdo sem capa do que gravar uma capa com as cores erradas. Por não desenhar a página, essa reserva também não traz o texto escrito por cima da ilustração (como o título do e-book), por isso é mesmo uma segunda opção, usada só quando o leitor do Windows não está disponível.

A.10 Telas e componentes (app/Views)

A view é a camada do MVC que só exhibe: ela recebe do controller as variáveis já prontas e monta o HTML, sem consultar o banco e sem aplicar regra de negócio. O ponto de entrada é a classe final View (app/Core/View.php), com o método estático render(string \$__tela, array \$__dados = []). Cada controller chama esse método por dentro do método protegido view() da classe abstrata Controller (app/Core/Controller.php), que os doze controllers estendem; a convenção da casa é terminar toda ação com \$this->view('pasta/tela', get_defined_vars()), de modo que a função nativa get_defined_vars() do PHP entregue à tela todas as variáveis que a ação calculou, sem precisar montar o array à mão nem listar cada uma. É por esse mecanismo que, no restante desta seção, dizemos que uma tela recebe tal variável do controller.

Dentro de render(), extract(\$__dados, EXTR_SKIP) transforma cada posição do array recebido numa variável local da função, e o unset(\$__dados) seguinte tira do caminho a própria variável do array. Os dois parâmetros do método usam o prefixo duplo sublinhado (__tela, __dados) de propósito: assim, mesmo que a tela precise de uma variável chamada \$dados ou \$tela, ela não é sobrescrita pelos parâmetros internos do método. Depois do extract, render() inclui, sempre na mesma ordem, o cabeçalho (app/Views/layouts/header.php), o arquivo da tela

dentro de `app/Views` (o parâmetro chega sem a extensão `.php`, como `vagas/lista`) e o rodapé (`app/Views/layouts/footer.php`). Como os três `require` ficam dentro do mesmo método, as variáveis extraídas continuam visíveis para os três arquivos, e é assim que o cabeçalho conhece, por exemplo, o `$title` da página que a tela ainda nem foi incluída para definir diretamente, ele vem do `controller`, junto com as demais variáveis.

O arquivo `View.php` guarda ainda duas funções globais que não pertencem à classe. A primeira, `pagina_erro(int $codigo, string $titulo, string $mensagem, string $detalhe = "")`, monta as páginas de erro (403, 404, 500 e 503) com a tela `app/Views/errors/erro.php`, que não usa o layout do site, porque precisa funcionar mesmo com o banco fora do ar; ela define o código HTTP, o cabeçalho `Content-Type` e `Cache-Control: no-store`, e encerra a requisição com `exit`. A segunda, `tratar_excecao(Throwable $ex)`, é registrada em `app/Core/bootstrap.php` como tratador global de exceções: ela grava o erro completo no log do PHP/Apache e decide qual `pagina_erro()` mostrar de acordo com o tipo da exceção (`DatabaseException` dá um 503 com dicas de instalação do XAMPP, mas só em ambiente local; `PDOException` dá um 500 genérico; qualquer outra dá um 500 de propósito geral), o detalhe técnico só aparece na tela quando a constante `DEBUG` está ativa, para não revelar informação interna a quem acessa de fora.

A.10.1 Layouts: `header.php`, `footer.php` e `admin_nav.php`

A pasta `app/Views/layouts` guarda as três peças que envolvem toda tela do site. O cabeçalho (`header.php`) é incluído por `View::render()` antes de cada tela e lê, se a ação do `controller` tiver definido, as variáveis `$title`, `$layoutLargo`, `$descricaoPagina`, `$ogImagem` e `$ptMenuTipo`. Ele começa chamando `getFlash()` para buscar a mensagem de uma ação anterior (o padrão `Post/Redirect/Get`) e decide o item ativo do menu principal a partir do endereço atual, devolvido por `Router::atual()`: as funções locais `$emPasta` e `$ehPagina` comparam o caminho do script com listas de nomes de arquivo, e o array `$menu` guarda, para cada item (`vagas`, `cursos`, `ebooks`, `cadastro`, `login`, `perfil`, `portfolio`), se ele deve ficar marcado com a classe `ativo` e o atributo `aria-current=page`. Para o candidato, a aba `Portfólio` só aparece depois que o cadastro do perfil é validado: o cabeçalho chama `Portfolio::validarCadastro()` sobre o perfil da sessão, dentro de um `try/catch` que, em caso de qualquer falha, simplesmente esconde a aba em vez de interromper a página. O restante do cabeçalho monta o `<head>` (título, meta description, meta Open Graph e os cinco links de folha de estilo, cada um com uma versão na própria URL do tipo `?v=10` para forçar o navegador a buscar a versão nova quando o CSS muda) e o menu de navegação, que troca os links de `Cadastro/Login` por `Meu perfil` ou `Painel`, `Portfólio` e `Sair` quando existe sessão aberta.

O rodapé (footer.php) é incluído por `View::render()` depois de cada tela. Ele exige o parcial `partials/doacao.php` e chama `Pix::doacao()` para montar o código copia e cola da doação; o rodapé tem cinco colunas (apresentação do projeto com o convite de doação e o QR Code, Navegação, Sua conta, Links úteis para as instituições parceiras e Contato, com o link para o termo de privacidade), fecha a tag `<main>` aberta no cabeçalho e, por fim, carrega os scripts da página: o `assets/js/vendor/qrcode.js` só é incluído quando existe de fato um código Pix para desenhar (evitando um script sem uso em quem nunca vai doar), e o `assets/js/app.js` é sempre carregado, porque concentra os comportamentos comuns a toda página, como descrevemos na seção A.11. O terceiro arquivo, `admin_nav.php`, é a barra de abas do painel administrativo e da empresa; ele próprio exige o parcial `partials/graficos.php`, de modo que toda tela do painel que o inclui já ganha, de graça, as funções de gráfico e das peças do dashboard. O array `$abas` é montado por regra de acesso: a aba Visão geral aparece sempre; Usuários, Categorias, Cursos e e-books e Assinaturas só para o administrador; Vagas, Candidaturas e Banco de talentos para os dois papéis; e Perfil da empresa e Planos só para a empresa (o administrador não assina plano). A variável `$abaAtiva`, que cada ação do controller define antes de chamar a view, marca qual aba fica destacada.

A.10.2 Partial de componentes do portal: `componentes.php`

O parcial `app/Views/partials/componentes.php` reúne as funções de apresentação que fazem o site parecer um portal de notícias: as vagas aparecem como matérias e os cursos e e-books como reportagens. Ele não acessa o banco em nenhum momento, os dados chegam prontos do controller, e se divide em duas famílias de funções, pelo prefixo do nome. As funções `pt_cuidam` de formatação solta: `pt_dt()` converte uma data do banco em `DateTimeImmutable`; `pt_tempo_relativo()` e `pt_time()` escrevem o tempo relativo (há 5 minutos, ontem, há 2 semanas) dentro de um elemento `<time>` acessível, com a data completa no atributo `title`; `pt_resumo()` corta um texto num tamanho máximo sem quebrar palavra, terminando em reticências; `pt_midia()` desenha a imagem de uma vaga ou curso inteira (`contain`), sobre um fundo desfocado feito da própria imagem, de forma que cartazes em pé e banners deitados fiquem bem enquadrados do mesmo jeito; `pt_local()` escreve a cidade e a UF no formato `Guará/DF`; `pt_lide_vaga()` gera o primeiro parágrafo da notícia a partir dos campos da vaga (empresa, cargo, cidade, regime de contratação, nível e modelo de trabalho); `pt_acesso_conteudo()` decide, pelo próprio endereço do conteúdo, se o botão deve ser Baixar (quando o link é um PDF já guardado em `assets/uploads/biblioteca_*.pdf`, nossa biblioteca) ou Acessar (quando é um link para outro site, que abre em nova aba); e `pt_contatos_vaga()` separa o texto de contato do

anúncio em itens clicáveis de WhatsApp, telefone, e-mail e link, reconhecendo números de celular e a palavra WhatsApp mesmo escrita de formas diferentes.

As funções `cv_` já devolvem blocos de HTML prontos, usados tanto na página inicial quanto nas listas e páginas de vagas e cursos. `cv_faixa()` monta a faixa de título das páginas internas, com o fundo de Brasília e a trilha de navegação (breadcrumb); `cv_titulo_secao()` monta o título de uma seção com ícone, subtítulo e o link ver todas; `cv_card_vaga()` é o cartão de vaga usado na capa, na lista e nas vagas relacionadas, ele mostra o cartaz inteiro (por `cv_cartaz_vaga()`, que segue a mesma lógica de fundo desfocado de `pt_midia()`), a fita de destaque, o selo de match quando existe um candidato logado, os chips de contratação, nível e modelo de trabalho, e os botões Saiba mais e Candidatar-se; e `cv_card_curso()` é o equivalente para cursos, e-books e vídeos, com a capa inteira (sem cortar) no caso dos e-books, o preço, a modalidade e os botões de acesso. Já `cv_paginacao()` monta a paginação numerada das listas (`< Anterior · 1 ... 4 5 6 ... 12 · Próxima >`) e escapa ela mesma cada endereço; quem chama passa o endereço puro, e nenhum link sai sem escape ou escapado duas vezes. Nenhuma dessas funções é um componente de framework: são funções PHP comuns que devolvem uma string de HTML já escapada pela função `e()`, chamadas dentro das próprias views com `<?= ?>`.

A.10.3 Partial dos gráficos e peças do painel: `graficos.php`

O parcial `app/Views/partials/graficos.php` é exigido por `layouts/admin_nav.php` e, por isso, está disponível em toda tela do painel do administrador e da empresa. Ele resolve, sem nenhuma biblioteca externa de gráficos, os quatro tipos de gráfico do dashboard, todos desenhados em SVG direto no servidor: `grafico_barras()` desenha barras horizontais de ranking, com o nome e o valor na linha de cima e a barra fina embaixo, numa largura em porcentagem que acompanha o cartão mesmo no celular; `grafico_linha()` desenha uma série diária com área preenchida, grade discreta, o valor do último ponto e do pico, e uma guia vertical que acompanha o mouse; `grafico_rosca()` desenha até seis fatias com o total no centro e a legenda com valor e percentual ao lado; e `grafico_tabela()` devolve, dentro de um `<details>` recolhido com o resumo Ver dados, uma tabela HTML com os mesmos números do gráfico, para quem prefere ler os valores ou usa leitor de tela, é a alternativa textual que a acessibilidade exige. Cada gráfico recebe um `aria-label` com o resumo dos dados em texto corrido, e cada marca (barra, ponto ou fatia) carrega os atributos `data-valor` e `data-rotulo`, que `assets/js/painel.js` lê para montar a dica (tooltip) que segue o ponteiro do mouse, como explicamos na seção A.11. As cores seguem três papéis fixos, nunca escolhidos à toa: uma cor para série única, uma rampa azul do claro para o escuro quando as categorias têm uma ordem, e três cores categóricas

validadas para daltonismo quando as categorias são papéis distintos (candidato, empresa, administrador, por exemplo), essas cores ficam definidas como variáveis CSS (--gf-s1, --gf-o1 e as demais) em `painel.css`.

O mesmo parcial reúne ainda as peças de interface do painel que não são gráfico, mas seguem o mesmo estilo visual de dashboard corporativo. `painel_kpi()` desenha o cartão de indicador (rótulo, valor grande e uma linha de contexto), que vira um link quando existe uma tela de detalhe; `painel_status()` e `painel_match()` desenharam os selos de situação e de nota de match, sempre com texto, nunca só com a cor; `painel_cabecalho()` monta o título de cada tela do painel, com a legenda Painel administrativo ou Painel da empresa; `painel_th()` desenha o cabeçalho de uma coluna que pode ser clicada para ordenar a tabela, com a seta e o `aria-sort`; `painel_paginacao()` e `painel_subabas()` cuidam da paginação e das abas internas de uma tela; `painel_chave()` desenha a chave liga-desliga (Aberta/Pausada, Publicado/Oculto) como um formulário POST protegido por token CSRF, nunca como um link GET, porque é uma ação que muda dado; `painel_botoes()` desenha a barra de ações de uma linha de tabela (ver, editar, excluir), com ícone e rótulo que encolhe só para o ícone em telas menores; e `painel_miniatra()` mostra a imagem pequena de uma vaga ou curso na lista, com o ícone do tipo quando não há imagem.

A.10.4 Partials de apoio: `icones.php` e `doacao.php`

O parcial `app/Views/partials/icones.php` tem uma única função pública, `icone(string $nome, int $tam = 20)`, que devolve um elemento `<svg>` com `viewBox` fixo de 24 por 24 e `fill=currentColor`, de modo que o ícone sempre herda a cor do texto ao redor, sem precisar de uma regra de CSS própria. Os desenhos de cada ícone (cerca de cinquenta, de vagas e maleta a check e lixeira, passando pela marca do site) ficam guardados num array estático dentro da função, como trechos de `<path>`, e são escritos à mão por nós, sem depender de uma fonte de ícones ou de um pacote externo, é a mesma filosofia do restante do projeto, de não depender de nada de fora para funcionar. O parcial `partials/doacao.php` guarda o texto fixo do convite de doação (as constantes `DOACAO_TITULO` e `DOACAO_TEXTO`) e duas funções: `cv_doacao_qr(string $pix, int $tam = 80)`, que desenha o quadro branco do QR Code, na verdade, um `<div data-qr-code>` vazio, que o JavaScript de `app.js` preenche depois com o SVG do QR Code gerado por `assets/js/vendor/qr-code.js` a partir do código copia e cola; sem uma chave Pix configurada em `config/config.php`, a função mostra no lugar uma imagem de QR fictício ou o aviso Pix em breve, mas o convite e o texto continuam visíveis, e

`cv_doacao_copiar()`, que desenha o botão Copiar código Pix, que também só aparece quando existe uma chave configurada.

A.10.5 Partial do relatório da extração: `relatorio_extracao.php` e o relatório da vaga

O parcial `app/Views/partials/relatorio_extracao.php` mostra o resultado da leitura do currículo, logo depois do envio. Ele é incluído por `require` nas duas telas do candidato que podem mostrá-lo (`perfil/index.php` e `perfil/portfolio.php`), sempre protegido por um `if ($relatorio)`, e devolve sem fazer nada quando a variável não é um array; os dois controllers (`CurriculoController::upload()` e `PerfilController`) montam esse array e o guardam na sessão, em `$_SESSION['relatorio_extracao']`. O parcial conta, a partir de `$relatorio['itens']`, quantos campos foram encontrados, quantos foram de fato aplicados ao perfil, quantos ficaram mantidos (porque o candidato já tinha preenchido e não marcou substituir) e quantos não foram encontrados no arquivo, e mostra esses quatro números no topo. Quando existem campos mantidos, aparece um formulário com uma caixa de marcar por campo, para o candidato escolher trocar pelo valor lido do currículo (ação `view/perfil/aplicar_extracao.php`); um `<details>` recolhido (aberto quando são até trinta itens) lista campo por campo, com o rótulo, o valor lido e um selo de situação (aplicado, mesclado, igual, mantido, nome diferente, lido do currículo ou não encontrado); e, por fim, um aviso lista os campos que faltaram, com um link de volta ao formulário do perfil.

Vale registrar uma decisão de arquitetura: o relatório da extração de uma vaga, mostrado depois que a empresa envia o cartaz ou cola o anúncio, não reaproveita esse parcial. Ele é montado inteiro dentro da própria tela `admin/vagas.php`, com a mesma classe CSS `pf-relatorio` (por isso os dois relatórios têm a mesma cara na tela), mas lendo uma variável diferente, `$relatorioVaga`, vinda de `EmpresaController::vagas()`, com as contagens lidos do anúncio, valor padrão e faltando, e um bloco próprio, `Padrões automáticos usados`, que lista as decisões que a classe `PadroesExtracao` tomou nesta leitura, algo que o relatório do currículo nunca mostra, porque o currículo tem dados pessoais e não participa dos padrões automáticos (ver `docs/PADROES_AUTOMATICOS.md`). Separar os dois evitou um parcial genérico demais, com muitos `if` para dois formatos de dado bem diferentes; o preço dessa escolha é que a estrutura visual das duas telas precisa ser mantida igual à mão, sempre que uma delas muda.

A.10.6 Quadro geral das telas

Arquivo (app/Views/...)	Tela	Quem vê	Dados recebidos do controller
admin/assinaturas.php	Assinaturas (CRUD)	Administrador	\$lista, \$totalLista, \$pagina, \$paginas, \$todas, \$resumo, \$edit, \$contas, \$filtroPlano, \$filtroStatus, \$filtroUsuario, \$busca, \$ordem, \$dir
admin/candidaturas.php	Candidaturas do painel	Administrador e Empresa	\$lista, \$totalLista, \$pagina, \$paginas, \$vagasFiltro, \$vagaId, \$status, \$ordem, \$ordensCand, \$statusPermitidos
admin/categorias.php	Categorias de vagas e cursos	Administrador	\$cats, \$todasCats, \$totalCats, \$pagina, \$paginas, \$edit, \$filtroTipo, \$busca, \$ordem, \$dir
admin/cursos.php	Cursos, e-books e vídeos (CRUD e extração de fichas)	Administrador	\$form, \$extraido, \$cats, \$lista, \$totalLista, \$pagina, \$paginas, \$porTipo, \$filtroTipo, \$filtroCat, \$filtroSituacao, \$busca, \$ordem, \$dir, \$comImagemPadrao, \$imagens, \$importacao, ver A.10.10
admin/empresa_perfil.php	Perfil da empresa	Empresa	\$p (perfil da empresa)
admin/painel.php	Visão geral do painel (dashboard)	Administrador e Empresa	\$resumoVagas, \$porStatus, \$porDia, \$dias, \$funil, \$recebidas, \$matchCandidaturas, \$recentes, \$isEmpresaPremium, \$perfil e, só para o administrador ou só para a empresa, vários outros, ver A.10.8
admin/talentos.php	Banco de talentos	Administrador e Empresa	\$talentos, \$isPremium, \$termo, \$cidade
admin/usuarios.php	Usuários	Administrador	\$usuarios, \$totalUsuarios, \$pagina, \$paginas, \$edit, \$ver, \$filtroTipo, \$filtroSituacao, \$busca, \$ordem, \$dir, \$meuld
admin/vagas.php	Vagas (CRUD e extração de cartaz/anúncio)	Administrador e Empresa	\$form, \$extraido, \$relatorioVaga, \$textoAnuncio, \$parecida, \$cats, \$empresas, \$lista, \$porSituacao, \$filtroStatus, \$busca, \$imagens, \$isPremium, \$ocrDisponivel, \$dinheiro, ver A.10.9
auth/cadastro.php	Criar conta	Qualquer pessoa (sem conta)	\$title (os erros voltam por mensagem flash; os campos, por old())
auth/esqueci_senha.php	Esqueci minha senha	Qualquer pessoa (sem conta)	\$enviado, \$linkDemo (link visível só em ambiente local com DEBUG)
auth/login.php	Entrar na plataforma	Qualquer pessoa (sem conta)	\$title, \$voltar (destino depois do login, ex.: planos)
auth/logout.php	Confirmar saída	Qualquer conta logada	\$title
auth/redefinir_senha.php	Redefinir senha	Qualquer pessoa (sem conta) com link válido	\$usuario (dono do token, ou null), \$token

Arquivo (app/Views/...)	Tela	Quem vê	Dados recebidos do controller
cursos/detalhe.php	Detalhe de curso, e-book ou vídeo	Todos	\$curso, \$id, \$competencias, \$vagasQuePedem, \$outros, \$mapaMatch, \$faltaParaMim, \$formato, \$secaoNome, \$secaoUrl, \$secaoIcône, \$instituicao, \$acesso, \$link, \$abaTipo
cursos/lista.php	Lista de cursos, e-books e vídeos	Todos	\$tipo, \$filtros, \$cursos, \$categorias, \$porTipo, \$recomendados, \$faltantesTop, \$tituloPag, \$subPag, \$nomeFormato, \$abaUrl, \$limparUrl, \$dbErro, \$ordem, \$ordens, \$encontrados, \$pagina, \$paginas, \$porPagina, \$qs
cursos/nao_encontrado.php	Conteúdo não encontrado (404)	Todos	\$dbErro
errors/erro.php	Página de erro (403, 404, 500 ou 503)	Todos (não usa o layout do site)	\$codigo, \$titulo, \$mensagem, \$detalhe, \$inicio
home/contrato.php	Termo de autorização e privacidade (LGPD)	Todos	nenhum (texto fixo)
home/index.php	Página inicial	Todos (dados extras para quem está logado)	\$dbErro, \$slides, \$creditos, \$vagasCapa, \$vagasFila, \$totalVagas, \$conteudosCapa, \$conteudosFila, \$totalConteudos, \$mapaMatch, \$minhas, \$plano
perfil/index.php	Meu perfil (cadastro do candidato)	Candidato	\$perfil, \$validacao, \$completude, \$isVip, \$assinaturaAtiva, \$candidaturasAtivas, \$cv, \$cands, \$relatorio, ver A.10.7
perfil/portfolio.php	Portfólio	O dono; pessoas sem conta e candidatos (completo); Empresa sem Premium (prévia)	\$p, \$ehDono, \$verContato, \$previa, \$nomeExibido, \$resumo, \$experiencias, \$formacao, \$habilidades, \$cursos, \$idiomas, \$extras, \$links, \$ehPcd, \$local, \$iniciais, \$titulo, \$relatorio, \$matches, \$exibirMatches, \$cursosAtivos, \$isVip, \$limiteGratis, ver A.10.7
planos/index.php	Planos e assinaturas	Todos veem; assinar exige Candidato ou Empresa	\$dao, \$usuarioLogado, \$usuarioId, \$assinaturaAtiva, \$historicoAssinaturas
vagas/candidatar.php	Candidatar-se à vaga	Candidato	\$vaga, \$vid, \$permissao, \$cv, \$isVip
vagas/detalhe.php	Detalhe da vaga	Todos (match só para o candidato logado)	\$vaga, \$id, \$aberta, \$ehDona, \$competencias, \$match, \$candidatura, \$recom, \$cursosRel, \$relacionadas, \$empresa, \$quando, \$atendidas, \$linkVaga, \$textoShare, \$trilha, \$salario, \$requisitos, \$beneficios, \$contatos, \$expira

Arquivo (app/Views/...)	Tela	Quem vê	Dados recebidos do controller
vagas/lista.php	Lista de vagas	Todos	\$filtros, \$vagas, \$total, \$pagina, \$paginas, \$porPagina, \$todas, \$categorias, \$catAtual, \$contagem, \$mapaMatch, \$minhas, \$ordenarMatch, \$filtrosAtivos, \$trilha, \$dbErro, \$qs
vagas/nao_encontrada.php	Vaga não encontrada (404)	Todos	nenhum

Quadro 97 – Todas as views de app/Views, a tela que mostram, quem vê e os dados recebidos do controller
Fonte: Autoria própria (2026).

A.10.7 A tela mais importante do candidato: perfil e portfólio

A tela Meu perfil (app/Views/perfil/index.php, rota view/perfil/index.php) funciona como o cadastro do candidato e é a porta de entrada para todo o resto do sistema do lado de quem procura emprego. Ela recebe de PerfilController::index() o próprio \$perfil, o resultado de Portfolio::validarCadastro() em \$validacao (que diz se o cadastro já está completo e, se não estiver, quais campos obrigatórios faltam, a partir da constante Portfolio::OBRIGATORIOS), a porcentagem \$completude para a barra de progresso, a situação do plano VIP (\$isVip, \$assinaturaAtiva, \$candidaturasAtivas), a lista de currículos enviados (\$cvcs), as candidaturas do candidato (\$cands) e, quando existe, o \$relatorio da última extração. A tela se divide em três partes: no topo, o cabeçalho do perfil com foto, nome, título profissional e a barra de progresso do cadastro, seguido do aviso de cadastro completo ou da lista do que falta; no meio, a máquina de extração do currículo (um formulário de envio de PDF, DOCX ou DOC de até 10 MB, com a opção substituir, que troca os dados já preenchidos pelos do currículo em vez de só completar os campos vazios) e, logo abaixo, o próprio formulário do cadastro, com todos os campos profissionais; e, na coluna lateral, o resumo do plano, o resumo dos números do cadastro, a lista de candidaturas com o botão de cancelar e o bloco de exclusão da própria conta, que pede a senha atual e exige a confirmação por caixa de marcar antes de apagar perfil, currículos, foto, candidaturas e match, a aplicação direta do direito de exclusão da LGPD.

Quando o cadastro é validado, a aba Portfólio é liberada no menu e o candidato chega à tela app/Views/perfil/portfolio.php (rota view/perfil/portfolio.php[?id=]), que é o portfólio público montado a partir do mesmo perfil. PerfilController::portfolio() decide, antes de chamar a view, quem pode ver o quê: \$ehDono diz se é o próprio candidato vendo o próprio portfólio; \$verContato libera e-mail, telefone e links para o dono, para o administrador, para quem já recebeu uma candidatura daquele candidato ou para empresa com plano Premium; e \$previa, quando verdadeiro, faz a tela mostrar só uma amostra (nome mascarado, título, local, habilidades e um resumo curto), reservando nome completo, foto, contato, experiências e

formação para quando a empresa tiver o Premium, é a mesma prévia que aparece no Banco de talentos. As experiências chegam como uma lista (\$experiencias) que a tela transforma numa linha do tempo, a formação (\$formacao) aparece organizada por curso e instituição, e habilidades, idiomas e cursos complementares aparecem em listas simples. Só o dono vê a seção de máquina de match, no final da página: ela compara o portfólio com cada vaga aberta nas quatro partes do cálculo (competências, cargo, localização e nível), mostra a nota em \$matches e \$exibirMatches, com um <details> Por que essa nota? por vaga, que explica cada parte da pontuação e sugere cursos do catálogo para cobrir as competências que faltam (Competencias::cursosPara()); quem não é VIP só vê as primeiras vagas compatíveis, até o limite \$limiteGratis, com um convite para assinar e ver todas. A tela também é a primeira a mostrar o relatório da extração do currículo (o mesmo parcial relatorio_extracao.php descrito em A.10.5), logo depois que o candidato envia um arquivo novo.

A.10.8 A visão geral com gráficos: admin/painel.php

A tela admin/painel.php (rota admin/index.php) é o painel de indicadores, no estilo dos dashboards do Power BI, e é a primeira coisa que o administrador e a empresa veem ao entrar na área logada. AdminController::painel() decide o escopo dos números pela variável \$pid: para a empresa, é o id do perfil dela; para o administrador, fica null, valor que os DAOs entendem como o sistema todo. A tela monta uma grade de cartões (\$resumoVagas, \$porStatus, \$porDia, \$funil, \$matchCandidaturas, \$recentes chegam para os dois papéis) com a linha de indicadores (KPI) no topo, para o administrador, usuários, vagas abertas, candidaturas, match médio, assinaturas vigentes e cursos publicados; para a empresa, vagas abertas, candidaturas, candidaturas aguardando análise, match médio, visualizações e taxa de candidatura, seguida dos gráficos de candidaturas por dia (grafico_linha) e do funil de seleção (grafico_barras), que valem para os dois papéis. Só o administrador vê, além disso, vagas abertas por área e por região, usuários por tipo (grafico_rosca), a compatibilidade de match entre candidatos e vagas abertas e as assinaturas vigentes por plano, todos vindos de variáveis que só existem no ramo isAdmin() da ação (\$usuariosPorTipo, \$novosUsuarios, \$vagasPorArea, \$vagasPorCidade, \$matchVagas, \$planos, \$cursosPublicados, \$ebooks, \$vagasRecentes). Só a empresa vê, no lugar desses, candidaturas por vaga e o match dos seus candidatos, a partir de \$desempenho, uma linha por vaga seguida de VagaDAO::desempenhoPorEmpresa().

Cada cartão de gráfico segue o mesmo padrão: um título com o resumo do período, o gráfico SVG desenhado pelas funções de partials/graficos.php (descritas em A.10.4) e, embaixo, a tabela Ver dados da mesma função, recolhida por padrão. No fim da página ficam

as tabelas com as candidaturas mais recentes (para os dois papéis) e, no lugar que sobra, as vagas cadastradas mais recentes para o administrador ou o desempenho detalhado de cada vaga para a empresa, com link direto para avaliar as candidaturas daquela vaga ou editá-la. A tela também dispara, de passagem, a manutenção diária do sistema: quando quem está olhando é o administrador, `AdminController::painel()` chama `manutencao_diaria()`, que no máximo uma vez por dia apaga de `storage/uploads` os arquivos enviados que já não pertencem a nenhum registro, sobra de contas excluídas, de cartazes lidos e não usados ou de capas tiradas de fichas de curso que não foram salvas. Por fim, a tela carrega o script `assets/js/painel.js`, responsável só pela dica (tooltip) que aparece ao passar o mouse sobre qualquer marca dos gráficos.

A.10.9 Vagas com o relatório da extração: `admin/vagas.php`

A tela `admin/vagas.php` (rota `admin/pages/vagas.php`) reúne o CRUD de vagas da empresa e do administrador com as duas máquinas de extração de anúncio: pelo cartaz (imagem) e pelo texto colado. `EmpresaController::vagas()` entrega a ela `$form` (os dados do formulário, já preenchidos pela extração ou pela edição de uma vaga existente), `$extraido` (o resultado bruto da extração, com avisos e, quando veio do cartaz, a confiança da leitura), `$relatorioVaga` (o relatório campo por campo, só quando algo foi extraído), `$textoAnuncio`, `$parecida` (uma vaga aberta igual, se o sistema encontrar antes de publicar), `$cats`, `$empresas` (só para o administrador escolher a empresa), `$lista` já filtrada e paginada, `$isPremium` e `$ocrDisponivel`, entre outras. A tela abre com uma caixa `<details>` de extração, com dois formulários: o primeiro envia a foto do cartaz (JPG, PNG ou WEBP de até 8 MB) e dispara a leitura assim que o arquivo é escolhido, sem precisar clicar em nada, o atributo `data-leitor-cartaz` no formulário aponta para a pasta do leitor (`assets/js/vendor/tesseract/`), que o script `leitor-cartaz.js` usa para rodar o OCR direto no navegador, como descrevemos em A.11; o segundo formulário recebe o texto do anúncio, seja colado pela empresa ou já preenchido com o texto que o OCR leu do cartaz, para ser corrigido e extraído de novo.

Quando existe um `$relatorioVaga`, a tela mostra, na mesma estrutura visual do relatório do currículo (classe CSS `pf-relatorio`, mas um bloco próprio desta tela, como explicamos em A.10.5), os números de campos lidos do anúncio, com valor padrão e faltando, os avisos da extração (por exemplo, quando o OCR não está disponível neste servidor) e, dentro de um `<details>` aberto, a tabela campo por campo com o que foi lido, o que ficou no valor padrão para o usuário confirmar e o que não foi encontrado. Quando a extração usou algum dos padrões automáticos da classe `PadroesExtracao`, por não ter uma palavra-chave fixa para uma linha, uma área ou um nome de empresa, , aparece ainda o bloco Padrões automáticos usados, com

cada decisão tomada, o texto da linha e o padrão que valeu, o que deixa visível para a banca e para a própria empresa como o sistema aprendeu com os cadastros anteriores. Se o sistema encontrar uma vaga aberta igual, a tela mostra um alerta com o link para a vaga existente antes de deixar publicar outra igual. Depois da extração e do relatório vem o formulário completo da vaga para revisão e, por fim, a lista das vagas já cadastradas, com os filtros, a situação (aberta, pausada, vencida) e os botões de ação de cada linha, montados por `painel_botoes()` e `painel_chave()`.

A.10.10 Cursos com a prévia das fichas: `admin/cursos.php`

A tela `admin/cursos.php` (rota `admin/pages/cursos.php`, só para o administrador) é enxuta de propósito: uma única caixa de extração, seguida do formulário ou da prévia, e depois a lista. `AdminController::cursos()` entrega a ela `$form` e `$extraido` (quando uma única ficha ou um texto de divulgação foi colado), `$importacao` (quando várias fichas foram coladas de uma vez, separadas por ---) e os dados de listagem (`$lista`, `$porTipo`, os filtros, a ordenação e a paginação). A caixa Extrair recebe tanto a ficha de pesquisa, com os campos Título, Tipo, Instituição, Modalidade, Cidade, Nível, Carga horária, Gratuito, Preço, Área, Link, PDF, Imagem e Descrição, quanto um texto de divulgação comum; o próprio placeholder do campo mostra o modelo da ficha, para orientar quem vai colar o resultado de uma pesquisa feita numa ferramenta de IA. Quando o texto colado é uma única ficha, a tela mostra o formulário já preenchido, do mesmo jeito que a extração de vaga faz com o anúncio, com um aviso de onde veio a imagem (capa tirada da primeira página do PDF, imagem achada na página do conteúdo ou nenhuma imagem, caso em que a imagem padrão da plataforma será usada até o administrador trocar).

Quando o texto colado traz várias fichas, a tela mostra, em vez do formulário, a prévia de importação em lote: uma tabela com uma linha por conteúdo, cada uma com uma caixa de marcar (já destravada e marcada quando a ficha não tem problema, e travada e desmarcada quando tem), a miniatura da imagem encontrada, o título, o tipo, a instituição, a modalidade, a área, o endereço do link de origem e um selo de situação, Pronto, em cor neutra quando há só um alerta a conferir (por exemplo, sem imagem ou um link que não abriu como deveria), ou bloqueado quando a ficha está sem título, sem link válido ou já existe um conteúdo cadastrado com o mesmo link ou o mesmo título e instituição. Essa prévia é resultado de o controller mandar `ImagemRemota::completar()` abrir cada link da ficha (o de imagem, o do PDF e o do próprio conteúdo) e descobrir o que cada um é: imagem, PDF (que vira a capa do e-book, tirada da primeira página do arquivo) ou página (de onde o sistema tenta tirar o PDF do e-book e a

imagem de divulgação, descartando logotipos e ícones). Quando existe algum e-book na prévia, aparece ainda a caixa de marcar Guardar na nossa biblioteca o PDF dos e-books, que faz o botão de acesso virar Baixar em vez de Acessar depois de salvo. Confirmando com o botão Cadastrar marcados, cada linha não bloqueada é gravada de uma vez; Descartar prévia joga tudo fora sem salvar nada.

A.11 JavaScript e estilos (public/assets)

O projeto não usa nenhum framework de front-end: todo o JavaScript da plataforma são cinco arquivos próprios, pequenos e sem build (nenhum passa por um compilador ou empacotador antes de ir ao ar), mais duas bibliotecas de terceiros guardadas dentro do próprio site, em public/assets/js/vendor. Os cinco arquivos próprios são carregados só onde são usados: app.js vem no rodapé do layout e por isso entra em toda página; anuncios.js só nas telas de vaga; leitor-cartaz.js só na tela de cadastro de vaga, que tem a máquina de extração pelo cartaz; e painel.js só nas telas do painel, que têm gráfico. Essa divisão evita carregar código sem uso e segue a mesma lógica das folhas de estilo, descritas no fim desta seção: cada arquivo cuida de um pedaço do site e nenhum deles depende de bibliotecas externas além das duas guardadas em vendor, para o projeto continuar funcionando sem internet, como o README explica.

A.11.1 app.js: comportamentos comuns de toda página

O arquivo public/assets/js/app.js é carregado no rodapé de toda página (layouts/footer.php) e reúne os comportamentos que não são de uma tela específica. Tudo roda dentro de um único ouvinte DOMContentLoaded, selecionando os elementos pelos atributos data- que as views já deixam prontos no HTML, em vez de por classe ou id, o que deixa claro no próprio HTML qual comportamento cada elemento tem. Os blocos são: a confirmação de ações destrutivas, pelo atributo data-confirm, que mostra um confirm() nativo do navegador antes de deixar o clique seguir; o esmaecimento da mensagem flash depois de cinco segundos (data-flash); o botão Copiar (data-copiar), usado no código Pix da doação e no link de compartilhar uma vaga ou curso, que tenta a API navigator.clipboard e, se o site estiver em HTTP comum (sem contexto seguro), cai para um textarea escondido com execCommand('copy') ou, no último caso, um prompt() com o texto para copiar à mão; o desenho do QR Code de doação (data-qrcode), que só roda se a função qrcode do vendor/qrcode.js estiver carregada, dentro de um try/catch que ignora qualquer falha de geração; o carregador das máquinas de extração, uma barra que fica amarela enquanto o

formulário com data-carregando está sendo enviado (e trava os botões, para impedir o clique duplo) e volta ao estado normal se a pessoa voltar pelo botão Voltar do navegador, graças ao evento pageshow; e o envio automático da máquina de extração de vaga (data-auto-envio), que mostra a prévia da imagem escolhida e envia o formulário sem precisar de um clique extra, exceto quando o leitor de cartaz do navegador (leitor-cartaz.js) já assumiu o controle do envio.

O restante do arquivo cuida dos componentes de rodízio e de acessibilidade da página inicial e do menu. O carrossel de fotos de Brasília (data-carrossel) troca de slide a cada cinco segundos, com os créditos da foto, respeitando prefers-reduced-motion para quem pede menos movimento no sistema; a vitrine rotativa de vagas e de conteúdos (data-rotativo) troca uma vaga por vez de um cartão da grade, com a vaga que saiu voltando para o fim de uma fila (um `<template>` escondido no HTML guarda as próximas), pausando com o mouse ou o foco em cima, com a aba escondida ou pelo botão Pausar; e o painel relâmpago (data-relampago) é o balão de avisos que troca de mensagem (quem somos, objetivo, missão, valores e o convite de doação por Pix) de tempos em tempos, lembrando por sessionStorage que foi fechado, para não voltar a aparecer por três minutos. Por fim, o script cuida de mostrar e ocultar a senha nos campos de login e cadastro (data-mostrar) e fecha o menu do celular com a tecla Esc.

A.11.2 anuncios.js: cartaz ampliado, compartilhar e contador de caracteres

O arquivo public/assets/js/anuncios.js atende só as telas de vaga (vaga.php e candidatar.php). Ele abre o cartaz da vaga ampliado dentro de um elemento `<dialog>` nativo do HTML (data-ampliar), sem nenhuma caixa de diálogo própria em JavaScript; se o navegador não suportar `<dialog>` (sem o método showModal), o link simplesmente abre a imagem numa aba nova, pelo próprio href, sem quebrar nada. Dentro da caixa ampliada, clicar na própria imagem alterna entre o tamanho ajustado e o tamanho real (classe zoom), clicar fora da imagem ou no botão de fechar encerra a caixa, e o foco volta para o link que abriu a caixa quando ela fecha, o que é importante para quem navega só pelo teclado. O script também liga o botão de Compartilhar, que só aparece (b.hidden=false) quando o navegador tem a Web Share API (navigator.share), abrindo o menu nativo de compartilhamento do aparelho com o título e o link da vaga; e liga o contador de caracteres da carta de apresentação da candidatura (data-contador), que atualiza a cada tecla digitada, mostrando quantos caracteres já foram usados do limite do campo.

A.11.3 leitor-cartaz.js e o Tesseract.js (vendor/tesseract): o OCR no navegador

O arquivo `public/assets/js/leitor-cartaz.js` é a parte de que mais nos orgulhamos no front-end: ele lê o texto do cartaz da vaga inteiramente no navegador de quem está enviando a foto, usando o Tesseract.js (uma biblioteca de OCR em WebAssembly) guardada em `public/assets/js/vendor/tesseract` e servida pelo próprio site, nada é baixado de um serviço externo, e nada precisa ser instalado no servidor. O script só assume o controle quando encontra na página um formulário com o atributo `data-leitor-cartaz` (presente em `admin/vagas.php`) e quando o navegador tem WebAssembly e Worker disponíveis; sem essas condições, o envio segue o caminho normal do `app.js` e o cartaz é mandado para o servidor, que tenta ler com o Tesseract do lado de lá, se estiver instalado (`OcrImagem`). Ao escolher o arquivo, o script trava os campos do formulário, mostra a prévia da imagem e passa por quatro etapas: primeiro carrega o leitor, criando um ou dois workers (threads separadas do navegador, cuja quantidade depende de a memória e os núcleos do aparelho serem suficientes) com o idioma português; depois prepara a imagem num `<canvas>`, convertendo para cinza com correção de gama (para a letra amarela ou laranja sobre fundo claro não desaparecer), ampliando para cerca de 2200 pixels de largura (cartazes de rede social chegam pequenos, com letra fina) e gerando também o negativo da imagem, com mais contraste, para quando a letra é clara sobre uma faixa escura; em seguida lê a imagem nos dois modos do Tesseract (página inteira e texto solto) para as duas versões (normal e negativo), quatro leituras ao todo, cada uma devolvendo uma tabela TSV de palavras, posição e confiança; e por fim anexa as quatro leituras como campos ocultos (`ocr_tsv[]`) e envia o formulário, para o servidor montar o resultado final com a mesma calibragem que o leitor do próprio servidor usaria. Se qualquer etapa falhar (falta de memória, imagem com tamanho inválido), o script desiste e envia o cartaz sem as leituras, deixando o servidor tentar.

O leitor começa a carregar antes mesmo de a pessoa escolher o arquivo, para já estar pronto na hora: ele é disparado quando a caixa de extração (o `<details>`) é aberta, quando o campo de arquivo recebe o foco ou, se a caixa já estiver aberta ao carregar a página, logo que o navegador estiver livre (`requestIdleCallback`). Enquanto carrega e lê, a barra do carregador (a mesma peça visual de `app.css`) fica amarela e mostra a porcentagem de progresso calculada a partir dos eventos do próprio Tesseract.js; ao terminar, fica azul com o tempo total da leitura. A pasta `vendor/tesseract` guarda os arquivos que o Tesseract.js precisa para funcionar sem acessar a internet: `tesseract.min.js` e `worker.min.js` (o código da biblioteca), a pasta `core` com três versões do motor de reconhecimento compiladas em WebAssembly (uma simples, uma com instruções SIMD e uma com SIMD relaxado, para o navegador escolher a mais rápida que

o aparelho suporta) e a pasta lang com o modelo de linguagem treinado para português (por.traineddata.gz), usado para reconhecer as palavras.

A.11.4 painel.js: a dica dos gráficos

O arquivo public/assets/js/painel.js é pequeno e tem uma única responsabilidade: desenhar a dica (tooltip) que aparece ao passar o mouse ou o ponteiro sobre uma barra, um ponto ou uma fatia dos gráficos SVG montados por partials/graficos.php (A.10.3), carregado só nas telas do painel que têm gráfico. O script cria, uma única vez, um elemento <div> fixo (fora do fluxo normal da página) com um valor em destaque e um rótulo embaixo, e o mantém escondido até o ponteiro passar sobre um elemento com o atributo data-valor, dentro de um gráfico (seletor .gf[data-valor]); o texto da dica vem sempre por textContent, nunca por HTML, o que evita qualquer risco de um valor malformatado quebrar a página. A dica segue o ponteiro (pointermove) e se ajusta sozinha para não sair da tela, tanto na borda direita quanto na borda de cima, e desaparece quando o ponteiro sai do elemento (pointerout) ou quando a página rola (scroll), para não ficar flutuando fora do lugar. Por usar os eventos pointerover, pointermove e pointerout em vez de mouseover e mouseout, o mesmo script funciona tanto com o mouse quanto, em telas sensíveis ao toque, com o dedo.

A.11.5 Bibliotecas de terceiros guardadas no site: vendor/qrcode.js

Além do Tesseract.js (descrito em A.11.3), a pasta public/assets/js/vendor guarda uma segunda biblioteca de terceiros, qrcode.js, de Kazuhiko Arase, que gera o desenho do QR Code inteiramente no navegador, em SVG, a partir de um texto qualquer. Ela é usada só pelo QR Code de doação por Pix, no rodapé de toda página: o parcial partials/doacao.php deixa pronto um <div data-qrcode> com o código copia e cola do Pix (montado por Pix::doacao(), no padrão do Banco Central) e, quando existe de fato uma chave Pix configurada, app.js chama a função global qrcode() dessa biblioteca para desenhar o QR Code dentro desse elemento. Como o QR Code nunca precisa ser gerado no servidor nem enviado para um serviço externo gerar a imagem, o código Pix do doador nunca sai do navegador dele antes de ser desenhado, e o arquivo assets/js/vendor/qrcode.js só é carregado pelo rodapé (footer.php) quando existe de fato uma chave Pix configurada, para não baixar um script sem uso.

A.11.6 app.css e site.css: a base visual e o layout do site

O arquivo public/assets/css/app.css é o mais antigo dos cinco e guarda a base de todo o visual: as variáveis de cor em :root (azul institucional, dourado, verde e vermelho), o estilo dos

botões (.btn e as variações .btn-outline, .btn-gold, .btn-danger), dos cartões, formulários, tabelas, alertas e do carregador das máquinas de extração (a barra que fica amarela enquanto carrega e azul quando termina, com uma animação de listras que para sozinha quando o sistema do usuário pede menos movimento), além dos estilos que o perfil e o portfólio ainda usam deste arquivo mais antigo (avatar, painel de perfil, grade de duas colunas). Já o arquivo site.css, maior e mais novo (identificado no próprio cabeçalho do arquivo como a versão 5 do layout, com o prefixo cv- em toda classe), cuida do cabeçalho, do rodapé, da página inicial e das páginas internas de vagas e cursos: a faixa de título com o fundo de Brasília, os cartões de vaga e de curso no estilo de notícia e reportagem, o carrossel de fotos do topo, a vitrine rotativa, o painel relâmpago dos avisos, a grade de planos e assinaturas e a aba Portfólio liberada no menu. Os dois arquivos convivem porque o site cresceu por partes, o layout mais antigo (app.css) nunca foi totalmente substituído onde ainda funciona bem, como no perfil, e as duas folhas são carregadas juntas em toda página, pelo cabeçalho do layout.

A.11.7 anuncios.css e portfolio.css: vagas e portfólio

O arquivo public/assets/css/anuncios.css, com o prefixo an-, cuida especificamente do cartão de vaga, da lista de vagas (vagas.php), da página de uma vaga (vaga.php, com o cartaz ampliado em <dialog>) e da tela de candidatura (candidatar.php); ele reaproveita as variáveis de cor já definidas em site.css, para manter a mesma identidade visual, e define os seus próprios pontos de quebra responsiva: a partir de 1180 pixels de largura a página da vaga vira duas colunas (cartaz à esquerda, título e candidatura à direita); abaixo de 760 pixels tudo volta a ficar numa coluna só, na ordem título, cartaz, candidatura, seções, match e cursos recomendados; e abaixo de 560 pixels os cartões da lista ficam deitados, com o cartaz pequeno à esquerda, para caber mais vaga por rolagem de tela no celular. O arquivo public/assets/css/portfolio.css, por sua vez, é o mais focado dos cinco: cuida só do relatório da extração do currículo (as classes pf-rel-, com a tabela campo por campo e os selos de situação) e da máquina de match dentro do portfólio (pf-match-, com a grade de vagas compatíveis e o <details> que explica cada parte da nota); os dois blocos têm uma regra @media print que os escondem por completo, porque o botão Salvar em PDF do portfólio imprime a página sem esses dois blocos, que não fazem sentido num currículo impresso.

A.11.8 painel.css: os dashboards do administrador e da empresa

O arquivo public/assets/css/painel.css, com os prefixos pn- para as telas do painel e gf- para os gráficos SVG de partials/graficos.php, define no seletor .pn as variáveis de cor só dos

gráficos: as cores de apoio (fundo, grade, linha de base), as três cores categóricas (--gf-s1 a --gf-s3, escolhidas e comentadas no próprio arquivo como validadas para daltonismo e com contraste de pelo menos 3 para 1 sobre fundo branco) e a rampa ordinal de azuis, do mais claro ao mais escuro (--gf-o1 a --gf-o4), usada quando as categorias têm uma ordem natural, como as faixas de match ou as etapas do funil de seleção. O arquivo também estiliza as abas do painel, o cabeçalho de cada tela, os cartões de indicador (KPI), a grade de cartões do dashboard, a tabela Ver dados escondida dentro de cada gráfico, a dica do mouse criada por painel.js, os selos de situação (sempre com texto, nunca só com a cor), as tabelas com ordenação por coluna, as subabas, a paginação, a chave liga-desliga e a barra de botões do CRUD, a ficha de uma conta de usuário, a máquina de extração de vagas e a prévia de importação de cursos em lote, e o Banco de talentos. A responsividade empilha a grade de cartões em telas menores (de doze colunas para seis, quatro e, por fim, uma só, conforme a largura encolhe) e, abaixo de 1200 pixels, escondemos o texto dos botões da barra de ações (deixando só o ícone, com o rótulo preservado para quem usa leitor de tela, pela classe sr-only), para a tabela não ficar apertada demais no notebook.

A.11.9 Responsividade e acessibilidade: um fio comum nas cinco folhas de estilo

Nenhuma das cinco folhas de estilo depende de uma biblioteca de componentes prontos (como Bootstrap): a responsividade é feita à mão, por @media, em pontos de quebra parecidos nos cinco arquivos (por volta de 1180, 980, 900, 760, 640 e 560 pixels de largura), sempre reduzindo o número de colunas da grade (de vaga, de curso, de indicador ou de botão) e empilhando o que não cabe mais numa linha, em vez de encolher a fonte a ponto de ficar difícil de ler. Vários componentes, como os atalhos de área nos filtros de curso e os pontos de um gráfico no celular, preferem a rolagem lateral de uma faixa só a quebrar em várias linhas, para caber a mesma informação numa tela estreita sem perder a organização. Em telas de impressão, uma regra @media print comum às cinco folhas esconde os elementos que não fazem sentido no papel: o relatório da extração, a máquina de match, o painel relâmpago e as barras de ação do painel, deixando o portfólio e o currículo imprimirem só o conteúdo profissional.

Do lado da acessibilidade, o cuidado aparece em cada arquivo: o link Pular para o conteúdo (.cv-pular), escondido fora da tela e visível só quando recebe foco pelo teclado; o contorno de foco bem visível em qualquer fundo claro ou escuro (body.cv :is(a,button,summary,[tabindex]):focus-visible, com um contorno azul e uma sombra dourada por cima); a classe utilitária .sr-only, repetida nas folhas que precisam dela, para texto que só o leitor de tela lê, usada em botões que ficam só com ícone, em links que abrem em nova

aba e nos selos de match e de empresa Premium; as cores de selo e de gráfico escolhidas para manter o contraste mínimo de 4,5 para 1 pedido pelo WCAG AA, com uma seção dedicada em `site.css` só para esse ajuste de contraste; e a regra `prefers-reduced-motion`, presente em quatro das cinco folhas (`app.css`, `site.css`, `anuncios.css` e `painel.css`), que desliga a animação do carregador, do carrossel, da vitrine rotativa e das transições de hover para quem configurou o sistema para reduzir movimento. É a mesma preocupação que já aparece no HTML das views, com o `aria-label` dos gráficos, o `aria-sort` das colunas ordenáveis e o `role=switch` da chave liga-desliga: o CSS só termina, visualmente, o que a marcação já começou a tornar acessível.

A.12 Testes automáticos e proteção do código

Como o projeto não usa framework, também escrevemos nossos próprios testes, em PHP puro, na pasta `tests/`. São três testes complementares (sintaxe, teste rápido e jornadas), um atalho que roda os três com um clique duplo (`verificar.bat`) e um gancho do Git (`pre-commit`) que roda a mesma verificação sozinho antes de cada commit, para nenhuma quebra chegar ao repositório sem ninguém perceber. Também documentamos aqui o `resetar_senhas.php`, que não é bem um teste, mas é a ferramenta de linha de comando que devolve o sistema a um estado conhecido quando algo dá errado com o login durante os ensaios da apresentação.

A.12.1 tests/lint.php

`lint.php` é o teste mais simples e mais rápido: confere se todo arquivo PHP do projeto tem sintaxe válida, antes de qualquer outra verificação. Ele percorre recursivamente as pastas `app`, `config`, `database`, `docs`, `public` e `tests` com um `RecursiveDirectoryIterator`, reúne todo arquivo terminado em `".php"` (mais o `index.php` da raiz) e roda `"php -l"` (lint, a própria checagem de sintaxe do PHP) em cada um deles com `exec()`. Qualquer arquivo com erro de sintaxe é listado com o caminho e a mensagem do PHP; no final imprime quantos arquivos têm erro (ou confirma que todos estão certos) e termina com código de saída 1 se houver qualquer falha, esse código de saída é o que `tests/verificar.bat` e o gancho `.github/hooks/pre-commit` usam para decidir se continuam ou param. Só roda pelo terminal (`PHP_SAPI !== 'cli'` devolve 403), porque não faz sentido nem é seguro expor essa checagem pela web.

A.12.2 tests/smoke.php

`smoke.php` é o teste rápido: confere, em um único arquivo, se o sistema "está de pé", sem gravar nada no banco (a única mudança é +1 no contador de visualizações da vaga 1, usada

como exemplo). Ele se divide em quatro partes, na ordem em que aparecem no arquivo e no texto que imprime no terminal.

A parte 1, Classes (autoloader), varre as pastas `app/Controllers`, `app/Models`, `app/DTO`, `app/Services` e `app/Services/Extracao` com `glob()`, pega o nome de cada arquivo e confere, com `class_exists()` ou `trait_exists()`, se o Autoloader (`app/Core/Autoloader.php`) realmente consegue carregar cada uma dessas classes quando usada pela primeira vez. Isso pega de cara um erro comum: um arquivo cujo nome não bate exatamente com o nome da classe lá dentro.

A parte 2, Regras de negócio, é a maior: dezenas de verificações diretas sobre as funções e classes de `app/Core` e `app/Services`, sem precisar do navegador nem do banco (os padrões automáticos são desligados no início dessa parte, com `PadroesExtracao::ligar(false)`, para o que já está cadastrado no banco não interferir no resultado esperado). Entre outras coisas, confere: as funções de apoio (`decimal_ou_null`, `salario_texto`, `rotulo`, `maskar_email`, `caminho_upload`, `caminho_imagem_valido`); o dicionário de competências (`Competencias::extrair`, inclusive casos em que um texto parecido NÃO deveria virar competência); a extração de vagas a partir de vários textos de anúncio e de cartazes reais do DF (título, salário sem confundir com vale-transporte, cidade, cargos escritos em várias linhas, códigos de vaga, prefixos como "Temporário -"); duas leituras reais de OCR do navegador, salvas em `tests/amostras/` (cartazes das empresas fictícias "Smile & Face" e "Mimória"), conferindo que palavras partidas pelo OCR são corrigidas e que a marca é reconhecida mesmo escrita em duas linhas; o cálculo do CRC16 do Pix contra o exemplo oficial do Banco Central; a extração de cursos e de fichas de e-books (instituição, duração, preço, link, imagem, PDF da biblioteca), inclusive o formato de ficha colado por diferentes IAs de pesquisa; a escolha da foto de um currículo (um "retrato" sintético vence um "logotipo" sintético, e uma foto embutida num PDF é encontrada); as regras de ordenação e paginação das tabelas do painel; e a nota de match de um candidato ideal, que deve fechar em 100 pontos.

A parte 2b, Padrões automáticos das máquinas de extração, testa a classe `PadroesExtracao` inteiramente em memória, com vagas e cursos inventados (sem tocar no banco): confere que um termo visto em pelo menos dois cadastros diferentes e no mesmo destino vira padrão, que um termo visto uma única vez não vira, que a normalização do texto (`PadroesExtracao::termos`) tira acento, maiúscula e pontuação e troca número por "#", que uma linha sem pista nas regras é decidida pelo padrão aprendido dos cadastros (mas a regra continua mandando quando ela já tem uma pista) e que nomes genéricos de empresa ("Loja", "Empresa Teste") nunca entram como nome conhecido.

A parte 3, Banco de dados, conecta de verdade no MySQL (`Database::getConexao()`) e confere: se as 11 tabelas do `database/schema.sql` existem; se os padrões automáticos montados a partir das vagas e cursos do banco não saem vazios; se as três contas de teste existem com as senhas do README (e se a tolerância do login a erros comuns de digitação funciona contra o hash salvo); se nenhuma vaga está numa categoria de curso (nem o contrário); e se toda vaga aberta e todo curso/e-book publicado tem uma imagem e o arquivo dela realmente existe no disco.

A parte 4, Páginas (HTTP), já faz pedidos de verdade pela rede (com `file_get_contents` e um `stream_context`, sem seguir redirecionamentos) contra um Apache ligado, por padrão calculando o endereço a partir da própria pasta do projeto dentro do `htdocs` (ou recebendo a URL base pelo primeiro argumento da linha de comando). Confere o código HTTP esperado de cerca de 20 endereços (200 nas páginas públicas e nos arquivos do leitor de cartaz `Tesseract.js`, 404 numa vaga inexistente e num endereço qualquer, 302 nas páginas que exigem login); confere que cada formato de conteúdo (curso, e-book, vídeo) só mostra cartões do próprio formato e que a página inicial tem a vitrine rotativa de vagas, cursos e e-books, cada uma com seu próprio ritmo; confere os cabeçalhos de segurança descritos na seção A.3 e a ausência de `X-Powered-By`; planta um arquivo `.php` dentro de `public/assets` na hora e confirma que ele recebe 403 em vez de ser executado (a blindagem do `public/.htaccess`); e confirma que arquivos internos como `config/config.php`, `cacert.pem`, `app/Core/Database.php`, `database/schema.sql` e o próprio `tests/smoke.php` respondem bloqueados (403 ou 404). No final, imprime RESULTADO com a contagem de falhas e termina com código de saída 1 se houver qualquer uma.

A.12.3 tests/jornadas.php

`jornadas.php` usa o sistema exatamente como uma pessoa usaria, pelo navegador: uma classe auxiliar chamada `Navegador` (definida no próprio arquivo, usando `cURL`) guarda os cookies da sessão entre os pedidos, segue redirecionamentos, lê o token CSRF escondido na página atual, lê o aviso (flash) exibido e lê o valor de um campo do formulário, tudo isso para simular, com requisições HTTP reais, alguém clicando pela interface. Uma segunda função auxiliar, `docx_de_teste()`, monta um arquivo `.docx` mínimo (mas válido, no mesmo formato do Word) com `ZipArchive`, para o teste poder enviar um "currículo" de verdade. O teste cria uma conta de candidato temporária, com um e-mail novo a cada execução (carimbo de data/hora + bytes aleatórios), e percorre sete etapas seguidas.

Etapa	O que o teste faz
1. Cadastro, saída e login	Cria a conta pelo formulário de cadastro.php e confirma que ela já entra direto no perfil; sai pelo logout e confirma que o perfil deixa de abrir; tenta entrar com a senha errada (recusado); entra com a senha certa, mas com a primeira letra trocada de maiúscula para minúscula, confirmando a tolerância do login a esse erro comum de digitação.
2. Currículo preenche o perfil	Monta um currículo DOCX sintético com nome, contato, resumo, experiência, formação e habilidades, envia por curriculo_upload.php e confere que o arquivo foi salvo no servidor e que a extração preencheu os campos de habilidades/competências do perfil (com "excel" entre elas).
3. Candidatura	Entra com a conta de empresa de teste, busca no banco uma vaga aberta dessa empresa, envia a candidatura em nome do candidato temporário por candidatar.php e confirma que ela existe no banco e aparece na lista de candidaturas que a empresa vê.
4. Empresa: extração de vagas	Cola um texto de anúncio em admin/pages/vagas.php (acao=extrair) e confere que o formulário é preenchido sem nada ser salvo; envia a caixa vazia e confere que a extração é barrada com o aviso certo; envia uma imagem de cartaz junto com as leituras reais de OCR do navegador (tests/amostras/ocr_navegador_smile.json) e confere que o cargo e a empresa anunciante saem certos; por fim, descarta o cartaz lido que não chegou a ser salvo.
5. Administrador: ficha de curso	Entra como administrador e cola uma ficha de curso em admin/pages/cursos.php (acao=extrair), conferindo que o formulário é preenchido sem nada ser salvo.
6. Esqueci minha senha	Pede o link de redefinição pelo e-mail da conta temporária e confere que, no modo de demonstração, o link aparece no log (storage/logs/redefinicoes_senha.log) com o e-mail mascarado e sem o e-mail completo; em modo de produção (APP_DEBUG=0), confere que nada é gravado no log.
7. LGPD: exclusão da própria conta	Tenta excluir a conta com a senha errada (recusado) e sem marcar a confirmação (recusado); exclui de verdade com a senha certa e a confirmação marcada, e confere que a conta, o arquivo do currículo e as tentativas de login registradas somem do banco e do disco; por fim, confirma que a conta excluída não consegue mais entrar.

Quadro 98 – Jornadas de tests/jornadas.php, passo a passo

Fonte: Autoria própria (2026).

O teste conta 18 verificações ao todo. Qualquer passo que falhe é registrado, mas a execução segue até o fim para não deixar a conta temporária pela metade; no bloco finally, a conta (se ainda existir), as tentativas de login dela e os arquivos temporários (o DOCX e o PNG de teste) são sempre apagados, e o log de redefinições de senha é cortado de volta ao tamanho que tinha antes do teste começar, mesmo numa falha no meio do caminho, nada do teste fica para trás no sistema. Como a extração do currículo chama ZipArchive e o php.ini padrão do XAMPP vem com a extensão zip desligada, o teste exige curl_init e ZipArchive logo no início e falha cedo, com uma mensagem clara, se alguma das duas não estiver disponível.

A.12.4 tests/verificar.bat

verificar.bat é o atalho pensado para quem vai usar o sistema sem abrir o terminal: um clique duplo entra na pasta do projeto e roda, em sequência, php tests\lint.php, depois tests\smoke.php e por fim tests\jornadas.php, parando no primeiro que falhar (if errorlevel 1 goto erro). Antes de rodar as jornadas, o script confere se a extensão ZipArchive já está ligada (com um teste de uma linha em php -r) e, se não estiver, liga a extensão zip só para aquela

execução (-d extension=zip), sem precisar editar o php.ini do XAMPP; o próprio site, fora dos testes, não depende dessa extensão. No final, mostra "TUDO CERTO: pode usar e fazer commit." ou, se algo quebrou, o aviso com a dica de voltar à tag tcc-final-v1.1 do Git e de restaurar o banco a partir de um dos backups em storage/backups.

A.12.5 .githooks/pre-commit

pre-commit é um script shell (chamado automaticamente pelo Git, pelo VS Code ou por qualquer outro cliente, logo antes de criar um commit) que roda a mesma blindagem do verificar.bat, mas de forma silenciosa e bloqueando o commit se algo falhar. Ele é ativado uma vez, por computador, com "git config core.hooksPath .githooks" (é por isso que ele não roda sozinho numa cópia nova do repositório sem esse comando). O script localiza o php.exe do XAMPP em um de dois caminhos prováveis (ou cai para "php" do PATH); roda tests/lint.php primeiro e já bloqueia o commit se houver erro de sintaxe; roda tests/smoke.php jogando a saída inteira em .git/smoke_ultimo.txt e só mostra no terminal as linhas com "FALHA" ou "RESULTADO" (para não poluir o terminal do commit com 500 linhas), bloqueando o commit se o código de saída não for zero; e por fim roda tests/jornadas.php do mesmo jeito, em .git/jornadas_ultimo.txt, ligando a extensão zip só para aquela execução. Qualquer uma das três etapas que falhe impede o commit (exit 1) e aponta onde está o relatório completo; em uma emergência (por exemplo, com o XAMPP desligado durante uma correção pequena), o próprio comentário do script documenta a saída de emergência: "git commit --no-verify". O .gitattributes do projeto garante que este arquivo seja sempre salvo com quebra de linha LF (um script shell não tolera CRLF), enquanto verificar.bat é mantido em CRLF, o padrão do Windows.

A.12.6 database/resetar_senhas.php

resetar_senhas.php não é um teste automático, mas a ferramenta de linha de comando para quando algo trava um ensaio ou uma apresentação: senha esquecida, conta bloqueada por engano ou login em pausa depois de muitas tentativas erradas (inclusive as que os próprios testes às vezes geram). Rodado com "C:\xampp\php\php.exe database\resetar_senhas.php", ele exige o mesmo app/Core/bootstrap.php do site, conecta no banco e faz duas coisas: primeiro, devolve as três contas de teste (administrador, empresa e candidato) às senhas publicadas no README, gravando um novo hash com password_hash() e reativando a conta (ativo=1) de uma vez; segundo, apaga todas as linhas da tabela tentativas_login, o que tira imediatamente qualquer pausa de segurança do login. O script só roda pelo terminal (a pasta database/ está

bloqueada ao navegador por database/.htaccess) e imprime, conta por conta, se a senha foi confirmada ou se a conta ainda não existe (sinal de que falta importar database/seed.sql). Como a troca de senha muda a "marca" conferida por Session::revalidar_sessao() a cada requisição (ver seção A.2.7), qualquer sessão que estivesse aberta com a senha antiga dessas contas é encerrada sozinha assim que for usada de novo, e a própria mensagem final do script avisa disso.